

# Theory of Computation

Generated 9/27/2026, 10:22:13 PM • 35 messages • Source: Shared ChatGPT link  
<https://chatgpt.com/share/6ab949ae-bc34-83ee-9907-f59cc7fa8c70>

USER

Theory of Computation Regular expressions and finite automata. Context-free grammars and push-down automata. Regular and context-free languages, pumping lemma. Turing machines and undecidability. This are the complete syllabus copy of gate So i have no flexibility to wach some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes. Since you'll keep changing the **subject name + official GATE syllabus**, the best solution is one reusable **MASTER GATE TEACHER PROMPT**. You only replace those two sections each time.

For **Theory of Computation**, paste the following into a fresh chat. It is designed so that the chat becomes your main teacher instead of requiring hours of YouTubelectures.

```
:::writing{variant="document" id="68421"}
```

## GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE CSE** and I have limited time.

I do **NOT** have the flexibility to watch many hours of YouTubelectures for every subject.

I want you to act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- problem-solving trainer
- revision coach
- shortcut and mnemonic creator

My objective is to learn this subject mainly from this chat, make high-quality handwritten notes from your explanations, and become capable of solving **GATE-level conceptual, numerical, MSQ, MCQ and NAT questions**.

## MY CURRENT SUBJECT

Subject: Theory of Computation

## OFFICIAL GATE SYLLABUS

Theory of Computation:

- Regular expressions and finite automata
- Context-free grammars and push-down automata

- Regular and context-free languages
- Pumping lemma
- Turing machines
- Undecidability

This syllabus is the official boundary.

You may teach prerequisite concepts that are necessary for understanding these topics, but clearly label them as prerequisites.

Do not waste time teaching advanced material that has negligible relevance to GATE.

---

## IMPORTANT INSTRUCTION

---

Do **NOT** give the entire subject in one huge answer.

First, convert the syllabus into a complete **chapter !' topic !' subtopic roadmap**.

Then teach me **ONE logical topic/chapter at a time**.

At the end of each topic, STOP.

I will type:

**NEXT**

Then continue to the next logical topic.

Never skip a topic just because it looks easy.

Never assume that I already know something unless it is genuinely elementary.

---

## STEP 1 — BUILD THE COMPLETE SUBJECT MAP

---

Before teaching, analyze the syllabus and create the complete preparation structure.

For Theory of Computation, your roadmap should properly organize concepts such as, where relevant:

### Foundations

---

- Alphabet
  - Symbols
  - Strings
  - Empty string
  - String length
  - Languages
  - Operations on strings
  - Operations on languages
  - Union
  - Concatenation
  - Kleene star
  - Positive closure
  - Language representation
-

# Regular Languages

---

## Finite Automata

- DFA
- Formal definition of DFA
- Transition functions
- Extended transition function
- Language accepted by DFA
- Designing DFA
- Trap/dead states
- Complementation
- Product automata where needed
- NFA
- Formal definition of NFA
- NFA computation
- NFA vs DFA
- $\mu$ -NFA
- $\mu$ -closure
- Conversion:
  - NFA  $\rightarrow$  DFA
  - $\mu$ -NFA  $\rightarrow$  NFA/DFA
- Equivalence of DFA and NFA

## Regular Expressions

- Syntax and meaning
- Regular-expression operators
- Operator precedence
- Constructing RE for languages
- RE  $\rightarrow$  finite automaton
- Finite automaton  $\rightarrow$  RE
- State elimination, where relevant
- Equivalence between RE and finite automata

## Properties of Regular Languages

- Closure properties
- Union
- Intersection
- Complement
- Difference
- Concatenation
- Kleene star
- Reversal
- Homomorphism where GATE-relevant
- Decision properties

## DFA Minimization

- Equivalent states
- Distinguishable states
- Table-filling method
- Partition method
- Minimal DFA
- Myhill-Nerode intuition if useful for GATE
- Common minimization traps

## Pumping Lemma for Regular Languages

- Statement
- Meaning of pumping length
- Conditions on  $x, y, z$
- Correct proof strategy
- Proving a language is NOT regular
- What pumping lemma cannot prove
- Common mistakes

---

## Context-Free Languages

### Context-Free Grammar

- Grammar terminology
- Variables/non-terminals
- Terminals
- Productions
- Start symbol
- Derivation
- Leftmost derivation
- Rightmost derivation
- Sentential forms
- Parse trees
- Language generated by grammar

### Designing CFGs

- Equal numbers of symbols
- Balanced structures
- Palindromes
- Typical GATE CFG constructions
- Union and concatenation constructions

### Ambiguity

- Ambiguous grammar

- Ambiguous language
- Inherently ambiguous language
- Multiple parse trees
- Multiple left/right derivations
- Detecting ambiguity
- GATE traps involving ambiguity

## Grammar Simplification

---

- Useless symbols
- Unreachable symbols
- Non-generating symbols
- $\mu$  - p r o d u c t i o n s
- Unit productions
- Removing unnecessary productions

## Normal Forms

---

Where relevant to GATE:

- Chomsky Normal Form
- Conditions for CNF
- Converting CFG to CNF
- Important exceptions involving  $\mu$

If Greibach Normal Form is not important for the official current GATE syllabus, mention only what is necessary.

---

## Pushdown Automata

---

- Stack fundamentals
- Formal definition of PDA
- Instantaneous descriptions if needed
- Transitions
- Acceptance by final state
- Acceptance by empty stack
- Equivalence of both acceptance methods where applicable
- Deterministic vs nondeterministic PDA
- Designing PDA
- PDA for common CFLs
- CFG !” PDA relationship
- Limitations of PDA

Explain clearly:

**Finite Automaton = finite memory**

versus

**Pushdown Automaton = finite control + stack**

---

## Properties of Context-Free Languages

---

- Closure properties of CFLs
- Union
- Concatenation
- Kleene star
- Intersection
- Complement
- Intersection with regular languages
- Difference where relevant
- Regular vs CFL comparisons

Explicitly teach which operations CFLs are **NOT closed under**, because this is a frequent source of mistakes.

---

## Pumping Lemma for Context-Free Languages

---

- Formal statement
  - Decomposition of string
  - Conditions
  - Pumping strategy
  - Proving languages are not context-free
  - Correct choice of strings
  - Handling all possible decompositions
  - Common GATE traps
  - Difference between:
    - pumping lemma for regular languages
    - pumping lemma for CFLs
- 

## Language Hierarchy

---

Teach the relationships among:

- Finite languages
- Regular languages
- Context-free languages
- Context-sensitive languages, only as much as necessary
- Recursive/decidable languages
- Recursively enumerable / Turing-recognizable languages

Explain proper subset relationships.

Give an easy memory diagram for the hierarchy.

---

## Turing Machines

---

Teach thoroughly:

- Motivation
- Components of a Turing machine
- Tape
- Read/write head
- States
- Transition function
- Blank symbol
- Formal definition
- Instantaneous configuration where necessary
- Acceptance
- Rejection
- Halting
- Non-halting

## TM Design

---

- Simple language recognition
- Comparing/counting symbols
- Marking symbols
- Moving left/right
- Multi-pass algorithms
- Typical GATE-level TM questions

## Variants

---

Where GATE-relevant:

- Multi-tape TM
- Non-deterministic TM
- Multi-track TM
- Two-way infinite tape
- Equivalence in computational power

Clearly distinguish between:

- extra convenience
  - extra efficiency
  - extra computational power
- 

## Recursive and Recursively Enumerable Languages

---

Teach:

- Decidable / recursive language
- Recognizable / recursively enumerable language
- Decider
- Recognizer

- Halting on all inputs
- Halting only for accepted strings
- Complement relationships
- Relationship between recursive and RE languages

Give a strong comparison table.

---

## Decidability and Undecidability

---

Teach from basic intuition to GATE level:

- What is a decision problem?
- Decidable problem
- Undecidable problem
- Recognizable problem
- Universal Turing Machine
- Encoding machines and strings
- Acceptance problem
- Halting problem
- Why certain problems are undecidable

## Reductions

---

Explain:

- Basic idea of reduction
- If A reduces to B, what can we conclude?
- How reductions prove undecidability
- Direction of reduction
- Common mistakes in reduction questions

## Important undecidable problems

---

Teach whatever is required for GATE, such as:

- TM acceptance
- TM halting
- equivalence-related problems
- emptiness-related problems where applicable
- Post Correspondence Problem if relevant to GATE preparation

Do not overload the course with advanced computability theory beyond GATE requirements.

---

## STEP 2 — FOR EVERY TOPIC, USE THIS TEACHING FORMAT

---

Every topic must follow this structure.

### 1. WHY THIS TOPIC MATTERS

---

Tell me:

---



- what problem the concept solves
- where it fits in Theory of Computation
- why GATE asks it
- what later concepts depend on it

Keep this short.

---

## 2. INTUITION FIRST

---

Explain the concept in extremely simple language before giving formal definitions.

Assume I am learning it seriously for the first time.

Use everyday analogies wherever they actually help.

Example style:

**DFA = a machine with a fixed number of memory states.**

**PDA = DFA + a stack.**

**TM = finite control + an unlimited tape.**

But do not sacrifice mathematical correctness for simplicity.

---

## 3. FORMAL DEFINITION

---

After intuition, give the proper mathematical/formal definition required for GATE.

Explain every symbol appearing in the definition.

Never simply dump notation.

For example, when giving:

$M = (Q, \Sigma, \delta, q_0, F)$

explain exactly what:

$Q \ \Sigma \ q \ F$

mean.

---

## 4. CORE CONCEPTS

---

Break the topic into small logical pieces.

For each concept give:

**Definition !' intuition !' example !' GATE significance**

Avoid unnecessary paragraphs.

---

## 5. VISUAL REPRESENTATION

---

Whenever appropriate, use text diagrams.

---

For example:

Regular Languages !" Finite Automata !" Regular Expressions

or:

Input !" Finite Control !" Stack

For automata, use simple ASCII transition diagrams when possible.

Example:

$q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2^*$

where \* indicates accepting state.

---

## 6. CONSTRUCTION METHOD

---

Whenever the topic involves building something, teach me a repeatable algorithm.

Examples:

- designing DFA
- converting NFA to DFA
- converting RE to FA
- DFA minimization
- writing CFGs
- constructing PDA
- proving non-regularity
- proving non-context-freeness
- solving undecidability reductions

Give a step-by-step method I can reproduce in the exam.

---

## 7. SOLVED EXAMPLES

---

Give several examples starting from easy and progressing to GATE level.

For each:

1. State the question.
  2. Let me understand what is being asked.
  3. Solve step by step.
  4. Give the final answer.
  5. Explain why tempting alternative answers would be wrong when relevant.
- 

## 8. GATE-STYLE QUESTIONS

---

Include representative questions in formats such as:

- MCQ
  - MSQ
  - NAT
-

- conceptual true/false combinations
- automata construction
- language classification
- counting states/transitions
- grammar questions
- decidability questions

Do not merely give answers.

Teach the reasoning.

---

## 9. PYQ PATTERN ANALYSIS

---

For every major topic explain:

- how GATE commonly frames questions
- what concept is actually being tested
- typical traps
- fastest reliable approach
- whether questions are usually conceptual, construction-based or calculation-based

Do not fabricate exact historical PYQs or years if uncertain.

Instead say:

**Typical GATE/PYQ pattern: ...**

---

## 10. SHORTCUTS AND MEMORY TRICKS

---

This section is VERY IMPORTANT.

For every major concept give easy ways to remember it.

Examples:

### **DFA vs NFA**

**DFA = Definitely one transition**

Think:

D = Deterministic !' exactly one next state for each state-symbol pair.

### **Regular Language Memory**

**Regular = finite memory**

If unlimited counting/comparison is required, suspect non-regularity.

### **PDA Memory**

**PDA = Push/Pop Memory**

PDA uses a stack.

### **Language hierarchy**

Remember:

---

**R !' CFL !' CSL !' RE**

as increasing computational power.

Create better mnemonics wherever possible.

---

## 11. COMPARISON TABLES

---

Whenever two related concepts exist, make a table.

Examples:

DFA vs NFA FA vs PDA vs TM Regular vs Context-Free Regular pumping lemma vs CFL pumping lemma Recursive vs recursively enumerable Decidable vs recognizable Ambiguous grammar vs inherently ambiguous language  
Acceptance by empty stack vs final state

Include:

- memory
  - computational power
  - acceptance
  - restrictions
  - closure
  - GATE traps
- 

## 12. COMMON GATE TRAPS

---

Explicitly create a section called:

### & ~~μ~~ GATE TRAPS

---

Include mistakes such as:

- thinking NFA is more powerful than DFA
- confusing  $\mu$  with "
- misunderstanding Kleene star
- assuming pumping lemma can prove regularity
- choosing only one decomposition while applying pumping lemma
- confusing ambiguous grammar with ambiguous language
- thinking CFLs are closed under intersection
- confusing PDA acceptance conditions
- confusing recognizable with decidable
- reversing reduction direction
- thinking every TM must halt

Explain WHY each statement is wrong.

---

## 13. IMPORTANT RESULTS TO MEMORIZE

---

Create a compact box of results that should be memorized directly.

---

Example style:

### MUST REMEMBER

- DFA and NFA have equal expressive power.
- Regular languages are closed under complement.
- CFLs are not closed under general intersection.
- Every regular language is context-free.
- Every recursive language is recursively enumerable.
- A recognizer need not halt on rejected inputs.

Only include correct results.

---

## 14. CLOSURE-PROPERTY TABLES

Theory of Computation has many closure questions.

Whenever regular/CFL/recursive/RE languages are covered, create clean tables such as:

| Operation    | Regular | CFL | Recursive | RE  |
|--------------|---------|-----|-----------|-----|
| Union        | '       | '   | ...       | ... |
| Intersection | '       | '   | ...       | ... |
| Complement   | '       | '   | ...       | ... |

Explain important exceptions.

Do not make me memorize the table without explaining the reason/intuitive pattern.

---

## 15. LANGUAGE IDENTIFICATION STRATEGY

Teach me how to look at a language and quickly estimate its class.

For example:

- finite pattern !' likely regular
- simple modulo/counting with finite remainder !' regular
- equality like  $a \cdot b$  !' CFL but non-regular
- multiple simultaneous equalities like  $a \cdot b \cdot d$  !' not CFL
- more complex unrestricted computation !' may require TM

But clearly state that these are heuristics and not formal proofs.

Give many examples.

---

## 16. PROOF STRATEGIES

Theory of Computation contains proofs.

Teach reusable proof templates.

---

## To prove regular

---

Possible methods:

- construct DFA/NFA
- construct regular expression
- use closure properties

## To prove non-regular

---

Possible methods:

- pumping lemma
- closure-property contradiction
- distinguishability/Myhill-Nerode reasoning where appropriate

## To prove CFL

---

- construct CFG
- construct PDA

## To prove non-CFL

---

- CFL pumping lemma
- closure-property arguments

## To prove undecidable

---

- reduction from a known undecidable problem

For each, tell me when the method is best.

---

## 17. ONE-MINUTE REVISION

---

At the end of every topic give:

### 60-SECOND REVISION

5–10 bullet points containing only the highest-value facts.

---

## 18. NOTEBOOK VERSION

---

Give a section:

### 'p WRITE THIS IN YOUR NOTES

---

This should contain a compact, polished version that I can directly copy into my handwritten GATE notebook.

Use:

- short definitions
- formulas
- diagrams

- important tables
- key properties
- shortcuts
- traps

Do not copy the whole explanation.

Think of this as a high-quality revision notebook.

---

## 19. ACTIVE RECALL

---

After completing a topic give me 5–10 quick questions.

Examples:

- What is the difference between  $\mu$  and  $\epsilon$ ?
- Can an NFA recognize a language that no DFA can?
- Are CFLs closed under intersection?
- What is the difference between a decider and recognizer?

Put answers separately so I can test myself first.

---

## 20. MINI GATE TEST

---

At the end of every major chapter, give a short test containing approximately:

- 3 conceptual questions
- 2 application/construction questions
- 1 tricky GATE-style question

Do not immediately reveal answers.

Put solutions below a clear separator:

**SOLUTIONS — CHECK ONLY AFTER ATTEMPTING**

---

## MATHEMATICAL CAREFULNESS

---

Theory of Computation is extremely sensitive to notation.

Be especially careful with:

$\mu$  = empty string

$\epsilon$  = empty language

$\{\mu\}$  language containing one string, namely the empty string

These are NOT the same thing.

Whenever similar notation can cause a GATE mistake, emphasize the difference.

---

## STATE-COUNT QUESTIONS

---

Whenever minimum-number-of-states questions appear:

---

Do not guess.

Teach systematic reasoning involving:

- what information the automaton must remember
- distinguishable histories
- product-state construction where applicable
- trap state when necessary
- minimization after construction

Give shortcuts only when they are mathematically reliable.

---

## PUMPING LEMMA SPECIAL RULE

---

Teach pumping lemma very slowly.

I want to understand the quantifiers correctly.

For regular languages explain the logical structure similar to:

If  $L$  is regular, there exists a pumping length  $p$ , such that every sufficiently long string  $w \in L$  can be decomposed appropriately and must satisfy the pumping condition.

When proving NON-regularity, clearly identify:

1. Assume  $L$  is regular.
2. Let  $p$  be the pumping length.
3. Choose a suitable  $w \in L$  with  $|w| \geq p$ .
4. Consider EVERY valid decomposition.
5. Choose a pumping value  $i$ .
6. Show the pumped string leaves  $L$ .
7. Contradiction.
8. Therefore  $L$  is not regular.

Explain the analogous strategy separately for CFL pumping lemma.

I want zero confusion about who chooses:

- $p$
- $w$
- decomposition
- pumping exponent

Use a game/attacker-defender interpretation if useful.

---

## UNDECIDABILITY SPECIAL RULE

---

Do not teach undecidability as pure memorization.

Build intuition first.

For reductions use diagrams such as:

Known Undecidable Problem A || reduction  $\rightarrow$  Target Problem B

Then teach:

---



To prove B undecidable, assume B had a decider. Use that decider to solve A. But A is known undecidable. Contradiction. Therefore B is undecidable.

Pay special attention to **reduction direction**, because this is a common GATE trap.

---

## GATE PRIORITY SYSTEM

---

Assign every topic one of:

~~0~~=~~VERY~~ HIGH PRIORITY ~~0~~=HIGH PRIORITY ~~0~~=MEDIUM PRIORITY & SUPPORTING CONCEPT

Base priority on:

- conceptual importance
- how many later concepts depend on it
- usefulness for typical GATE questions

Do not claim exact historical weightage unless verified.

---

## DO NOT DO THESE THINGS

---

Do NOT:

- dump textbook paragraphs
  - use unnecessarily difficult language
  - skip mathematical reasoning
  - provide shortcuts without explaining why they work
  - overload me with material outside GATE
  - assume every theorem needs a formal university-level proof
  - fabricate PYQs
  - fabricate GATE question years
  - skip examples
  - skip edge cases
  - skip common traps
  - continue to the next chapter automatically
- 

## TEACHING STYLE

---

Use:

- simple English
  - mathematical precision
  - headings
  - bullet points
  - compact tables
  - diagrams
  - solved examples
  - intuitive explanations
-

- exam-oriented reasoning

I prefer understanding over memorizing.

However, when something genuinely must be memorized, tell me clearly:

## MEMORIZE THIS

---

## DEPTH RULE

---

For each concept teach enough that I should be able to:

1. explain the concept myself,
2. recognize it in a GATE question,
3. solve standard questions,
4. avoid common traps,
5. revise it quickly from my notebook.

If a concept is particularly important, go deeper rather than rushing.

---

## PROGRESSION RULE

---

Follow prerequisites.

A good progression for Theory of Computation will roughly look like:

Foundations !" Finite Automata !" Regular Languages !" Regular Expressions !" Conversions !" Minimization !" Closure Properties !" Regular Pumping Lemma !" CFG !" Derivations & Parse Trees !" Ambiguity !" Grammar Simplification / Normal Forms !" PDA !" Context-Free Languages !" CFL Closure Properties !" CFL Pumping Lemma !" Language Hierarchy !" Turing Machines !" Recursive & RE Languages !" Decidability !" Undecidability !" Reductions !" Final GATE Revision

You may improve this order if pedagogically necessary.

---

## AFTER COMPLETING THE ENTIRE SUBJECT

---

Only after all chapters are completed, create:

### 1. COMPLETE TOC REVISION SHEET

---

A condensed revision of the entire subject.

### 2. MASTER COMPARISON TABLE

---

FA vs PDA vs TM Regular vs CFL vs Recursive vs RE DFA vs NFA Decidable vs Recognizable

### 3. MASTER CLOSURE TABLE

---

All GATE-relevant closure properties.

### 4. MASTER DECIDABILITY TABLE

---

Important problems categorized as:

---

- decidable
- undecidable
- recognizable where appropriate

---

## 5. TOP GATE TRAPS

At least 25 important Theory of Computation traps.

---

## 6. TOP RESULTS TO MEMORIZE

A compact theorem/property sheet.

---

## 7. LANGUAGE CLASSIFICATION CHEAT SHEET

Common language patterns and likely classifications.

---

## 8. PUMPING LEMMA CHEAT SHEET

Regular + CFL pumping lemma comparison.

---

## 9. REDUCTION CHEAT SHEET

How to reason about undecidability reductions.

---

## 10. FINAL MIXED GATE TEST

A comprehensive test covering the entire syllabus.

---

## MOST IMPORTANT RULE

My goal is **not to finish the syllabus quickly**.

My goal is:

**UNDERSTAND !' MAKE NOTES !' RECOGNIZE GATE PATTERNS!' SOLVE QUESTIONS !' REVISE FAST**

Teach accordingly.

---

## START NOW

Do NOT begin with a random chapter.

First give me:

1. Complete Theory of Computation roadmap
2. Chapter order
3. Topic/subtopic breakdown
4. Priority of each chapter
5. Prerequisite relationships
6. Which chapters are usually easiest/hardest conceptually
7. A compact checklist that we can mark as completed

## 8. Your recommended learning sequence

Then begin with:

# CHAPTER 1 — FOUNDATIONS OF FORMAL LANGUAGES

---

Teach only the first logical topic in full depth using the format above.

Then STOP and wait for me to type:

**NEXT** :::

## How to reuse this for another GATE subject

You do **not** need a completely new prompt every time. Change only:

**Subject:** Theory of Computation

and

**Official GATE Syllabus:** Regular expressions and finite automata...

For example, later you can replace it with **Algorithms, Digital Logic, Computer Organization**, etc., and tell ChatGPT: “**Adapt the same master teaching system to this syllabus.**”

For TOC specifically, one shortcut to keep in your head from Day 1 is:

**FA !' PDA !' TM = increasing memory/power**

- **FA** !' finite memory !' Regular Languages
- **PDA** !' stack memory !' Context-Free Languages
- **TM** !' unlimited tape !' general computation

And the broad hierarchy is:

**Regular ", CFL ", CSL ", Recursive ", RE**

This single picture will connect a large part of the subject.

Paste the master prompt into this chat if you want to continue here; after that you can simply keep typing **NEXT**, exactly like your DBMS, OS, and Computer Networks preparation.

---

## ASSISTANT

I'll follow the teaching system in your uploaded master prompt: first the complete TOC roadmap, then only the **first logical topic**, and I'll stop for your **NEXT**.

# THEORY OF COMPUTATION— COMPLETE GATE ROADMAP

---

## Big picture first

---

Almost the entire subject can be understood through one question:

**What kind of machine is powerful enough to recognize a particular language?**

The central chain is:

## TEXT

Regular Language

!•

Finite Automaton

!•

Regular Expression

!" more memory

Context-Free Language

!•

Pushdown Automaton

!•

Context-Free Grammar

!" much more power

Turing Machine

!"

Decidability / Recognizability

!"

Undecidability

A useful first memory:

**FA !' PDA !' TM = Finite memory !' Stack memory !' Unlimited tape**

## 1. COMPLETE CHAPTER ORDER

| Ch. | Chapter                                | Priority                  | Conceptual Difficulty |
|-----|----------------------------------------|---------------------------|-----------------------|
| 1   | Foundations of Formal Languages        | & <sup>a</sup> Supporting | Easy                  |
| 2   | Deterministic Finite Automata — DFA    | Ø=Ý% Very Hig             | Easy–Medium           |
| 3   | NFA and $\mu$ -NFA                     | Ø=Ý% Very Hig             | Medium                |
| 4   | Regular Expressions                    | Ø=Ý% Very Hig             | Medium                |
| 5   | FA !" RE and Automata Conversions      | Ø=Ý% Very Hig             | Medium                |
| 6   | DFA Minimization & State Equivalence   | Ø=Ý% Very Hig             | Medium–Hard           |
| 7   | Regular Languages & Closure Properties | Ø=ßà High                 | Medium                |
| 8   | Pumping Lemma for Regular Languages    | Ø=Ý% Very Hig             | Hard initially        |
| 9   | Context-Free Grammars                  | Ø=Ý% Very Hig             | Medium                |
| 10  | Derivations, Parse Trees & Ambiguity   | Ø=Ý% Very Hig             | Medium                |
| 11  | CFG Simplification & Normal Forms      | Ø=ßà High                 | Medium                |
| 12  | Pushdown Automata                      | Ø=Ý% Very Hig             | Hard                  |
| 13  | CFL Properties & Closure               | Ø=Ý% Very Hig             | Medium–Hard           |
| 14  | Pumping Lemma for CFL                  | Ø=Ý% Very Hig             | Hard                  |

| Ch. | Chapter                                      | Priority       | Conceptual Difficulty |
|-----|----------------------------------------------|----------------|-----------------------|
| 15  | Language Hierarchy                           | Ø=Ý% Very Hig  | Medium                |
| 16  | Turing Machines                              | Ø=Ý% Very Hig  | Medium–Hard           |
| 17  | Recursive & Recursively Enumerable Languages | Ø=Ý% Very Hig  | Hard                  |
| 18  | Decidability                                 | Ø=Ý% Very Hig  | Hard                  |
| 19  | Undecidability & Reductions                  | Ø=Ý% Very Hig  | Hardest               |
| 20  | Complete TOC Revision & Mixed GATE Practice  | Ø=Ý% Very High | –                     |

## 2. DETAILED TOPIC MAP

### CHAPTER 1 — Foundations of Formal Languages &<sup>a</sup>

We will learn:

1. Symbol
2. Alphabet
3. String
4. Empty string  $\mu$
5. Length of a string
6. Concatenation of strings
7. String powers
8. Language
9. Empty language "
10.  $\{ \mu \}$  versus "
11. Operations on languages
12. Union
13. Intersection
14. Difference
15. Complement
16. Concatenation
17. Kleene star  $L^*$
18. Positive closure  $L^+$

### CHAPTER 2 — DFA Ø=Ý%

- Basic idea of finite automata
- States
- Input alphabet
- Start state
- Final states

- Transition function
- Formal 5-tuple
- Extended transition function
- Acceptance/rejection
- Designing DFA
- Dead/trap state
- Complement DFA
- Languages involving:
  - prefixes/suffixes
  - substrings
  - parity
  - modulo counting
  - occurrence counting
- Minimum-state intuition

## CHAPTER 3 — NFA & $\mu$ -NFA

- NFA definition
- Multiple possible transitions
- Missing transitions
- NFA acceptance
- DFA vs NFA
- $\mu$ -transitions
- $\mu$ -closure
- NFA  $\neq$  DFA
- $\mu$ -NFA  $\neq$  DFA
- Subset construction
- Number of possible DFA states
- Equivalence of DFA/NFA

### Memory

NFA gives choices, not extra power.

## CHAPTER 4 — Regular Expressions

- Basic RE
- `"`
- `μ`
- alphabet symbols
- union
- concatenation
- Kleene star

- precedence
- building RE
- interpreting RE
- typical language patterns

## CHAPTER 5 — FA !”RE Ø=Ý%

- RE !’  $\mu$  - N F A
- NFA !’ DFA
- FA !’ RE
- state elimination
- equivalence of:
  - DFA
  - NFA
  - $\mu$  - N F A
  - Regular Expression

This gives one of the most important facts in TOC:

TEXT

DFA

%

NFA

%

$\mu$  - N F A

%

Regular Expression

A L L   H A V E   S A M E  
E X P R E S S I V E   P O W E R

## CHAPTER 6 — DFA MINIMIZATION Ø=Ý%

- Reachable states
- Unreachable states
- Equivalent states
- Distinguishable states
- Table-filling
- Partition refinement
- Minimum DFA
- Unique minimal DFA up to renaming
- State-equivalence questions
- Myhill-Nerode intuition

## CHAPTER 7 — REGULAR LANGUAGE PROPERTIES Ø=ßà

Closure under:

- union
- intersection



- complement
- difference
- concatenation
- Kleene star
- reversal
- homomorphism where relevant

Also:

- decision properties
- product DFA
- regular-language proof techniques

## CHAPTER 8 — PUMPING LEMMA FOR REGULAR LANGUAGES Ø=Ý%

One of the first conceptually difficult portions.

We will learn:

TEXT

```
Assume regular
! "
Get pumping length p
! "
Choose clever string w
! "
Analyze every valid decomposition
! "
Pump
! "
Contradiction
! "
Not regular
```

Special attention to:

- quantifiers
- who chooses what
- $|x y| \geq p$
- $|y| > 0$
- $x y^i z \in L$
- common invalid proofs

## CHAPTER 9 — CFG Ø=Ý%

- Grammar
- terminals
- non-terminals
- productions
- start variable
- derivation

- sentential forms
- language generated
- constructing grammars
- common patterns such as:

#### TEXT

a • b •  
palindromes  
balanced structures

## CHAPTER 10 — DERIVATION, PARSE TREES & AMBIGUITY Ø=Ý%

- leftmost derivation
- rightmost derivation
- parse tree
- ambiguous grammar
- ambiguous strings
- inherently ambiguous language
- detecting ambiguity

## CHAPTER 11 — CFG SIMPLIFICATION & NORMAL FORMS Ø=ßà

- useless variables
- non-generating variables
- unreachable variables
- $\mu$  - p r o d u c t i o n s
- unit productions
- CNF
- CNF conversion
- important exceptions

## CHAPTER 12 — PDA Ø=Ý%

Core intuition:

**PDA = Finite Automaton + Stack**

Topics:

- stack
- push
- pop
- transition
- PDA definition
- instantaneous description

- acceptance by:
  - final state
  - empty stack
- NPDA
- DPDA basics
- designing PDA
- CFG  $\Rightarrow$  PDA
- PDA limitations

## CHAPTER 13 — CFL PROPERTIES $\emptyset \neq \Sigma^*$

Key closure questions.

Particularly important:

**CFL is not closed under general intersection.**

But:

**CFL  $\cap$  Regular = CFL.**

We will build the complete closure table later.

## CHAPTER 14 — CFL PUMPING LEMMA $\emptyset \neq \Sigma^*$

Regular pumping:

TEXT  
xyz

CFL pumping:

TEXT  
uvwxy

We will learn why two portions can be pumped and how this differs fundamentally from regular pumping.

## CHAPTER 15 — LANGUAGE HIERARCHY $\emptyset \neq \Sigma^*$

Broad hierarchy:

TEXT  
Finite  
  ",  
Regular  
  ",  
Context-Free  
  ",

```
Context-Sensitive
  ",
Recursive
  ",
Recursively Enumerable
```

All inclusions above are proper in the standard hierarchy.

This chapter connects the entire subject.

---

## CHAPTER 16 — TURING MACHINES Ø=Ý%

**TM = Finite Control + Unlimited Read/Write Tape**

Topics:

- tape
- head
- states
- transition function
- read/write
- left/right movement
- acceptance
- rejection
- looping
- TM design
- variants
- multitape TM
- nondeterministic TM
- computational equivalence

---

## CHAPTER 17 — RECURSIVE & RE LANGUAGES Ø=Ý%

Major distinction:

### **Decider**

Always halts.

### **Recognizer**

May loop forever on strings not in the language.

This distinction becomes extremely important for undecidability.

---

## CHAPTER 18 — DECIDABILITY Ø=Ý%

- decision problems
- decidable problems

- recognizability
- machine encodings
- Universal TM
- acceptance problem
- algorithmic properties

## CHAPTER 19 — UNDECIDABILITY & REDUCTIONS Ø=Ý%

Potentially the hardest TOC chapter.

Topics:

- halting problem
- acceptance problem
- undecidable problems
- mapping reductions
- reduction direction
- contradiction
- PCP where GATE-relevant
- equivalence/emptiness questions involving TMs

Main reasoning:

TEXT

```
Known undecidable A
      |
      | reduce
      v
New problem B
```

If solving **B** would solve **A**, then B cannot be decidable.

## 3. PREREQUISITE FLOW

Follow this order:

TEXT

```
Symbols / Strings / Languages
      !"
      DFA
      !"
      N F A / μ - N F A
      !"
Regular Expressions
      !"
Conversions + Minimization
      !"
Regular Properties
      !"
Pumping Lemma
      !"
      CFG
      !"
Derivation / Parse Tree / Ambiguity
```

|                             |    |
|-----------------------------|----|
| PDA                         | !" |
| CFL Properties              | !" |
| CFL Pumping Lemma           | !" |
| Language Hierarchy          | !" |
| Turing Machine              | !" |
| Recursive / RE              | !" |
| Decidability                | !" |
| Undecidability + Reductions | !" |

Do **not** jump directly to pumping lemma or undecidability.

Their difficulty drops dramatically when prerequisites are strong.

## 4. CONCEPTUALLY EASIEST AREAS

Usually easier:

- symbols/alphabet/strings
- basic DFA
- basic NFA
- basic regular expressions
- left/right derivations
- basic CFG construction
- basic Turing-machine operation

## 5. CONCEPTUALLY HARDEST AREAS

Spend extra attention on:

### 1. Pumping lemma

Because the issue is not formula memorization—it's understanding quantifiers.

### 2. DFA minimum-state questions

Because intuition can produce wrong answers.

### 3. CFG ambiguity

Because one grammar and one language are different objects.

### 4. PDA construction

Because input and stack must be tracked together.

### 5. Recursive vs RE

Because halting behavior matters.

## 6. Reductions and undecidability

Because the **direction of implication** is easy to reverse.

## 6. MASTER CHECKLIST

Use this throughout our course.

### TEXT

```
[ ] 01 Foundations
[ ] 02 DFA
[ ] 03 NFA
[ ] 04  $\mu$  - NFA
[ ] 05 NFA  $\rightarrow$  DFA
[ ] 06 Regular Expressions
[ ] 07 RE  $\rightarrow$  FA
[ ] 08 DFA Minimization
[ ] 09 Regular Closure Properties
[ ] 10 Regular Pumping Lemma

[ ] 11 CFG Basics
[ ] 12 CFG Construction
[ ] 13 Derivations
[ ] 14 Parse Trees
[ ] 15 Ambiguity
[ ] 16 CFG Simplification
[ ] 17 CNF
[ ] 18 PDA
[ ] 19 CFG  $\rightarrow$  PDA
[ ] 20 CFL Closure Properties
[ ] 21 CFL Pumping Lemma

[ ] 22 Language Hierarchy
[ ] 23 Turing Machines
[ ] 24 TM Variants
[ ] 25 Recursive Languages
[ ] 26 Recursively Enumerable Languages
[ ] 27 Decidability
[ ] 28 Halting / Acceptance Problems
[ ] 29 Reductions
[ ] 30 Undecidability

[ ] 31 Master Closure Table
[ ] 32 Master Comparison Table
[ ] 33 Master Decidability Table
[ ] 34 Final Revision
[ ] 35 Mixed GATE Test
```

## CHAPTER 1 — FOUNDATIONS OF FORMAL LANGUAGES

### Topic 1: Symbol, Alphabet and String

#### Priority: **SUPPORTING CONCEPT**

These look elementary, but incorrect understanding here causes mistakes later with:

- regular expressions
- automata
- CFG

- pumping lemma
- language operations

So we will make the foundation precise.

---

## 1. WHY THIS TOPIC MATTERS

---

Theory of Computation studies **languages**.

Languages contain **strings**.

Strings contain **symbols**.

And symbols are selected from an **alphabet**.

Therefore:

TEXT

Alphabet

!"

Symbols

!"

Strings

!"

Language

!"

Machines recognize languages

This is where TOC begins.

---

## 2. INTUITION FIRST

---

Consider English.

English uses letters such as:

TEXT

a, b, c, ..., z

You combine letters to create words:

TEXT

cat

gate

computer

TOC generalizes exactly this idea.

Suppose our allowed symbols are only:

TEXT

0 and 1

Then:



TEXT

0  
1  
01  
101  
110011

are strings over that alphabet.

### 3. SYMBOL

A **symbol** is a basic indivisible character used to construct strings.

Examples:

TEXT

0  
1  
a  
b  
#

Each is one symbol.

### 4. ALPHABET

An **alphabet** is a finite, non-empty set of symbols.

It is conventionally denoted by:

$\Sigma$

Sigma.

Examples:

$\Sigma = \{0,1\}$

Binary alphabet.

Another:

$\Sigma = \{a,b\}$

Another:

$\Sigma = \{a,b,c\}$

### Important distinction

If:

$\Sigma = \{0,1\}$

then:

0 and 1 are symbols.

But:

TEXT

01  
101  
110

are **strings**, not individual symbols.

## 5. STRING

A **string** is a finite sequence of symbols chosen from an alphabet.

If:

$\Sigma = \{a, b\}$

examples of strings are:

TEXT

a  
b  
ab  
ba  
aab  
abba  
bbbb

## FORMAL IDEA

If:

$w = a_1 a_2 a_3 \dots a_n$

where every:

$a_i \in \Sigma$

then  $w$  is a string over  $\Sigma$

## Example

Given:

$\Sigma = \{0, 1\}$

Is:

TEXT

101101

a valid string?

Yes.

Every character belongs to  $\{0, 1\}$ .

What about:

TEXT

10201

No.

Because:

$\nexists \Sigma$

## 6. LENGTH OF A STRING

The length of string  $w$  is written:

$|w|$

$w$

$|w|$

It means:

number of symbols contained in the string.

Example:

$w=10101$

Therefore:

$|w|=5$

### Example

$w=abab$

Then:

$|w|=4$

### & Important

Length counts **symbols**, not the numeric value.

For:

TEXT

111111

length is:

$|w|=6$

not 111111.

---

## 7. EMPTY STRING $\mu$

---

The **empty string** is the string containing **zero symbols**.

Represented by:

$\epsilon$

Therefore:

$|\epsilon|=0$

Think:

$\mu$  = a valid string, but nothing has been written yet.

---

### Important

---

$\mu$  is still a **string**.

It is the unique string of length 0.

---

## 8. EPSILON IS NOT AN ALPHABET SYMBOL BY DEFAULT

---

Suppose:

$\Sigma = \{a, b\}$

Then  $\mu$  doesn't need to appear inside  $\Sigma$  for the empty string to exist.

The empty string is automatically part of:

$\Sigma^*$

which we will study shortly.

---

## 9. STRING CONCATENATION

---

Concatenation means:

join two strings one after another.

Suppose:

$x = ab$

and:

$y = baa$

Then:

$xy = abbbaa$

while:

$\lceil yx = baaab \rceil$

Therefore, generally:

$\lceil xy \neq yx \rceil$

---

## & IMPORTANT PROPERTY

String concatenation is **not commutative**.

Generally:

$\lceil xy \neq yx \rceil$

But it is associative:

$\lceil (xy)z = x(yz) \rceil$

---

## 10. EPSILON AND CONCATENATION

For every string  $w$ :

$\lceil w \text{varepsilon} = \text{varepsilon} w = w \rceil$

Therefore  $\mu$  behaves as the **identity element** for string concatenation.

Example:

$\lceil w = abc \rceil$

Then:

TEXT

|             |     |         |
|-------------|-----|---------|
| $a b c \mu$ | $=$ | $a b c$ |
| $\mu a b c$ | $=$ | $a b c$ |

## MEMORY TRICK

Think:

$\mu =$  invisible string

Attaching nothing before or after a string changes nothing.

---

## 11. POWERS OF A STRING

If  $w$  is a string:

$\lceil w^0 = \text{varepsilon} \rceil$

$\lceil w^1 = w \rceil$

$\lceil w^2 = ww \rceil$

---

$\backslash[ w^3=www \backslash]$

and so on.

---

## Example

If:

$\backslash[ w=ab \backslash]$

then:

$\backslash[ w^0=\text{\textbackslash varepsilon} \backslash]$

$\backslash[ w^1=ab \backslash]$

$\backslash[ w^2=abab \backslash]$

$\backslash[ w^3=ababab \backslash]$

---

## Important formula

$\backslash[$

|       |      |     |
|-------|------|-----|
| $w^n$ | $=n$ | $w$ |
|-------|------|-----|

$\backslash]$

Example:

$\backslash[ w=abc \backslash]$

so:

$\backslash[ |w|=3 \backslash]$

Then:

$\backslash[ |w^4|=4(3)=12 \backslash]$

---

## 12. REVERSE OF A STRING

The reverse of string  $w$  is commonly written:

$\backslash[ w^R \backslash]$

Example:

$\backslash[ w=abc \backslash]$

Then:

$\backslash[ w^R=cba \backslash]$

If:

$\backslash[ w=10110 \backslash]$

then:

$\forall w^R = 01101$

## SPECIAL CASE

$\forall \epsilon^R = \epsilon$

## 13. PALINDROME

A string is a palindrome if:

$\forall w = w^R$

Examples:

TEXT

0  
1  
00  
101  
abba  
0110

### Example

$\forall w = 101$

Then:

$\forall w^R = 101$

So it is a palindrome.

## 14. PREFIX

A string  $x$  is a prefix of  $w$  if there exists a string  $y$  such that:

$\forall w = xy$

Example:

TEXT

$w = abcde$

Prefixes include:

TEXT

$\mu$   
a  
ab  
abc  
abcd  
abcde

---

## 15. SUFFIX

A string  $y$  is a suffix of  $w$  if:

$$\exists [w=xy]$$

for some string  $x$ .

For:

```
TEXT
abcde
```

suffixes include:

```
TEXT
μ
e
de
cde
bcde
abcde
```

---

## 16. SUBSTRING

A string  $y$  is a substring of  $w$  if:

$$\exists [w=xyz]$$

for some strings  $x, z$ .

Example:

```
TEXT
w = abcdef
```

Then:

```
TEXT
bcd
cde
abc
def
```

are substrings.

---

## PREFIX VS SUFFIX VS SUBSTRING

Take:



TEXT

w = abcde

| Concept   | Meaning                      | Example |
|-----------|------------------------------|---------|
| Prefix    | beginning portion            | abc     |
| Suffix    | ending portion               | cde     |
| Substring | consecutive portion anywhere | bcd     |

Memory:

**PRE = beginning, SUF = ending, SUB = somewhere inside**

## SOLVED EXAMPLE 1

Given:

$\Sigma = \{a, b\}$

Which of these are strings over  $\Sigma$ ?

TEXT

abba  
abab  
abc  
 $\mu$

### Solution

abba is valid.

abab is valid.

abc is invalid because:

$c \notin \Sigma$

$\mu$  is valid empty string.

### Answer

TEXT

abba, abab,  $\mu$

## SOLVED EXAMPLE 2

Let:

$x = ab, y = baa$

Find:

$$\lceil xy, \text{quad } yx \rceil$$

### Solution

$$\lceil xy = abbaa \rceil$$

$$\lceil yx = baaab \rceil$$

Thus:

$$\lceil xy \neq yx \rceil$$

This demonstrates non-commutativity.

## SOLVED EXAMPLE 3

If:

$$\lceil w = 101 \rceil$$

find:

$$\lceil w^3 \rceil$$

### Solution

$$\lceil w^3 = www \rceil$$

Therefore:

$$\lceil w^3 = 101101101 \rceil$$

And:

$$\lceil |w^3| = 9 \rceil$$

We can verify using:

$$\lceil |w^3| = 3|w| = 3(3) = 9 \rceil$$

## SOLVED EXAMPLE 4 — GATE STYLE

Let:

$$\lceil |x| = 5, \text{quad } |y| = 8 \rceil$$

What is:

$$\lceil$$

xy

$$\rceil$$

We know:

$$\lceil$$

xy

=

x

+

y

$$\rceil$$

Therefore:

$$\lfloor |xy|=5+8=\boxed{13} \rfloor$$

## MUST REMEMBER

For strings  $x, y$ :

$$\lfloor \begin{array}{|c|c|c|c|c|} \hline xy & = & x & + & y \\ \hline \end{array} \rfloor$$

$\rfloor$   
For string  $w$ :

$$\lfloor \begin{array}{|c|c|c|} \hline w^n & =n & w \\ \hline \end{array} \rfloor$$

$\rfloor$   
And:  
 $\lfloor |\varepsilon|=0 \rfloor$

## GATE-STYLE QUESTION 1

Suppose:

$$\lfloor x=01, \text{ } \sqcup y=10 \rfloor$$

Which statement is true?

- A.  $xy = yx$  B.  $|xy| = 2$  C.  $|xy| = 4$  D.  $xy = 1001$

**Solution**

$$\lfloor xy=0110 \rfloor$$

$$\lfloor yx=1001 \rfloor$$

Hence:

$$\lfloor xy \neq yx \rfloor$$

and:

$$\lfloor |xy|=4 \rfloor$$

**Answer**

$$\lfloor \boxed{C} \rfloor$$

## GATE-STYLE QUESTION 2

If:

$\forall |w|=4$

then find:

$\forall$

$w^7$

$\forall$

Using:

$\forall$

$w^n$

$=n$

$w$

$\forall$

$\forall = 7 \times 4$

$\forall = \boxed{28}$

### GATE-STYLE QUESTION 3

Which statement is correct?

A.  $\mu$  B.  $\mu$  C.  $\mu$  D.  $\mu$  cannot be a string

Answer:

$\boxed{B}$

The distinction between  $\mu$  and " becomes even more important in the next topic.

### Typical GATE/PYQ Pattern

GATE usually doesn't spend a full difficult question simply asking:

**"What is a string?"**

Instead, these basics are embedded inside questions involving:

- regular expressions
- language cardinality
- Kleene star
- DFA acceptance
- CFG
- pumping lemma
- $\mu$ transitions
- closure properties

So these concepts are **foundational rather than isolated**.

## SHORTCUTS & MEMORY TRICKS

### Alphabet

**Alphabet = Allowed characters**

If:

$\Sigma = \{0,1\}$

only 0 and 1 can appear in its strings.

### String

**String = Sequence of alphabet symbols**

$\mu$

**$\mu$  = Empty string = zero characters**

Therefore:

$|\mu| = 0$

### Concatenation

**$XY$  = X first, Y next**

Never mentally sort or rearrange the symbols.

### Reverse

**R = Read backwards**

$(abc)^R = cba$

## & µGATE TRAPS

### Trap 1 — $\mu$ has length 1

Wrong.

$\mu$  is notation for the empty string.

$\forall |\epsilon|=0$

---

## Trap 2 — $\mu$ and $\epsilon$ are identical

---

Wrong.

This is extremely important.

$\mu$

A **string**.

$\epsilon$

A **set/language** containing no strings.

We will study this carefully in the next topic.

---

## Trap 3 — Concatenation is commutative

---

Wrong.

Generally:

$xy \neq yx$

Example:

TEXT

$x = ab$

$y = cd$

$xy = abcd$

but:

$yx = cdab$

---

## Trap 4 — Any character can occur in a string

---

Wrong.

Every symbol must come from its alphabet.

---

## Trap 5 — $w^0$ is an empty language

---

Wrong.

$w^0 = \epsilon$

It is the **empty string**.

---

## IMPORTANT RESULTS TO MEMORIZE

---

## MUST REMEMBER

$\lvert \Sigma \rvert = 0$

$\lvert w \rvert = \lvert w \rvert$

$w^0 = \epsilon$

$\lvert$

|      |     |     |     |     |
|------|-----|-----|-----|-----|
| $xy$ | $=$ | $x$ | $+$ | $y$ |
|------|-----|-----|-----|-----|

$\rvert$

$\lvert$

|       |      |     |
|-------|------|-----|
| $w^n$ | $=n$ | $w$ |
|-------|------|-----|

$\rvert$

$(w^R)^R = w$

and generally:

$xy \neq yx$

## ' p WRITE THIS IN YOUR NOTES

### Symbols, Alphabet & Strings

**Symbol:** Basic character used to construct strings.

**Alphabet**  $\Sigma$  Finite non-empty set of symbols.

Example:

$\Sigma = \{0,1\}$

**String:** Finite sequence of symbols from  $\Sigma$ .

Example:

TEXT

10110

**Empty string:**

$\lvert \epsilon \rvert = 0$

**Length:**

$\lvert w \rvert = \text{number of symbols in } w$

**Concatenation:**

If  $x=ab$ ,  $y=cd$ :

$xy=abcd$

Generally:

$\lceil xy \rceil \neq \lceil yx \rceil$

but:

$\lceil (xy)z \rceil = \lceil x(yz) \rceil$

**Identity:**

$\lceil w \rceil \text{varepsilon} = \lceil \text{varepsilon} w \rceil$

**String power:**

$\lceil w^0 \rceil = \lceil \text{varepsilon} \rceil$

$\lceil w^n \rceil = \underbrace{ww \dots w}_n \text{ times}$

$\lceil$

|       |      |     |
|-------|------|-----|
| $w^n$ | $=n$ | $w$ |
|-------|------|-----|

$\rceil$

**Reverse:**

$\lceil w^R \rceil = \text{symbols of } w \text{ in reverse order}$

Palindrome:

$\lceil w = w^R \rceil$

**Memory**

TEXT

Alphabet !' allowed symbols  
String !' sequence  
 $\mu$  !' zero-length string

## 60-SECOND REVISION

- Symbol = basic character.
- Alphabet  $\Sigma$  = finite non-empty set of symbols.
- String = finite sequence over  $\Sigma$
- $\mu$  = empty string.
- $|\mu| = 0$
- $|xy| = |x| + |y|$
- String concatenation is generally not commutative.
- $wp = \mu$
- $|w \cdot | = n + |w|$
- $w^R$  means reverse.
- Palindrome satisfies  $w = w^R$
- Prefix starts from the beginning.
- Suffix reaches the end.
- Substring is consecutive symbols anywhere.



## ACTIVE RECALL

---

Try these without looking above.

### Q1

What is an alphabet?

### Q2

If:

$\Sigma = \{0,1\}$

is `1021` a valid string?

### Q3

What is:

$\epsilon$

`\varepsilon`

$\epsilon$

### Q4

If:

$x = abc, y = de$

what is `xy`?

### Q5

If:

$|w| = 5$

what is:

$w^4$

`w^4`

$w^4$

### Q6

Is string concatenation commutative?

### Q7

What is:

$(11001)^R$

### Q8

What condition defines a palindrome?

---

## ANSWERS

1. A finite non-empty set of symbols.
2. No, because  $2 \neq \epsilon$ .
3.  $\emptyset$ .
4. abcde.
5. 20.
6. No, generally  $xy \neq yx$ .
7. 10011.
8.  $w = w$ ?

## CHAPTER 1 PROGRESS

TEXT

```
[ ' ] S y m b o l
[ ' ] A l p h a b e t
[ ' ] S t r i n g
[ ' ] E m p t y   s t r i n g
[ ' ] S t r i n g   l e n g t h
[ ' ] C o n c a t e n a t i o n
[ ' ] S t r i n g   p o w e r s
[ ' ] R e v e r s e   /   P r e f i x   /   S u f f i x   /   S u b s t r i n g
```

```
[ ] L a n g u a g e
[ ] E m p t y   l a n g u a g e
[ ]  $\mu$  v s " v s {  $\mu$  }
[ ] O p e r a t i o n s   o n   l a n g u a g e s
[ ]  $\epsilon^*$ 
[ ]  $\epsilon^+$ 
[ ] K l e e n e   c l o s u r e
[ ] P o s i t i v e   c l o s u r e
```

## Next topic

\*Languages, " , {  $\mu$  }  $\epsilon$ ,  $\epsilon^*$  and the extremely important difference between  $\mu$  , and {  $\mu^*$  }

Type NEXT.

USER

next

ASSISTANT

Continuing exactly from your uploaded TOC master prompt and the previous chapter sequence.

## CHAPTER 1 — FOUNDATIONS OF FORMAL LANGUAGES

### Topic 2: Language, " , { $\mu$ } $\epsilon$ , $\epsilon^*$ and Language Operations

Priority: **SUPPORTING CONCEPT** — but extremely important for avoiding GATE traps

This topic looks basic, but many later questions in:

- Regular Expressions
- DFA/NFA
- CFG
- Pumping Lemma
- Closure Properties

depend on getting these symbols exactly right.

---

## 1. WHY THIS TOPIC MATTERS

---

A machine in TOC does not recognize a single string.

It recognizes a **language**.

So the core chain is:

```
TEXT
Alphabet
    !"
Strings
    !"
Language
    !"
Automaton / Grammar / TM
```

Example:

```
TEXT
A l p h a b e t   £   =   { 0 , 1 }

Possible strings:
µ
0
1
00
01
10
11
000
...
```

Now we can choose some of those strings and form a language.

---

## 2. INTUITION FIRST

---

Think of:

**Alphabet = available letters**

**String = one word**

**Language = collection of words**

Example:

$$\Sigma = \{a, b\}$$

Possible strings include:

TEXT

$\mu$   
a  
b  
aa  
ab  
ba  
bb  
aaa  
...

A language could be:

$$L = \{a, ab, abb\}$$

Another language:

$$L = \{a^n b^n \mid n \geq 0\}$$

which contains:

TEXT

$\mu$   
ab  
aabb  
aaabbb  
...

### 3. FORMAL DEFINITION OF LANGUAGE

A **language over an alphabet**  $\Sigma$  is any set of strings formed using symbols from  $\Sigma$ .

Formally:

$$L \subseteq \Sigma^*$$

This is a very important statement.

#### Meaning

$\Sigma^*$  contains **all possible finite strings** over alphabet  $\Sigma$

A language chooses some of those strings.

### 4. EXAMPLE OF A LANGUAGE

Suppose:

$$\Sigma = \{0, 1\}$$

Define:

$L = \{w \mid w \text{ ends with } 1\}$

Then:

TEXT

1  
01  
11  
001  
101  
111  
...

belong to  $L$ .

But:

TEXT

$\mu$   
0  
10  
100

do not.

---

## 5. FINITE LANGUAGE

A language having finitely many strings is a **finite language**.

Example:

$L = \{a, ab, ba\}$

It has exactly 3 strings.

Therefore:

$|L| = 3$

---

## 6. INFINITE LANGUAGE

A language may contain infinitely many strings.

Example:

$L = \{a^n \mid n \geq 0\}$

Expanding:

TEXT

$\mu$   
a  
aa  
aaa  
aaaa  
...

There is no largest string.  
Hence the language is infinite.

---

## 7. EMPTY LANGUAGE $\emptyset$

This is one of the most important distinctions in TOC.

The **empty language** is:

$\emptyset$

It contains **zero strings**.

Therefore:

$|\emptyset|=0$

Think:

$\emptyset$  = empty box

There is nothing inside it.

---

## 8. EMPTY STRING $\epsilon$

The empty string:

$\epsilon$

is a **string of length zero**.

Therefore:

$|\epsilon|=0$

But  $\epsilon$  itself exists.

---

## 9. LANGUAGE $\{\epsilon\}$

Now consider:

$\{\epsilon\}$

This is a **language containing exactly one string**.

That one string happens to be the empty string.

Therefore:

$|\{\epsilon\}|=1$

This is an extremely important GATE distinction.

---

$\emptyset$  vs  $\{\epsilon\}$

---

## MEMORIZE THIS

| Expression | Meaning                   | Type     | Cardinality |
|------------|---------------------------|----------|-------------|
| $\mu$      | Empty string              | String   | length = 0  |
| "          | Empty language            | Language | 0 strings   |
| $\{\mu\}$  | Language containing $\mu$ | Language | 1 string    |

## VISUAL MEMORY

TEXT

$\mu$   
= ""

one string  
with zero symbols

TEXT

"  
= { }

zero strings

TEXT

$\{\mu\}$   
= { "" }

one string  
and that string is empty

## BEST MEMORY TRICK

Think of boxes.

"

TEXT

[       ]

Empty box.

$\{\mu\}$

TEXT

[    $\mu$    ]

Box contains one object.

That object is an empty string.

So:

Empty box "" box containing an empty object

Therefore:

$\emptyset \neq \{\emptyset\}$

## 10. CARDINALITY

For a finite language  $L$ :

$|L|$

$L$

$|L|$

means number of strings in  $L$ .

Example:

$L = \{\epsilon, a, aa\}$

Then:

$|L| = 3$

Be careful:

$|w|$  for a string = number of symbols.

$|L|$  for a language = number of strings.

## & pGATE TRAP

Suppose:

$L = \{\epsilon\}$

Then:

$|L| = 1$

but:

$|\epsilon| = 0$

These are completely different quantities.

## 11. £ \\* KLEENE STAR OF AN ALPHABET

Now one of the most important objects in TOC.

For alphabet  $\Sigma$

$\Sigma^*$



means:

all possible finite strings over  $\Sigma$  including  $\mu$ .

## Example

Let:

$\Sigma = \{a, b\}$

Then:

$\Sigma^*$

contains:

TEXT

$\mu$

length 1:

a

b

length 2:

aa

ab

ba

bb

length 3:

aaa

aab

aba

abb

baa

bab

bba

bbb

...

Therefore:

$\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$

## FORMAL EXPRESSION

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

where:

$\Sigma^0 = \{\epsilon\}$

This is crucial.

## 12. WHAT IS $\Sigma^p$ ?

For an alphabet:

$$\Sigma^0 = \{\epsilon\}$$

Not:

$$\{\epsilon\}$$

and not:

$$\emptyset$$

Why?

Because  $\Sigma$  is a **language**, so its result must be a set of strings.

Hence:

$$\Sigma^0 = \{\epsilon\}$$

---

### 13. WHAT IS $\Sigma^1$ ?

$$\Sigma^1 = \Sigma$$

For:

$$\Sigma = \{a, b\}$$

$$\Sigma^1 = \{a, b\}$$

---

### 14. WHAT IS $\Sigma^2$ ?

It contains all strings of length exactly 2.

For:

$$\Sigma = \{a, b\}$$

$$\Sigma^2 = \{aa, ab, ba, bb\}$$

---

### 15. WHAT IS $\Sigma^3$ ?

All strings of length exactly 3:

$$\Sigma^3 = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

---

### 16. NUMBER OF STRINGS IN $\Sigma^n$

Suppose:

$$|\Sigma| = k$$

Then the number of strings of length exactly  $n$  is:

$$k^n$$

This is very important for NAT/counting questions.

## Example

---

If:

$$\Sigma = \{0,1\}$$

then:

$$|\Sigma| = 2$$

Number of length-5 strings:

$$2^5 = 32$$

---

## Example

---

If:

$$\Sigma = \{a,b,c\}$$

number of strings of length 4:

$$3^4 = 81$$

---

## Why?

---

For each position, we have  $k$  choices.

For  $n$  positions:

$$k \times k \times \cdots \times k$$

$n$  times.

Therefore:

$$k^n$$

---

## 17. STRINGS OF LENGTH AT MOST $n$

---

Suppose:

$$|\Sigma| = k$$

Number of strings of length:

$$0, 1, 2, \dots, n$$

is:

$$k^0 + k^1 + k^2 + \cdots + k^n$$

---

## Example

---

Binary alphabet:

$$\Sigma = \{0,1\}$$

How many strings have length at most 3?

---

$$\{ 2^0 + 2^1 + 2^2 + 2^3 \}$$

$$\{ 1 + 2 + 4 + 8 \}$$

$$\{ \boxed{15} \}$$

Notice:

$$2^p = 1$$

because there is exactly one string of length zero:

$$\{ \epsilon \}$$

## GATE SHORTCUT

For  $k \geq 1$

$$\{ 1 + k + k^2 + \dots + k^n = \frac{k^{n+1} - 1}{k - 1} \}$$

Use only when it saves time.

## 18. $\Sigma^+$ POSITIVE CLOSURE

$$\{ \Sigma^+ \}$$

means:

**all non-empty finite strings over  $\Sigma$ .**

Formally:

$$\{ \Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots \}$$

So:

$$\{ \boxed{\Sigma^+ = \Sigma^* - \{ \epsilon \}} \}$$

provided  $\Sigma$  is an ordinary alphabet.

## Example

For:

$$\{ \Sigma = \{0, 1\} \}$$

$$\Sigma^+$$

TEXT

$\mu$   
 $0$   
 $1$   
 $00$   
 $01$   
 $10$   
 $11$   
 $000$   
 $\dots$

$\epsilon^+$

TEXT

0  
1  
00  
01  
10  
11  
000  
...

Only  $\epsilon$  is missing.

## MEMORY TRICK

Star allows zero repetitions.

Therefore  $\epsilon$  appears.

Plus means at least one repetition.

Therefore  $\epsilon$  does not appear.

## Symbolic memory

TEXT

\* !' 0 o r m o r e  
+ !' 1 o r m o r e

You will see exactly the same idea in regular expressions later.

## 19. RELATION BETWEEN $\epsilon^*$ AND $\epsilon^+$

For a nonempty alphabet:

$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$

and:

$\Sigma^+ = \Sigma^* - \{\epsilon\}$

Therefore:

$\epsilon \in \Sigma^*$

but:

$\epsilon \notin \Sigma^+$

## & pMAJOR GATE TRAP

Does  $\Sigma$  always contain  $\mu$  ?

**YES.**

Always.

Even if:

$\Sigma = \{0,1\}$

$\mu$  is still included.

---

## 20. LANGUAGE OPERATIONS

Now suppose:

$L_1, L_2$

are languages.

We can perform set-like and string-based operations.

Main ones:

1. Union
2. Intersection
3. Difference
4. Complement
5. Concatenation
6. Power
7. Kleene star
8. Positive closure

---

## 21. UNION

$L_1 \cup L_2$

contains every string belonging to at least one of the languages.

Example:

$L_1 = \{a, ab\}$

$L_2 = \{b, ab\}$

Then:

$L_1 \cup L_2 = \{a, b, ab\}$

Duplicate strings appear only once because a language is a set.

---

## 22. INTERSECTION

$L_1 \cap L_2$

contains strings present in both.

Using:

$$L_1 = \{a, ab\}$$

$$L_2 = \{b, ab\}$$

we get:

$$L_1 \cap L_2 = \{ab\}$$

---

## 23. DIFFERENCE

---

$$L_1 - L_2$$

means:

**strings present in  $L_1$  but not in  $L_2$ .**

Using the same languages:

$$L_1 - L_2 = \{a\}$$

and:

$$L_2 - L_1 = \{b\}$$

Therefore, generally:

$$L_1 - L_2 \neq L_2 - L_1$$

---

## 24. COMPLEMENT

---

If:

$$L \subseteq \Sigma^*$$

then complement of  $L$  with respect to  $\Sigma$  is:

$$\overline{L} = \Sigma^* - L$$

Meaning:

**all strings over  $\Sigma$  that are not in  $L$ .**

---

### Example

---

Let:

$$\Sigma = \{0, 1\}$$

and let  $L$  contain all strings ending in 1.

Then:

$$\overline{L}$$

contains strings **not ending in 1**.

This includes:

TEXT

$\mu$   
0  
00  
10  
100  
...

## & IMPORTANT

Complement depends on the universe.

In TOC the universe is usually:

$\Sigma^*$

So always identify the alphabet.

## 25. LANGUAGE CONCATENATION

This is different from union.

For two languages:

$L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$

Meaning:

take every string from  $L_1$ , followed by every string from  $L_2$ .

### Example

Let:

$L_1 = \{a, b\}$

and:

$L_2 = \{0, 1\}$

Then:

$L_1 L_2 = \{a0, a1, b0, b1\}$

### Example 2

$L_1 = \{a, ab\}$

$L_2 = \{b, c\}$

Then:



#### TEXT

$a + b = ab$   
 $a + c = ac$   
 $ab + b = abb$   
 $ab + c = abc$

Hence:

$$L_1 L_2 = \{ab, ac, abb, abc\}$$

## 26. IS LANGUAGE CONCATENATION COMMUTATIVE?

Generally:

$$L_1 L_2 \neq L_2 L_1$$

Example:

$$L_1 = \{a\}$$

$$L_2 = \{b\}$$

Then:

$$L_1 L_2 = \{ab\}$$

while:

$$L_2 L_1 = \{ba\}$$

## 27. CONCATENATION WITH $\{\epsilon\}$

For every language  $L$ :

$$L\{\epsilon\} = \{\epsilon\}L = L$$

So:

$$\{\epsilon\}$$

is the **identity language** for concatenation.

## 28. CONCATENATION WITH $\emptyset$

For every language  $L$ :

$$L\emptyset = \emptyset L = \emptyset$$

Why?

To form  $xy$ , we need:

$$y \in \emptyset$$

But no such  $y$  exists.

Therefore no strings can be produced.

## THIS IS EXTREMELY IMPORTANT

---

Compare:

$$\forall \epsilon L \setminus \{\epsilon\} = L$$

but:

$$\forall \emptyset = \emptyset$$

This again proves why:

$$\emptyset \neq \{\epsilon\}$$

## MEMORY TRICK

---

**{ 1 }**

Acts like multiplication by **1**:

$$\forall \epsilon L \setminus \{\epsilon\} = L$$

**"**

Acts like multiplication by **0**:

$$\forall \emptyset = \emptyset$$

Very useful memory.

## 29. POWER OF A LANGUAGE

---

For a language **L**:

$$L^0 = \{\epsilon\}$$

$$L^1 = L$$

$$L^2 = LL$$

$$L^3 = LLL$$

and so on.

### Example

---

Let:

$$L = \{a, b\}$$

Then:

$$L^2 = \{aa, ab, ba, bb\}$$

and:

$$L^0 = \{\epsilon\}$$

## & pGATE TRAP

For a string:

$$\forall w^0 = \{\epsilon\}$$

For a language:

$$\forall L^0 = \{\epsilon\}$$

Why difference?

Because:

- $w$  must be a string.
- $L$  must be a language.

## 30. KLEENE STAR OF A LANGUAGE

For a language  $L$ :

$$\forall L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$$

Since:

$$\forall L^0 = \{\epsilon\}$$

we always have:

$$\forall \boxed{\epsilon \in L^*}$$

for **every language**  $L$ .

### Example

Let:

$$\forall L = \{ab\}$$

Then:

$$\forall L^* = \{\epsilon, ab, abab, ababab, \dots\}$$

## 31. POSITIVE CLOSURE OF A LANGUAGE

$$\forall L^+ = L^1 \cup L^2 \cup L^3 \cup \dots$$

Meaning:

**concatenate one or more strings from  $L$ .**

Generally:

$$\forall L^+ = LL^*$$

and:

$$\forall L^* = L^+ \cup \{\epsilon\}$$

but there is an important edge case.

## 32. IMPORTANT EDGE CASE — WHAT IF $\mu \in L$ ?

Suppose:

$$L = \{\epsilon, a\}$$

Then  $\mu$  can already be selected from  $L$  once.

Therefore:

$$\epsilon \in L^+$$

This means the statement:

**" $L^+$  never contains  $\mu$ "**

is not universally true.

The correct statement is:

**$L^+$  excludes zero repetitions, but  $\mu$  can still belong to  $L^+$  if  $\mu$  is already obtainable using one or more members of  $L$ .**

For an alphabet  $\Sigma$ , however:

$$\epsilon \notin \Sigma$$

under the normal alphabet definition, so:

$$\epsilon \notin \Sigma^+$$

## & #pEXCELLENT GATE TRAP

For:

$$L = \{\epsilon, a\}$$

is:

$$\epsilon \in L^+?$$

**YES.**

Because  $\mu$  itself belongs to  $L$ .

## 33. SPECIAL CASE: $\emptyset^*$

This is frequently confusing.

Find:

$$\emptyset^*$$

By definition:

$$\emptyset^* = \emptyset^0 \cup \emptyset^1 \cup \emptyset^2 \cup \dots$$

We know:

$$\emptyset^0 = \{\varnothing\}$$

and for every positive  $n$ :

$$\emptyset^n = \emptyset$$

Therefore:

$$\boxed{\emptyset^* = \{\varnothing\}}$$

---

## 34. WHAT IS $\cup$ ?

$$\emptyset^+ = \emptyset^1 \cup \emptyset^2 \cup \dots$$

Every term is:

$$\emptyset$$

Therefore:

$$\boxed{\emptyset^+ = \emptyset}$$

---

## MUST MEMORIZE

$$\boxed{\emptyset^* = \{\varnothing\}}$$

$$\boxed{\emptyset^+ = \emptyset}$$

This is a classic conceptual trap.

---

## 35. WHAT IS $\mu$ ?

Let:

$$L = \{\varnothing\}$$

Then:

$$L^1 = \{\varnothing\}$$

$$L^2 = \{\varnothing, \varnothing\} = \{\varnothing\}$$

and similarly every power remains:

$$\{\varnothing\}$$

Therefore:

$$\boxed{\{\varnothing\}^* = \{\varnothing\}}$$

Likewise:

$$\boxed{\{\varnothing\}^+ = \{\varnothing\}}$$

---

## MASTER SPECIAL-CASE TABLE

---

| Expression  | Result    |
|-------------|-----------|
| " *         | { $\mu$ } |
| " +         | "         |
| { $\mu$ } * | { $\mu$ } |
| { $\mu$ } + | { $\mu$ } |
| L p         | { $\mu$ } |
| w p         | $\mu$     |

Memorize this table.

### 36. SOLVED EXAMPLE 1

Given:

$\Sigma = \{0, 1\}$

find:

$\Sigma^2$

Each string must have exactly 2 symbols.

Therefore:

$\Sigma^2 = \{00, 01, 10, 11\}$

### SOLVED EXAMPLE 2

If:

$|\Sigma| = 3$

how many strings have length exactly 5?

Formula:

$|\Sigma|^n$

Therefore:

$3^5 = 243$

**Answer**

$243$

### SOLVED EXAMPLE 3

Binary alphabet.

How many strings have length at most 4?

$$\lceil 2^0+2^1+2^2+2^3+2^4 \rceil$$

$$\lceil =1+2+4+8+16 \rceil$$

$$\lceil =\boxed{31} \rceil$$

## SOLVED EXAMPLE 4

Let:

$$\lceil L_1=\{a,b\} \rceil$$

$$\lceil L_2=\{0,1\} \rceil$$

Find:

$$\lceil L_1L_2 \rceil$$

Combine every first-language string with every second-language string:

$$\lceil \boxed{L_1L_2= \{a0,a1,b0,b1\}} \rceil$$

## SOLVED EXAMPLE 5

Let:

$$\lceil L=\{ab\} \rceil$$

Find the first four members generated by  $L^*$ .

$$\lceil L^0=\{\epsilon\} \rceil$$

$$\lceil L^1=\{ab\} \rceil$$

$$\lceil L^2=\{abab\} \rceil$$

$$\lceil L^3=\{ababab\} \rceil$$

Therefore:

$$\lceil L^*= \{\epsilon,ab,abab,ababab,\ldots\} \rceil$$

## SOLVED EXAMPLE 6 — IMPORTANT TRAP

Find:

$$\lceil \emptyset^* \rceil$$

Many students incorrectly answer:

$$\lceil \emptyset \rceil$$

But:

$$\lceil L^* \rceil$$

always contains:

$$\lceil L^0=\{\epsilon\} \rceil$$

Thus:

$\boxed{\emptyset^* = \{\epsilon\}}$

---

## SOLVED EXAMPLE 7 — CARDINALITY TRAP

---

Given:

$L = \{\epsilon, a, aa\}$

Find:

$|$

L

$|$

There are three strings:

1.  $\mu$
2.  $a$
3.  $aa$

Therefore:

$|L| = 3$

Even though:

$|\epsilon| = 0$

$\mu$  still counts as one member of the language.

---

## 37. GATE-STYLE MCQ

---

Which of the following is correct?

A.

$\emptyset = \{\epsilon\}$

B.

$|\{\epsilon\}| = 0$

C.

$\emptyset^* = \{\epsilon\}$

D.

$|\epsilon| = 1$

**Answer**

$\boxed{C}$

---

## 38. GATE-STYLE MSQ

---

Which are true?



1.

$\epsilon \in \Sigma^*$

2.

$\emptyset \subseteq \Sigma^*$

3.

$\emptyset \in \Sigma^*$

4.

$\{\epsilon\} \subseteq \Sigma^*$

### Reasoning

1. True.  $\mu$  is always in  $\Sigma^*$ .
2. True. Empty set is a subset of every set.
3. False.  $\Sigma^*$  is a set/language, while elements of  $\Sigma^*$  are strings.
4. True because  $\mu$  belongs to  $\Sigma^*$ .

### Answer

$\boxed{1,2,4}$

This is a very good type-checking question.

---

## 39. TYPE-CHECKING SHORTCUT

When confused, first ask:

**Is this object a string or a set of strings?**

Example:

$\epsilon$

String.

$\{\epsilon\}$

Set/language.

$\emptyset$

Set/language.

$\Sigma^*$

Set/language.

This catches many mistakes immediately.

---

## 40. GATE-STYLE NAT

Suppose:

$|\Sigma|=4$

How many strings have length exactly 3?

$\boxed{4^3=64}$

---

## 41. GATE-STYLE NAT

For:

$\Sigma=\{0,1\}$

how many strings have length less than or equal to 5?

$2^0+2^1+\cdots+2^5$

$=1+2+4+8+16+32$

$\boxed{63}$

---

## 42. TYPICAL GATE/PYQ PATTERNS

These concepts commonly appear inside questions involving:

### Pattern 1 — Cardinality

How many strings exist of length:

- exactly  $n$
- at most  $n$
- between two lengths

Core formula:

$k^n$

---

### Pattern 2 — $\mu/''$ confusion

Examples:

$\emptyset^*$

$\{\epsilon\}^*$

$L\emptyset$

$L\{\epsilon\}$

---

### Pattern 3 — Kleene closure

Determine whether:

$\epsilon \in L^*$

Answer:

**Always yes.**

## Pattern 4 — Language concatenation

Compute:

$\backslash [L_1 L_2 \backslash]$

or reason about its members.

## Pattern 5 — Type errors

Distinguishing:

TEXT

$\mu$  " L

from:

TEXT

{  $\mu$  } "+" L

These can mean related things but use different operators because one compares an element, while the other compares sets.

## 43. SHORTCUTS & MEMORY TRICKS

$\mu$  vs " vs {  $\mu$  }

$\mu$  = nothing written

" = nothing inside

{  $\mu$  } one empty thing inside

## Star vs Plus

STAR starts from zero

TEXT

\* = zero or more

PLUS starts from one

TEXT

+ = one or more

$\Sigma \bullet$

$n$  = exact length.

$\forall \Sigma^n$

means:

all strings of exactly length  $n$ .

$\Sigma^*$

All finite lengths from zero upward

TEXT

$0, 1, 2, 3, \dots$

$\Sigma^+$

All positive lengths

TEXT

$1, 2, 3, \dots$

## 44. & pGATE TRAPS

### Trap 1

$\emptyset \in \Sigma^*$

'L False.

### Trap 2

$|\Sigma^0| = 0$

'L False.

It contains one member.

$|\Sigma^1| = 1$

### Trap 3

$\forall \emptyset^* = \emptyset$

'LFalse.

Correct:

$\forall \emptyset^* = \{\epsilon\}$

### Trap 4

$L^*$  may not contain  $\mu$ .

'LFalse.

Always:

$\forall \epsilon \in L^*$

### Trap 5

$L^+$  never contains  $\mu$ .

'LNot universally true.

If:

$\forall \epsilon \in L$

then:

$\forall \epsilon \in L^+$

### Trap 6

$L^0 = \epsilon$

'LFalse.

$L$  must be a language:

$L^0 = \{\epsilon\}$

### Trap 7

$w^0 = \{\epsilon\}$

'LFalse.

$w$  is a string.

Therefore:

$w^0 = \epsilon$

## Trap 8

$\mathbb{L} \setminus \emptyset = \mathbb{L}$

'LFalse.

Correct:

$\mathbb{L} \setminus \emptyset = \emptyset$

## Trap 9

$\mathbb{L} \setminus \{\varepsilon\} = \emptyset$

'LFalse.

Correct:

$\mathbb{L} \setminus \{\varepsilon\} = \mathbb{L}$

## 45. MUST REMEMBER

$\mathbb{L} \subseteq \Sigma^*$

$\Sigma^0 = \{\varepsilon\}$

$|\Sigma^n| = |\Sigma|^n$

$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$

$\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n$

$\mathbb{L}^0 = \{\varepsilon\}$

$\mathbb{L}^* = \bigcup_{n \geq 0} \mathbb{L}^n$

$\mathbb{L}^+ = \bigcup_{n \geq 1} \mathbb{L}^n$

$\mathbb{L} \setminus \{\varepsilon\} = \mathbb{L}$

$\emptyset \setminus \emptyset = \emptyset$

$\emptyset^* = \{\varepsilon\}$

$\emptyset^+ = \emptyset$

## ' p WRITE THIS IN YOUR NOTES

### Language

A language over alphabet  $\Sigma$  is a set of strings:

$\mathbb{L} \subseteq \Sigma^*$

### Important distinction

| Object     | Meaning                          |
|------------|----------------------------------|
| $\mu$      | empty string                     |
| $\epsilon$ | empty language                   |
| $\{\mu\}$  | language containing empty string |

$\backslash[\backslash\text{varepsilon}=0\backslash]$

$\backslash[\backslash\text{emptyset}=0\backslash]$

$\backslash[\backslash\{\text{varepsilon}\}=1\backslash]$

## Alphabet powers

$\backslash[\backslash\text{Sigma}^0=\backslash\{\text{varepsilon}\}\backslash]$

$\backslash[\backslash\text{Sigma}^1=\backslash\text{Sigma}\backslash]$

$\backslash[\backslash\text{Sigma}^n=\backslash\text{Sigma}^n\backslash]$

## Kleene Star

$\backslash[\backslash\text{Sigma}^* = \backslash\text{Sigma}^0\cup\backslash\text{Sigma}^1\cup\backslash\text{Sigma}^2\cup\backslash\text{cdots}\backslash]$

Includes  $\mu$ .

Memory:

**\* = 0 or more**

## Positive Closure

$\backslash[\backslash\text{Sigma}^+ = \backslash\text{Sigma}^1\cup\backslash\text{Sigma}^2\cup\backslash\text{cdots}\backslash]$

For ordinary alphabets:

$\backslash[\backslash\text{Sigma}^+=\backslash\text{Sigma}^*-\backslash\{\text{varepsilon}\}\backslash]$

Memory:

**+ = 1 or more**

## Language concatenation

$\backslash[\backslash L_1 L_2 = \backslash\{xy\mid x\in L_1, y\in L_2\}\backslash]$

Generally:

$\backslash[\backslash L_1 L_2 \neq L_2 L_1\backslash]$





## Q2

Is:

$$\forall \epsilon \in \Sigma^*$$

always true?

## Q3

What is:

$$\Sigma^0$$

## Q4

If:

$$|\Sigma|=3$$

how many strings exist of length 4?

## Q5

If:

$$L = \{a, b\}$$

what is  $L^2$ ?

## Q6

Find:

$$L \setminus \emptyset$$

## Q7

Find:

$$L \setminus \{\epsilon\}$$

## Q8

Find:

$$\emptyset^*$$

## Q9

Find:

$$\emptyset^+$$

## Q10

Can  $\mu$  belong to  $L^+$ ?

---

## ANSWERS

### A1

- $\mu$  = empty string
- $\epsilon$  = empty language

- $\{\mu\}$  language containing  $\mu$

**A2**

Yes.

**A3**

$\{\epsilon\}$

**A4**

$3^4=81$

**A5**

$\{aa,ab,ba,bb\}$

**A6**

$\emptyset$

**A7**

$L$

**A8**

$\{\epsilon\}$

**A9**

$\emptyset$

**A10**

Yes, if  $\mu$  can be produced using one or more strings from  $L$ , especially if:

$\epsilon \in L$

## CHAPTER 1 MINI GATE TEST

Try these before reading the solutions.

### Q1 — MCQ

Which is correct?

A.

$\emptyset^* = \emptyset$

B.

$\emptyset^* = \{\epsilon\}$

C.

$\emptyset^* = \epsilon$

D.

$\emptyset^* = \Sigma^*$

---

## Q2 — NAT

---

Let:

$$\Sigma = \{a, b, c\}$$

How many strings have length exactly 5?

---

## Q3 — MCQ

---

For every language  $L$ :

A.

$$L \cap \emptyset = L$$

B.

$$L \cap \{\epsilon\} = \emptyset$$

C.

$$L \cap \emptyset = \emptyset$$

D.

$$L^0 = \{\epsilon\}$$

---

## Q4 — MSQ

---

Which are always true?

A.

$$\epsilon \in L^*$$

B.

$$L \subseteq L^*$$

C.

$$\epsilon \in L^+$$

D.

$$L^0 = \{\epsilon\}$$

---

## Q5 — NAT

---

For binary alphabet:

$$\Sigma = \{0, 1\}$$

how many strings have length at most 6?

---

## Q6 — Tricky

---

Which pair represents two different languages?

A.

$\{\}$

B.

$\{\epsilon\}$

C.

$\{\}$

D. None

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

**Q1**

$\boxed{B}$

**Q2**

$3^5 = \boxed{243}$

**Q3**

$\boxed{C}$

**Q4**

A, B and D.

Why B?

Because:

$L = L^1$

and  $L^1$  is included in  $L^*$ .

C is not always true.

**Q5**

$2^0 + 2^1 + \dots + 2^6$

$= 1 + 2 + 4 + 8 + 16 + 32 + 64$

$= \boxed{127}$

**Q6**

$\boxed{C}$

because:

$\{\}$

contains zero strings while:

$\{\epsilon\}$

contains one.

## CHAPTER 1 — COMPLETE '

TEXT

```
[ ' ] S y m b o l
[ ' ] A l p h a b e t
[ ' ] S t r i n g
[ ' ] E m p t y   s t r i n g
[ ' ] S t r i n g   l e n g t h
[ ' ] C o n c a t e n a t i o n
[ ' ] S t r i n g   p o w e r s
[ ' ] R e v e r s e
[ ' ] P r e f i x   /   S u f f i x   /   S u b s t r i n g
[ ' ] L a n g u a g e
[ ' ] E m p t y   l a n g u a g e
[ ' ]  $\mu$    v s   "   v s   {  $\mu$  }
[ ' ] £ •
[ ' ] £ *
[ ' ] £ +
[ ' ] L a n g u a g e   o p e r a t i o n s
[ ' ] L a n g u a g e   c o n c a t e n a t i o n
[ ' ] L *
[ ' ] L +
[ ' ] I m p o r t a n t   s p e c i a l   c a s e s
```

Next

## CHAPTER 2 — DETERMINISTIC FINITE AUTOMATA(DFA)

We will begin from:

**Why finite automata are needed !' states !' transitions !' formal 5-tuple !' acceptance/rejection !' first DFA constructions.**

Type NEXT.

USER

Next

ASSISTANT

Continuing with your uploaded TOC master structure. We have completed **Chapter 1 — Foundations**, so we now start **Chapter 2 — Deterministic Finite Automata (DFA)**.

## CHAPTER 2 — DETERMINISTIC FINITE AUTOMATA(DFA)

### Topic 1: DFA Fundamentals — States, Transitions, Formal Definition & Acceptance

Priority: ~~0~~=~~VERY~~ HIGH

DFA is one of the most important foundations of Theory of Computation.

Everything below later connects to:

TEXT

DFA

!"

NFA

!"

Regular Expressions

!"

Regular Languages

!"

Minimization

!"

Pumping Lemma

So do not rush this chapter.

## 1. WHY DO WE NEED A FINITE AUTOMATON?

Suppose I ask:

**Does a binary string end with 1 ?**

Examples:

TEXT

|         |    |       |
|---------|----|-------|
| 1       | !' | Y E S |
| 1 0 1   | !' | Y E S |
| 1 1 0 1 | !' | Y E S |
| 0       | !' | N O   |
| 1 1 0   | !' | N O   |
| 1 0 1 0 | !' | N O   |

We can solve this by remembering only one thing:

**What was the latest symbol?**

We don't need unlimited memory.

That is exactly the type of problem a **Finite Automaton** can solve.

## 2. INTUITION FIRST

A DFA is basically a tiny machine with:

- a finite number of states
- an input string
- rules telling it where to move
- a starting state
- one or more accepting states

It reads input:

one symbol at a time, from left to right.

## SIMPLE EXAMPLE

Suppose:

$\Sigma = \{0,1\}$

We want to accept strings ending in 1.

We need only two situations.

### State $q_0$

Last symbol is not 1, or nothing has been read.

### State $q_1$

Last symbol is 1.

Then:

TEXT

```
      1
      % % % % % % % % % %
      %                               %4
! '  q 0                               q 1 *
      %2                               %
      % % % % % % % % % %
      0
```

But we also need self/other transitions:

TEXT

```
q0 --0--> q0
q0 --1--> q1

q1 --0--> q0
q1 --1--> q1
```

$q_1^*$  is the accepting state.

## 3. WHAT DOES "STATE" MEAN?

A **state** represents the limited information the machine currently remembers.

This is one of the most useful ways to understand DFA design.

**State = memory situation**

For the previous problem:

TEXT

q0 = current string does NOT end in 1  
q1 = current string ends in 1

## MEMORY TRICK

When designing a DFA, ask:

**What information about the past do I need to remember to decide correctly at the end?**

Each required memory situation usually becomes a state.

This shortcut will become extremely powerful.

## 4. WHAT DOES "FINITE" MEAN?

Finite Automaton has only a **finite number of states**.

Example:

TEXT

q0, q1

or:

TEXT

q0, q1, q2, q3

But not:

TEXT

q0, q1, q2, q3, ...

infinitely many states.

## WHY IS THIS IMPORTANT?

Suppose we need to remember an arbitrary number  $n$ .

A finite-state machine cannot create a separate state for every possible:

$\forall n=0,1,2,3,\dots$

This eventually explains why languages such as:

$\{a^n b^n \mid n \geq 0\}$

are not regular.

We'll prove that properly later.



## 5. WHAT DOES "DETERMINISTIC" MEAN?

This is extremely important.

In a DFA:

**For every state and every input symbol, there is exactly one next state.**

Suppose:

$\Sigma = \{0,1\}$

At state  $q_0$ , there must be:

- exactly one transition for 0
- exactly one transition for 1

Not zero.

Not two.

Exactly one.

## BEST MEMORY TRICK

**DFA = Definitely one transition**

More formally:

TEXT

D = Deterministic  
! ' one definite next state

## 6. COMPONENTS OF A DFA

A DFA has five components.

Formally:

$M = (Q, \Sigma, \delta, q_0, F)$

This 5-tuple is extremely important.

### MEMORIZE IT

$M = (Q, \Sigma, \delta, q_0, F)$

## 7. FORMAL DEFINITION

A Deterministic Finite Automaton is:

$M = (Q, \Sigma, \delta, q_0, F)$

where:

## 1. $Q$

$Q$

is a finite set of states.

Example:

$Q = \{q_0, q_1\}$

---

## 2. $\Sigma$

$\Sigma$

is the input alphabet.

Example:

$\Sigma = \{0, 1\}$

---

## 3. $\delta$

$\delta$

is the transition function.

For a DFA:

$\delta: Q \times \Sigma \rightarrow Q$

This is extremely important.

We'll understand it slowly.

---

## 4. $q_0$

$q_0 \in Q$

is the **initial/start state**.

The machine begins here before reading any input.

---

## 5. $F$

$F \subseteq Q$

is the set of accepting/final states.

Example:

$F = \{q_1\}$

---

# 8. MEMORY FOR THE 5-TUPLE

---

Remember:

$Q, \Sigma, \delta, q_0, F$

Think:

## Q Students Decide Quickly Finally

A better conceptual memory:

TEXT

```
Q      !'   s t a t e s
ε      !'   s y m b o l s
'      !'   d i r e c t i o n / t r a n s i t i o n
q 0    !'   s t a r t
F      !'   f i n i s h / a c c e p t
```

## 9. UNDERSTANDING THE TRANSITION FUNCTION

The DFA transition function is:

$\delta: Q \times \Sigma \rightarrow Q$

Meaning:

Give the machine:

TEXT

(Current State, Input Symbol)

and it returns:

TEXT

Exactly One Next State

## EXAMPLE

Suppose:

$Q = \{q_0, q_1\}$

and:

$\Sigma = \{0, 1\}$

We may define:

$\delta(q_0, 0) = q_0$

$\delta(q_0, 1) = q_1$

$\delta(q_1, 0) = q_0$

$\delta(q_1, 1) = q_1$

## TRANSITION TABLE

The same DFA can be represented:

| Present State | Input 0 | Input 1 |
|---------------|---------|---------|
| ! q €         | q €     | q •     |
| * q •         | q €     | q •     |

Symbols:

- !' = initial state
- \* = final/accepting state

So:

TEXT

```
! ' q 0
* q1
```

means:

- q0 is start
- q1 is accepting

## 10. TRANSITION DIAGRAM

Same machine:

TEXT

```

      1
      % % % % % % % % °   q 1 *
      %                     %²
      %                     % 1
      %                     %
! '  q 0  %Ä% % % % % % % %
      %²                0
      %
      % % %  0

```

A simpler textual version:

TEXT

```

q0 --0--> q0
q0 --1--> q1

q1 --0--> q0
q1 --1--> q1

```

## 11. HOW DOES DFA PROCESS AN INPUT?

A DFA:

1. begins at q €
2. reads first symbol
3. follows corresponding transition
4. reads next symbol

5. continues
6. after consuming the entire string:
  - if current state " F !" ACCEPT
  - otherwise !" REJECT

---

## IMPORTANT RULE

---

Acceptance is checked:

**after the entire input is consumed.**

Simply entering a final state halfway through does **not** automatically mean the string is accepted.

This is a major beginner trap.

---

## 12. SOLVED TRACE

---

Take our DFA:

TEXT

```
q0 --0--> q0
q0 --1--> q1

q1 --0--> q0
q1 --1--> q1
```

where:

$\setminus [ F = \{q_1\} \setminus ]$

Test:

TEXT

1011

Start:

TEXT

q0

Read 1:

TEXT

q0 --1--> q1

Read 0:

TEXT

q1 --0--> q0

Read 1:

TEXT

q0 --1--> q1

Read final 1 :

TEXT

q1 --1--> q1

Final state:

$\{q_1\}$

Since:

$\{q_1\} \in F$

the string is:

$\boxed{\text{ACCEPTED}}$

## TRACE TABLE METHOD

This is often safer in exams.

| Input consumed | Current state |
|----------------|---------------|
| $\mu$          | q $\epsilon$  |
| 1              | q •           |
| 10             | q $\epsilon$  |
| 101            | q •           |
| 1011           | q •           |

Final state = q • .

Therefore accepted.

## 13. ANOTHER TRACE

Test:

TEXT

1010

| Input | State        |
|-------|--------------|
| $\mu$ | q $\epsilon$ |
| 1     | q •          |
| 10    | q $\epsilon$ |

| Input | State        |
|-------|--------------|
| 101   | $q \bullet$  |
| 1010  | $q \epsilon$ |

Final:

$\{ q_0 \notin F \}$

Therefore:

$\boxed{\text{REJECTED}}$

## WHAT LANGUAGE DOES THIS DFA ACCEPT?

Observe:

TEXT

```
1 0 1 1  !'  a c c e p t e d
1 1 1    !'  a c c e p t e d
0 0 1    !'  a c c e p t e d
1        !'  a c c e p t e d
```

while:

TEXT

```
0
10
110
μ
```

are rejected.

Pattern:

**Strings ending in 1.**

Therefore:

$L(M) = \{ w \in \{0,1\}^* \mid w \text{ ends with } 1 \}$

## 14. LANGUAGE ACCEPTED BY A DFA

The language accepted by machine  $M$  is written:

$L(M)$

Formally:

$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \}$

Meaning:

**all strings that finish in an accepting state.**

---

## 15. EXTENDED TRANSITION FUNCTION

---

The ordinary transition function:

$$\delta(q, a)$$

processes **one symbol**.

But GATE questions often require processing an entire string.

So we use:

$$\delta^*$$

called the **extended transition function**.

---

## BASIC IDEA

---

Ordinary:

$$\delta: Q \times \Sigma \rightarrow Q$$

Extended:

$$\delta^*: Q \times \Sigma^* \rightarrow Q$$

Meaning:

TEXT

State + entire string

!"

final state

---

## EXAMPLE

---

For our DFA:

$$\delta^*(q_0, 1011) = q_1$$

because after processing 1011, the machine reaches  $q_1$ .

---

## 16. FORMAL DEFINITION OF $\delta^*$

---

Base condition:

$$\delta^*(q, \epsilon) = q$$

Why?

Reading no input causes no movement.

Then recursively:

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

where:

---



- $w$  is a string
- $a$  is the final symbol

## DON'T PANIC ABOUT THIS FORMULA

It simply means:

**Process  $w$  first, then process the final symbol  $a$ .**

## Example

Calculate:

$$\delta^*(q_0, 101)$$

First:

TEXT

```
q0 --1--> q1
q1 --0--> q0
q0 --1--> q1
```

Thus:

$$\boxed{\delta^*(q_0, 101) = q_1}$$

## 17. FORMAL ACCEPTANCE CONDITION

A string  $w$  is accepted iff:

$$\boxed{\delta^*(q_0, w) \in F}$$

This is one of the central formulas of DFA.

### Meaning:

Start at  $q_0$ .

Process entire string  $w$ .

If final state belongs to  $F$  !' accept.

## 18. WHAT HAPPENS FOR $\epsilon$ ?

This is an excellent GATE concept.

For:

$$w = \epsilon$$

no input is read.

So:

$$\delta^*(q_0, \epsilon) = q_0$$

Therefore  $q_0$  is accepted iff the start state is also a final state.

So:

$$\forall \epsilon \in L(M) \text{ iff } q_0 \in F$$

**MEMORIZE THIS.**

## VISUAL SHORTCUT

If DFA starts like:

TEXT

! ' \* q 0

then:

$$\forall \epsilon \in L$$

is accepted.

Because without reading anything, machine is already in an accepting state.

## 19. COMPLETE TRANSITION REQUIREMENT

Suppose:

$$\Sigma = \{0,1\}$$

Every DFA state must specify a transition for:

TEXT

0  
and  
1

Example:

TEXT

$q_0 \xrightarrow{0} q_0$

but no transition for 1.

This is **not a complete DFA**.

## Why?

Because:

$$\delta: Q \times \Sigma \rightarrow Q$$

is a total function.

Every pair:

$$(q, a)$$

must produce exactly one next state.

## 20. DEAD / TRAP STATE

Suppose once a condition is violated, the string can never become acceptable again.

We often use a **trap state**.

Example:

```
TEXT
    0,1
    qdead  !°
```

Once entered, all inputs keep the DFA there.

## EXAMPLE LANGUAGE

Suppose we want strings containing **no** `11`.

Once we see:

```
TEXT
11
```

condition is permanently violated.

So machine enters a trap state.

Conceptually:

```
TEXT
q0 = previous symbol was not 1
q1 = previous symbol was 1
qd = 11 has occurred
```

Then:

```
TEXT
qd --0--> qd
qd --1--> qd
```

## MEMORY TRICK

**Trap = no escape**

Once entered:

all roads lead back to trap

## 21. DFA VS SIMPLE FLOWCHART

A DFA is not just any state diagram.

It has strict rules.

For every state and every alphabet symbol:

$\boxed{\text{exactly one outgoing transition}}$

## 22. HOW TO CHECK IF A MACHINE IS A DFA

Use this checklist:

### Step 1

Identify alphabet.

Example:

$\Sigma = \{0,1\}$

### Step 2

For every state, check transition on 0.

Exactly one?

### Step 3

For every state, check transition on 1.

Exactly one?

### Step 4

No  $\mu$ -transitions exist.

### Step 5

Only one initial state.

If all conditions hold !' DFA.

## 23. DFA CANNOT HAVE $\mu$ - TRANSITIONS

A DFA transition consumes exactly one symbol from:

$\Sigma$

An  $\mu$ -transition means moving without reading input.

That is allowed in an  $\mu$  - NFA. Not an ordinary DFA.

So:

$\boxed{\text{DFA has no } \varepsilon\text{-transitions}}$

## 24. CAN TWO SYMBOLS LEAD TO THE SAME STATE?

Yes.

Example:

TEXT

```
q0 --0--> q1
q0 --1--> q1
```

This is perfectly valid.

Why?

For each individual input:

- 0 has exactly one destination
- 1 has exactly one destination

Determinism is not violated.

## 25. CAN ONE SYMBOL LEAD TO TWO STATES?

Example:

TEXT

```
q0 --0--> q1
q0 --0--> q2
```

This is **not** DFA.

Because on input 0, next state is not uniquely determined.

This is NFA-style behavior.

## COMPARISON

### Valid DFA

TEXT

```
q0 --0--> q1
q0 --1--> q2
```

### Invalid DFA

TEXT

```
q0 --0--> q1
q0 --0--> q2
```

## 26. FIRST DFA CONSTRUCTION

### Problem

Construct a DFA over:

$\Sigma = \{0,1\}$

that accepts all strings ending in 1.

### STEP 1 — ASK WHAT MUST BE REMEMBERED

At the end we only need to know:

Is the latest symbol 1?

So two memory situations:

TEXT

$q_0$  = string does not currently end in 1  
 $q_1$  = string currently ends in 1

### STEP 2 — INITIAL STATE

Before reading anything:

$\epsilon$

does not end in 1.

So:

$q_0$

is the start state.

### STEP 3 — ACCEPTING STATE

If current string ends in 1:

$q_1$

should be final.

Thus:

$F = \{q_1\}$

### STEP 4 — TRANSITIONS

From  $q_0$ :

Read 0:

TEXT

still not ending in 1

Therefore:

$$\forall \delta(q_0, 0) = q_0$$

Read 1:

TEXT

now ends in 1

Therefore:

$$\forall \delta(q_0, 1) = q_1$$

From  $q_1$ :

Read 0:

TEXT

now ends in 0

Therefore:

$$\forall \delta(q_1, 0) = q_0$$

Read 1:

TEXT

still ends in 1

Therefore:

$$\forall \delta(q_1, 1) = q_1$$

## FINAL DFA

| State        | 0   | 1           |
|--------------|-----|-------------|
| $!q$         | $q$ | $q \bullet$ |
| $*q \bullet$ | $q$ | $q \bullet$ |

## FORMAL 5-TUPLE

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

$\Sigma = \{0, 1\}$

and  $\delta$  is given by the transition table.

## 27. SECOND DFA CONSTRUCTION

### Language

All binary strings ending in 0.

Again two memory situations:

TEXT

$q_0$  = does not end in 0  
 $q_1$  = ends in 0

Transition table:

| State | 0     | 1     |
|-------|-------|-------|
| $q_0$ | $q_0$ | $q_1$ |
| $q_1$ | $q_1$ | $q_0$ |

Notice how similar this is to the previous construction.

## 28. THIRD DFA CONSTRUCTION — EVEN NUMBER OF 1s

This introduces a very important GATE pattern.

Language:

$L = \{w \in \{0,1\}^* \mid w \text{ contains an even number of 1s}\}$

## WHAT MUST WE REMEMBER?

Not the exact number of 1s.

We only need:

TEXT

even  
or  
odd

Therefore two states:

TEXT

$q_E$  = even number of 1s  
 $q_O$  = odd number of 1s



## START STATE

Initially:

$\{0\}$

ones have occurred.

Zero is even.

Therefore:

$\{q_E\}$

is the start state.

## FINAL STATE

We accept even number of 1s.

Therefore:

$\{q_E\}$

is also final.

TEXT

! ' \* q E

## TRANSITIONS ON 0

Reading 0 does not change number of 1s.

Therefore:

TEXT

qE --0--> qE  
q0 --0--> q0

## TRANSITIONS ON 1

Each 1 flips parity:

TEXT

E v e n ! ' o d d  
o d d ! ' E v e n

Therefore:

TEXT

qE --1--> q0  
q0 --1--> qE

## FINAL TABLE

| State   | 0  | 1  |
|---------|----|----|
| ! * q E | qE | qO |
| qO      | qO | qE |

## BEAUTIFUL MEMORY RULE

For parity DFAs:

**Relevant symbol toggles. Irrelevant symbol stays.**

Here:

- **1** changes parity !' toggle.
- **0** doesn't !' stay.

This pattern appears constantly.

## 29. TEST SOME STRINGS

**μ**

Number of 1s:

$\{ 0 \}$

even !' accepted.

**0**

0 ones !' accepted.

**1**

1 one !' rejected.

**101**

2 ones !' accepted.

**111**

3 ones !' rejected.

## 30. GATE DESIGN STRATEGY

Whenever constructing a DFA:

## Step 1 — Read the language carefully

Find exactly what must be checked.

## Step 2 — Ask:

What minimum information about the prefix read so far must be remembered?

## Step 3

Create one state for each meaningful memory situation.

## Step 4

Identify initial memory situation.

That becomes start state.

## Step 5

Identify memory situations satisfying the language.

Those become final states.

## Step 6

For every state and every alphabet symbol, ask:

What will my memory become after reading this symbol?

Add transition.

## Step 7

Check:

Does every state have exactly one transition per alphabet symbol?

## Step 8

Test with:

- one accepted string
- one rejected string

- $\mu$
- boundary cases

## THIS IS THE CORE DFA DESIGN ALGORITHM

TEXT

```
Language condition
    !"
What must I remember?
    !"
Create memory states
    !"
Choose start state
    !"
Choose accepting states
    !"
Fill transitions
    !"
Test strings
```

## 31. TYPICAL GATE/PYQ PATTERNS

GATE frequently tests DFA through:

### Pattern 1 — Identify accepted language

Given a DFA, determine what it accepts.

### Pattern 2 — Trace a string

Calculate:

$\delta^*(q_0, w)$

### Pattern 3 — Minimum states

Examples:

- parity
- divisibility
- suffix
- substring
- exact/count constraints

This becomes very important later.

### Pattern 4 — Construct/combine DFA

Such as:

TEXT

```
number of 0s even
AND
```

number of 1s odd

Later we'll use **product automata**.

### Pattern 5 — Complement

Swap final/non-final states of a **complete DFA**.

We'll study this carefully.

## 32. COMMON GATE TRAPS

### & pTrap 1 — Final state reached halfway = accepted

Wrong.

Entire input must be consumed.

### & pTrap 2 — DFA may have two transitions for same symbol

Wrong.

Example:

TEXT

```
q0 --0--> q1
q0 --0--> q2
```

Not DFA.

### & pTrap 3 — A transition can be missing

Not in a complete DFA.

Every:

$\forall (q,a)$

must have a destination.

### & pTrap 4 — DFA can use $\mu$ -transition

Wrong.

$\mu$ -transition belongs to  $\mu$ -NFA.

### & pTrap 5 — Start state cannot be final

Wrong.

It can.

If:

$$\forall q_0 \in F$$

then  $\mu$  is accepted.

---

## & ⚠Trap 6 — More states automatically means more computational power

---

Wrong.

A DFA with 100 states and DFA with 2 states belong to the same computational model.

More states may recognize a more complicated regular language, but DFA expressive class stays **regular languages**.

---

## & ⚠Trap 7 — DFA remembers entire input

---

Wrong.

It only stores information through its finite state.

The actual past input isn't separately available.

---

## 33. MUST REMEMBER

---

### Formal DFA

$$\boxed{M = (Q, \Sigma, \delta, q_0, F)}$$

### Transition function

$$\boxed{\delta: Q \times \Sigma \rightarrow Q}$$

### Extended transition

$$\boxed{\delta^*: Q \times \Sigma^* \rightarrow Q}$$

### Empty input

$$\boxed{\delta^*(q, \epsilon) = q}$$

### Acceptance

$$\boxed{w \in L(M) \text{ iff } \delta^*(q_0, w) \in F}$$

### $\mu$ accepted iff:

$$\boxed{q_0 \in F}$$

---

## 34. COMPARISON — $\cdot$ VS $\cdot^*$

---

| Feature         | $\cdot$           | $\cdot^*$           |
|-----------------|-------------------|---------------------|
| Input processed | One symbol        | Entire string       |
| Domain          | $Q \times \Sigma$ | $Q \times \Sigma^*$ |

| Feature | $\delta$         | $\delta^*$            |
|---------|------------------|-----------------------|
| Output  | State            | State                 |
| Example | $\delta(q_0, 1)$ | $\delta^*(q_0, 1011)$ |

Memory:

$\delta$  = one step

$\delta^*$  = many steps

## 35. SHORTCUTS & MEMORY TRICKS

### DFA

Definitely one transition

### State

State = memory

### Acceptance

Finish in F

F = Final.

### Empty string

No movement

Therefore:

$\delta^*(q, \epsilon) = q$

### Parity DFA

Matching symbol toggles parity

### Trap state

Trap = once in, never out

## 36. 'p WRITE THIS IN YOUR NOTES

### Deterministic Finite Automaton

A DFA is:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

- $Q$  = finite states
- $\Sigma$  = input alphabet
- $\delta$  = transition function
- $q_0$  = initial state
- $F$  = final states

#### DFA transition function

$$\delta: Q \times \Sigma \rightarrow Q$$

For every state-symbol pair:

exactly one next state

No  $\mu$ -transition.

### Extended transition

$$\delta^*: Q \times \Sigma^* \rightarrow Q$$

$$\delta^*(q, \epsilon) = q$$

Acceptance:

$$w \in L(M) \text{ iff } \delta^*(q_0, w) \in F$$

### $\mu$ rule

$$\epsilon \in L(M) \text{ iff } q_0 \in F$$

### DFA design idea

TEXT

Language condition

!"

Required memory

!"

States



Transitions !"

### Golden question:

What information must the DFA remember about the prefix read so far?

## Parity pattern

For even/odd count:

TEXT

Relevant symbol !' toggle  
Other symbol !' stay

## 37. 60-SECOND REVISION

- DFA = deterministic finite automaton.
- State represents finite memory.
- DFA is a 5-tuple.
- $M = (Q, \Sigma, \delta, q_0, F)$
- $\delta : Q \times \Sigma \rightarrow Q$
- Exactly one transition for every state-symbol pair.
- DFA has no  $\mu$ -transition.
- Entire input must be consumed before checking acceptance.
- Accept iff final state belongs to  $F$ .
- $\delta$  processes strings.
- $\delta^*(q, \mu) = q$
- $\mu$  is accepted iff start state is accepting.
- Trap state cannot escape.
- DFA design begins by asking what information must be remembered.

## 38. ACTIVE RECALL

Try before reading the answers.

### Q1

What are the five components of a DFA?

### Q2

What does deterministic mean?

### Q3

Write the DFA transition-function type.

#### Q4

Can a DFA have an  $\mu$ -transition?

#### Q5

Can the start state also be final?

#### Q6

When does a DFA accept  $\mu$ ?

#### Q7

What does:

$\delta^*(q_0, 101)$

represent?

#### Q8

What does a state conceptually represent?

#### Q9

Can this occur in a DFA?

TEXT

```
q0 --0--> q1
q0 --0--> q2
```

#### Q10

For a parity DFA, what usually happens when the counted symbol appears?

## ANSWERS

1.

$\{Q, \Sigma, \delta, q_0, F\}$

2. Exactly one next state for each state-symbol pair.

3.

$\delta: Q \times \Sigma \rightarrow Q$

4. No.

5. Yes.

6. If:

$q_0 \in F$

7. State reached after reading the complete string **101** starting from  $q \in$ .

8. Required finite memory about the input read so far.

9. No. That would introduce nondeterminism.

10. The parity state toggles.

## 39. MINI PRACTICE

---

### Q1 — MCQ

---

A DFA transition function is:

A.

$$\mathbb{Q} \times \Sigma \rightarrow \mathbb{Q}$$

B.

$$\mathbb{Q} \times \Sigma \rightarrow \mathbb{Q}$$

C.

$$\mathbb{Q} \rightarrow \Sigma$$

D.

$$\Sigma \rightarrow \mathbb{Q}$$

**Answer**

$$\boxed{B}$$

---

### Q2 — MCQ

---

If the initial state of a DFA is also accepting, which must be accepted?

A.  $\emptyset$  B.  $\{1\}$  C.  $\mu$  D. Every string

**Answer**

$$\boxed{C}$$

---

### Q3 — True/False

---

A DFA may temporarily enter a final state, leave it later, and reject the final string.

**Answer**

$$\boxed{\text{True}}$$

Acceptance depends on the state after **all** input symbols are consumed.

---

### Q4 — MCQ

---

A DFA over alphabet  $\{0, 1\}$  has a state  $q$ . How many defined outgoing transitions must  $q$  have, counting input-symbol choices?

$$\boxed{2}$$

One for  $0$ , one for  $1$ .

They may point to the same destination.

---

## CHAPTER 2 PROGRESS

### TEXT

```
[' ] Why finite automata are needed
[' ] State intuition
[' ] Determinism
[' ] Formal DFA 5-tuple
[' ] Transition function
[' ] Transition table
[' ] Transition diagram
[' ] Acceptance and rejection
[' ] Extended transition '*'
[' ] Empty-string acceptance
[' ] Complete-transition requirement
[' ] Trap/dead state basics
[' ] First DFA constructions
[' ] Parity DFA intuition
```

```
[ ] DFA construction patterns
[ ] Prefix languages
[ ] Suffix languages
[ ] Substring languages
[ ] Contains / does-not-contain patterns
[ ] Exact occurrence questions
[ ] At least / at most occurrence questions
[ ] Modulo counting
[ ] Binary-number divisibility DFA
[ ] Complement
[ ] Product automata
[ ] Minimum-state intuition
```

### Next topic

## DFA CONSTRUCTION PATTERNS

We will systematically learn how to build DFAs for:

**starts with !' ends with !' contains substring !' avoids substring !' exact counts !' at least/at most counts !'**  
**parity/modulo conditions**, including the shortcuts for deciding the correct number of states.

Type **NEXT**.

USER

NEXT

### ASSISTANT

Continuing from your master prompt and the completed DFA fundamentals section.

## CHAPTER 2 — DETERMINISTIC FINITE AUTOMATA

### Topic 2: DFA Construction Patterns

Priority: ~~0~~=~~VERY~~ HIGH

This is where DFA becomes useful for GATE.

The goal is not to memorize dozens of diagrams.

The goal is to learn a few **construction patterns** so that you can build an unfamiliar DFA yourself.

## 1. THE GOLDEN DFA QUESTION

Whenever you see:

**Construct a DFA for language L**

ask:

**What minimum information about the input read so far must I remember?**

That information becomes your states.

## MASTER DFA DESIGN FORMULA

### TEXT

Language condition

!"

What information matters?

!"

Memory situations

!"

States

!"

Transitions

!"

Final states

## 2. THE MAIN DFA PATTERNS

Most GATE-level DFA constructions fall into these families:

1. Starts with a pattern
2. Ends with a pattern
3. Contains a substring
4. Does not contain a substring
5. Exact number of occurrences
6. At least / at most occurrences
7. Even / odd occurrence
8. Count modulo  $k$
9. Length modulo  $k$
10. Binary number modulo/divisibility
11. Multiple conditions using product construction

We will build these systematically.

## PART A — STARTS WITH

### Example 1

Construct DFA over:

$\Sigma = \{0,1\}$

accepting all strings that **start with 1**.

### Intuition

After the **first character**, our fate is already decided.

If first character is:

TEXT

```
1 !' condition satisfied forever
0 !' condition violated forever
```

So we need:

TEXT

```
q0 = nothing read yet
qA = started with 1
qD = started with 0
```

### DFA

TEXT

```
q0 --1--> qA*
q0 --0--> qD

qA --0--> qA
qA --1--> qA

qD --0--> qD
qD --1--> qD
```

### Transition table

| State | 0  | 1  |
|-------|----|----|
| ! q € | qD | qA |
| *qA   | qA | qA |
| qD    | qD | qD |

## WHY DO qA AND qD LOOP?

Once the first symbol has been read:

- if it was 1, later symbols cannot change the fact that the string started with 1
- if it was 0, later symbols cannot repair it

## MEMORY TRICK

Starts with = decision happens early

Once prefix condition is known, often move to:

- permanent accept state
- permanent reject/trap state

## Example strings

TEXT

|             |   |
|-------------|---|
| 1           | ' |
| 1 0         | ' |
| 1 0 1 1 0 1 | ' |
| 0           | ' |
| 0 1         | ' |
| 0 0 1       | ' |
| μ           | ' |

μ is rejected because no first symbol exists.

## PART B — ENDS WITH

Ending conditions are different.

Why?

Because the machine cannot know the end until input finishes.

Therefore, it must continuously remember the **relevant suffix**.

## Example 2

Construct DFA for strings ending with 1.

We already saw:

TEXT

q0 = current string does not end in 1  
q1 = current string ends in 1

Transitions:

TEXT

```
q0 --0--> q0
q0 --1--> q1*

q1 --0--> q0
q1 --1--> q1*
```

## Important intuition

For suffix problems:

**Remember the useful ending of the prefix read so far.**

## Example 3 — Ends with 01

Construct DFA over  $\{0, 1\}$  accepting strings ending with:

TEXT

01

## Step 1 — What must we remember?

We care about whether the current input suffix matches part of 01.

Useful situations:

TEXT

```
q0 = no useful suffix
q1 = current string ends in 0
q2 = current string ends in 01
```

q2 is accepting.

## Step 2 — Build transitions

### From q0

Read 0:

TEXT

current suffix = 0

So:

$\backslash$  q0 to q1  $\backslash$



Read 1:

No useful part of 01.

So remain q0.

TEXT

```
q0 --0--> q1
q0 --1--> q0
```

## From q1

Current string ends in 0.

Read 1:

Now ends in:

TEXT

01

So:

TEXT

```
q1 --1--> q2
```

Read 0:

Current string still ends in 0.

Example:

TEXT

...00

The last 0 can start a future 01.

Therefore:

TEXT

```
q1 --0--> q1
```

## From q2

Current string ends in 01.

Read 0:

New ending:

TEXT

010

Its useful suffix is:

TEXT

0

So:

TEXT

q2 --0--> q1

Read 1:

New ending:

TEXT

011

No useful suffix matching beginning of 01 .

Therefore:

TEXT

q2 --1--> q0

## Final table

| State | 0   | 1   |
|-------|-----|-----|
| ! q € | q • | q € |
| q •   | q • | q , |
| * q , | q • | q € |

## Test

101

TEXT

q0 --1--> q0  
q0 --0--> q1  
q1 --1--> q2\*

Accepted.

001

TEXT

q0 --0--> q1  
q1 --0--> q1  
q1 --1--> q2\*

Accepted.

010

ends with 10.

Rejected.

## POWERFUL SUFFIX SHORTCUT

For a target suffix:

TEXT

P

states usually mean:

length of the longest suffix of what I have read that is also a prefix of P.

This idea will return when we study substring DFAs.

## PART C — CONTAINS A SUBSTRING

### Example 4

Construct DFA accepting strings containing:

TEXT

01

somewhere.

Examples:

TEXT

```

0 1
0 0 1
1 0 1 1
1 1 0 1 0 0

```

Once 01 appears, later symbols cannot destroy that fact.

So after seeing it:

permanent accept state.

## States

#### TEXT

q0 = have not seen useful 0  
q1 = most recent useful suffix is 0  
q2 = substring 01 has occurred

q2 = accepting.

## Transitions

#### TEXT

q0 --0--> q1  
q0 --1--> q0  
  
q1 --0--> q1  
q1 --1--> q2\*  
  
q2 --0--> q2  
q2 --1--> q2

## Transition table

| State | 0   | 1   |
|-------|-----|-----|
| ! q € | q • | q € |
| q •   | q • | q , |
| * q , | q , | q , |

## IMPORTANT DIFFERENCE

Compare:

### Ends with 01

After reaching accepting state, later characters can make us leave it.

### Contains 01

Once substring appears, we stay accepting forever.

## MEMORY

**Contains = achievement cannot be undone**

So accepting state often becomes an absorbing state.

## PART D — DOES NOT CONTAIN A SUBSTRING

### Example 5

Construct DFA for binary strings that do **not** contain:

TEXT

11

### What must be remembered?

We only need to know:

TEXT

q0 = previous symbol is not 1

q1 = previous symbol is 1

qD = 11 has appeared

If 11 ever appears, condition can never recover.

Therefore qD is trap.

### Start/final states

Before input:

TEXT

no 11 has occurred

so q0 is accepting.

q1 also represents no 11 yet, so q1 is accepting.

Trap qD rejects.

### Transitions

TEXT

q0 --0--> q0

q0 --1--> q1

q1 --0--> q0

q1 --1--> qD

qD --0--> qD

qD --1--> qD

### Table

| State | 0     | 1     |
|-------|-------|-------|
| $q_0$ | $q_0$ | $q_1$ |
| $q_1$ | $q_0$ | $q_2$ |
| $q_2$ | $q_2$ | $q_2$ |

## Test

TEXT

```

μ      '
0      '
1      '
1 0 1 0 1  '
1 1 0      '
0 1 1 0    '

```

## MEMORY TRICK

Avoid pattern = trap when pattern appears

## PART E — EXACT NUMBER OF OCCURRENCES

### Example 6

Construct DFA accepting strings over  $\{0, 1\}$  having **exactly two 1s**.

### What must we remember?

Count 1s up to 2.

After 3 or more, rejection is permanent.

States:

TEXT

```

q0 = seen 0 ones
q1 = seen 1 one
q2 = seen exactly 2 ones
q3 = seen 3 or more ones

```

$q_2$  is accepting.

$q_3$  is trap.

## What does 0 do?

Zero does not change number of 1s.

So every state loops on 0.

## What does 1 do?

Moves count forward:

TEXT

$q_0 \xrightarrow{!} q_1 \xrightarrow{!} q_2 \xrightarrow{!} q_3$

and  $q_3$  remains  $q_3$ .

## Table

| State       | 0           | 1           |
|-------------|-------------|-------------|
| $! q \in$   | $q \in$     | $q \bullet$ |
| $q \bullet$ | $q \bullet$ | $q ,$       |
| $* q ,$     | $q ,$       | $q f$       |
| $q f$       | $q f$       | $q f$       |

## General shortcut

For **exactly k occurrences** of some symbol, normally need:

$\lfloor \boxed{k+2} \rfloor$

memory categories:

TEXT

$0, 1, 2, \dots, k, >k$

So:

$\lfloor k+1 \rfloor$

count states plus one overflow/trap state.

## Example

Exactly 5 ones:

TEXT

0  
1  
2  
3  
4  
5  
>5

Total:

$\lfloor \boxed{7} \rfloor$

states.

This is a useful shortcut, but later we'll discuss minimization formally.

## PART F — AT MOST k OCCURRENCES

### Example 7

Strings containing **at most two 1s**.

States:

TEXT

q0 = 0 ones  
q1 = 1 one  
q2 = 2 ones  
q3 = 3 or more

Accept:

TEXT

q0, q1, q2

Reject:

TEXT

q3

Same transitions as previous DFA.

Only final-state selection changes.

## General pattern

For:

**at most k occurrences**

track:



TEXT

$0, 1, \dots, k, >k$

Normally:

$\boxed{k+2}$

states.

Accept first:

$k+1$

states.

Overflow state rejects.

## PART G — AT LEAST $k$ OCCURRENCES

### Example 8

Strings having **at least two 1s**.

We need:

TEXT

$q_0$  = 0 ones

$q_1$  = 1 one

$q_2$  = 2 or more ones

Once  $q_2$  is reached, exact number no longer matters.

### Table

| State | 0   | 1   |
|-------|-----|-----|
| $!q$  | $q$ | $q$ |
| $q$   | $q$ | $q$ |
| $*q$  | $q$ | $q$ |

### General shortcut

For:

**at least  $k$  occurrences**

states represent:

TEXT

$0, 1, 2, \dots, k-1, \text{ "e k}$

Therefore normally:

$\boxed{k+1}$

states.

## Important comparison

| Condition  | Typical states |
|------------|----------------|
| Exactly k  | $k + 2$        |
| At most k  | $k + 2$        |
| At least k | $k + 1$        |

Why?

For exactly/at-most, we need an overflow state.

For at-least, after reaching  $k$ , all larger counts are equivalent.

## PART H — EVEN / ODD COUNT

We already studied even number of 1s.

States:

TEXT

$q_E$  = even  
 $q_O$  = odd

Relevant symbol toggles.

Other symbol stays.

## Even number of 1s

| State              | 0     | 1     |
|--------------------|-------|-------|
| $! \text{ } * q_E$ | $q_E$ | $q_O$ |
| $q_O$              | $q_O$ | $q_E$ |

## Odd number of 1s

Same transitions.

Only final state changes:

TEXT

$F = \{q_0\}$

## MEMORY

Parity only needs remainder modulo 2.

We do not care whether count is:

TEXT

2  
4  
6  
8  
...

They are all equivalent from the DFA's perspective.

## PART I — COUNT MODULO k

This generalizes parity.

### Example 9

Construct DFA accepting binary strings where number of 1s is divisible by 3.

## What must be remembered?

Only the remainder after dividing count of 1s by 3.

Possible remainders:

TEXT

0  
1  
2

States:

TEXT

|       |   |     |    |   |     |   |
|-------|---|-----|----|---|-----|---|
| $q_0$ | = | # 1 | "a | 0 | mod | 3 |
| $q_1$ | = | # 1 | "a | 1 | mod | 3 |
| $q_2$ | = | # 1 | "a | 2 | mod | 3 |

On 0

Count of 1s does not change.  
So stay.

On 1

Remainder cycles:

TEXT

0 1' 1 1' 2 1' 0

Table

| State    | 0   | 1   |
|----------|-----|-----|
| 1' * q € | q € | q • |
| q •      | q • | q , |
| q ,      | q , | q € |

Why q0 accepting?

Because:  
 $\{0,3,6,9,\ldots\}$   
are divisible by 3.  
Also note:  
 $\{0\}$   
ones is divisible by 3.  
So  $\mu$ is accepted.

General modulo-count rule

For:  
 $\{a \equiv r \pmod{k}\}$   
create:  
 $\{k\}$   
states:

TEXT

$q_0, \dots, q_{k-1}, \dots, q_{(k-1)}$

where  $q_i$  represents remainder  $i$ .

On each counted symbol:  
$$i \mapsto (i+1) \bmod k$$

## MEMORY TRICK

**Modulo k !' think k remainder states**

This is one of the strongest DFA shortcuts.

## PART J — LENGTH MODULO k

Another very common pattern.

### Example 10

Accept all binary strings whose length is divisible by 3.

We don't care whether the symbol is 0 or 1.

Every symbol increases length by 1.

States:

TEXT

|     |   |        |    |   |     |   |
|-----|---|--------|----|---|-----|---|
| q 0 | = | length | "a | 0 | mod | 3 |
| q 1 | = | length | "a | 1 | mod | 3 |
| q 2 | = | length | "a | 2 | mod | 3 |

Every symbol causes:

TEXT

|     |   |     |   |     |   |     |
|-----|---|-----|---|-----|---|-----|
| q 0 | ! | q 1 | ! | q 2 | ! | q 0 |
|-----|---|-----|---|-----|---|-----|

## Table

| State   | 0   | 1   |
|---------|-----|-----|
| ! * q € | q • | q • |
| q •     | q , | q , |
| q ,     | q € | q € |

## Major difference

### Count of 1s modulo 3

Only input 1 changes state.

## Length modulo 3

Every symbol changes state.

## MEMORY

Length counts every symbol.

## PART K — ENDS WITH 00

Let's build another suffix problem because these are important.

Language:

$L = \{w \mid w \text{ ends with } 00\}$

States:

TEXT

q0 = no useful trailing 0  
q1 = ends in one 0  
q2 = ends in 00

q2 accepting.

## Transitions

From q0:

TEXT

|   |   |     |
|---|---|-----|
| 0 | ! | q 1 |
| 1 | ! | q 0 |

From q1:

TEXT

|   |   |     |
|---|---|-----|
| 0 | ! | q 2 |
| 1 | ! | q 0 |

From q2:

Read 0:

String still ends in 00.

Example:

TEXT

000

So:

TEXT

0 !' q 2

Read 1:

No longer has useful trailing zero.

TEXT

1 !' q 0

## Table

| State  | 0   | 1   |
|--------|-----|-----|
| !' q € | q • | q € |
| q •    | q , | q € |
| * q ,  | q , | q € |

## PART L — CONTAINS 101

This shows how to track a longer substring.

We want substring:

TEXT

101

States correspond to progress:

TEXT

q0 = matched nothing  
q1 = matched 1  
q2 = matched 10  
q3 = found 101

q3 accepting and absorbing.

## From q0

TEXT

1 !' q 1  
0 !' q 0

## From q1

We've matched:

TEXT  
1

Read 0:

TEXT  
10

So:

TEXT  
q1 --0--> q2

Read 1:

Current ending is still:

TEXT  
1

because in:

TEXT  
11

the second 1 can begin a new 101.

So:

TEXT  
q1 --1--> q1

---

## From q2

We've matched:

TEXT  
10

Read 1:

TEXT  
101

found.

TEXT  
q2 --1--> q3



Read 0:

Current ending:

TEXT

100

There is no suffix that is a prefix of 101.

So:

TEXT

q2 --0--> q0

q3

Once found:

TEXT

q3 --0--> q3  
q3 --1--> q3

Final table

| State | 0   | 1   |
|-------|-----|-----|
| ! q € | q € | q • |
| q •   | q , | q • |
| q ,   | q € | q f |
| * q f | q f | q f |

EXTREMELY USEFUL SUBSTRING PRINCIPLE

For finding pattern P:

Each state stores how much of P has currently been matched as a useful suffix.

Example pattern:

TEXT

101

progress states:

TEXT

```
" "      !'   q 0
" 1 "    !'   q 1
" 1 0 "  !'   q 2
" 1 0 1 "!'   q 3
```

But after mismatches, do not always go to q0.

You may already have a suffix that begins another possible match.

This is a common GATE trap.

## Example

Suppose target is:

TEXT

101

and you've matched 1.

Then read another 1.

You have:

TEXT

11

Although the first attempt failed, the final 1 itself may start a fresh 101.

Therefore:

TEXT

q1 --1--> q1

not q0.

## PART M — EXACTLY ONE 1

Let's apply the counting rule.

States:

TEXT

```
q0 = zero 1s
q1 = exactly one 1
q2 = two or more 1s
```

q1 accepting.

## Table

| State | 0   | 1   |
|-------|-----|-----|
| ! q € | q € | q • |
| * q • | q • | q , |
| q ,   | q , | q , |

## PART N — NO 1

Language consists only of zeroes:

$\{0^*\}$

States:

TEXT

qA = no 1 encountered  
qD = a 1 occurred

## Table

| State   | 0  | 1  |
|---------|----|----|
| ! * q A | qA | qD |
| qD      | qD | qD |

µaccepted.

Why?

It contains no 1.

## PART O — AT LEAST ONE 1

States:

TEXT

q0 = no 1 seen  
q1 = at least one 1 seen

## Table

| State | 0   | 1   |
|-------|-----|-----|
| ! q € | q € | q • |
| * q • | q • | q • |

Once one 1 appears:

condition permanently satisfied.

## PART P — STARTS WITH 01

Language:

TEXT

01...

We must check the first two symbols.

States:

TEXT

q0 = nothing read  
q1 = first symbol was 0  
qA = first two are 01  
qD = condition permanently failed

## Transitions

TEXT

q0 --0--> q1  
q0 --1--> qD  
  
q1 --1--> qA  
q1 --0--> qD  
  
qA --0--> qA  
qA --1--> qA  
  
qD --0--> qD  
qD --1--> qD

## Important

The string:

TEXT

0

is rejected.

Because it does not yet start with the complete prefix 01.

## PART Q — GATE STATE-COUNT SHORTCUTS

These are useful but always verify with logic.

## 1. At least k occurrences

Usually:

$\forall \boxed{k+1}$

states.

## 2. At most k occurrences

Usually:

$\forall \boxed{k+2}$

states.

## 3. Exactly k occurrences

Usually:

$\forall \boxed{k+2}$

states.

## 4. Count modulo k

Usually:

$\forall \boxed{k}$

states.

## 5. Length modulo k

Usually:

$\forall \boxed{k}$

states.

## 6. Contains a fixed pattern of length m

Often approximately:

$\forall m+1$

states:

TEXT

matched 0

matched 1

...

matched  $m$

but pattern overlap must be handled correctly.

## 7. Ends with a fixed pattern

Often states correspond to useful prefixes of the target suffix.

Do not blindly say  $m+1$ ; analyze overlaps.

## 3. THE MOST IMPORTANT DESIGN IDEA — STATES ARE EQUIVALENCE CLASSES OF MEMORY

Consider language:

**even number of 1s.**

Suppose one prefix contains:

TEXT  
2 ones

and another contains:

TEXT  
100 ones

Do we need separate states?

No.

For future acceptance, both are equivalent because both counts are even.

So one state represents both.

## This is the essence of DFA minimization

Two histories can share a state when:

**No possible future continuation requires the machine to treat them differently.**

We will make this formal later with distinguishable states and Myhill-Nerode intuition.

## 4. SOLVED GATE-STYLE QUESTION

A DFA accepts binary strings with an odd number of zeros.

How many states are required?

We only need to remember parity:

TEXT

even zeros  
odd zeros

Thus:

$\boxed{2}$

states.

## SOLVED QUESTION 2

Minimum memory categories for strings containing exactly 4 ones?

Need:

TEXT

0  
1  
2  
3  
4  
>4

Therefore:

$\boxed{6}$

states.

## SOLVED QUESTION 3

Strings containing at least 4 zeros?

Need:

TEXT

0  
1  
2  
3  
"e 4

Therefore:

$\boxed{5}$

states.

## SOLVED QUESTION 4

Strings where number of a s is congruent to 2 modulo 5.

Possible remainders:

TEXT

0  
1  
2  
3  
4

Therefore:

$\boxed{5}$

states.

Accepting:

TEXT

q2

## SOLVED QUESTION 5

Binary strings whose length is:

$1 \pmod 4$

States:

TEXT

q0 = length mod 4 = 0  
q1 = 1  
q2 = 2  
q3 = 3

Accept q1.

Total:

$\boxed{4}$

states.

## 5. GATE/PYQ PATTERN ANALYSIS

Typical questions usually ask one of four things.

### Pattern 1 — Draw/identify DFA

You are given a language condition.

Translate it into memory states.

### Pattern 2 — Find minimum states

Common conditions:



TEXT

exactly k  
at least k  
modulo k  
suffix  
substring

Do not guess from pattern length alone.

Ask how many distinct memory situations genuinely matter.

### Pattern 3 — Identify language from diagram

Translate each state into meaning.

Example:

TEXT

q0 = even  
q1 = odd

Then observe which transition toggles.

### Pattern 4 — Find accepted/rejected strings

Trace input carefully to the final state.

## 6. & pGATE TRAPS

### Trap 1 — "Contains 01" and "ends with 01" use same DFA

Wrong.

For contains 01:

once found '1' accepting forever.

For ends with 01:

later input can destroy the suffix.

### Trap 2 — Every pattern mismatch goes to start state

Wrong.

Overlapping pattern information may survive.

Example target:

TEXT

101

After seeing:

TEXT

11

the last 1 can start a new pattern.

### Trap 3 — Exactly k and at least k need same number of states

Usually false.

At least k can collapse all:

TEXT

k, k+1, k+2, ...

into one state.

Exactly k must distinguish:

TEXT

exactly k

from:

TEXT

greater than k

### Trap 4 — For even number of 1s, start state is odd

Wrong.

Initially:

$\lceil 0 \rceil$

ones have occurred.

Zero is even.

### Trap 5 — Length modulo k only changes on a particular symbol

Wrong.

Length increases on **every** symbol.

### Trap 6 — A substring DFA never needs self-loops before completion

Wrong.

Overlap often creates self-loops.

Example:

TEXT

contains 101

At state representing 1:

TEXT

i n p u t 1 ! ' r e m a i n a t t h a t s t a t e

## Trap 7 — Minimum states can always be obtained by a memorized formula

Wrong.

Formulas are shortcuts for common patterns.

Actual minimum-state proof requires distinguishability/minimization reasoning.

## 7. MASTER COMPARISON TABLE

| Language condition | What DFA remembers                             |
|--------------------|------------------------------------------------|
| Starts with P      | Whether initial prefix matched or failed       |
| Ends with P        | Relevant suffix matching prefix of P           |
| Contains P         | Progress toward P / whether P already occurred |
| Avoids P           | Progress toward P + trap if P occurs           |
| Exactly k a's      | $0, 1, \dots, k, >k$                           |
| At most k a's      | $0, 1, \dots, k, >k$                           |
| At least k a's     | $0, 1, \dots, k-1, \geq k$                     |
| Even/odd a's       | count mod 2                                    |
| Count mod k        | count remainder mod k                          |
| Length mod k       | length remainder mod k                         |

This table is worth writing in your notebook.

## 8. DFA CONSTRUCTION DECISION GUIDE

When you see the language, ask:

**Is it a START condition?**

Example:

TEXT

starts with 01

!' check beginning, then permanent accept/reject.

---

### Is it an END condition?

Example:

TEXT

ends with 101

!' track relevant suffix continuously.

---

### Is it a CONTAINS condition?

Example:

TEXT

contains 101

!' track pattern progress; once found, accept forever.

---

### Is it an AVOIDS condition?

Example:

TEXT

does not contain 11

!' track progress; once forbidden pattern appears, trap.

---

### Is it a COUNT condition?

Example:

TEXT

exactly 3 ones

!' count up to threshold.

---

### Is it a MODULO condition?

Example:

TEXT

# 1 "a 2 m o d 5

!' remainder states.

---

## 9. 'p WRITE THIS IN YOUR NOTES

### DFA Construction Golden Rule

A DFA state represents the minimum information about the prefix read so far needed to decide future acceptance.

### Construction procedure

TEXT

1. Identify language condition
2. Find required memory
3. Create one state per memory situation
4. Choose start state
5. Choose accepting state(s)
6. Fill every transition
7. Add trap state if necessary
- 8 .   T e s t   μ   +   a c c e p t e d   +   r e j e c t e d   s t r i n g s

### Common state-count patterns

$\backslash \text{At least } k \rightarrow k+1 \backslash$

$\backslash \text{Exactly } k \rightarrow k+2 \backslash$

$\backslash \text{At most } k \rightarrow k+2 \backslash$

$\backslash \text{Modulo } k \rightarrow k \backslash$

Use as shortcuts, not as blind proofs.

### Pattern memory

TEXT

STARTS WITH

! '   d e c i s i o n   h a p p e n s   e a r l y

ENDS WITH

! '   r e m e m b e r   u s e f u l   s u f f i x

CONTAINS

! '   o n c e   f o u n d ,   s t a y   a c c e p t e d

DOES NOT CONTAIN

! '   o n c e   f o u n d ,   t r a p

COUNT

! '   s t a t e s   r e p r e s e n t   c o u n t   c a t e g o r i e s

MODULO

! '   s t a t e s   r e p r e s e n t   r e m a i n d e r s

## 10. 60-SECOND REVISION

---

- DFA construction = identify minimum memory.
  - States are memory situations.
  - Starts-with condition often becomes permanently decided early.
  - Ends-with condition requires continuous suffix tracking.
  - Contains condition normally has permanent accepting state after pattern occurs.
  - Avoid-pattern DFA uses trap state after violation.
  - Exactly  $k$  occurrences usually needs  $k+2$  states.
  - At most  $k$  usually needs  $k+2$ .
  - At least  $k$  usually needs  $k+1$ .
  - Parity = modulo 2.
  - Modulo  $k$  generally uses  $k$  remainder states.
  - Length modulo changes on every symbol.
  - Pattern overlap matters in substring/suffix DFA.
  - Never blindly reset to start state after mismatch.
- 

## 11. ACTIVE RECALL

---

Try these without looking back.

### Q1

What is the golden question when constructing a DFA?

### Q2

For strings containing at least 5 ones, how many standard memory states are needed?

### Q3

For exactly 5 ones?

### Q4

For number of zeros divisible by 4?

### Q5

Why does a DFA for contains 01 stay accepting after finding 01?

### Q6

Why can a DFA for ends with 01 leave its accepting state?

### Q7

What usually happens once a forbidden substring appears in a does not contain DFA?

### Q8

For length divisible by 5, how many states?

### Q9

Which symbols change state in a length-modulo DFA?

### Q10

For even number of 1s, is  $\mu$  accepted?

---

## ANSWERS

---

### A1

Ask:

What minimum information about the prefix read so far must be remembered?

### A2

$\lfloor 5+1=\boxed{6} \rfloor$

### A3

$\lfloor 5+2=\boxed{7} \rfloor$

### A4

$\lfloor \boxed{4} \rfloor$

remainder states.

### A5

Because once the substring has occurred, later symbols cannot undo its occurrence.

### A6

Because later symbols can change the ending.

### A7

Enter a rejecting trap state.

### A8

$\lfloor \boxed{5} \rfloor$

### A9

Every symbol, because every symbol increases string length.

### A10

Yes. Zero 1s is even.

---

## 12. MINI GATE TEST

---

Try first.

### Q1 — MCQ

---

Minimum standard DFA states required to recognize strings containing at least 3 occurrences of 1:

A. 3 B. 4 C. 5 D. 6

---

---

## Q2 — NAT

Minimum standard count-memory states for exactly 7 zeros:

---

## Q3 — MCQ

A DFA accepts strings with number of `a`s congruent to 3 modulo 6.

How many remainder states are required?

A. 3 B. 4 C. 6 D. 7

---

## Q4 — MSQ

Which statements are correct?

A. A contains-pattern DFA may have an absorbing final state. B. An ends-with-pattern DFA must always stay final after first reaching a final state. C. An avoid-pattern DFA may use a trap state. D. Modulo-5 counting naturally suggests five remainder states.

---

## Q5 — NAT

A language contains binary strings whose length is divisible by 7.

How many remainder states are needed?

---

## Q6 — TRICKY

Consider the language of strings containing substring `101`.

The DFA has reached a state representing that the current useful suffix is `1`.

It reads another `1`.

Should it necessarily return to the start state?

A. Yes B. No

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

At least 3:

TEXT

0

1

2

"e 3

Therefore:

---



$\boxed{B=4}$

## Q2

Exactly 7:

TEXT

0, 1, 2, 3, 4, 5, 6, 7, >7

Total:

$\boxed{9}$

## Q3

Modulo 6 needs six remainder classes.

$\boxed{C}$

## Q4

Correct:

$\boxed{A, C, D}$

## Q5

$\boxed{7}$

## Q6

$\boxed{B}$

After **11**, the final **1** may begin a new occurrence of **101**, so useful partial matching remains.

## CHAPTER 2 PROGRESS

TEXT

```
[ ' ] D F A m o t i v a t i o n
[ ' ] S t a t e s a n d f i n i t e m e m o r y
[ ' ] F o r m a l D F A d e f i n i t i o n
[ ' ] T r a n s i t i o n f u n c t i o n
[ ' ] E x t e n d e d t r a n s i t i o n
[ ' ] A c c e p t a n c e / r e j e c t i o n
[ ' ] T r a p s t a t e s
[ ' ] B a s i c D F A c o n s t r u c t i o n
[ ' ] S t a r t s - w i t h D F A
[ ' ] E n d s - w i t h D F A
[ ' ] C o n t a i n s - s u b s t r i n g D F A
[ ' ] A v o i d - s u b s t r i n g D F A
[ ' ] E x a c t - c o u n t D F A
[ ' ] A t - l e a s t / a t - m o s t D F A
[ ' ] P a r i t y D F A
[ ' ] C o u n t m o d u l o k
[ ' ] L e n g t h m o d u l o k
```

[ ] Overlap intuition

- [ ] Binary-number divisibility DFA
- [ ] Complement of DFA
- [ ] Product automata
- [ ] Multiple simultaneous conditions
- [ ] State equivalence intuition
- [ ] DFA minimization

## NEXT TOPIC

### 0=100% Binary Number Divisibility Using DFA + Complement + Product Automata

This is especially important because GATE can combine several conditions such as:

[ \text{even number of 0s AND odd number of 1s} ]

or ask for divisibility of a binary number represented by the input string.

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing from the same TOC master structure.

## CHAPTER 2 — DETERMINISTIC FINITE AUTOMATA

### Topic 3: Binary Divisibility DFA, Complement & Product Automata

Priority: 0=100% VERY HIGH

This topic connects three powerful ideas:

1. Remainder-based DFA
2. Complement of a DFA
3. Combining two DFAs using product states

These patterns are extremely useful in GATE questions involving:

- divisibility
- modulo conditions
- AND / OR combinations
- simultaneous parity conditions
- state-count questions

## PART A — BINARY NUMBER DIVISIBILITY USING DFA

## 1. WHY THIS LOOKS DIFFICULT

Suppose input is a binary string:

```
TEXT
1011
```

It represents decimal:

$\lfloor 11 \rfloor$

Now suppose we want a DFA that accepts binary numbers divisible by 3.

At first it may seem that the DFA must remember the entire number.

It does **not**.

It only needs the current remainder modulo 3.

That is the key.

## 2. THE MAIN IDEA

Suppose the binary prefix read so far represents number:

$\lfloor N \rfloor$

Now we read one more bit:

$\lfloor b \in \{0,1\} \rfloor$

The new number is:

$\lfloor \boxed{2N+b} \rfloor$

Why?

Appending a binary digit shifts the old number left by one position.

That multiplies it by 2.

Then we add the new bit.

## Example

Binary:

```
TEXT
10
```

represents:

$\lfloor 2 \rfloor$

Append **1**:

```
TEXT
101
```

New value:

$$\lfloor 2(2)+1=5 \rfloor$$

Correct.

---

### 3. REMAINDER TRANSITION FORMULA

---

If current remainder modulo  $m$  is:

$$\lfloor r \rfloor$$

and next bit is:

$$\lfloor b \rfloor$$

new remainder is:

$$\lfloor \boxed{(2r+b) \bmod m} \rfloor$$

This is one of the most useful formulas in binary-divisibility DFA questions.

---

### MEMORIZE THIS

---

For binary input:

$$\lfloor \boxed{r_{\text{new}} = (2r+b) \bmod m} \rfloor$$

---

### 4. DFA FOR BINARY NUMBERS DIVISIBLE BY 3

---

We need remainder classes:

TEXT

```
q0 = remainder 0 mod 3
q1 = remainder 1 mod 3
q2 = remainder 2 mod 3
```

Since a number is divisible by 3 iff remainder is 0:

$$\lfloor F = \{q_0\} \rfloor$$

---

### START STATE

---

Before reading input, numeric value is 0.

$$\lfloor 0 \bmod 3 = 0 \rfloor$$

Therefore:

$$\lfloor q_0 \rfloor$$

is start state.

It is also accepting.

---

## 5. CALCULATE TRANSITIONS

---

Use:

$$\lfloor r' = (2r + b) \bmod 3 \rfloor$$

---

### From q0

---

Current remainder:

$$\lfloor r = 0 \rfloor$$

#### Input 0

$$\lfloor (2 \cdot 0 + 0) \bmod 3 = 0 \rfloor$$

So:

$$\lfloor q_0 \xrightarrow{0} q_0 \rfloor$$

#### Input 1

$$\lfloor (2 \cdot 0 + 1) \bmod 3 = 1 \rfloor$$

So:

$$\lfloor q_0 \xrightarrow{1} q_1 \rfloor$$

---

### From q1

---

$$\lfloor r = 1 \rfloor$$

#### Input 0

$$\lfloor (2 \cdot 1 + 0) \bmod 3 = 2 \rfloor$$

So:

$$\lfloor q_1 \xrightarrow{0} q_2 \rfloor$$

#### Input 1

$$\lfloor (2 \cdot 1 + 1) \bmod 3 = 3 \bmod 3 = 0 \rfloor$$

So:

$$\lfloor q_1 \xrightarrow{1} q_0 \rfloor$$

---

### From q2

---

$$\lfloor r = 2 \rfloor$$

#### Input 0

$$\lfloor (2 \cdot 2 + 0) \bmod 3 = 4 \bmod 3 = 1 \rfloor$$

So:

$$\lfloor q_2 \xrightarrow{0} q_1 \rfloor$$

---

## Input 1

$\forall (2 \cdot 2 + 1) \bmod 3 = 5 \bmod 3 = 2$

So:

$\forall q_2 \rightarrow q_2$

## FINAL TRANSITION TABLE

| State | Meaning     | 0     | 1     |
|-------|-------------|-------|-------|
| $q_0$ | remainder 0 | $q_0$ | $q_1$ |
| $q_1$ | remainder 1 | $q_0$ | $q_2$ |
| $q_2$ | remainder 2 | $q_1$ | $q_2$ |

## 6. TEST STRING 110

Binary:

$\forall 110_2 = 6$

6 is divisible by 3.

Trace:

TEXT

```
q0 --1--> q1
q1 --1--> q0
q0 --0--> q0
```

Final state:

$\forall q_0$

Accepted.

## 7. TEST STRING 101

Binary:

$\forall 101_2 = 5$

5 is not divisible by 3.

Trace:

TEXT

```
q0 --1--> q1
q1 --0--> q2
q2 --1--> q2
```

Final state:

$q_2$

Rejected.

---

## 8. GENERAL METHOD FOR BINARY DIVISIBILITY BY $m$

---

To recognize binary numbers divisible by:

$m$

create:

$m$

remainder states:

$q_0, q_1, \dots, q_{m-1}$

where:

$q_r = \text{current number has remainder } r$

For input bit  $b$ :

$\Delta(q_r, b) = q_{(2r+b) \bmod m}$

Final state:

$q_0$

---

## MEMORY TRICK

---

**Binary append = Double + Bit**

So:

TEXT

$\text{new} = 2 \times \text{old} + \text{bit}$

Then take modulo.

---

## 9. DECIMAL-DIGIT GENERALIZATION

---

This is useful conceptually.

If digits are in base  $B$ :

$N_{\text{new}} = BN + d$

Therefore:

$r_{\text{new}} = (Br + d) \bmod m$

For binary:

$B=2$

So:

$$\lfloor (2r+b) \bmod m \rfloor$$

## 10. SPECIAL NOTE ABOUT LEADING ZEROS

Suppose:

TEXT

0011

and:

TEXT

11

Both represent decimal 3.

The remainder DFA naturally treats leading zeros correctly.

For  $q_0$ :

$$\lfloor q_0 \xrightarrow{0} q_0 \rfloor$$

So leading zeros don't alter divisibility.

## & IMPORTANT EMPTY-STRING ISSUE

If start state  $q_0$  is accepting, then:

$$\lfloor \epsilon \rfloor$$

is accepted.

Mathematically, this corresponds to treating the empty prefix as numeric value 0.

But a question may define "binary representations" to require at least one bit.

If so,  $\mu$  may need to be excluded with an additional start condition/state.

**GATE rule:**

**Read the language definition carefully.**

Do not automatically assume  $\mu$  is intended as a valid binary numeral.

## PART B — COMPLEMENT OF A DFA

### 11. INTUITION

Suppose DFA  $M$  accepts:

**strings ending in 1.**

Then its complement should accept:



strings that do not end in 1.

For a **complete DFA**, this is extremely easy.

Just swap:

TEXT

f i n a l ! "   n o n - f i n a l

## 12. FORMAL IDEA

Suppose:

$M = (Q, \Sigma, \delta, q_0, F)$

recognizes:

$L$

Then a DFA for:

$\overline{L}$

is:

$M' = (Q, \Sigma, \delta, q_0, Q - F)$

Only the accepting set changes.

## MEMORIZE

$\boxed{F' = Q - F}$

## 13. EXAMPLE

Original DFA:

Binary strings ending in 1.

| State           | 0       | 1           |
|-----------------|---------|-------------|
| $! \ q \in$     | $q \in$ | $q \bullet$ |
| $* \ q \bullet$ | $q \in$ | $q \bullet$ |

Here:

$F = \{q_1\}$

Complement accepting set:

$F' = \{q_0\}$

So complement recognizes:

**binary strings not ending in 1.**

That includes:

TEXT

$\mu$   
0  
00  
10  
110

## 14. WHY DOES SWAPPING WORK?

A DFA has exactly one computation path for every string.

Every input ends in exactly one state.

Therefore each string is either:

- in a final state
- in a non-final state

Swapping these decisions reverses acceptance for every input.

## & **CRITICAL CONDITION**

This simple complement method assumes the DFA is **complete**.

Every state must have exactly one transition for every input symbol.

## 15. WHY INCOMPLETE AUTOMATA CAUSE A PROBLEM

Suppose a diagram omits a transition.

That missing transition may implicitly mean:

**go to a rejecting trap state.**

If you simply swap shown final states but forget the trap state, the complement may be wrong.

So:

### Safe procedure

1. Complete the DFA.
2. Add trap state if needed.
3. Then swap final and non-final states.

## GATE SHORTCUT

Complement DFA = complete first, flip finals second.

## 16. COMPLEMENT AND REGULAR LANGUAGES

Since every regular language has a DFA, and complement can be obtained by flipping final states:

$\boxed{\text{Regular languages are closed under complement}}$

We will revisit this formally in closure properties.

## PART C — PRODUCT AUTOMATA

### 17. WHY PRODUCT AUTOMATA?

Suppose we want strings satisfying two conditions simultaneously:

#### Condition A

Number of 0s is even.

#### Condition B

Number of 1s is odd.

One DFA can track A.

Another DFA can track B.

How do we make one DFA track both?

Use a **product automaton**.

### 18. INTUITION

Suppose DFA A has states:

TEXT

E0 = even number of 0s

O0 = odd number of 0s

DFA B has:

TEXT

E1 = even number of 1s

O1 = odd number of 1s

To know both conditions at the same time, the combined machine must know a pair:

TEXT

(E0, E1)

(E0, O1)

(O0, E1)

(00, 01)

Each pair becomes one state.

## FORMAL STATES SET

If DFA 1 has state set:

$\{Q_1\}$

and DFA 2 has:

$\{Q_2\}$

then product DFA states are:

$\{Q_1 \times Q_2\}$

## MAXIMUM NUMBER OF STATES

If:

$|Q_1|=m$

and:

$|Q_2|=n$

then product construction gives at most:

$\{mn\}$

states.

Some may be unreachable and later removed.

## 19. EXAMPLE — EVEN 0s AND ODD 1s

We need:

TEXT

```
qEE = even 0s, even 1s
qEO = even 0s, odd 1s
qOE = odd 0s, even 1s
qOO = odd 0s, odd 1s
```

## START STATE

Initially:

TEXT

```
# 0 = 0 !' e v e n
# 1 = 0 !' e v e n
```

Therefore start:

$\lceil q_{\{EE\}} \rceil$

## ACCEPTING STATE

We want:

TEXT

even number of 0s  
AND  
odd number of 1s

So accepting state:

$\lceil q_{\{EO\}} \rceil$

## 20. TRANSITIONS

Reading 0 changes parity of zero count but not one count.

So:

TEXT

EE --0--> 0E  
EO --0--> 00  
OE --0--> EE  
OO --0--> E0

Reading 1 changes one parity but not zero parity:

TEXT

EE --1--> E0  
EO --1--> EE  
OE --1--> 00  
OO --1--> 0E

## FINAL TABLE

| State   | 0   | 1   |
|---------|-----|-----|
| ! q E E | qOE | qEO |
| *qEO    | qOO | qEE |
| qOE     | qEE | qOO |
| qOO     | qEO | qOE |

## TEST STRING 101

Count:

TEXT

```
0 s = 1 !' o d d
1 s = 2 !' e v e n
```

So should reject.

Trace:

TEXT

```
EE --1--> E0
E0 --0--> 00
00 --1--> 0E
```

Final:

TEXT

0E

Not accepting.

Rejected.

## TEST STRING 001

Count:

TEXT

```
0 s = 2 !' e v e n
1 s = 1 !' o d d
```

Should accept.

Trace:

TEXT

```
EE --0--> 0E
0E --0--> EE
EE --1--> E0
```

Final:

$\backslash [q_{\{EO\}}]$

Accepted.

## PART D — AND / OR / DIFFERENCE USING PRODUCT DFA

Suppose:

$\backslash [L_1]$

recognized by DFA 1 and:

$\setminus [L_2 \setminus]$

recognized by DFA 2.

The transitions of the product machine are the same.

Only the final-state choice changes depending on the operation.

## 21. INTERSECTION — AND

We want:

$\setminus [L_1 \cap L_2 \setminus]$

A pair state is final only when:

TEXT

first component is final

AND

second component is final

Formally:

$\setminus [\boxed{F=F_1 \times F_2} \setminus]$

## MEMORY

**Intersection = both final**

## 22. UNION — OR

For:

$\setminus [L_1 \cup L_2 \setminus]$

pair is accepting when:

TEXT

first final

OR

second final

OR both

## MEMORY

**Union = at least one final**

## 23. DIFFERENCE

For:

$\setminus [L_1 - L_2 \setminus]$

accept when:

TEXT

L1 accepts  
AND  
L2 rejects

So:

$\setminus [\boxed{F = F_1 \times (Q_2 - F_2)} \setminus]$

---

## 24. SYMMETRIC DIFFERENCE

Sometimes conceptually useful.

Accept when exactly one DFA accepts.

TEXT

Accept/Reject  
or  
Reject/Accept

Not both.

---

## 25. COMPLEMENT THROUGH PRODUCT LOGIC

By De Morgan:

$\setminus [\overline{L_1 \cup L_2} = \overline{L_1} \cap \overline{L_2} \setminus]$

and:

$\setminus [\overline{L_1 \cap L_2} = \overline{L_1} \cup \overline{L_2} \setminus]$

These become useful in closure-property questions.

---

## PART E — PRODUCT AUTOMATON FORMAL DEFINITION

Suppose:

$\setminus [M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1) \setminus]$

and:

$\setminus [M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2) \setminus]$

Combined machine:

$\setminus [M = (Q_1 \times Q_2, \Sigma, \delta, (q_{01}, q_{02}), F) \setminus]$

Transition:

$\setminus [\boxed{\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))} \setminus]$



In simple language:

**Feed the same input symbol to both machines simultaneously.**

## 26. STATE-COUNT EXAMPLE

Suppose:

- DFA A has 3 states
- DFA B has 4 states

Product construction has at most:

$3 \times 4 = 12$

states.

But if some state pairs are unreachable, actual DFA can contain fewer.

And after minimization, it may reduce further.

## & P GATE TRAP

Product states = exactly  $mn$  minimum states.

Wrong.

Correct:

**Product construction gives at most  $mn$  reachable candidates before reachability/minimization analysis.**

## PART F — MULTIPLE MODULO CONDITIONS

### Example

Find a DFA for strings where:

TEXT

number of 0s is even

AND

number of 1s is divisible by 3

First condition requires:

$2$

states.

Second requires:

$3$

states.

Product:

$$\lfloor 2 \times 3 = 6 \rfloor$$

candidate states.

They represent:

TEXT

(E, 0)

(E, 1)

(E, 2)

(0, 0)

(0, 1)

(0, 2)

Accepting:

TEXT

(E, 0)

## GATE SHORTCUT

Independent modulo conditions often multiply.

Example:

TEXT

#0 mod 2

and

#1 mod 3

naturally suggests:

$$\lfloor 2 \times 3 = 6 \rfloor$$

memory combinations.

## 27. LENGTH DIVISIBLE BY 2 AND 3

Suppose condition is:

$$\lfloor |w| \text{ divisible by 2 AND 3} \rfloor$$

One brute-force product might suggest:

$$\lfloor 2 \times 3 = 6 \rfloor$$

states.

But observe:

A number divisible by both 2 and 3 is divisible by:

$$\lfloor \operatorname{lcm}(2, 3) = 6 \rfloor$$

Therefore a direct remainder-mod-6 DFA also uses 6 states.

## Example where simplification matters

Suppose length divisible by 2 AND 4.

Product:

$$\sqrt[2]{2 \times 4 = 8}$$

would be unnecessary.

Because divisibility by 4 already implies divisibility by 2.

So condition reduces to:

$$\sqrt[w]{w \equiv 0 \pmod{4}}$$

Only 4 states required.

## IMPORTANT GATE LESSON

Before blindly multiplying state counts:

**Simplify the language condition first.**

## PART G — OR CONDITION EXAMPLE

Suppose language:

**Number of 0s is even OR number of 1s is odd.**

Use same four parity-pair states:

TEXT

EE  
EO  
OE  
OO

Now decide finals.

Condition:

TEXT

zero parity even  
OR  
one parity odd

Evaluate:

**EE**

Zero even !' accept.

**EO**

Zero even and one odd !' accept.

## OE

Zero odd and one even !' reject.

## OO

One odd !' accept.

Therefore:

$\mathcal{L} = \{EE, EO, OO\}$

Only OE rejects.

## PART H — COMPLEMENT EXAMPLE USING PARITY

Language:

**even number of 1s.**

DFA:

TEXT

qE = final  
qO = non-final

Complement:

**odd number of 1s.**

Just swap:

TEXT

qE = non-final  
qO = final

Transitions unchanged.

This is why complement questions can sometimes be solved instantly.

## PART I — SOLVED GATE-STYLE QUESTIONS

### Q1 — Binary divisibility

A DFA recognizes binary numbers divisible by 5.

How many remainder states does the standard construction need?

Possible remainders:

TEXT

0, 1, 2, 3, 4

Therefore:

$\boxed{5}$

---

## Q2 — Transition calculation

---

For divisibility by 5 DFA, current remainder is 3.

Read bit **1**.

New remainder:

$$(2 \cdot 3 + 1) \bmod 5$$

$$= 7 \bmod 5$$

$$= \boxed{2}$$

So:

$$q_3 \xrightarrow{1} q_2$$

---

## Q3 — Product states

---

DFA A has 4 states.

DFA B has 5 states.

Maximum product-state count:

$$4 \times 5 = \boxed{20}$$

---

## Q4 — Combined parity

---

How many natural states are needed to track:

- parity of zeros
- parity of ones

Each has 2 possibilities.

Therefore:

$$2 \times 2 = \boxed{4}$$

---

## Q5 — Combined modulo

---

Track:

$$\#0 \bmod 3$$

and:

$$\#1 \bmod 4$$

Possible combinations:

$$3 \times 4 = \boxed{12}$$

candidate states.

---

---

## Q6 — Complement

---

A complete DFA has:

$$Q = \{q_0, q_1, q_2, q_3\}$$

and:

$$F = \{q_1, q_3\}$$

Complement finals:

$$Q - F = \{q_0, q_2\}$$

Therefore:

$$\boxed{\{q_0, q_2\}}$$

---

## 28. GATE/PYQ PATTERN ANALYSIS

---

Typical questions include:

### Pattern 1 — Remainder transition

Given current remainder and next bit, find next state.

Use:

$$(2r + b) \bmod m$$

---

### Pattern 2 — Draw divisibility DFA

Create remainder states.

---

### Pattern 3 — Complement

Given DFA for  $L$ , identify DFA for:

$$\overline{L}$$

Usually flip finals after ensuring completeness.

---

### Pattern 4 — Product-state count

If one property needs  $m$  memory states and another needs  $n$ , product construction may give:

$$mn$$

states.

---

### Pattern 5 — AND/OR accepting pairs

Given two DFA final sets, determine final state pairs for intersection or union.

---

### Pattern 6 — Multiple parity/modulo constraints

---

Examples:

TEXT

```
#0 even AND #1 odd  
#0 mod 3 = 1 AND #1 mod 2 = 0
```

Use combined remainder states.

## 29. & pGATE TRAPS

### Trap 1 — Binary append formula is $N+b$

Wrong.

Appending a bit means:

$\lfloor 2N+b \rfloor$

### Trap 2 — Divisibility-by- $m$ DFA needs to remember whole number

Wrong.

Only remainder modulo  $m$  is necessary.

### Trap 3 — Complement means reverse transitions

Wrong.

Transitions remain exactly the same.

Only final/non-final states swap.

### Trap 4 — Complement can be performed safely on an incomplete DFA diagram

Potentially wrong.

Complete the DFA first.

### Trap 5 — Product DFA always has exactly $mn$ reachable states

Wrong.

$mn$  is an upper bound for pairs.

Some may be unreachable.

### Trap 6 — Intersection accepts if either component is final

Wrong.

Intersection:

both final.

### Trap 7 — Union accepts only if both are final

Wrong.

Union:

at least one final.

### Trap 8 — For binary divisibility by $m$ , transition is $(r+b) \bmod m$

Wrong.

Correct:

$\lfloor (2r+b) \bmod m \rfloor$

### Trap 9 — Start remainder should be 1

Wrong.

Before input:

$\lfloor N=0 \rfloor$

so remainder is 0.

### Trap 10 — Multiplying state counts always gives minimum DFA

Wrong.

Product construction and minimum-state proof are different things.

## 30. MASTER COMPARISON TABLE

| Concept                       | Core memory                          |
|-------------------------------|--------------------------------------|
| Count modulo $m$              | count remainder                      |
| Binary value divisible by $m$ | numeric remainder                    |
| Complement                    | same states/transitions, flip finals |
| Intersection                  | product states, both final           |
| Union                         | product states, at least one final   |
| Difference $L1-L2$            | first final, second non-final        |



---

## 31. IMPORTANT FORMULAS

---

### Binary append

$$\boxed{N' = 2N + b}$$

### Remainder transition

$$\boxed{r' = (2r + b) \bmod m}$$

### Product state count

$$\boxed{|Q_1 \times Q_2| = |Q_1| |Q_2|}$$

before reachability/minimization reductions.

### Complement finals

$$\boxed{F' = Q - F}$$

### Intersection final set

$$\boxed{F_1 \times F_2}$$

---

## 32. SHORTCUTS & MEMORY TRICKS

---

### Binary divisibility

Double + Bit + Mod

TEXT

$2r + \text{bit}$   
then modulo  $m$

### Complement

Keep machine, flip finish

Transitions unchanged.

Finals flipped.

---

### Product

Pair the memories

If:

---

TEXT

Machine A remembers X  
Machine B remembers Y

combined machine remembers:

TEXT

(X, Y)

## AND

Both final

## OR

One or both final

## 33. 'p WRITE THIS IN YOUR NOTES

### Binary Divisibility DFA

For binary prefix value  $N$ , after reading bit  $b$ :

$$\lfloor N' = 2N + b \rfloor$$

For modulo  $m$ :

$$\lfloor \boxed{r' = (2r + b) \bmod m} \rfloor$$

Use states:

$$\lfloor q_0, q_1, \dots, q_{m-1} \rfloor$$

where  $qr$  = current remainder  $r$ .

For divisibility by  $m$ :

$$\lfloor F = \{q_0\} \rfloor$$

### Complement DFA

For complete:

$$\lfloor M = (Q, \Sigma, \delta, q_0, F) \rfloor$$

complement:

$$\lfloor M' = (Q, \Sigma, \delta, q_0, Q - F) \rfloor$$

### Rule

TEXT

Complete DFA  
!"  
Swap final/non-final

## Product DFA

If:

$$\forall |Q_1|=m, \text{quad} |Q_2|=n \setminus$$

candidate pairs:

$$\forall m \times n \setminus$$

Transition:

$$\forall \delta((p,q),a) = (\delta_1(p,a), \delta_2(q,a)) \setminus$$

## Intersection

TEXT

final AND final

## Union

TEXT

final OR final

## 34. 60-SECOND REVISION

- Binary append means  $2N + b$ .
- Binary remainder transition is  $(2r+b) \bmod m$ .
- Divisibility-by-m naturally uses m remainder states.
- $q_0$  represents remainder zero.
- Complement does not change transitions.
- Complement flips final/non-final states.
- Complete DFA before complementing.
- Product states are ordered pairs.
- $m \times n$  is the product-construction upper bound.
- Intersection requires both components final.
- Union requires at least one component final.
- Difference means first accepts and second rejects.
- Simplify conditions before multiplying state counts.

## 35. ACTIVE RECALL

**Q1**

What is the binary append formula?

**Q2**

What is the remainder transition formula modulo  $m$  ?

**Q3**

How many states does standard divisibility-by-7 DFA use?

**Q4**

What is the start remainder?

**Q5**

For complement, do transitions change?

**Q6**

Why should a DFA be completed before taking complement?

**Q7**

If DFA A has 3 states and DFA B has 5, what is the product upper bound?

**Q8**

For intersection, when is pair  $(p, q)$  accepting?

**Q9**

For union?

**Q10**

If current remainder mod 4 is 3 and input bit is 1, what is new remainder?

---

**ANSWERS**

---

**A1**

$\lfloor 2N+b \rfloor$

**A2**

$\lfloor (2r+b) \bmod m \rfloor$

**A3**

$\lfloor 7 \rfloor$

**A4**

$\lfloor 0 \rfloor$

**A5**

No.

**A6**

Because omitted transitions may represent an implicit rejecting trap that must also become accepting in the complement.

**A7**

$\lceil 3 \times 5 = 15 \rceil$

**A8**

When both  $p$  and  $q$  are final in their original DFAs.

**A9**

When at least one is final.

**A10**

$\lceil (2 \cdot 3 + 1) \bmod 4 = 7 \bmod 4 = \boxed{3} \rceil$

---

## 36. MINI GATE TEST

---

### Q1 — NAT

---

A DFA tracks divisibility of binary numbers by 6.

How many natural remainder states are required?

---

### Q2 — NAT

---

Current remainder modulo 7 is 5.

Next binary input is 1.

Find the new remainder.

---

### Q3 — MCQ

---

For a complete DFA recognizing  $L$ , which operation constructs a DFA for complement of  $L$ ?

A. Reverse every transition B. Swap start and final states C. Swap final and non-final states D. Delete final states

---

### Q4 — NAT

---

Two DFAs have 4 and 6 states.

How many product-state pairs are possible before removing unreachable states?

---

### Q5 — MSQ

---

For a product DFA recognizing:

$\lceil L_1 \cap L_2 \rceil$

which pair states are accepting?

A. First final only B. Second final only C. Both final D. Neither final

---

## Q6 — TRICKY

---

A language requires:

$\{$

w

$\}$

to be divisible by both 2 and 4.

Do we necessarily need:

$\{ 2 \times 4 = 8 \}$

minimum states?

A. Yes B. No

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

$\boxed{6}$

---

### Q2

$(2 \cdot 5 + 1) \bmod 7 = 11 \bmod 7 = \boxed{4}$

---

### Q3

$\boxed{C}$

---

### Q4

$4 \times 6 = \boxed{24}$

---

### Q5

$\boxed{C}$

---

### Q6

$\boxed{B}$

Divisible by 4 already implies divisible by 2.

So the condition reduces to divisibility by 4.

---

## CHAPTER 2 PROGRESS

### TEXT

```
[ ' ] D F A   f u n d a m e n t a l s
[ ' ] D F A   c o n s t r u c t i o n   p a t t e r n s
[ ' ] P r e f i x   /   s u f f i x   p a t t e r n s
[ ' ] C o n t a i n s   /   a v o i d s   p a t t e r n s
[ ' ] E x a c t   /   a t   l e a s t   /   a t   m o s t
[ ' ] P a r i t y   a n d   m o d u l o
[ ' ] B i n a r y   d i v i s i b i l i t y   D F A
[ ' ] C o m p l e m e n t
[ ' ] P r o d u c t   a u t o m a t a
[ ' ] A N D   /   O R   /   D i f f e r e n c e
[ ' ] M u l t i p l e   s i m u l t a n e o u s   c o n d i t i o n s
```

```
[ ] R e a c h a b l e   /   u n r e a c h a b l e   s t a t e s
[ ] E q u i v a l e n t   /   d i s t i n g u i s h a b l e   s t a t e s
[ ] D F A   m i n i m i z a t i o n
[ ] T a b l e - f i l l i n g   m e t h o d
[ ] P a r t i t i o n   m e t h o d
[ ] M i n i m u m - s t a t e   r e a s o n i n g
```

## NEXT TOPIC

### 0=~~1~~ DFA MINIMIZATION — Equivalent States, Distinguishable States, Reachability, Table-Filling and Partition Method

This is the next major high-priority section because it teaches how to answer:

What is the minimum number of states required?

Type NEXT.

USER

NEXT

ASSISTANT

Continuing exactly from the TOC roadmap in your uploaded master prompt.

## CHAPTER 2 — DFA MINIMIZATION

### Topic 4: Reachable States, Equivalent States, Distinguishable States, Table-Filling & Partition Method

Priority: 0=~~1~~ VERY HIGH

This topic answers one of the most common GATE questions:

What is the minimum number of states required for this DFA/language?

The most important idea is:

**Two states can be merged only if no future input can make them behave differently with respect to acceptance.**

## 1. WHY DFA MINIMIZATION MATTERS

Two different DFAs can recognize the same language.

Example:

TEXT

```
D F A   1   !'   6   s t a t e s
D F A   2   !'   4   s t a t e s
```

Both may recognize exactly the same language.

But the second one may be the **minimal DFA**.

A minimal DFA:

- recognizes the same language
- has no unnecessary states
- has no mergeable equivalent states

## 2. BIG PICTURE

Minimization usually has two major steps:

TEXT

```
Original DFA
! "
Remove unreachable states
! "
Find equivalent states
! "
Merge equivalent states
! "
Minimal DFA
```

Memory:

**Remove first, Merge second**

## 3. WHAT IS A REACHABLE STATE?

A state  $q$  is reachable if there exists some string:

$\forall w \in \Sigma^*$

such that:

$\forall \delta^*(q_0, w) = q$



In simple language:

**Starting from  $q_0$  can some input take us to  $q_3$ ?**

If yes !' reachable.

If no !' unreachable.

## 4. EXAMPLE

Suppose:

TEXT

```
! ' q_0 - - 0 - - > q_1
  |
  1
  v
  q_2
```

$q_3$  exists separately

and no transition from  $q_0$ ,  $q_1$ , or  $q_2$  ever reaches  $q_3$ .

Then:

$\neg [q_3]$

is unreachable.

It can be deleted without changing the language.

## GATE RULE

**Always remove unreachable states before minimization.**

An unreachable state cannot affect any computation from the start state.

## 5. HOW TO FIND REACHABLE STATES

Use this simple algorithm.

### Step 1

Mark start state.

### Step 2

From every marked state, follow every possible transition.

### Step 3

Mark newly reached states.

Step 4

Repeat until no new states appear.  
Unmarked states are unreachable.

Example

Suppose transition table:

| State | 0 | 1 |
|-------|---|---|
| ! A   | B | C |
| B     | B | D |
| C     | C | A |
| *D    | D | D |
| E     | E | E |

Start at A.  
From A:

TEXT

0 !' B  
1 !' C

Mark:

TEXT

A, B, C

From B:

TEXT

0 !' B  
1 !' D

Mark D.

From C:

TEXT

0 !' C  
1 !' A

Nothing new.  
From D:

TEXT

|   |   |   |   |
|---|---|---|---|
| 0 | ! | ' | D |
| 1 | ! | ' | D |

Nothing new.

State E never reached.

Therefore:

$\boxed{E \text{ is unreachable}}$

Delete E.

## 6. WHAT ARE EQUIVALENT STATES?

This is the heart of minimization.

Two states:

$\{p, q\}$

are equivalent if **every possible future string** causes both states to agree on acceptance.

Formally:

$\boxed{p \equiv q \text{ iff } \forall w \in \Sigma^*, \delta^*(p, w) \in F \Leftrightarrow \delta^*(q, w) \in F}$

## INTUITION

Suppose we are currently in state p or q.

If no possible continuation can ever reveal which one we were in, then keeping them separate is unnecessary.

So we merge them.

## MEMORY TRICK

Equivalent = Future cannot tell them apart

## 7. DISTINGUISHABLE STATES

States p and q are distinguishable if there exists **at least one string** w such that:

$\delta^*(p, w)$

is accepting and:

$\delta^*(q, w)$

is rejecting,

or vice versa.

Formally:

$\neg \exists w \in \Sigma^* \text{ such that acceptance differs}$

That  $w$  is called a **distinguishing string**.

## MEMORY

**One witness is enough to prove distinguishability.**

To prove equivalent:

**must agree for every future string.**

To prove distinguishable:

**one counterexample string is enough.**

## 8. FIRST EASY DISTINGUISHABILITY RULE

If:

TEXT

$p = \text{final}$   
 $q = \text{non-final}$

then  $p$  and  $q$  are immediately distinguishable using:

$w = \epsilon$

Why?

Without reading anything:

- $p$  already accepts
- $q$  rejects

Therefore:

$\neg \text{final state and non-final state can never be equivalent}$

This is the foundation of minimization methods.

## 9. EXAMPLE OF EQUIVALENT STATES

Suppose:

| State | 0 | 1 |
|-------|---|---|
| *A    | A | B |
| *B    | A | B |

Both A and B are final.

Their transitions behave identically:

TEXT

```
o n 0 !' A
o n 1 !' B
```

From either state, every future string finishes in the same acceptance category.

Therefore:

$\lceil A \equiv B \rceil$

They can be merged.

## 10. EXAMPLE OF DISTINGUISHABLE STATES

Suppose:

| State | 0 | 1 |
|-------|---|---|
| A     | C | A |
| B     | D | B |
| *C    | C | C |
| D     | D | D |

A and B are both initially non-final.

Can they be merged?

Check input:

TEXT

0

From A:

$\lceil A \xrightarrow{0} C \rceil$

C is final.

From B:

$\lceil B \xrightarrow{0} D \rceil$

D is non-final.

So 0 distinguishes A and B.

Thus:

$\lceil \boxed{A \not\equiv B} \rceil$

## 11. WHAT DOES MINIMAL DFA MEAN?

A DFA is minimal if:

1. every state is reachable
2. no two distinct states are equivalent

That is the core criterion.

## MUST REMEMBER

**Minimal DFA = Reachable + Pairwise distinguishable**

## 12. VERY IMPORTANT RESULT

For every regular language:

**the minimal DFA is unique up to renaming of states.**

Example:

TEXT

q0, q1, q2

and:

TEXT

A, B, C

can be structurally identical minimal DFAs.

State names do not matter.

Behavior does.

## 13. TABLE-FILLING METHOD

Now the standard exam method.

This method systematically finds distinguishable state pairs.

Suppose states are:

TEXT

A, B, C, D

We examine every unordered pair:

TEXT

(A, B)

(A, C)

(A, D)

(B, C)

(B, D)

(C, D)

## 14. TABLE-FILLING CORE IDEA

Initially:

**Mark every pair where one state is final and the other is non-final.**

Then repeatedly:

**If some input sends a pair  $(p, q)$  to an already marked pair, mark  $(p, q)$  too.**

At the end:

- marked pairs = distinguishable
- unmarked pairs = equivalent

## 15. WHY THIS WORKS

Suppose:

$\forall \delta(p, a) = r$

and:

$\forall \delta(q, a) = s$

If  $r$  and  $s$  are already distinguishable, then after reading  $a$ ,  $p$  and  $q$  reach distinguishable states.

Therefore  $p$  and  $q$  are also distinguishable.

So distinction propagates backward.

## MEMORY

**Different future !' different current states**

## 16. COMPLETE TABLE-FILLING ALGORITHM

### Step 0

Remove unreachable states.

### Step 1

List every unordered pair of states.

## Step 2

Mark every:

TEXT  
(final, non-final)

pair.

## Step 3

For every unmarked pair  $(p, q)$ , for each symbol  $a$ , calculate:

$\delta(p, a), \delta(q, a)$

If that destination pair is marked, mark  $(p, q)$ .

## Step 4

Repeat until no new markings appear.

## Step 5

Unmarked pairs are equivalent.

## Step 6

Merge equivalent states.

## 17. SOLVED EXAMPLE — TABLE FILLING

Consider:

| State | 0 | 1 |
|-------|---|---|
| ! A   | B | C |
| B     | A | D |
| C     | D | A |
| *D    | D | D |

Only D is final.

All states reachable.

We test pairs:

TEXT  
(A, B)  
(A, C)  
(A, D)  
(B, C)  
(B, D)



(C, D)

## STEP 1 — MARK FINAL/NON-FINAL PAIRS

D is final.

A, B, C non-final.

So immediately mark:

TEXT

(A, D) X  
(B, D) X  
(C, D) X

Remaining unmarked:

TEXT

(A, B)  
(A, C)  
(B, C)

## STEP 2 — CHECK (A,B)

On input  $\emptyset$  :

$\lceil A \rightarrow B \rceil$

$\lceil B \rightarrow A \rceil$

Destination pair:

$\lceil (A, B) \rceil$

itself.

No information.

On input  $1$  :

$\lceil A \rightarrow C \rceil$

$\lceil B \rightarrow D \rceil$

Destination:

$\lceil (C, D) \rceil$

Already marked.

Therefore:

$\lceil \boxed{(A, B)} \text{ marked} \rceil$

## STEP 3 — CHECK (A,C)

On input  $\emptyset$  :

$\lceil A \rceil \rightarrow B \lfloor$

$\lceil C \rceil \rightarrow D \lfloor$

Destination:

$\lceil (B,D) \rfloor$

marked.

Therefore:

$\lceil \boxed{(A,C)} \text{ marked} \rfloor$

### STEP 4 — CHECK (B,C)

On input  $\emptyset$  :

$\lceil B \rceil \rightarrow A \lfloor$

$\lceil C \rceil \rightarrow D \lfloor$

Destination:

$\lceil (A,D) \rfloor$

marked.

Therefore:

$\lceil \boxed{(B,C)} \text{ marked} \rfloor$

### FINAL RESULT

All state pairs are marked.

Therefore:

**No states are equivalent.**

The DFA is already minimal.

Minimum states:

$\lceil \boxed{4} \rfloor$

### 18. EXAMPLE WHERE STATESMERGE

Consider:

| State              | 0 | 1 |
|--------------------|---|---|
| $\lceil A \rfloor$ | B | C |
| B                  | B | D |

| State | 0 | 1 |
|-------|---|---|
| C     | B | D |
| *D    | D | D |

Only D is final.

All states reachable.

Pairs:

TEXT

(A, B)  
(A, C)  
(A, D)  
(B, C)  
(B, D)  
(C, D)

## INITIAL MARKS

Final/non-final:

TEXT

(A, D) X  
(B, D) X  
(C, D) X

Now investigate:

TEXT

(A, B)  
(A, C)  
(B, C)

## Pair (B,C)

On 0:

TEXT

B !' B  
C !' B

destination:

$\setminus [(B,B) \setminus]$

same state.

On 1:

TEXT

|   |   |   |   |
|---|---|---|---|
| B | ! | ' | D |
| C | ! | ' | D |

same state.

No distinction exists.

So:

$\boxed{B \equiv C}$

They remain unmarked.

Thus B and C can be merged.

## 19. MERGED DFA

Replace B and C by:

TEXT

BC

States:

TEXT

A  
BC  
D

Then rebuild transitions.

This gives smaller DFA.

## 20. TABLE-FILLING SHORTCUT

If two states have:

- same final/non-final status
- identical transitions to the same equivalence groups

they will be equivalent.

But be careful:

**identical immediate destinations are sufficient, not necessary.**

Two states can still be equivalent even if they go to different state names, provided those destination states themselves are equivalent.

## PART B — PARTITION METHOD

## 21. WHY PARTITION METHOD?

Instead of comparing every pair directly, we group states by behavior.

Start with:

TEXT

Final states  
Non-final states

Then repeatedly split groups whenever states behave differently.

Eventually no further split is possible.

Each final group becomes one state of minimal DFA.

## 22. INITIAL PARTITION

Suppose:

$Q = \{A, B, C, D, E\}$

and:

$F = \{D, E\}$

Initial partition:

$P_0 = \{\{D, E\}, \{A, B, C\}\}$

Why?

Final and non-final states can never be equivalent.

## 23. SPLITTING A GROUP

Take states A, B, C.

Check where each goes under each symbol in terms of **current groups**.

Suppose:

TEXT

```
A :  0 !'  n o n f i n a l   g r o u p ,   1 !'   f i n a l   g r o u p
B :  0 !'  n o n f i n a l   g r o u p ,   1 !'   f i n a l   g r o u p
C :  0 !'   f i n a l   g r o u p ,           1 !'   f i n a l   g r o u p
```

Then C behaves differently from A and B.

Split:

TEXT

{A, B, C}

into:

TEXT

{A, B}  
{C}

## 24. KEEP REFINING

Now check groups again using the new partition.

If A and B still have identical group-transition signatures, keep together.

If not, split them.

Continue until stable.

## 25. PARTITION SIGNATURE

For each state, write:

TEXT

(destination group on 0,  
destination group on 1,  
...)

States in the same block must have identical signatures.

## 26. SOLVED PARTITION EXAMPLE

Consider:

| State | 0 | 1 |
|-------|---|---|
| ! A   | B | C |
| B     | B | D |
| C     | B | D |
| *D    | D | D |

Only D final.

Initial partition:

$\mathcal{P}_0 = \{ \{D\}, \{A, B, C\} \}$

Call:

TEXT

G1 = {D}  
G2 = {A, B, C}

## STATE SIGNATURES

### A

0 !' B !' G2 1 !' C !' G2

Signature:

TEXT

(G2, G2)

### B

0 !' B !' G2 1 !' D !' G1

Signature:

TEXT

(G2, G1)

### C

0 !' B !' G2 1 !' D !' G1

Signature:

TEXT

(G2, G1)

So A differs.

Split:

$\backslash P\_1 = \backslash \{ \{ D \}, \{ A \}, \{ B, C \} \} \backslash$

## CHECK B AND C

Both:

TEXT

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | ! | ' | B | ! | ' | { | B | , | C | } |
| 1 | ! | ' | D | ! | ' | { | D | } |   |   |

same behavior.

No further split.

Final partition:

$\backslash \boxed{ \{ A \}, \{ B, C \}, \{ D \} } \backslash$

Minimum DFA has:

$\backslash \boxed{3} \backslash$

states.

## 27. TABLE FILLING VS PARTITION

| Feature      | Table Filling         | Partition                |
|--------------|-----------------------|--------------------------|
| Main object  | State pairs           | State groups             |
| Starts with  | Final/non-final pairs | Final/non-final blocks   |
| Finds        | Distinguishable pairs | Equivalent classes       |
| Good for     | Small/medium DFA      | Structured DFA           |
| Final result | Unmarked pairs merge  | Each block becomes state |

Both are correct.

For GATE, use whichever is faster.

## 28. EQUIVALENT VS SAME STATE

Two different states can be equivalent.

Example:

```
TEXT
B  "  C
```

as labels, but:

$[B \equiv C]$

if they accept exactly the same set of future suffixes.

Then minimal DFA merges them.

## 29. DISTINGUISHING STRING

Suppose states  $p$  and  $q$ .

If:

```
TEXT
input 101
```

causes:

```
TEXT
p  !'  f i n a l
q  !'  n o n - f i n a l
```

then `101` is a distinguishing string.

Thus:

$[p \not\equiv q]$



---

## 30. LENGTH OF DISTINGUISHING STRINGS

---

For exam reasoning:

- distinguishes final vs non-final immediately.
- one-symbol strings often distinguish states next.
- longer strings may be required.

Table-filling essentially discovers such distinctions indirectly.

---

## 31. MINIMUM-STATE REASONING WITHOUT A GIVEN DFA

---

This is extremely important for GATE.

Sometimes no DFA is given.

Question asks:

**Minimum states required for language L?**

You need to reason about how many distinct histories must remain distinguishable.

Example:

**Strings with exactly two 1s.**

Histories:

```
TEXT
0 ones
1 one
2 ones
>2 ones
```

Why must these be separate?

Because future extensions affect them differently.

---

## Example

---

State representing:

```
TEXT
1 one
```

cannot merge with:

```
TEXT
2 ones
```

because suffix:

$\backslash \text{varepsilon} \backslash$

already distinguishes:

- exactly 2 !' accept
- exactly 1 !' reject

Likewise:

TEXT  
0 ones

and:

TEXT  
1 one

can be distinguished by appending one 1.

## 32. MINIMUM DFA FOR "AT LEAST 2 ONES"

Memory states:

TEXT  
0 ones  
1 one  
"e 2 o n e s

Can any merge?

No.

### 0 vs 1

Append:

TEXT  
1

Then:

- from 0 !' total 1 !' reject
- from 1 !' total 2 !' accept

distinguishable.

### 1 vs "e 2

μdistinguishes:

- 1 !' reject
- "e 2' accept

### 0 vs "e 2

μdistinguishes.

Therefore minimum:

$\boxed{}$

### 33. MINIMUM DFA FOR LENGTH MODULO 3

States:

TEXT

```
q 0 = length "a" 0
q 1 = length "a" 1
q 2 = length "a" 2
```

Can q1 and q2 merge?

No.

Append one symbol:

- q2 !' q0 !' accepting
- q1 !' q2 !' rejecting

So distinguishable.

Likewise all three remainder classes are distinct.

Minimum:

$\boxed{}$

### 34. VERY IMPORTANT CONCEPT — FUTURE BEHAVIOR

For prefix  $x$ , define:

**what suffixes can still make  $x$  accepted?**

Two prefixes can share a DFA state iff their future behavior is identical.

This is the intuition behind **Myhill-Nerode equivalence**.

We don't need a highly formal theorem yet, but remember:

**Different future acceptance possibilities require different DFA states.**

### 35. MYHILL-NERODE INTUITION

For strings  $x$  and  $y$ :

$x \sim_L y$

if:

$\forall z, xz \in L \iff yz \in L$

Meaning:

Appending any possible suffix  $z$  cannot distinguish  $x$  and  $y$ .

Each equivalence class corresponds to one state in the minimal DFA.

## WHY THIS MATTERS

If you can find  $k$  pairwise distinguishable prefixes, then every DFA for the language needs at least:

$\lceil k \rceil$

states.

This is one of the strongest ways to reason about minimum-state questions.

## 36. EXAMPLE — STRINGS ENDING WITH 01

Why do we need 3 states?

Possible meaningful histories:

TEXT

$q_0$  = no useful suffix  
 $q_1$  = suffix 0  
 $q_2$  = suffix 01

These must be distinguishable.

### $q_2$ vs $q_0$

$\mu$ distinguishes because  $q_2$  accepts and  $q_0$  rejects.

### $q_2$ vs $q_1$

$\mu$ distinguishes.

### $q_0$ vs $q_1$

Append:

TEXT

1

Then:

- $q_1 + 1$  !' suffix 01 !' accept
- $q_0 + 1$  !' not ending 01 !' reject

So  $q_0$  and  $q_1$  are distinguishable.

Therefore 3 states are necessary.

## 37. UNREACHABLE VS DEAD STATE

Do not confuse them.

## Unreachable state

Cannot be reached from start state at all.

Remove it.

## Dead/trap state

May be reachable, but once entered, acceptance may become impossible.

A dead state can be necessary.

## Example

Exactly one 1 DFA:

TEXT

q0 = 0 ones  
q1 = exactly 1  
qD = >1

qD is reachable.

It is a trap.

You cannot simply delete it from a complete DFA.

## COMPARISON

| State type  | Reachable?  | Needed?        |
|-------------|-------------|----------------|
| Unreachable | No          | Remove         |
| Trap/dead   | Usually yes | Often required |

## 38. COMMON MINIMIZATION ERROR

Students sometimes merge two rejecting states just because both are non-final.

Wrong.

Two non-final states may have different futures.

Example:

TEXT

q0 = seen 0 ones  
q1 = seen 1 one

for exactly two ones.

Both currently reject.

But append:

TEXT

1

From  $q_1$   $!$  accept.

From  $q_0$   $!$  still reject.

So they are distinguishable.

### 39. ANOTHER COMMON ERROR

Two final states are not automatically equivalent either.

They may accept  $\mu$  equally but react differently to future input.

Example:

TEXT

p final  
q final

If input  $\emptyset$  sends:

TEXT

p  $!$  final  
q  $!$  non-final

then  $\emptyset$  distinguishes them.

### 40. GATE-STYLE QUESTION 1

A DFA has 6 states, one of which is unreachable. After removing it, two remaining states are equivalent.

Minimum number of states?

Start:

$\boxed{6}$

Remove unreachable:

$\boxed{5}$

Merge one equivalent pair:

$\boxed{4}$

Answer:

$\boxed{4}$

Assuming no additional equivalences.

### 41. GATE-STYLE QUESTION 2

Two states p and q are both non-final.

On input  $\emptyset$ :

TEXT

|   |   |   |
|---|---|---|
| p | ! | r |
| q | ! | s |

and  $(r, s)$  is already known distinguishable.

Then p and q are:

A. Equivalent B. Distinguishable

Answer:

$\boxed{B}$

because input  $\emptyset$  leads to distinguishable behavior.

---

## 42. GATE-STYLE QUESTION 3

What string immediately distinguishes a final state from a non-final state?

$\boxed{\varepsilon}$

No input is needed.

---

## 43. GATE-STYLE QUESTION 4

Can two unreachable states affect the language recognized by the DFA?

No.

Because no input starting from q can reach them.

---

## 44. GATE/PYQ PATTERNS

Typical GATE questions include:

### Pattern 1

Given transition table, find number of states after minimization.

### Pattern 2

Identify equivalent states.

### Pattern 3

Use table-filling.

### Pattern 4

Count unreachable states.

### Pattern 5

Find minimum states directly from a language property.

## Pattern 6

Given several strings/prefixes, determine whether they are distinguishable.

---

## 45. FAST EXAM STRATEGY

---

For a given DFA:

TEXT

1. Remove unreachable states
2. Separate final/non-final
3. Check distinguishability
4. Merge equivalent states
5. Count resulting classes

For direct minimum-state questions:

TEXT

1. Identify required memory categories
2. Show categories are reachable
3. Find distinguishing suffixes
4. Conclude minimum

## 46. & pGATE TRAPS

---

### Trap 1

---

All non-final states are equivalent.

'L False.

---

### Trap 2

---

All final states are equivalent.

'L False.

---

### Trap 3

---

Unreachable states should participate in final minimization.

'L Remove them first.

---

### Trap 4

---

Trap state should always be deleted.

'L A reachable trap state may be necessary.

---



## Trap 5

If two states have different names, they cannot be equivalent.

'LNames are irrelevant.

---

## Trap 6

If two states have same immediate final/non-final status, they are equivalent.

'LFuture behavior also matters.

---

## Trap 7

If product construction gives 12 states, minimum DFA must have 12.

'LUnreachable and equivalent states may reduce it.

---

## Trap 8

Final and non-final state might merge if transitions are identical.

'LNever.

$\mu$ distinguishes them.

---

## Trap 9

To show two states distinguishable, must test all strings.

'LOne distinguishing string is enough.

---

## Trap 10

To show two states equivalent, checking one or two short strings is enough.

'LIn principle they must agree for all future strings; use a valid minimization method.

---

## 47. IMPORTANT RESULTS TO MEMORIZE

$$\boxed{p \equiv q \text{ iff } \forall w, \delta^*(p, w) \in F \iff \delta^*(q, w) \in F}$$

Distinguishable:

$$\boxed{\exists w \text{ causing different acceptance}}$$

Minimal DFA:

$$\boxed{\text{all states reachable} + \text{no equivalent pair}}$$

Final vs non-final:

$$\boxed{\forall \epsilon \text{ distinguishes them}}$$

---

Minimal DFA is unique up to state renaming.

## 48. TABLE-FILLING MEMORY TRICK

Remember:

**F/N !' mark first**

Then:

**If transition reaches marked pair !' mark current pair**

Finally:

**Unmarked = merge**

Compact form:

TEXT

F i n a l   v s   N o n - f i n a l   !'   X

Destination pair X

!"

Current pair X

No X at end

!"

Equivalent

## 49. PARTITION METHOD MEMORY TRICK

TEXT

Final / Non-final

!"

Split by transition behavior

!"

Split again

!"

Stable groups

!"

One minimal state per group

## 50. COMPARISON — REACHABLE / DISTINGUISHABLE / EQUIVALENT

| Concept     | Meaning                                 |
|-------------|-----------------------------------------|
| Reachable   | Some input can reach state from $q \in$ |
| Unreachable | No input reaches it                     |

| Concept         | Meaning                                           |
|-----------------|---------------------------------------------------|
| Equivalent      | No suffix can separate acceptance behavior        |
| Distinguishable | At least one suffix produces different acceptance |
| Minimal DFA     | Reachable states with no equivalent pair          |

## 51. 'p WRITE THIS IN YOUR NOTES

### DFA Minimization

Goal:

Remove unnecessary states without changing language.

#### Steps

TEXT

1. Remove unreachable states
2. Find equivalent states
3. Merge equivalent states

### Reachable state

$q \text{ reachable} \iff \exists w: \delta^*(q_0, w) = q$

### Equivalent states

$p \equiv q \iff \forall w, \delta^*(p, w) \in F \iff \delta^*(q, w) \in F$

Memory:

Same future behavior

### Distinguishable states

$p, q \text{ distinguishable} \iff \exists w$

such that one accepts after  $w$  and the other rejects.

One witness is enough.

### Important

Final and non-final states are always distinguishable by:

$\epsilon$

---

## Table filling

---

TEXT

1. Remove unreachable
2. List all pairs
3. Mark Final/Non-final pairs
4. Mark pair if some transition leads to marked pair
5. Repeat
6. Unmarked pairs are equivalent

---

## Partition

---

TEXT

$P_0 = \{\text{Final states}\}, \{\text{Non-final states}\}$

Repeatedly split blocks by transition destinations.

Stable blocks = minimal states.

---

## Minimal DFA

---

$\boxed{\text{Reachable + pairwise distinguishable}}$

Minimal DFA unique up to renaming.

---

## 52. 60-SECOND REVISION

---

- Remove unreachable states first.
  - Reachable means some input reaches the state from  $q \in$ .
  - Equivalent states have identical future acceptance behavior.
  - Distinguishable states differ for at least one suffix.
  - $\mu$ distinguishes final vs non-final.
  - One distinguishing string is enough to prove difference.
  - Table-filling marks distinguishable pairs.
  - Initially mark final/non-final pairs.
  - Distinction propagates backward through transitions.
  - Unmarked pairs are equivalent.
  - Partition method repeatedly splits groups.
  - Each stable partition block becomes one minimal state.
  - Minimal DFA has only reachable, pairwise distinguishable states.
  - Minimal DFA is unique up to state names.
- 

## 53. ACTIVE RECALL

---

Try before checking answers.

### Q1

What are the two major operations in DFA minimization?

### Q2

What is an unreachable state?

### Q3

Can an unreachable state change the language?

### Q4

What is the difference between equivalent and distinguishable states?

### Q5

What string distinguishes any final state from any non-final state?

### Q6

In table filling, which pairs are marked first?

### Q7

What do unmarked pairs represent at the end?

### Q8

In partition method, what are the first two groups?

### Q9

Is a reachable trap state automatically removable?

### Q10

What two conditions characterize a minimal DFA?

---

## ANSWERS

1. Remove unreachable states; merge equivalent states.
2. A state that cannot be reached from  $q_0$  by any input string.
3. No.
4. Equivalent states agree for all continuations; distinguishable states differ for at least one.
5.  $\mu$ .
6. Final/non-final pairs.
7. Equivalent state pairs.
8. Final states and non-final states.
9. No.
10. All states reachable and no two distinct states equivalent.

---

## 54. MINI GATE TEST

### Q1 — MCQ

Two states  $p$  and  $q$  are one final and one non-final.

They are:

A. Always equivalent B. Always distinguishable C. Sometimes equivalent D. Cannot determine

### Q2 — MCQ

In DFA minimization, which should normally be done first?

A. Add final states B. Remove unreachable states C. Reverse transitions D. Convert to NFA

### Q3 — NAT

A DFA has 8 states. Two are unreachable. Of the remaining states, exactly two disjoint equivalent pairs are merged and no other equivalences exist.

How many states remain?

### Q4 — MSQ

Which statements are correct?

A. Every unreachable state is unnecessary for language recognition. B. Every trap state is unreachable. C. Equivalent states can be merged. D. Two final states may still be distinguishable.

### Q5 — MCQ

In table filling, pair  $(p, q)$  is currently unmarked. For input  $1$ , the states move to  $(r, s)$ , and  $(r, s)$  is marked.

What should happen?

A. Keep  $(p, q)$  unmarked B. Mark  $(p, q)$  C. Merge  $p$  and  $q$  immediately D. Delete  $p$

### Q6 — CONCEPTUAL

If you find  $k$  pairwise distinguishable reachable histories/prefixes for a language, what lower bound follows?

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\mu$  distinguishes them.

### Q2

### Q3

Start:

$\{ 8 \}$

Remove 2 unreachable:

$\{ 6 \}$

Each merged pair reduces state count by 1.

Two pairs:

$\{ 6-2=\boxed{4} \}$

### Q4

Correct:

$\{ \boxed{A,C,D} \}$

B is false.

A trap state can be reachable.

### Q5

$\{ \boxed{B} \}$

### Q6

Every DFA for that language needs at least:

$\{ \boxed{k} \}$

states.

## CHAPTER 2 — COMPLETED '

#### TEXT

```
[ ' ] D F A   f u n d a m e n t a l s
[ ' ] F o r m a l   5 - t u p l e
[ ' ] T r a n s i t i o n   f u n c t i o n
[ ' ] A c c e p t a n c e
[ ' ] E x t e n d e d   t r a n s i t i o n
[ ' ] D F A   c o n s t r u c t i o n
[ ' ] P r e f i x   p a t t e r n s
[ ' ] S u f f i x   p a t t e r n s
[ ' ] C o n t a i n s   /   A v o i d s
[ ' ] E x a c t   /   A t   l e a s t   /   A t   m o s t
[ ' ] P a r i t y
[ ' ] M o d u l o   c o u n t i n g
[ ' ] B i n a r y   d i v i s i b i l i t y
[ ' ] C o m p l e m e n t
[ ' ] P r o d u c t   a u t o m a t a
[ ' ] R e a c h a b i l i t y
[ ' ] E q u i v a l e n t   s t a t e s
[ ' ] D i s t i n g u i s h a b l e   s t a t e s
[ ' ] T a b l e - f i l l i n g   m e t h o d
[ ' ] P a r t i t i o n   m e t h o d
[ ' ] M i n i m u m - s t a t e   r e a s o n i n g
[ ' ] M y h i l l - N e r o d e   i n t u i t i o n
```

## NEXT CHAPTER

### CHAPTER 3 — NFA & $\mu$ - N F A

Next we will learn:

why nondeterminism is allowed !' formal NFA definition !' multiple transitions !' missing transitions !' NFA acceptance !' DFA vs NFA !' why NFA is not more powerful than DFA.

Type NEXT.

USER

NEXT

ASSISTANT

Continuing from your uploaded TOC master structure and the completed DFA chapter.

### CHAPTER 3 — NONDETERMINISTIC FINITE AUTOMATA (NFA)

#### Topic 1: NFA Fundamentals — Definition, Acceptance, DFA vs NFA

Priority: ~~0~~=~~V~~ERY HIGH

NFA is one of the most important ideas in TOC because it teaches something surprising:

**A machine can have multiple possible next states, yet it recognizes exactly the same class of languages as a DFA.**

That means:

$\boxed{\text{DFA and NFA have equal expressive power}}$

NFA is often easier to **design**, while DFA is easier to **execute uniquely**.

#### 1. WHY DO WE NEED NFA?

Consider the language:

**All binary strings ending with 01.**

We built a DFA earlier.

Now suppose the language is:

**All binary strings that contain 101 somewhere.**

A DFA must always keep track of the exact useful suffix.

An NFA can take a more convenient approach:



Whenever it sees a 1, it can guess that this might be the beginning of 101.

If the guess fails, another computation path may still succeed.

This is the intuition behind nondeterminism.

## 2. INTUITION FIRST

A DFA says:

**“For this state and this input symbol, I know exactly one next state.”**

An NFA says:

**“For this state and this input symbol, I may have several possible next states.”**

Example:

TEXT

q0 --0--> q0

q0 --1--> q0

q0 --1--> q1

On input 1 from q0, the NFA can move to:

TEXT

q0

OR

q1

So it explores multiple possibilities conceptually.

## 3. IMPORTANT WARNING

NFA does **not** mean:

**machine randomly chooses one path and hopes it works.**

The correct mathematical interpretation is:

**A string is accepted if at least one possible computation path ends in a final state after the entire input is consumed.**

This is crucial.

## 4. DFA VS NFA — FIRST LOOK

| Feature                             | DFA                         | NFA                          |
|-------------------------------------|-----------------------------|------------------------------|
| Next state                          | Exactly one                 | Zero, one, or many           |
| Missing transition                  | Not allowed in complete DFA | Allowed                      |
| Multiple transitions on same symbol | No                          | Yes                          |
| Computation path                    | Exactly one                 | Possibly many                |
| Acceptance                          | Unique path ends final      | At least one path ends final |
| Expressive power                    | Regular languages           | Regular languages            |

## MEMORY TRICK

**DFA = Definitely one path**

**NFA = Numerous possible paths**

But remember:

**Same power**

## 5. FORMAL DEFINITION OF NFA

An NFA is also represented by:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Same five components:

- $Q$  = finite states
- $\Sigma$  = input alphabet
- $\delta$  = transition function
- $q_0 \in Q$  start state
- $F \subseteq Q$  = final states

The difference is in:

$$\delta$$

## 6. NFA TRANSITION FUNCTION

For a DFA:

$$\delta: Q \times \Sigma \rightarrow Q$$

For an NFA:

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

This is extremely important.

## 7. WHAT DOES $2^Q$ MEAN?

$\mathcal{P}(2^Q)$

means:

**power set of  $Q$**

or:

**set of all subsets of  $Q$ .**

Suppose:

$\mathcal{P}(Q = \{q_0, q_1\})$

Then:

$\mathcal{P}(2^Q = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\})$

So an NFA transition can return:

- no state
- one state
- several states

## EXAMPLE

$\mathcal{P}(\delta(q_0, 1) = \{q_0, q_1\})$

means:

**From  $q_0$  on input 1, NFA can move to  $q_0$  or  $q_1$ .**

## Another example

$\mathcal{P}(\delta(q_1, 0) = \emptyset)$

means:

**There is no transition from  $q_1$  on 0.**

This is allowed in an NFA.

## 8. WHY DFA'S OUTPUT IS $Q$ BUT NFA'S IS $2^Q$

DFA:

TEXT

```
one current state + symbol
! "
one next state
```

NFA:

TEXT

```
one current state + symbol
! "
set of possible next states
```

Therefore:

$\{ Q \}$

versus:

$\{ 2^Q \}$

## 9. SIMPLE NFA EXAMPLE

Construct an NFA for strings ending in `01`.

Use:

TEXT

```
q0 = searching
q1 = guessed a possible starting 0
q2 = found ending 01
```

Transitions:

TEXT

```
q0 --0--> q0
q0 --0--> q1
q0 --1--> q0

q1 --1--> q2*
```

q2 has no outgoing transitions for this construction.

## WHY DOES THIS WORK?

At every `0`, q0 can do two things:

1. stay in q0 and keep searching
2. guess that this `0` may be the second-last symbol

Then if the next symbol is `1`:

TEXT

```
q1 --1--> q2
```

If that `1` is the final symbol, one path ends in q2 and accepts.

## 10. TRACE INPUT 1101

Start:

$\{q_0\}$

Read first 1:

$\{q_0\}$

Read second 1:

$\{q_0\}$

Read 0:

From  $q_0$  on 0:

$\{q_0, q_1\}$

Read final 1:

From  $q_0$ :

$\{q_0\}$

From  $q_1$ :

$\{q_2\}$

Therefore current possible state set:

$\{q_0, q_2\}$

Since:

$q_2 \in F$

the string is accepted.

## 11. TRACE TABLE

| Input consumed | Possible states |
|----------------|-----------------|
| $\mu$          | $\{q_0\}$       |
| 1              | $\{q_0\}$       |
| 11             | $\{q_0\}$       |
| 110            | $\{q_0, q_1\}$  |
| 1101           | $\{q_0, q_2\}$  |

Because  $q_2$  is final:

$\boxed{\text{Accepted}}$

## 12. NFA ACCEPTANCE RULE

A string is accepted if:

**At least one possible path finishes in an accepting state after the entire input is consumed.**

Formally, if:

$$\delta^*(q_0, w)$$

is the set of states reachable after reading  $w$ , then:

$$\boxed{w \in L(M) \text{ iff } \delta^*(q_0, w) \cap F \neq \emptyset}$$

## UNDERSTAND THIS FORMULA

Suppose after processing  $w$ :

$$\delta^*(q_0, w) = \{q_1, q_3, q_5\}$$

and:

$$F = \{q_3, q_4\}$$

Intersection:

$$\{q_1, q_3, q_5\} \cap \{q_3, q_4\} = \{q_3\}$$

Non-empty.

Therefore accepted.

## 13. REJECTION RULE

An NFA rejects if:

**none of its possible computation paths finish in a final state.**

It is not enough that one path fails.

All paths must fail.

## EXAMPLE

Possible final set after input:

$$\{q_1, q_2\}$$

and:

$$F = \{q_3\}$$

Then:

$$\{q_1, q_2\} \cap \{q_3\} = \emptyset$$

So reject.

---

## 14. MAJOR GATE TRAP

---

Suppose an NFA has three paths:

TEXT

```
P a t h   1   ! '   r e j e c t
P a t h   2   ! '   r e j e c t
P a t h   3   ! '   a c c e p t
```

Overall result?

$\boxed{\text{ACCEPT}}$

Because one accepting path is enough.

---

## MEMORY TRICK

---

**NFA acceptance = existential**

Think:

$\exists \text{ accepting path}$

DFA has only one path, so this issue does not arise.

---

## 15. CAN NFA HAVE NO TRANSITION?

---

Yes.

Example:

$\delta(q_2, 1) = \emptyset$

That path simply dies.

Other paths may continue.

---

## 16. DEAD COMPUTATION PATH

---

Suppose current possible states:

$\{q_1, q_2\}$

Input is 0.

From  $q_1$ :

$\emptyset$

From  $q_2$ :

$\{q_3\}$

Then  $q_1$  path dies, but  $q_2$  survives.

---

New possible state set:

$\{q_3\}$

---

## 17. CAN AN NFA HAVE MULTIPLE TRANSITIONS WITH SAME LABEL?

Yes.

Example:

TEXT

```
q0 --1--> q1
q0 --1--> q2
q0 --1--> q3
```

Equivalent transition function:

$\delta(q_0, 1) = \{q_1, q_2, q_3\}$

Perfectly valid NFA.

---

## 18. CAN AN NFA HAVE ONE TRANSITION ONLY?

Yes.

NFA does not require multiple transitions.

A DFA is actually a special type of NFA.

This leads to:

$\boxed{\text{Every DFA can be viewed as an NFA}}$

---

## 19. DFA "NFA AS MACHINE DESCRIPTIONS

A DFA obeys stricter rules.

NFA allows more transition flexibility.

So every DFA satisfies NFA rules.

But this does **not** mean NFA recognizes more languages.

---

## 20. EXPRESSIVE POWER

This is one of the most important results:

$\boxed{\text{DFA and NFA recognize exactly the regular languages}}$

So:

$\boxed{\text{NFA is not more powerful than DFA}}$

NFA may be:

- easier to construct
- smaller in number of states



but not more expressive.

## 21. WHY ARE THEY EQUIVALENT?

Because every NFA can be converted into an equivalent DFA.

The conversion is called:

**subset construction**

or:

**powerset construction**

We will study it in the next topic.

## BIG IDEA

An NFA may simultaneously be in:

$\{q_1, q_3, q_4\}$

A DFA cannot be in three states at once.

So the DFA creates **one new state** representing that whole subset:

TEXT

$\{q_1, q_3, q_4\}$

That is why:

$2^Q$

appears naturally.

## 22. NFA FOR "CONTAINS 01"

Let's build a very simple NFA.

Language:

**strings over  $\{0, 1\}$  containing substring 01.**

States:

TEXT

q0 = searching  
q1 = guessed a 0 that may begin 01  
q2 = found 01

Transitions:

#### TEXT

```
q0 --0--> q0
q0 --0--> q1
q0 --1--> q0

q1 --1--> q2*

q2 --0--> q2
q2 --1--> q2
```

## WHY q2 LOOPS?

Once 01 has occurred:

**condition is permanently satisfied.**

## 23. DFA VS NFA FOR CONTAINS 01

The DFA needed carefully designed suffix memory.

NFA can say:

**“At each zero, I'll nondeterministically guess that this one begins the target pattern.”**

This often makes NFA design easier.

## 24. NFA FOR "ENDS WITH 101"

Construct:

#### TEXT

```
q0 = searching
q1 = guessed first 1
q2 = matched 10
q3 = matched 101
```

Transitions:

#### TEXT

```
q0 --0--> q0
q0 --1--> q0
q0 --1--> q1

q1 --0--> q2

q2 --1--> q3*
```

q3 has no outgoing transitions.

## WHY NO LOOP AT q3?

---

Because we need the matched `101` to occur **at the end**.

If more symbols remain, this successful path should die.

Meanwhile q0 may still search for another later occurrence.

---

## 25. TRACE `110101`

---

We don't need to track every branch individually.

Use state sets.

Start:

$\mathbb{V} \{q_0\}$

After first 1:

$\mathbb{V} \{q_0, q_1\}$

After second 1:

- $q_0 \neq q_0, q_1$
- $q_1$  on 1  $\neq$  nothing

Result:

$\mathbb{V} \{q_0, q_1\}$

After 0:

- $q_0 \neq q_0$
- $q_1 \neq q_2$

Result:

$\mathbb{V} \{q_0, q_2\}$

After 1:

- $q_0 \neq q_0, q_1$
- $q_2 \neq q_3$

Result:

$\mathbb{V} \{q_0, q_1, q_3\}$

But input is not finished.

Read 0:

- $q_3$  dies
- $q_0 \neq q_0$
- $q_1 \neq q_2$

Result:

$\mathbb{V} \{q_0, q_2\}$

Read final 1:

$\mathbb{V} \{q_0, q_1, q_3\}$

q3 final, so accepted.

---

## 26. NFA DESIGN PATTERN— GUESS AND VERIFY

---

This is a very useful mental model.

For substring/suffix problems:

```
TEXT
Search
  !"
Guess start
  !"
Verify remaining pattern
  !"
Accept if one guess succeeds
```

Memory:

**NFA = Guess + Verify**

This is conceptual, not actual randomness.

---

## 27. FORMAL NFA EXTENDED TRANSITION

---

For an NFA:

$\delta^*(q, w)$

returns a **set of states**.

So:

$\boxed{\delta^*: Q \times \Sigma^* \rightarrow 2^Q}$

Compare DFA:

$\delta^*: Q \times \Sigma^* \rightarrow Q$

---

## 28. BASE CASE

---

For ordinary NFA without  $\mu$ -transitions:

$\delta^*(q, \epsilon) = \{q\}$

Why braces?

Because NFA extended transition returns a **set** of states.

---

## IMPORTANT TYPE DIFFERENCE

---

DFA:

$\delta^*(q, \epsilon) = q$

NFA:

$$\delta^*(q, \epsilon) = \{q\}$$

This is a useful mathematical distinction.

## 29. RECURSIVE IDEA

Suppose we have state set  $S$  after reading prefix  $w$ .

For next symbol  $a$ , take all transitions from all states in  $S$  and union them.

Conceptually:

$$\delta(S, a) = \bigcup_{q \in S} \delta(q, a)$$

### Example

Suppose:

$$S = \{q_1, q_2\}$$

and:

$$\delta(q_1, 0) = \{q_1, q_3\}$$

$$\delta(q_2, 0) = \{q_4\}$$

Then:

$$\delta(S, 0) = \{q_1, q_3\} \cup \{q_4\}$$

$$= \boxed{\{q_1, q_3, q_4\}}$$

## 30. IMPORTANT DIFFERENCE — NFA STATE VS STATE SET

The NFA itself has states:

TEXT

$q_0, q_1, q_2$

During analysis, we may track a set:

$$\{q_0, q_2\}$$

This does **not** mean  $\{q_0, q_2\}$  was originally an NFA state.

It means:

**after the input, the NFA has possible computations in  $q_0$  and  $q_2$ .**

When converting to DFA, however, that subset may become one actual DFA state.

## 31. NFA ACCEPTS $\mu$ WHEN?

For an ordinary NFA without  $\mu$  - m o v e s :

$$\delta^*(q_0, \epsilon) = \{q_0\}$$

Therefore:

$$\epsilon$$

is accepted iff:

$$q_0 \in F$$

Same practical rule as DFA.

## 32. IMPORTANT: NFA VS $\mu$ - N F A

Today's NFA transitions consume input symbols.

Later we'll study  $\mu$  - N F A where transitions like:

TEXT

$q_0 \xrightarrow{\epsilon} q_1$

are permitted.

That means the machine can move without consuming input.

For now:

**Ordinary NFA has no  $\mu$  - t r a n s i t i o n .**

## 33. COMPARISON — DFA VS NFA FORMALLY

| Property                       | DFA                             | NFA                               |
|--------------------------------|---------------------------------|-----------------------------------|
| Tuple                          | $(Q, \Sigma, \delta, q_0, F)$   | $(Q, \Sigma, \delta, q_0, F)$     |
| Transition                     | $Q \times \Sigma \rightarrow Q$ | $Q \times \Sigma \rightarrow 2^Q$ |
| Next states                    | Exactly 1                       | 0, 1, or many                     |
| Extended $\delta^*$ output     | $Q$                             | $2^Q$                             |
| Missing transition             | No                              | Yes                               |
| Same symbol to multiple states | No                              | Yes                               |
| $\mu$ - t r a n s i t i o n    | No                              | No in ordinary NFA                |
| Acceptance                     | unique path final               | any path final                    |
| Language class                 | Regular                         | Regular                           |

## 34. THE MOST IMPORTANT THEOREM

---

$\boxed{L \text{ is accepted by some NFA} \iff L \text{ is accepted by some DFA}}$

Therefore:

$\boxed{\text{DFA} = \text{NFA in expressive power}}$

---

## 35. STATE COMPLEXITY DIFFERENCE

---

Even though power is equal, number of states can differ dramatically.

If an NFA has:

$n$

states, an equivalent DFA may require as many as:

$2^n$

states in the worst case.

This comes from possible subsets of NFA states.

---

## & IMPORTANT

---

It does **not** mean every  $n$ -state NFA converts to exactly  $2^n$  DFA states.

Only:

$\boxed{\text{at most } 2^n}$

subset states are possible.

Many may:

- never be reached
  - later be equivalent/minimized
- 

## 36. THE EMPTY SET AS A DFA STATE DURING CONVERSION

---

Suppose an NFA state set has no possible transition on some input.

Then next subset is:

$\emptyset$

During NFA  $\rightarrow$  DFA conversion:

$\emptyset$

may become the DFA's trap state.

This is very important.

---

## 37. SOLVED QUESTION 1

---

Suppose:

$$Q = \{q_0, q_1, q_2\}$$

How many subsets exist?

$$2^3 = 8$$

They are:

$$\emptyset$$

$$\{q_0\}, \{q_1\}, \{q_2\}$$

$$\{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}$$

$$\{q_0, q_1, q_2\}$$

So equivalent DFA has at most:

$$8$$

subset states.

## SOLVED QUESTION 2

An NFA has 5 states.

Maximum possible number of states in subset-construction DFA:

$$2^5 = 32$$

## SOLVED QUESTION 3

An NFA reaches possible states:

$$\{q_1, q_2, q_4\}$$

after input  $w$ .

Final states:

$$F = \{q_2, q_3\}$$

Is  $w$  accepted?

Intersection:

$$\{q_1, q_2, q_4\} \cap \{q_2, q_3\} = \{q_2\}$$

Non-empty.

Therefore:

$$\boxed{\text{Accepted}}$$

## SOLVED QUESTION 4

After input  $w$ , possible NFA states are:

$$\{q_1, q_4\}$$

and:



$$F = \{q_2, q_3\}$$

Intersection:

$$\emptyset$$

Therefore:

$$\boxed{\text{Rejected}}$$

## 38. GATE/PYQ PATTERN ANALYSIS

Typical GATE questions test:

### Pattern 1 — Is this DFA or NFA?

Look for:

- missing transitions
- multiple same-symbol transitions

### Pattern 2 — Acceptance

Given several computation paths, determine whether string is accepted.

Remember:

**one accepting path is enough.**

### Pattern 3 — Transition-function type

DFA:

$$Q \times \Sigma \rightarrow Q$$

NFA:

$$Q \times \Sigma \rightarrow 2^Q$$

### Pattern 4 — Maximum DFA states

For n-state NFA:

$$2^n$$

possible subsets.

### Pattern 5 — Identify NFA language

Often easier if you interpret each branch as:

TEXT

guess something  
then verify

## 39. & μGATE TRAPS

---

### Trap 1 — NFA is more powerful than DFA

---

'L False.

Same expressive power.

---

### Trap 2 — NFA must choose exactly one path

---

'L False.

All possible paths are conceptually considered.

---

### Trap 3 — If one path rejects, whole NFA rejects

---

'L False.

One accepting path is enough.

---

### Trap 4 — NFA transition must always exist

---

'L False.

It may be:

$\emptyset$

---

### Trap 5 — NFA transition function returns one state

---

'L False.

It returns a subset:

$2^Q$

---

### Trap 6 — Every n-state NFA produces exactly $2^n$ DFA states

---

'L False.

$2^n$  is the maximum number of possible subsets.

---

### Trap 7 — Ordinary NFA automatically includes $\mu$ -transitions

---

'L False.

$\mu$ -transitions are not in NFA.

---

### Trap 8 — NFA can accept before input finishes

---

'LFalse.

An accepting path must end in a final state **after all input is consumed**.

---

### Trap 9 — A branch reaching a final state early guarantees acceptance

---

'LFalse.

If it must consume further symbols and has no valid path afterward, that branch can die.

---

### Trap 10 — $\{q_1, q_2\}$ is automatically an original NFA state

---

'LFalse.

It's a set of possible current NFA states.

It may later become a DFA state in subset construction.

---

## 40. SHORTCUTS & MEMORY TRICKS

---

### NFA

Numerous Future Alternatives

Or simply:

NFA = Guess + Verify

### Acceptance

One good path is enough

### Transition function

NFA returns a SET

$\mathcal{P}(2^Q)$

### Conversion size

n NFA states  $\rightarrow$  at most  $2^n$  DFA subset states

## Power

More choices "more language power"

## 41. IMPORTANT RESULTS TO MEMORIZE

$\delta_{\text{DFA}}: Q \times \Sigma \rightarrow Q$

$\delta_{\text{NFA}}: Q \times \Sigma \rightarrow 2^Q$

NFA acceptance:

$\delta^*(q_0, w) \cap F \neq \emptyset$

NFA and DFA:

same expressive power

n-state NFA:

equivalent DFA has at most  $2^n$  subset states

## 42. WRITE THIS IN YOUR NOTES

### Nondeterministic Finite Automaton

NFA:

$M = (Q, \Sigma, \delta, q_0, F)$

Transition:

$\delta: Q \times \Sigma \rightarrow 2^Q$

So for a state-symbol pair, next-state set may contain:

TEXT

0 states

1 state

many states

### Acceptance

A string  $w$  is accepted if at least one computation path ends in final state.

$w \in L(M) \text{ iff } \delta^*(q_0, w) \cap F \neq \emptyset$

Memory:

NFA = one successful path is enough

## DFA vs NFA

### TEXT

D F A ! ' e x a c t l y o n e n e x t s t a t e  
N F A ! ' z e r o / o n e / m a n y n e x t s t a t e s

But:

$\boxed{\text{DFA and NFA have equal expressive power}}$

Both recognize:

$\boxed{\text{Regular Languages}}$

## State conversion fact

If NFA has  $n$  states:

$\boxed{\leq 2^n}$

DFA subset states may be needed.

Each DFA state represents a subset of NFA states.

## 43. 60-SECOND REVISION

- NFA has the same five components as DFA.
- Main difference is transition function.
- DFA:  $Q \times \Sigma \rightarrow Q$
- NFA:  $Q \times \Sigma \rightarrow 2^Q$
- NFA can have zero, one, or many next states.
- Missing transition is allowed.
- Multiple same-symbol transitions are allowed.
- Ordinary NFA has no  $\mu$ -transition.
- NFA accepts if at least one computation path accepts.
- All input must still be consumed.
- DFA and NFA have equal expressive power.
- Both recognize exactly regular languages.
- Every NFA can be converted to a DFA.
- $n$ -state NFA can generate at most  $2^n$  subset states.

## 44. ACTIVE RECALL

### Q1

What is the formal transition function of an NFA?

### Q2

What does  $2^Q$  mean?

### Q3

Can an NFA have no outgoing transition for an input symbol?

### Q4

Can an NFA have two outgoing transitions with the same input symbol?

### Q5

If five possible paths exist and exactly one ends in a final state, is the input accepted?

### Q6

Are NFAs more powerful than DFAs?

### Q7

What language class do both recognize?

### Q8

For an n-state NFA, what is the maximum subset count?

### Q9

What does subset  $\{q_1, q_3\}$  mean while tracing an NFA?

### Q10

Does ordinary NFA permit  $\mu$  - t r a n s i t i o n s ?

---

## ANSWERS

1.

$\mathcal{P}[Q \times \Sigma^*]$

2. Power set of  $Q$ , i.e. set of all subsets of  $Q$ .

3. Yes.

4. Yes.

5. Yes.

6. No.

7. Regular languages.

8.

$\mathcal{P}[2^n]$

9. Both  $q_1$  and  $q_3$  are possible current states across different computation paths.

10. No. That is  $\mu$  - N F A .

---

## 45. MINI GATE TEST

### Q1 — MCQ

The transition function of an NFA is:

A.

$\{ Q \times \Sigma^* \}$

B.

$\{ Q \times \Sigma^{2^*} \}$

C.

$\{ 2^*Q \times \Sigma^* \}$

D.

$\{ Q \times \Sigma \}$

---

## Q2 — NAT

An NFA has 6 states.

Maximum number of possible subset states in the equivalent DFA:

---

## Q3 — MCQ

An NFA has four possible computation paths for string w. Three reject and one accepts.

The NFA:

A. Rejects B. Accepts C. Loops D. Cannot determine

---

## Q4 — MSQ

Which are allowed in an ordinary NFA?

A. Multiple transitions with same label from one state B. Missing transition C.  $\mu$  - t r a n s i t i o n s D. One transition only

---

## Q5 — True/False

Every language recognized by an NFA can also be recognized by some DFA.

---

## Q6 — Tricky

An NFA reaches a final state before input is completely consumed, but no continuation from that branch can read the remaining input.

Does that branch accept the string?

A. Yes B. No

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\boxed{B}$

---

## Q2

$\boxed{2^6=64}$

## Q3

$\boxed{B}$

## Q4

Correct:

$\boxed{A,B,D}$

C belongs to  $\mu$ -NFA.

## Q5

$\boxed{\text{True}}$

## Q6

$\boxed{B}$

The full input must be consumed.

## CHAPTER 3 PROGRESS

### TEXT

```
[ ] NFA motivation
[ ] Formal NFA definition
[ ] Transition function
[ ] Power set  $2^Q$ 
[ ] Multiple transitions
[ ] Missing transitions
[ ] NFA acceptance
[ ] State-set tracing
[ ] Guess-and-verify intuition
[ ] DFA vs NFA
[ ] Equal expressive power
[ ]  $2^n$  subset upper bound

[ ] NFA  $\rightarrow$  DFA subset construction
[ ] Empty-set DFA state
[ ] Reachable subset optimization
[ ]  $\mu$ -NFA
[ ]  $\mu$ -closure
[ ]  $\mu$ -NFA  $\rightarrow$  DFA
```

## NEXT TOPIC

### ~~Q=17%~~ NFA $\rightarrow$ DFA SUBSET CONSTRUCTION

Next we will convert complete NFAs into equivalent DFAs step-by-step, including:

- how to form subset states
- why  $\{q_0, q_1\}$  becomes one DFA state
- when  $\emptyset$  becomes a trap state
- how to choose final DFA subsets



- how to avoid generating unnecessary 2 states

Type NEXT.

USER  
NEXT

ASSISTANT

Continuing from the uploaded TOC master prompt and the NFA fundamentals section.

## CHAPTER 3 — NFA & $\mu$ - N F A

### Topic 2: NFA !' DFA Conversion Using Subset Construction

Priority: ~~0~~ = VERY HIGH

This topic proves the central result:

$\boxed{\text{Every NFA has an equivalent DFA}}$

That is why:

$\boxed{\text{NFA and DFA have equal expressive power}}$

The conversion method is called:

**Subset Construction or Powerset Construction**

### 1. WHY DO WE NEED SUBSET CONSTRUCTION?

An NFA can be in several possible states after reading the same input.

Example:

$\{q_0, q_1, q_3\}$

A DFA cannot do that.

A DFA must always be in exactly **one state**.

So we create one DFA state representing the entire NFA state-set:

TEXT

NFA possible states  
 $\{q_0, q_1, q_3\}$

!"

One DFA state  
 $\{q_0, q_1, q_3\}$

That is the whole idea.

## 2. CORE INTUITION

Suppose an NFA has:

$$Q = \{q_0, q_1\}$$

The possible subsets are:

$$\emptyset$$

$$\{q_0\}$$

$$\{q_1\}$$

$$\{q_0, q_1\}$$

Any of these subsets can potentially become a state of the equivalent DFA.

## MEMORY TRICK

**NFA keeps many possibilities separately.**

**DFA bundles those possibilities into one subset-state.**

## 3. WHY THE NAME "POWERSET CONSTRUCTION"?

If an NFA has state set  $Q$ , the DFA state set is built from subsets of  $Q$ .

Therefore:

$$Q_D = \text{subsets of } 2^Q$$

where:

$$2^Q$$

is the power set.

If:

$$|Q| = n$$

then:

$$|2^Q| = 2^n$$

Hence:

$$\boxed{\text{'n-state NFA' at most } 2^n \text{ DFA states}}$$

## 4. VERY IMPORTANT WORD: "AT MOST"

An NFA with 4 states has:

$$2^4 = 16$$

possible subsets.

But the DFA does **not necessarily have 16 reachable states**.

Maybe only:

TEXT

```
{q0}  
{q0, q1}  
{q2}  
"
```

are reachable.

Then only those are needed.

## GATE RULE

**Do not generate all  $(2^n)$  subsets blindly.**

Generate only subsets reachable from the start state.

## 5. FORMAL CONSTRUCTION

Suppose NFA:

$N = (Q, \Sigma, \delta, q_0, F)$

We construct DFA:

$D = (Q_D, \Sigma, \delta_D, S_0, F_D)$

where:

### DFA states

$Q_D \subseteq 2^Q$

Each state is a subset of NFA states.

### DFA start state

For an ordinary NFA without  $\mu$ -transitions:

$S_0 = \{q_0\}$

### DFA transition

For subset:

$S \subseteq Q$

and input symbol  $a$ :

$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$

### DFA final states

A DFA subset-state is accepting if it contains **at least one NFA final state**.

So:

$$\boxed{F_D = \{S \subseteq Q \mid S \cap F \neq \emptyset\}}$$

This rule is extremely important.

## 6. WHY DOES A SUBSET BECOME FINAL IF IT CONTAINS ONE FINAL NFA STATE?

Because NFA acceptance is existential.

If current NFA possibilities are:

$$\{q_1, q_2, q_4\}$$

and:

$$q_2 \in F$$

then one possible computation ends in final state.

Therefore the NFA accepts.

So the corresponding DFA state:

$$\{q_1, q_2, q_4\}$$

must also be accepting.

## MEMORY

**Subset final if it contains a final.**

## 7. COMPLETE SUBSET-CONSTRUCTION ALGORITHM

Use this every time.

### Step 1

Start with:

$$\{q_0\}$$

### Step 2

For every input symbol, calculate all states reachable from every state in the subset.

Take union.

### Step 3

Every newly obtained subset becomes a DFA state.

Step 4

Repeat for every new subset until no new subsets appear.

Step 5

Mark every subset containing at least one NFA final state as final.

Step 6

If:  
 $\emptyset$   
appears, include it as the DFA trap state.

COMPACT MEMORY

```
TEXT
Start with {q0}
!
Take union of NFA moves
!
Create new subset
!
Repeat until stable
!
Subset containing final ! final
```

8. SOLVED EXAMPLE 1 — SIMPLE NFA

Consider NFA:  
 $Q = \{q_0, q_1\}$   
 $\Sigma = \{0, 1\}$   
Final:  
 $F = \{q_1\}$   
Transitions:

| NFA State   | 0                      | 1               |
|-------------|------------------------|-----------------|
| $q \in$     | $\{q \in, q \bullet\}$ | $\{q \in\}$     |
| $q \bullet$ | "                      | $\{q \bullet\}$ |

Convert to DFA.

---

## STEP 1 — START STATE

---

Ordinary NFA, so:

$$\Delta(q_0, 0) = \{q_0\}$$

Let us give subset states short labels:

TEXT

$$A = \{q_0\}$$

---

## STEP 2 — TRANSITIONS FROM A

---

### Input 0

$$\Delta(q_0, 0) = \{q_0, q_1\}$$

So:

$$A \xrightarrow{0} \{q_0, q_1\}$$

Call:

TEXT

$$B = \{q_0, q_1\}$$

---

### Input 1

$$\Delta(q_0, 1) = \{q_0\}$$

Thus:

$$A \xrightarrow{1} A$$

---

## STEP 3 — PROCESS B

---

$$B = \{q_0, q_1\}$$

### Input 0

Take union:

$$\Delta(q_0, 0) \cup \Delta(q_1, 0)$$

$$= \{q_0, q_1\} \cup \emptyset$$

$$= \{q_0, q_1\}$$

Thus:

$$B \xrightarrow{0} B$$

## Input 1

$\delta(q_0, 1) \cup \delta(q_1, 1)$

$= \{q_0\} \cup \{q_1\}$

$= \{q_0, q_1\}$

Thus:

$B \rightarrow 1B$

## 9. FINAL DFA STATES

Original NFA final:

$q_1$

Which DFA subsets contain  $q_1$ ?

TEXT

$B = \{q_0, q_1\}$

So:

$B$

is final.

A is not.

## FINAL DFA

| DFA State | NFA subset                  | 0 | 1 |
|-----------|-----------------------------|---|---|
| ' A       | $\{q_\epsilon\}$            | B | A |
| *B        | $\{q_\epsilon, q_\bullet\}$ | B | B |

## WHAT LANGUAGE IS THIS?

Once a **0** occurs, NFA can reach  $q_1$ .

After that  $q_1$  survives on **1**.

Because B loops on both symbols, the DFA accepts:

**all strings containing at least one 0.**

## 10. SOLVED EXAMPLE 2 — EMPTY SET APPEARS

Consider:

| NFA State         | 0               | 1               |
|-------------------|-----------------|-----------------|
| $!^{\circ} q \in$ | $\{q \bullet\}$ | $"$             |
| $* q \bullet$     | $"$             | $\{q \bullet\}$ |

Final:

$\backslash F=\backslash \{q_1\} \backslash$

START

$\backslash A=\backslash \{q_0\} \backslash$

FROM A

On 0:

$\backslash \backslash \{q_1\} \backslash$

Call:

TEXT

B = {q1}

On 1:

$\backslash \backslash \text{emptyset} \backslash$

Call:

TEXT

D = "

FROM B

On 0:

$\backslash \backslash \text{emptyset} \backslash$

so:

$\backslash B \backslash \rightarrow 0D \backslash$

On 1:

$\backslash \backslash \{q_1\} \backslash$

so:

$\backslash B \backslash \rightarrow 1B \backslash$

FROM D = "



There are no NFA states to move from.

Therefore on either input:

$\emptyset$

So:

TEXT

D --0--> D

D --1--> D

Thus  $\emptyset$  becomes a trap state.

## FINAL STATE

B contains  $q_1$ .

So B is accepting.

## FINAL TABLE

| DFA state               | 0 | 1 |
|-------------------------|---|---|
| $! A = \{ q \in \}$     | B | D |
| $* B = \{ q \bullet \}$ | D | B |
| $D = \emptyset$         | D | D |

## 11. WHAT DOES $\emptyset$ MEAN IN SUBSET DFA?

It means:

The NFA has no surviving computation path after the input read so far.

Once there are no possible NFA states:

no future input can revive a computation in an ordinary NFA.

So:

$\emptyset$

loops to itself on all symbols.

Hence it acts like a DFA trap state.

## MEMORY

## 12. SOLVED EXAMPLE 3 — THREE-STATE NFA

Consider:

$$Q = \{q_0, q_1, q_2\}$$

Alphabet:

$$\Sigma = \{0, 1\}$$

Final:

$$F = \{q_2\}$$

Transitions:

| State | 0              | 1         |
|-------|----------------|-----------|
| $q_0$ | $\{q_0, q_1\}$ | $\{q_2\}$ |
| $q_1$ | "              | $\{q_1\}$ |
| $q_2$ | $\{q_1\}$      | $\{q_1\}$ |

### START

$$A = \{q_0\}$$

### FROM A

On 0:

$$B = \{q_0, q_1\}$$

Call:

$$B = \{q_0, q_1\}$$

On 1:

$$A = \{q_0\}$$

### FROM B = {q0,q1}

#### Input 0

$$\delta(q_0, 0) \cup \delta(q_1, 0)$$

$$= \{q_0, q_1\} \cup \emptyset$$

$$= \{q_0, q_1\} = B$$

---

### Input 1

$$\mathbb{V}[\delta(q_0,1) \cup \delta(q_1,1)]$$

$$\mathbb{V} = \{q_0\} \cup \{q_2\}$$

$$\mathbb{V} = \{q_0, q_2\}$$

Call:

$$\mathbb{V} \text{ C} = \{q_0, q_2\}$$

---

### FROM C = {q0,q2}

---

#### Input 0

$$\mathbb{V}[\delta(q_0,0) \cup \delta(q_2,0)]$$

$$\mathbb{V} = \{q_0, q_1\} \cup \{q_2\}$$

$$\mathbb{V} = \{q_0, q_1, q_2\}$$

Call:

$$\mathbb{V} \text{ D} = \{q_0, q_1, q_2\}$$

---

#### Input 1

$$\mathbb{V}[\delta(q_0,1) \cup \delta(q_2,1)]$$

$$\mathbb{V} = \{q_0\} \cup \{q_2\}$$

$$\mathbb{V} = \text{C}$$

---

### FROM D = {q0,q1,q2}

---

#### Input 0

$$\mathbb{V}[\{q_0, q_1\} \cup \text{emptyset} \cup \{q_2\}]$$

$$\mathbb{V} = \text{D}$$

#### Input 1

$$\mathbb{V}[\{q_0\} \cup \{q_2\} \cup \{q_2\}]$$

$$\mathbb{V} = \{q_0, q_2\} = \text{C}$$

---

### FINAL SUBSETS

Original final:

$$\mathbb{V} \text{ q}_2$$

---

Therefore any DFA state containing  $q_2$  is final:

$$C = \{q_0, q_2\}$$

$$D = \{q_0, q_1, q_2\}$$

## FINAL DFA TABLE

| DFA       | Subset              | 0 | 1 |
|-----------|---------------------|---|---|
| $\{q_0\}$ | $\{q_0\}$           | B | A |
| B         | $\{q_0, q_1\}$      | B | C |
| *C        | $\{q_0, q_2\}$      | D | C |
| *D        | $\{q_0, q_1, q_2\}$ | D | C |

## 13. NOTICE SOMETHING IMPORTANT

The NFA had 3 states.

Maximum theoretical DFA subsets:

$$2^3 = 8$$

But only four subsets appeared:

TEXT

```
{q0}
{q0, q1}
{q0, q2}
{q0, q1, q2}
```

So actual reachable DFA states:

$$4$$

not 8.

## 14. REACHABLE SUBSET OPTIMIZATION

This is the fastest exam method.

Do **not** list the entire power set first.

Instead:

TEXT

```
Start subset
!"
Calculate its transitions
!"
Only process new subsets
!"
```

Stop when nothing new appears

This automatically produces only reachable subsets.

## 15. GATE EXAM TABLE METHOD

Create table progressively:

| DFA subset | 0 | 1 |
|------------|---|---|
| {q0}       | ? | ? |

Calculate.

If new subset appears, add row:

| DFA subset | 0       | 1    |
|------------|---------|------|
| {q0}       | {q0,q1} | {q0} |
| {q0,q1}    | ?       | ?    |

Continue.

This is much safer than drawing a diagram immediately.

## 16. FINAL-STATE IDENTIFICATION SHORTCUT

Suppose NFA finals:

$\{q_2, q_4\}$

Then subset states such as:

TEXT

{q0, q2}  
{q4}  
{q1, q3, q4}  
{q2, q4}

are all accepting.

Why?

Each contains:

$q_2$

or:

$q_4$

## Non-final examples

TEXT

$\{q_0\}$   
 $\{q_1, q_3\}$   
"

contain no NFA final state.

## 17. MULTIPLE NFA FINAL STATES

Nothing changes.

A DFA subset is final if:

$$\mathbb{I} \ S \cap F \neq \emptyset \mathbb{I}$$

Example:

$$\mathbb{I} \ F = \{q_1, q_3\} \mathbb{I}$$

Subset:

$$\mathbb{I} \ S = \{q_0, q_2, q_3\} \mathbb{I}$$

Since:

$$\mathbb{I} \ S \cap F = \{q_3\} \mathbb{I}$$

it is accepting.

## 18. WHY NFA !' DFA PRESERVES LANGUAGE

After every prefix  $w$ :

**DFA subset-state equals exactly the set of all NFA states reachable after reading  $w$ .**

So if the NFA has at least one accepting possibility:

$$\mathbb{I} \ S \cap F \neq \emptyset \mathbb{I}$$

the DFA marks that subset accepting.

Therefore both agree on every input string.

Hence:

$$\mathbb{I} \ \boxed{L(D) = L(N)} \mathbb{I}$$

## 19. DOES THE DFA NEED TO BE MINIMIZED AFTER CONVERSION?

Not necessarily for correctness.

Subset construction gives an **equivalent DFA**.

But that DFA may not be minimal.

So pipeline can be:

TEXT

NFA

!"

Subset construction

!"

Equivalent DFA

!"

Remove unreachable states

!"

Minimize

!"

Minimal DFA

Note: reachable-subset construction already avoids unreachable subset states, but equivalent subset states may still exist.

## 20. NFA STATECOUNT VS DFA STATECOUNT

An NFA may be much smaller.

Example:

TEXT

NFA = 5 states

DFA might require up to 32 states

because:

$\lfloor 2^5=32 \rfloor$

This difference is called an exponential state blow-up in the worst case.

## IMPORTANT GATE FACT

**NFA can be exponentially more succinct than DFA.**

But:

**not more computationally expressive.**

## 21. VERY IMPORTANT DISTINCTION

### Expressive power

What languages can be recognized?

DFA = NFA.

### Description/state complexity

How many states might be needed?

NFA may need far fewer.

These are different ideas.

---

## 22. SOLVED GATE-STYLE QUESTION

---

An NFA has:

$n=7$

states.

What is the maximum number of subsets?

$2^7=128$

Therefore subset construction has at most:

$\boxed{128}$

possible DFA states.

---

## 23. ANOTHER GATE QUESTION

---

Suppose NFA has 4 states.

During conversion only these subsets appear:

TEXT

```
{q0}
{q0, q1}
{q2}
"
{q1, q3}
```

How many DFA states are generated?

$\boxed{5}$

Not:

$2^4=16$

because only reachable subsets count in the constructed DFA.

---

## 24. TRICKY QUESTION — FINAL SUBSET

---

NFA final states:

$F=\{q_2\}$

Which subset is accepting?

A.

$\{q_0, q_1\}$

B.

---



$\{q_1, q_2\}$

C.

$\emptyset$

D.

$\{q_0\}$

Answer:

$\boxed{B}$

because it contains  $q_2$ .

---

## 25. TRICKY QUESTION — "

Suppose DFA subset state:

$\emptyset$

What is:

$\delta_D(\emptyset, 0)$ ?

There are no NFA states to contribute transitions.

So:

$\boxed{\emptyset}$

Similarly:

$\delta_D(\emptyset, 1) = \emptyset$

Thus it is a trap.

---

## 26. & pGATE TRAPS

### Trap 1 — Every power-set subset becomes a DFA state

'L False.

Only reachable subsets need to appear.

---

### Trap 2 — A subset is final only if all its NFA states are final

'L False.

One final NFA state is enough.

Correct:

$\cap F \neq \emptyset$

---

### Trap 3 — $\{q_1, q_2\}$ means DFA is simultaneously in two states

---

'L False.

In the DFA,  $\{q_1, q_2\}$  is **one DFA state**.

It represents two possible NFA states.

---

#### Trap 4 — " can be omitted even when reached

'L Not from a complete DFA.

If reachable, " functions as a trap state and needs its transitions.

---

#### Trap 5 — NFA with $n$ states always gives $2 \cdot$ DFA states

'L False.

At most  $2 \cdot$

---

#### Trap 6 — Subset conversion changes the language

'L False.

Equivalent DFA recognizes exactly the same language.

---

#### Trap 7 — Final DFA subsets are only singleton final states

'L False.

Any subset containing an NFA final state is accepting.

---

#### Trap 8 — Union is unnecessary when calculating a subset transition

'L False.

You must combine transitions from **all** states currently in the subset.

---

## 27. COMMON CALCULATION ERROR

Suppose:

$$\backslash [ S = \backslash \{q_0, q_1\} \backslash ]$$

and:

$$\backslash [ \backslash \delta(q_0, 1) = \backslash \{q_1, q_2\} \backslash ]$$

$$\backslash [ \backslash \delta(q_1, 1) = \backslash \{q_2, q_3\} \backslash ]$$

Then:

$$\backslash [ \backslash \delta_D(S, 1) \backslash ]$$

is not one of these individually.

---

Take union:

$$\mathbb{V} \{q_1, q_2\} \cup \{q_2, q_3\} \mathbb{V}$$

$$\mathbb{V} = \boxed{\{q_1, q_2, q_3\}} \mathbb{V}$$

Duplicates do not matter because it is a set.

---

## 28. SUBSET-CONSTRUCTION PROOF INTUITION

---

Suppose after reading  $w$ , the NFA has possible states:

$$\mathbb{V} S \mathbb{V}$$

The equivalent DFA is in exactly the subset-state:

$$\mathbb{V} S \mathbb{V}$$

After reading symbol  $a$ , every NFA path makes its own transition.

The collection of all resulting states is:

$$\mathbb{V} \bigcup_{q \in S} \delta(q, a) \mathbb{V}$$

The DFA moves to exactly that new subset.

So both machines remain synchronized in meaning after every input prefix.

---

## 29. GATE/PYQ PATTERNS

---

Typical questions include:

### Pattern 1 — Convert a given NFA to DFA

Use table-based subset construction.

---

### Pattern 2 — Count possible DFA states

For  $n$  NFA states:

$$\mathbb{V} 2^n \mathbb{V}$$

is the maximum candidate count.

---

### Pattern 3 — Count actual reachable subset states

Follow transitions.

Do not simply use  $2^n$ .

---

### Pattern 4 — Identify accepting subsets

Any subset containing at least one original final state.

---

### Pattern 5 — Determine " behavior

It becomes a trap.

---

## Pattern 6 — Determine equivalence of NFA and DFA

Every NFA can be converted to DFA.

Therefore same expressive power.

## 30. SHORTCUTS & MEMORY TRICKS

### Subset construction

Bundle all possible NFA states

### Transition

Move each, then union

TEXT

subset

!"

apply symbol to every member

!"

union all results

### Final DFA state

Contains Final !' Final

### Empty subset

No path !' Trap

### Size

n !' at most  $2^n$

## 31. MASTER FORMULAS

### DFA state set

$$\{ Q_D \subseteq 2^Q \}$$

### Start state

$$\{ q_0 \}$$

for ordinary NFA.

### Transition

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$$

### Finals

$$F_D = \{ S \mid S \cap F \neq \emptyset \}$$

### Maximum state count

$$2^{|Q|}$$

## 32. COMPARISON TABLE

| Feature               | NFA                    | Converted DFA         |
|-----------------------|------------------------|-----------------------|
| Current configuration | Set of possible states | One subset-state      |
| Transition result     | Set                    | One subset            |
| Start                 | $q \in$                | $\{ q \in \}$         |
| Accept                | Any path final         | Subset contains final |
| Missing path          | Can disappear          | May lead toward "     |
| "                     | No surviving path      | Trap-state subset     |

## 33. 'p WRITE THIS IN YOUR NOTES

### NFA !' DFA: Subset Construction

For:

$$N = (Q, \Sigma, \delta, q_0, F)$$

equivalent DFA uses subsets of NFA states.

### Start state

$$\{ q_0 \}$$

### Transition

For subset  $S$ :

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$$

### Final subset

$\boxed{\text{final} \iff S \cap F \neq \emptyset}$

### Empty subset

$\boxed{\emptyset}$

acts as trap if reached.

### Number of possible states

If NFA has  $n$  states:

$\boxed{\leq 2^n}$

## Procedure

TEXT

1. Start with  $\{q_0\}$
2. Find moves for every symbol
3. Add new subsets
4. Repeat
5. Mark subsets containing any final NFA state
6. Include " if reached

## 34. 60-SECOND REVISION

- Every NFA can be converted to DFA.
- Method = subset/powerset construction.
- Each DFA state represents a subset of NFA states.
- Start state is  $\{q_0\}$  for ordinary NFA.
- For each symbol, move from every state in subset.
- Union all destination states.
- New subsets become DFA states.
- Generate reachable subsets only.
- A subset is final if it contains at least one NFA final state.
- " represents no surviving NFA paths.
- If reached, " behaves as trap.
- $n$ -state NFA has at most  $2^n$  subset states.
- Converted DFA may still be non-minimal.
- DFA and NFA recognize exactly the same language class.

## 35. ACTIVE RECALL

### Q1

Why are DFA states subsets during NFA conversion?

### Q2

What is the start subset for an ordinary NFA?

### Q3

For subset  $S$ , how is transition on  $a$  calculated?

### Q4

When is a DFA subset accepting?

### Q5

What does " $\cap$ " mean?

### Q6

If an NFA has 8 states, what is the maximum number of subsets?

### Q7

Do all subsets necessarily appear?

### Q8

If NFA finals are  $\{q_2, q_4\}$ , is subset  $\{q_0, q_4\}$  final?

### Q9

Can the converted DFA later be minimized?

### Q10

What does  $\{q_1, q_3\}$  mean in the converted DFA?

---

## ANSWERS

1. Because one DFA state records all possible NFA states after the same input.

2.

$\bigcup_{q \in S} \delta(q, a)$

3.

$\bigcup_{q \in S} \delta(q, a)$

4. If it contains at least one NFA final state.

5. No NFA computation path survives.

6.

$2^8 = 256$

7. No.

8. Yes.

9. Yes.

10. One DFA state representing the NFA possibly being in  $q_1$  or  $q_3$ .

---

## 36. MINI GATE TEST

### Q1 — NAT

An NFA has 5 states.

Maximum number of possible subset states?

---

## Q2 — MCQ

If:

$$F = \{q_3\}$$

which subset is accepting?

- A.  $\{q_0, q_1\}$  B.  $\{q_1, q_2\}$  C.  $\{q_0, q_3\}$  D.  $\emptyset$

---

## Q3 — MCQ

The DFA state  $\emptyset$  obtained during subset construction usually acts as:

- A. Start state B. Final state C. Trap state D. NFA final state

---

## Q4 — MSQ

Suppose:

$$S = \{q_0, q_1\}$$

and:

$$\delta(q_0, a) = \{q_1, q_2\}$$

$$\delta(q_1, a) = \{q_2, q_3\}$$

Which states belong to:

$$\delta_D(S, a)?$$

- A.  $q$  B.  $q, C$  C.  $q, D$  D.  $q \in$

---

## Q5 — True/False

The subset-construction DFA is always minimal.

---

## Q6 — Tricky

An NFA has 6 states but only 9 reachable subsets during subset construction.

How many states does the constructed DFA need before minimization?

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

$$2^5 = \boxed{32}$$



## Q2

$\boxed{C}$

## Q3

$\boxed{C}$

## Q4

Union:

$\{q_1, q_2\} \cup \{q_2, q_3\} = \{q_1, q_2, q_3\}$

Correct:

$\boxed{A, B, C}$

## Q5

$\boxed{\text{False}}$

## Q6

Only reachable subsets matter:

$\boxed{9}$

## CHAPTER 3 PROGRESS

### TEXT

```
[ ] NFA fundamentals
[ ] NFA transition function
[ ] NFA acceptance
[ ] State-set tracing
[ ] DFA vs NFA
[ ] Equal expressive power
[ ] Subset construction
[ ] Reachable subsets
[ ] Final-subset rule
[ ] Empty-set trap state
[ ]  $2^n$  upper bound

[ ]  $\mu$ -NFA
[ ]  $\mu$ -transition
[ ]  $\mu$ -closure
[ ]  $\mu$ -NFA tracing
[ ]  $\mu$ -NFA  $\neq$  DFA
[ ]  $\mu$ -NFA  $\neq$  ordinary NFA
```

## NEXT TOPIC

### $\emptyset = \epsilon$ NFA AND $\mu$ -CLOSURE

Next we will learn the important difference:

### TEXT

NFA:  
transition consumes an input symbol

$\mu$  - N F A :  
can move without consuming input

Then we will master:

- $\mu$  - t r a n s i t i o n
- $\mu$  - c l o s u r e
- how to trace  $\mu$  - N F A
- why start DFA state is no longer simply  $\{ q \in \}$
- $\mu$  - N F A ! DFA conversion

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing from the same TOC master prompt and the completed NFA ! DFA conversion section.

## CHAPTER 3 — $\mu$ - N F A

### Topic 3: $\mu$ - T r a n s i t i o n C l o s u r e T r a c i n g and $\mu$ - N F A ! DFA

Priority: ~~0~~ = **VERY HIGH**

This is the final major part of finite-automata nondeterminism.

The core difference is simple:

TEXT

DFA

! ' e x a c t l y o n e n e x t s t a t e  
! ' c o n s u m e s o n e i n p u t s y m b o l

NFA

! ' z e r o / o n e / m a n y n e x t s t a t e s  
! ' c o n s u m e s o n e i n p u t s y m b o l

$\mu$  - N F A

! ' m a y a l s o m o v e w i t h o u t c o n s u m i n g i n p u t

That last line is the whole idea.

## 1. WHAT IS AN $\mu$ - T R A N S I T I O N ?

An  $\mu$  - t r a n s i t i o n state transition that consumes **no input symbol**.

Example:

TEXT

q 0 - - μ - - > q 1

means:

**The machine may move from q0 to q1 without reading anything from the input.**

So if the remaining input is:

TEXT

101

after the μ - t r a n s i t i o n s t i l l ;

TEXT

101

Nothing was consumed.

## 2. INTUITION FIRST

Imagine you are standing in room q0 .

Normally, to move to another room, you must hand over an input symbol.

But an μ - t r a n s i t i o n i s l i k e a f r e e p a s s a g e :

TEXT

q 0 - - μ - - > q 1

You move to q1 without spending any input.

Memory:

**μ - m o v e f r e e m o v e**

## 3. WHY ARE μ - T R A N S I T I O N S U S E F U L ?

They make automata construction much easier.

Suppose we want to combine two machines for:

$L_1 \cup L_2$

We can create a new start state and use:

TEXT

new start            μ ! '    M a c h i n e    1  
                         μ ! '    M a c h i n e    2

The automaton can nondeterministically choose either branch without consuming input.

This is extremely useful later when converting regular expressions into automata.

## 4. FORMAL $\mu$ - NFA

An  $\mu$  - NFA can still be written:

$$M = (Q, \Sigma, \delta, q_0, F)$$

But transition function becomes:

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

Compare carefully.

### Ordinary NFA

$$Q \times \Sigma \rightarrow 2^Q$$

### $\mu$ - NFA

$$Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

The extra  $\mu$  is the difference.

## 5. IMPORTANT

$\mu$  is **not necessarily part of the input alphabet**.

Suppose:

$$\Sigma = \{0, 1\}$$

Then  $\mu$  - transitions are still allowed in an  $\mu$  - NFA.

We do **not** redefine the alphabet as:

$$\Sigma = \{0, 1, \epsilon\}$$

because  $\mu$  is not an ordinary input symbol.

It means:

**consume nothing.**

## & **TRAP**

$\mu$  is not a character that physically appears in the input string.

So an input such as:

TEXT

0  $\mu$  1

is not being read in the usual sense.

$\mu$  is a transition label meaning **no input consumed**.

## 6. WHAT IS $\mu$ -CLOSURE?

This is the most important concept of this topic.

For a state  $q$ :

$\epsilon\text{-closure}(q)$

is:

the set of all states reachable from  $q$  using zero or more  $\mu$ -transitions.

## IMPORTANT WORDS

**Zero or more.**

Because zero  $\mu$ -moves are allowed:

$\boxed{q \in \epsilon\text{-closure}(q)}$

always.

## 7. SIMPLE EXAMPLE

Suppose:

TEXT

```
q 0  - -  $\mu$  - - >  q 1
q 1  - -  $\mu$  - - >  q 2
```

Then:

$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$

Why?

From  $q_0$ :

- take zero  $\mu$ -moves  $\Rightarrow q_0$
- one  $\mu$ -move  $\Rightarrow q_1$
- two  $\mu$ -moves  $\Rightarrow q_2$

## 8. ANOTHER EXAMPLE

Suppose:

TEXT

```
q 0  - -  $\mu$  - - >  q 1
q 0  - -  $\mu$  - - >  q 2
q 1  - -  $\mu$  - - >  q 3
q 2  h a s   n o    $\mu$  - e d g e
q 3  h a s   n o    $\mu$  - e d g e
```

Then:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

## MEMORY TRICK

$\mu$  - c l o s u r e = all free places I can reach

including the place where I already am.

## 9. $\mu$ - C L O S U R E SET

Suppose:

$$S = \{q_1, q_3\}$$

Then:

$$\epsilon\text{-closure}(S)$$

means union of  $\mu$  - c l o s u r e of all states in S:

$$\epsilon\text{-closure}(S) = \bigcup_{q \in S} \epsilon\text{-closure}(q)$$

## 10. SOLVED $\mu$ - C L O S U R E EXAMPLE

Suppose:

TEXT

```
q 0 - -  $\mu$  - - > q 1
q 1 - -  $\mu$  - - > q 2
q 1 - -  $\mu$  - - > q 3
q 3 - -  $\mu$  - - > q 4
```

Find:

$$\epsilon\text{-closure}(q_1)$$

Start with q1 itself:

$$\{q_1\}$$

From q1:

TEXT

```
 $\mu$  ! ' q 2
 $\mu$  ! ' q 3
```

Add:

$$\{q_2, q_3\}$$

From q3:

TEXT

$\mu$  ! ' q 4

Add q4.

Therefore:

$\epsilon$ -closure( $q_1$ ) = { $q_1, q_2, q_3, q_4$ }

## 11. HOW TO CALCULATE $\mu$ - CLOSURE

Use this algorithm:

TEXT

1. Put q itself in the set
2. Follow every  $\mu$ -edge
3. Add newly reached states
4. From those states, follow  $\mu$ -edges again
5. Repeat until nothing new appears

Do **not** follow transitions labelled 0, 1, a, etc.

Only  $\mu$ .

## 12. $\mu$ - N ~~FA~~ ACCEPTANCE

Because  $\mu$ -moves can occur:

- before reading input
- between input symbols
- after consuming the entire input

So acceptance must account for all possible  $\mu$ -reach states

A string is accepted if some computation can:

1. consume the entire string
2. take any allowable  $\mu$ -moves
3. finish in a final state

## 13. CAN $\mu$ BE ACCEPTED EVEN IF $q_0$ IS NOT FINAL?

For DFA and ordinary NFA:

$\epsilon$ -NFA  $L$  iff  $q_0 \in F$

But in an  $\mu$ -NFA this can change.

Suppose:

TEXT

! ' q 0 - -  $\mu$  - - > q 1 \*

$q_0$  is not final.

But without consuming any input:

TEXT

$q_0 \xrightarrow{\mu} q_1^*$

So  $\mu$  is accepted.

Therefore:

$\boxed{\epsilon \in L(M) \text{ iff } \epsilon \in \text{closure}(q_0) \cap F \neq \emptyset}$

This is extremely important.

## 14. EXAMPLE

TEXT

$q_0 \xrightarrow{\mu} q_1 \xrightarrow{\mu} q_2^*$

Then:

$\epsilon \in \text{closure}(q_0) = \{q_0, q_1, q_2\}$

Since  $q_2$  is final:

$\boxed{\epsilon \text{ is accepted}}$

## 15. TRACING AN $\epsilon$ -NFA

The safest method is:

**Closure !' Move !' Closure**

This is the central algorithm.

For each input symbol:

TEXT

```
Current states
    !"
μ - closure
    !"
consume one symbol
    !"
μ - closure again
```

Memory:

**FREE !' READ !' FREE**

## 16. WHY DO WE NEED CLOSURE AFTER READING?

Suppose:



TEXT

```
q0 --0--> q1
q 1 - - μ - - > q 2 *
```

Input:

TEXT

0

After consuming 0, you're at q1.

But you may then take:

TEXT

```
q 1 - - μ - - > q 2
```

So the string is accepted.

If you forget the second closure, you'll wrongly reject.

## 17. SOLVED TRACE

Consider:

TEXT

```
! ' q 0 - - μ - - > q 1
q1 --0--> q2
q 2 - - μ - - > q 3 *
```

Input:

TEXT

0

### Initial $\mu$ -closure

$\epsilon\text{-closure}(q_0) = \{q_0, q_1\}$

### Read 0

From  $q_0$  on 0:

$\emptyset$

From  $q_1$  on 0:

$\{q_2\}$

Move result:

$\{q_2\}$

### Take $\mu$ -closure again

$\epsilon\text{-closure}(q_2) = \{q_2, q_3\}$

$q_3$  is final.

Therefore:

$\boxed{\text{Accepted}}$

---

## 18. GENERAL MOVE FUNCTION

If current set is  $S$  and input symbol is  $a$ , first calculate:

$$\text{move}(S, a) = \bigcup_{q \in S} \delta(q, a)$$

Then take:

$$\boxed{\text{move}(S, a) \text{-closure}}$$

This gives the next configuration.

---

## 19. $\mu$ - NFA DFA CONVERSION

Now we combine:

- subset construction
- $\mu$  - closure

This is very important.

For ordinary NFA, DFA start state was:

$$\{q_0\}$$

But for  $\mu$  - NFA :

$$\boxed{S_0 = \text{move}(q_0) \text{-closure}}$$

This is the biggest change.

---

## 20. TRANSITION RULE

For DFA subset  $S$  and input symbol  $a$ :

### Step 1

Move on  $a$ :

$$\text{move}(S, a)$$

### Step 2

Take  $\mu$  - closure :

$$\boxed{\delta_D(S, a) = \text{move}(S, a) \text{-closure}}$$

Memory:

**Move first, then close**

---

## 21. FINAL-STATE RULE

Same as ordinary subset construction.

A DFA subset is accepting if it contains at least one  $\mu$  - N Final state:

$$\boxed{S \cap F \neq \emptyset}$$

## 22. COMPLETE CONVERSION ALGORITHM

### Step 1

Calculate:

$$A = \mu\text{-closure}(q_0)$$

This is DFA start state.

### Step 2

For every DFA subset  $S$  and every input  $a$ :

$$T = \text{move}(S, a)$$

### Step 3

Calculate:

$$\mu\text{-closure}(T)$$

That becomes destination DFA state.

### Step 4

Add new subset states as they appear.

### Step 5

Repeat until no new subsets appear.

### Step 6

Mark every subset containing any original final state as accepting.

## COMPACT MEMORY

TEXT

Start =  $\mu\text{-closure}(q_0)$

For each symbol:

```
Subset
!"
MOVE(symbol)
!"
μ - C L O S U R E
!"
Next subset
```

## 23. SOLVED CONVERSION EXAMPLE

Consider  $\mu$  - N F A :

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

Final:

$F = \{q_2\}$

Transitions:

```
TEXT
q 0 - - μ - - > q 1

q1 --0--> q1
q1 --1--> q2

q2 --1--> q2
```

No other transitions.

### STEP 1 — $\mu$ - C L O S U R E S

$\text{Epsilon-closure}(q_0) = \{q_0, q_1\}$

$\text{Epsilon-closure}(q_1) = \{q_1\}$

$\text{Epsilon-closure}(q_2) = \{q_2\}$

### STEP 2 — DFA START

$A = \{q_0, q_1\}$

### STEP 3 — FROM A ON 0

From  $q_0$  on 0:

$\emptyset$

From  $q_1$  on 0:

$\{q_1\}$

So:

$\text{move}(A, 0) = \{q_1\}$

Now closure:

$\text{closure}(\{q_1\}) = \{q_1\}$

Call:

$B = \{q_1\}$

Therefore:

$A \rightarrow B$

---

## FROM A ON 1

---

From  $q_0$  on 1:

$\emptyset$

From  $q_1$  on 1:

$\{q_2\}$

Move:

$\{q_2\}$

Closure:

$\{q_2\}$

Call:

$C = \{q_2\}$

Thus:

$A \rightarrow C$

---

## 24. PROCESS $B = \{q_1\}$

---

On 0:

$q_1 \rightarrow q_1$

Closure:

$\{q_1\} = B$

So:

$B \rightarrow B$

On 1:

$q_1 \rightarrow q_2$

Closure:

$\{q_2\} = C$

Thus:

$B \rightarrow C$

---

## 25. PROCESS C = {q2}

On 0:

No transition.

So:

$\emptyset$

Call:

$D = \emptyset$

On 1:

$q_2 \rightarrow q_2$

So:

$C \rightarrow C$

## PROCESS D

$\emptyset$

loops on all inputs:

TEXT

D --0--> D

D --1--> D

## 26. FINAL STATES

Original final:

$q_2$

Therefore subset containing  $q_2$  is final:

$C = \{q_2\}$

## FINAL DFA TABLE

| DFA state | NFA subset     | 0 | 1 |
|-----------|----------------|---|---|
| ! A       | $\{q_0, q_1\}$ | B | C |
| B         | $\{q_1\}$      | B | C |
| *C        | $\{q_2\}$      | D | C |
| D         | "              | D | D |

## 27. WHAT LANGUAGE DOES IT ACCEPT?

Let's inspect.

From A/B:

- any number of 0s can occur
- then a 1 reaches  $q_2$
- once in  $q_2$ , only 1s can continue

Therefore language resembles:

$\{0^*1^+\}$

except  $\mu$  is not accepted because start subset does not contain  $q_2$ .

Examples:

TEXT

```
1           '
0 1         '
0 0 1       '
0 1 1       '
0 0 0 1 1 1 '

μ           '
0           '
1 0         '
1 0 1       '
```

## 28. ANOTHER $\mu$ - CLOSURE EXAMPLE

Suppose:

TEXT

```
q 0  - - μ - - >  q 1
q 0  - - μ - - >  q 2
q 1  - - μ - - >  q 3
q 2  - - μ - - >  q 3
q 3  - - μ - - >  q 4
```

Then:

$\{\text{closure}\}(q_0) = \{q_0, q_1, q_2, q_3, q_4\}$

Even if multiple paths reach  $q_3$ ,  $q_3$  appears only once because closure is a set.

## 29. CYCLE IN $\mu$ - TRANSITIONS

Suppose:

TEXT

```
q 0  - - μ - - >  q 1
q 1  - - μ - - >  q 2
q 2  - - μ - - >  q 0
```

Then:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

Do not loop forever while calculating.

Once a state is already visited, don't process it again unnecessarily.

### 30. $\mu$ - CLOSURE DOES NOT FOLLOW INPUT EDGES

Suppose:

TEXT

```
q0 --μ--> q1
q1 --0--> q2
q2 --μ--> q3
```

Then:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1\}$$

NOT:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

Why?

The edge from  $q_1$  to  $q_2$  requires reading  $0$ .

$\mu$ -closure allows only  $\mu$ -moves.

### & μ - HUGE GATE TRAP

Do not mix:

$$\epsilon\text{-closure}$$

with ordinary reachability.

Closure =  $\mu$ only.

### 31. $\mu$ - NFAs VS DFA

| Feature                         | DFA                | NFA      | $\mu$ - NFA       |
|---------------------------------|--------------------|----------|-------------------|
| One transition per state-symbol | Exactly one        | 0/1/many | 0/1/many          |
| Missing transition              | No in complete DFA | Yes      | Yes               |
| Same symbol to multiple states  | No                 | Yes      | Yes               |
| $\mu$ -transition               | No                 | No       | Yes               |
| Acceptance paths                | One                | Many     | Many + free moves |
| Expressive power                | Regular            | Regular  | Regular           |



---

## THE IMPORTANT RESULT

$\boxed{\text{DFA} = \text{NFA} = \text{NFA}}$

in expressive power.

All recognize exactly:

$\boxed{\text{Regular Languages}}$

---

## 32. DOES $\mu$ -N HAVE MORE POWER?

No.

$\mu$ -transitions are just a convenience.

They make construction easier.

Any  $\mu$ -N can be converted to DFA.

So:

**More convenient syntax does not mean greater computational power.**

This distinction appears repeatedly in TOC.

---

## 33. $\mu$ -N IS AN ORDINARY NFA

You can also eliminate  $\mu$ -transitions without directly constructing a DFA.

The basic idea:

For each state  $q$  and symbol  $a$ :

1. take  $\mu\text{-closure}(q)$
2. from all those states, move on  $a$
3. take  $\mu\text{-closure}$  again
4. use those destinations as ordinary NFA transitions

Symbolically:

$\boxed{\delta'(q,a) = \mu\text{-closure}(\text{move}(\mu\text{-closure}(q), a))}$

---

## 34. FINAL STATES WHEN REMOVING $\mu$ -TRANSITIONS

This is important.

If:

$\mu\text{-closure}(q)$

contains some original final state, then  $q$  may need to become final in the  $\mu$ -NFA.

Why?

Because from  $q$ , the original  $\mu$ -N could reach final state without consuming input.

---

So:

$\mathbb{Q} \cap F'$

if:

$\mathbb{Q} \cap F \neq \emptyset$

## 35. EXAMPLE

Original:

TEXT

$q_0 \xrightarrow{\mu} q_1^*$

$q_0$  is non-final.

If we remove  $\mu$  without changing finals,  $\mu$  would stop being accepted.

So in equivalent ordinary NFA:

TEXT

$q_0$

must also be marked final.

## 36. GATE-STYLE QUESTION 1

Given:

TEXT

$q_0 \xrightarrow{\mu} q_1$   
 $q_1 \xrightarrow{\mu} q_2$   
 $q_2 \xrightarrow{1} q_3$

Find:

$\mathbb{Q} \cap F'$

Answer:

$\mathbb{Q} \cap F = \{q_0, q_1, q_2\}$

$q_3$  is not included because reaching it requires input 1.

## 37. GATE-STYLE QUESTION 2

Suppose:

$\mathbb{Q} \cap F = \{q_0, q_1, q_2\}$

and:

$F = \{q_2\}$

Is  $\mu$  accepted?

Yes.

Because:

$\epsilon\text{-closure}(q_0) \cap F = \{q_2\}$   
non-empty.

---

### 38. GATE-STYLE QUESTION 3

During  $\mu$ -NFA to DFA conversion:

$S = \{q_0, q_1\}$

For input  $a$ :

$\text{move}(S, a) = \{q_2\}$

and:

$\epsilon\text{-closure}(q_2) = \{q_2, q_3, q_4\}$

What is next DFA state?

$\boxed{\{q_2, q_3, q_4\}}$

---

### 39. GATE-STYLE QUESTION 4

An  $\mu$ -NFA has 4 states.

What is the maximum theoretical number of subset states after DFA conversion?

Still:

$2^4 = \boxed{16}$

$\mu$ -m o v e change how subsets are generated, not the basic maximum number of state subsets.

---

### 40. TYPICAL GATE/PYQ PATTERNS

#### Pattern 1 — Calculate $\mu$ -closure

Very common.

Be careful to include original state itself.

---

#### Pattern 2 — Does $\mu$ belong to the language?

Check:

$\epsilon\text{-closure}(q_0) \cap F$

---

#### Pattern 3 — Convert $\mu$ -NFA to DFA

Start with:

$\epsilon\text{-closure}(q_0)$

---

not  $\{q_0\}$ .

### Pattern 4 — Find next subset

Use:

TEXT  
MOVE  
then  
 $\mu$  - C L O S U R E

### Pattern 5 — Remove $\mu$ -transitions

Propagate symbol transitions through closures and update final states where necessary.

## 41. & pGATE TRAPS

## Trap 1

$\mu$  - trans consumes  $\mu$  as an actual input character.

'L Wrong.

It consumes nothing.

## Trap 2

$\mu$  must belong to  $\mathcal{E}$ .

'L Wrong.

$\mu$  is not an ordinary input symbol.

### Trap 3

$$\backslash[\backslash varepsilon\text{-closure}(q)\backslash]$$

doesn't contain  $q$  unless there's an  $\mu$ self-loop.

'LWrong.

Zero  $\mu$  - m o v a e s a l l o w e d.

Therefore  $q$  is always included.

## Trap 4

$\mu$  - c l o s e s all outgoing transitions.

'LWrong.

Only  $\mu$ -transitions.

---

### Trap 5

For  $\mu$  - NFA DFA, start state is  $\{q_0\}$ .

'L Usually wrong.

Correct:

$\{\text{closure}(q_0)\}$

---

### Trap 6

After consuming an input symbol, no closure is needed.

'L Wrong.

Take  $\mu$  - closure again.

---

### Trap 7

$\mu$  is accepted only if  $q_0$  is final.

'L Wrong for  $\mu$  - NFA.

Some final state may be  $\mu$  - reachable from  $q_0$ .

---

### Trap 8

$\mu$  - NFA is more powerful than NFA.

'L Wrong.

Same expressive power.

---

### Trap 9

An  $\mu$  - cycle causes infinitely many states in closure.

'L Wrong.

There are finitely many states; closure is a finite set.

---

### Trap 10

When removing  $\mu$  - transitions, final states never change.

'L Wrong.

A state whose  $\mu$  - closure reaches a final state may need to become final.

---

## 42. MEMORY TRICKS

---

## $\mu$ - transition

$\mu$  = free move

## $\mu$ - closure

All states reachable for free

## Conversion

FREE !' READ !' FREE

or:

Closure !' Move !' Closure

## Start subset

Start free

$\{ \text{closure}(q_0) \}$

## Acceptance of $\mu$

Can I reach final for free?

## 43. IMPORTANT RESULTS TO MEMORIZE

$\mu$  - NFA transition:

$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$

$\mu$  - closure always contains state itself:

$q \in \text{closure}(q)$

DFA start state:

$\{ \text{closure}(q_0) \}$

DFA transition:

$\delta_D(S, a) = \text{closure}(\text{move}(S, a))$

$\mu$  acceptance:

$\boxed{\epsilon \in L \iff \epsilon \in \text{closure}(q_0) \cap F \neq \emptyset}$

Power:

$\boxed{DFA = NFA = \epsilon\text{-NFA}}$

## 44. ' $\epsilon$ WRITE THIS IN YOUR NOTES

### $\epsilon$ - N F A

An  $\epsilon$  - N F A allows transitions without consuming input.

$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$

### $\epsilon$ - c l o s u r e

$\epsilon\text{-closure}(q)$

= all states reachable from  $q$  using **zero or more  $\epsilon$  - m o v e s**

Always:

$q \in \epsilon\text{-closure}(q)$

## Tracing rule

TEXT

```

 $\epsilon$  - c l o s u r e
    !
consume symbol
    !
 $\epsilon$  - c l o s u r e

```

Memory:

**FREE !' READ !' FREE**

### $\epsilon$ - N F A DFA

Start:

$\boxed{S_0 = \epsilon\text{-closure}(q_0)}$

Transition:

$\boxed{\delta_D(S, a) = \epsilon\text{-closure}(\text{move}(S, a))}$

Final subset:

$\boxed{S \cap F \neq \emptyset}$

## $\mu$ acceptance

$\boxed{\forall \epsilon \in L \iff \epsilon \in \text{closure}(q_0) \cap F \neq \emptyset}$

## Expressive power

$\boxed{\text{DFA} = \text{NFA} = \epsilon\text{-NFA}}$

All recognize regular languages.

## 45. 60-SECOND REVISION

- $\mu$ -transition consumes no input.
- $\mu$  is not an ordinary input symbol.
- $\mu$ -NFA allows  $\mu$ -moves.
- $\mu$ -closure( $q$ ) all states reachable using only  $\mu$ -moves.
- $q$  itself is always in  $\mu$ -closure( $q$ ).
- $\mu$ -cycles do not create infinite closure sets.
- For tracing: closure != symbol move != closure.
- For  $\mu$ -NFA DFA, start state is  $\mu$ -closure( $q_0$ ).
- Destination =  $\mu$ -closure(move(symbol, symbol)).
- Subset containing a final state is accepting.
- $\mu$  can be accepted even if  $q$  is non-final.
- $\mu$  accepted iff  $q \in \mu$ -closure( $q$ ) contains a final state.
- DFA, NFA and  $\mu$ -NFA have equal expressive power.

## 46. ACTIVE RECALL

### Q1

What does an  $\mu$ -transition consume?

### Q2

Is  $\mu$  necessarily part of  $\Sigma$ ?

### Q3

What is  $\mu$ -closure( $q$ )?

### Q4

Does  $q$  always belong to its own  $\mu$ -closure?

### Q5

During  $\mu$ -NFA DFA conversion, what is the start state?

### Q6

After moving on an input symbol, what must we do?



### Q7

Can  $\mu$  be accepted when  $q$  is non-final?

### Q8

How do you test whether  $\mu$  is accepted?

### Q9

Does  $\mu$  - NFA recognize more languages than DFA?

### Q10

What is the memory phrase for tracing?

## ANSWERS

1. Nothing.
2. No.
3. All states reachable from  $q$  using zero or more  $\mu$  - t r a n s i t i o n s .
4. Yes.
5.  
 $\bigcup \{ \text{\texttt{\textbackslash varepsilon}} \text{-closure}\}(q_0) \}$
6. Take  $\mu$  - c l o s u r e .
7. Yes.
8.  
 $\bigcup \{ \text{\texttt{\textbackslash varepsilon}} \text{-closure}\}(q_0) \} \cap F \neq \emptyset$
9. No.
10. FREE !' READ !' FREE.

## 47. MINI GATE TEST

### Q1 — MCQ

If:

TEXT

```
q 0  - - μ - - >  q 1
q 1  - - μ - - >  q 2
q2  --0--> q3
```

then:

$\bigcup \{ \text{\texttt{\textbackslash varepsilon}} \text{-closure}\}(q_0) \}$

is:

- A.  $\{q_0\}$  B.  $\{q_0, q_1\}$  C.  $\{q_0, q_1, q_2\}$  D.  $\{q_0, q_1, q_2, q_3\}$

## Q2 — MCQ

During  $\mu$ -NFA to DFA conversion, initial DFA state is:

- A.  $\{q_0\}$  always B.  $F$  C.  $\mu$ -closure of  $\{q_0\}$  D.  $(q_0)$

## Q3 — MSQ

Which statements are true?

- A.  $\mu$  transitions consumes no symbol. B.  $\mu$ -closure always contains  $q$ . C.  $\mu$ -NFA is strictly more powerful than DFA. D. A final state reachable from  $q$  using only  $\mu$ -moves causes  $\mu$  to be accepted.

## Q4 — NAT

An  $\mu$ -NFA has 5 states. What is the maximum number of possible DFA subset states?

## Q5 — TRICKY

Suppose:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_4\}$$

and:

$$F = \{q_4\}$$

Does the machine accept  $\mu$ ?

- A. Yes B. No

## Q6 — MCQ

For subset  $S$  and symbol  $a$ , equivalent DFA destination is:

A.

$$\text{move}(S, a)$$

only

B.

$$\epsilon\text{-closure}(S)$$

only

C.

$$\epsilon\text{-closure}(\text{move}(S, a))$$

D.

$$S \cup F$$

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\boxed{C}$

q3 requires consuming 0.

---

### Q2

$\boxed{C}$

---

### Q3

Correct:

$\boxed{A,B,D}$

---

### Q4

$2^5 = \boxed{32}$

---

### Q5

$\boxed{A}$

---

### Q6

$\boxed{C}$

---

## CHAPTER 3 — COMPLETED '

---

#### TEXT

```
[ ' ] N F A   f u n d a m e n t a l s
[ ' ] N F A   t r a n s i t i o n   f u n c t i o n
[ ' ] A c c e p t a n c e   b y   a n y   p a t h
[ ' ] S t a t e - s e t   t r a c i n g
[ ' ] D F A   v s   N F A
[ ' ] N F A   ! '   D F A
[ ' ] S u b s e t   c o n s t r u c t i o n
[ ' ] E m p t y   s u b s e t
[ ' ] 2 ^ n   u p p e r   b o u n d
[ ' ] μ - N F A
[ ' ] μ - t r a n s i t i o n s
[ ' ] μ - c l o s u r e
[ ' ] μ - N F A   t r a c i n g
[ ' ] μ   a c c e p t a n c e
[ ' ] μ - N F A   ! '   D F A
[ ' ] μ - t r a n s i t i o n   e l i m i n a t i o n   b a s i c s
[ ' ] D F A   =   N F A   =   μ - N F A   i n   e x p r e s s i v e   p o w e r
```

## NEXT CHAPTER

---

## CHAPTER 4 — REGULAR EXPRESSIONS

---

Next we will start from the very beginning:

---

**What is a Regular Expression !' " ,µ and alphabet-symbol REs !' union, concatenation, Kleene star !' operator precedence !' how to read REs !' how to construct REs for GATE languages.**

The most important connection we will eventually establish is:

$$\boxed{\text{Regular Expression}} \iff \text{Finite Automaton} \iff \text{Regular Language}$$

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing exactly from your uploaded TOC master prompt and the completed finite automata chapters.

## CHAPTER 4 — REGULAR EXPRESSIONS

### Topic 1: Regular Expression Fundamentals — " ,µ Symbols, Union, Concatenation, Kleene Star & Precedence

Priority: ~~0~~=~~VERY~~ HIGH

Regular Expressions are one of the three equivalent ways to describe a **regular language**:

$$\boxed{\text{Regular Expression}} \iff \text{Finite Automaton} \iff \text{Regular Language}$$

This connection is fundamental for GATE.

## 1. WHY DO WE NEED REGULAR EXPRESSIONS?

Suppose we want to describe:

**All binary strings ending in 01.**

We can draw a DFA.

But sometimes a symbolic description is much shorter.

For example:

$$(0+1)^*01$$

This is a regular expression.

It describes all binary strings that have:

TEXT

anything over {0,1}  
followed by  
01

## 2. INTUITION FIRST

Think of a Regular Expression as:

**a formula for generating strings in a regular language.**

Example:

$\backslash [ a^* \backslash ]$

generates:

TEXT

$\mu$   
a  
aa  
aaa  
aaaa  
...

Another:

$\backslash [ a+b \backslash ]$

means:

TEXT

choose a  
OR  
choose b

Another:

$\backslash [ ab \backslash ]$

means:

TEXT

a followed by b

## 3. VERY IMPORTANT — RE IS NOT A STRING

Suppose:

$\backslash [ R=(a+b)^* \backslash ]$

**R** is a **regular expression**.

It describes the language:

$\backslash [ L(R)=\{a,b\}^* \backslash ]$

which contains:

TEXT

$\mu$   
a

b  
aa  
ab  
ba  
bb  
aaa  
...

So distinguish:

TEXT

Regular Expression !' description  
Language !' set of strings

## 4. BASIC REGULAR EXPRESSIONS

For alphabet:

$\Sigma$

the simplest REs are:

1.  $\emptyset$
2.  $\epsilon$
3. each symbol  $a \in \Sigma$

Then larger REs are built using:

- union
- concatenation
- Kleene star

## 5. REGULAR EXPRESSION $\emptyset$

The RE:

$\emptyset$

represents the empty language:

$\boxed{L(\emptyset) = \emptyset}$

It generates no string.

## 6. REGULAR EXPRESSION $\epsilon$

The RE:

$\epsilon$

represents:

$\boxed{L(\epsilon) = \{\epsilon\}}$

Notice carefully:

The **expression** is:

$\mathbb{V}$

but the **language it denotes** is:

$\mathbb{V}$

---

## & pGATE TRAP

---

Do not write:

$L(\mathbb{V}) = \mathbb{V}$

if you mean a language.

Correct:

$\boxed{L(\mathbb{V}) = \mathbb{V}}$

because a language is a set.

---

## 7. SYMBOL AS REGULAR EXPRESSION

---

If:

$a \in \Sigma$

then  $a$  is an RE representing:

$\boxed{L(a) = \{a\}}$

Similarly:

$L(0) = \{0\}$

$L(1) = \{1\}$

---

## 8. THREE FUNDAMENTAL OPERATIONS

---

If  $R$  and  $S$  are regular expressions, then we can form:

### Union

$R + S$

or sometimes:

$R \mid S$

### Concatenation

$RS$

### Kleene Star

$R^*$

These three operations generate all regular expressions.

---

## MEMORY

RE = Choice + Sequence + Repetition

TEXT

```
+      !'   c h o i c e
R S    !'   s e q u e n c e
*      !'   r e p e t i t i o n
```

## PART A — UNION

### 9. UNION / ALTERNATION

$[ R+S ]$

means:

strings generated by R OR strings generated by S.

Formally:

$[ L(R+S)=L(R)\cup L(S) ]$

### Example

$[ R=a+b ]$

Then:

$[ L(R)=\{a,b\} ]$

It generates either:

TEXT

```
a
or
b
```

### Example

$[ R=ab+ba ]$

Then:

$[ L(R)=\{ab,ba\} ]$

## MEMORY



**+ = OR**

## 10. UNION DOES NOT MEAN CONCATENATION

$\{ a+b \}$

does **not** mean:

TEXT

ab

It means:

TEXT

a OR b

Whereas:

$\{ ab \}$

means:

TEXT

a followed by b

## PART B — CONCATENATION

### 11. CONCATENATION

If:

$\{ R \}$

and:

$\{ S \}$

are REs, then:

$\{ RS \}$

means:

**choose a string from  $L(R)$ , followed by a string from  $L(S)$ .**

Formally:

$\{ L(RS) = L(R)L(S) \}$

### Example

$\{ R=a \}$

$\{ S=b \}$

Then:

$\{ RS=ab \}$

and:

$\{ L(ab)=\{ab\} \}$

---

## Example

$\{ (a+b)c \}$

First choose:

TEXT

a or b

then append:

TEXT

c

So language:

$\{ \boxed{\{ac,bc\}} \}$

---

## 12. ANOTHER CONCATENATIONEXAMPLE

$\{ a(b+c) \}$

means:

TEXT

a followed by  
either b or c

So:

$\{ L(a(b+c)) = \{ab,ac\} \}$

---

## 13. CONCATENATIONIS GENERALLY NOT COMMUTATIVE

$\{ RS \neq SR \}$

in general.

Example:

$\{ R=a \}$

$\{ S=b \}$

Then:

$\{ RS=ab \}$

but:

$\{ SR=ba \}$

So different languages.

## PART C — KLEENE STAR

### 14. KLEENE STAR

$\{ R^* \}$

means:

zero or more repetitions of strings generated by R.

Formally:

$\{ L(R^*) = (L(R))^* \}$

## MEMORY

STAR = 0 or more

### 15. EXAMPLE — $a^*$

$\{ a^* \}$

generates:

$\{ \epsilon, a, aa, aaa, \dots \}$

Because we may use **a**:

TEXT

|     |   |   |   |   |   |   |   |       |
|-----|---|---|---|---|---|---|---|-------|
| 0   | t | i | m | e | s | ! | ' | $\mu$ |
| 1   | t | i | m | e |   | ! | ' | a     |
| 2   | t | i | m | e | s | ! | ' | a a   |
| 3   | t | i | m | e | s | ! | ' | a a a |
| ... |   |   |   |   |   |   |   |       |

### 16. EXAMPLE — $(ab)^*$

$\{ (ab)^* \}$

generates:

TEXT

μ  
ab  
abab  
ababab  
...

So:

$L((ab)^*) = \{\epsilon, ab, abab, ababab, \dots\}$

## 17. HUGE DIFFERENCE

Compare:

$(ab)^*$

and:

$a^*b^*$

They are completely different.

### $(ab)^*$

Strings:

TEXT

μ  
ab  
abab  
ababab  
...

Pattern **ab** repeats as a block.

### $a^*b^*$

Strings:

TEXT

μ  
a  
aa  
b  
bb  
ab  
aab  
abb  
aaabbb  
...

Any number of **a** s followed by any number of **b** s.

## & μGATE TRAP

$\nmid (ab)^n \neq a^nb^{n^*}$

## 18. STAR ALWAYS INCLUDES μ

For every RE  $R$ :

$\epsilon \in L(R^*)$

Why?

Because zero repetitions are allowed.

## Example

Even if:

$R = ab$

then:

$(ab)^*$

still includes  $\mu$ .

## 19. WHAT IS $\emptyset^*$ ?

From language closure rules:

$\emptyset^* = \{\epsilon\}$

As a regular expression,  $\emptyset^*$  denotes exactly the language containing  $\mu$ .

Therefore:

$L(\emptyset^*) = \{\epsilon\}$

## 20. WHAT IS $\mu^*$ ?

Repeated concatenation of  $\mu$  changes nothing:

$\epsilon\epsilon = \epsilon$

So:

$L(\epsilon^*) = \{\epsilon\}$

Thus:

$\emptyset^*$

and:

$\epsilon^*$

denote the same language.

---

## 21. POSITIVE CLOSURE IN RE

Sometimes:

$\backslash[ R^+ \backslash]$

means one or more repetitions.

Thus:

$\backslash[ \boxed{R^+ = RR^*} \backslash]$

Example:

$\backslash[ a^+ \backslash]$

generates:

TEXT

a  
aa  
aaa  
...

No  $\mu$  unless  $\mu$  can already arise from  $R$ .

---

## 22. STAR VS PLUS

| Operation | Meaning      |
|-----------|--------------|
| $R^*$     | zero or more |
| $R^+$     | one or more  |

Memory:

TEXT

\* starts from 0  
+ starts from 1

---

## PART D — OPERATOR PRECEDENCE

This is extremely important.

Suppose:

$\backslash[ a + b^* c \backslash]$

How should we interpret it?

Regular-expression operators have precedence.

Standard order:

$\backslash[ \boxed{\{$

- $\backslash; > \backslash; \text{concatenation} \backslash; > \backslash; +$   
} \]

Meaning:

### Highest

Kleene star.

### Next

Concatenation.

### Lowest

Union.

## MEMORY TRICK

### STAR !' STICK !' SELECT

- STAR first
- STICK together = concatenation
- SELECT using + last

Or simply:

$\backslash \boxed{* > \cdot > +} \backslash$

## 23. EXAMPLE OF PRECEDENCE

Expression:

$\backslash [ a + b^*c ] \backslash$

Step 1:

$\backslash [ b^* \backslash$

Step 2:

$\backslash [ b^*c \backslash$

Step 3:

$\backslash [ a + (b^*c) \backslash$

So it means:

#### TEXT

either a  
OR  
zero or more b's followed by c

## 24. ANOTHER EXAMPLE

$\forall ab^*$

means:

$\forall a(b^*)$

NOT:

$\forall (ab)^*$

Because  $*$  binds tighter than concatenation.

## Compare

$\forall (ab^*)$

Strings:

TEXT

a  
ab  
abb  
abbb  
...

$\forall ((ab)^*)$

Strings:

TEXT

$\mu$   
ab  
abab  
ababab  
...

Very different.

## 25. PARENTHESES CHANGE MEANING

Consider:

$\forall (a+b)^*$

Here star applies to whole group.

This generates:

**every finite string over  $\{a, b\}$ .**

So:

$\forall \boxed{(a+b)^* = \{a, b\}^*}$

as languages.



## Examples

TEXT

$\mu$   
a  
b  
aa  
ab  
ba  
bb  
aba  
bab  
...

## 26. WHAT DOES $a+b^*$ MEAN?

Because star has higher precedence:

$\backslash[ a+b^* \backslash]$

means:

$\backslash[ a+(b^*) \backslash]$

Language:

TEXT

a  
 $\mu$   
b  
bb  
bbb  
...

So:

$\backslash[ \boxed{L(a+b^*)} = \{a, \text{\texttt{\textbackslash varepsilon}}, b, bb, bbb, \text{\texttt{\textbackslash dots}}\} \backslash]$

## 27. WHAT DOES $(a+b)^*$ MEAN?

Completely different:

$\backslash[ (a+b)^* \backslash]$

means any sequence of  $a$  and  $b$ .

Therefore:

$\backslash[ \boxed{\Sigma^*} \backslash]$

for:

$\backslash[ \Sigma = \{a, b\} \backslash]$

## PART E — HOW TO READ AN RE

Use this decoding strategy.

Suppose:

$\mathcal{V}[(0+1)^*01]$

Read from structure.

**$(0+1)$**

choose:

TEXT

0 or 1

**$(0+1)^*$**

repeat that choice zero or more times.

Therefore:

**any binary string.**

Then append:

TEXT

01

So:

**all binary strings ending in 01.**

## 28. EXAMPLE

$\mathcal{V}[1(0+1)^*]$

Interpret:

TEXT

first symbol = 1  
then any binary string

Therefore:

**all non-empty binary strings starting with 1.**

## 29. EXAMPLE

$\mathcal{V}[(0+1)^*1]$

Interpret:

TEXT

any binary string  
followed by 1

Therefore:

**all binary strings ending in 1.**

## 30. EXAMPLE

$\mathbb{V}[(0+1)^*01(0+1)^*]$

Interpret:

TEXT

anything  
then 01  
then anything

Therefore:

**all binary strings containing substring 01 .**

This is a very important template.

## 31. MASTER PATTERN TEMPLATES

For:

$\mathbb{V}[\Sigma=\{0,1\}]$

we can write:

$\mathbb{V}[\Sigma^*=(0+1)^*]$

### Starts with P

$\mathbb{V}[\boxed{P(0+1)^*}]$

Example starts with 01 :

$\mathbb{V}[01(0+1)^*]$

### Ends with P

$\mathbb{V}[\boxed{(0+1)^*P}]$

Example ends with 01 :

$\mathbb{V}[(0+1)^*01]$

## Contains P

$\boxed{(0+1)^*P(0+1)^*}$

Example contains 101:

$(0+1)^*101(0+1)^*$

## MEMORY

### TEXT

Starts with P:

P  $\epsilon$  \*

Ends with P:

$\epsilon$  \* P

Contains P:

$\epsilon$  \* P  $\epsilon$  \*

This is worth memorizing.

## 32. EXACT STRING

If language contains only:

### TEXT

101

then RE is simply:

$\boxed{101}$

No star is needed.

## 33. FINITE SET OF STRINGS

Suppose:

$L = \{0, 11, 101\}$

Then:

$\boxed{0+11+101}$

is a regular expression.

Union simply lists the alternatives.

## 34. ZERO OR ONE OCCURRENCE

Regular-expression notation in pure TOC often avoids ?.

If we want optional a:

TEXT

$\mu$    o   r   a

write:

$\boxed{\text{\texttt{\textbackslash varepsilon+a}}}$

Example:

$\text{\texttt{b(\textbackslash varepsilon+a)c}}$

generates:

TEXT

bc  
bac

## 35. EXACTLY $n$ REPETITIONS

For exactly three a's:

$\text{\texttt{aaa}}$

or:

$\text{\texttt{a^3}}$

if exponent shorthand is permitted.

But be careful:

In formal regular expressions, powers are shorthand rather than primitive operators.

## 36. AT LEAST ONE a

$\text{\texttt{a^+}}$

or:

$\text{\texttt{aa^*}}$

Since:

TEXT

one mandatory a  
then zero or more additional a's

Thus:

$\boxed{\text{\texttt{a^+=aa^*}}}$

## 37. AT LEAST TWO a's CONSECUTIVELY

$\boxed{\text{\texttt{aaa^*}}}$

which is:

TEXT

aa  
aaa  
aaaa  
...

Another way:

$\boxed{a^2a^*}$

## PART F — IMPORTANT ALGEBRAIC IDENTITIES

These are very useful for simplifying REs.

### 38. UNION WITH $\emptyset$

$\boxed{R + \emptyset = R}$

Because union with empty language adds nothing.

### 39. CONCATENATION WITH $\epsilon$

$\boxed{R \epsilon = \epsilon R = R}$

because  $\epsilon$  is concatenation identity.

### 40. CONCATENATION WITH $\emptyset$

$\boxed{R \emptyset = \emptyset R = \emptyset}$

because there is no string to concatenate from empty language.

### 41. DUPLICATE UNION

$\boxed{R + R = R}$

Because union is set union.

### 42. UNION COMMUTATIVITY

$\boxed{R + S = S + R}$

as languages.

### 43. CONCATENATION IS NOT COMMUTATIVE

Generally:

$\boxed{RS \neq SR}$

---

## 44. STAR OF STAR

---

$\boxed{(R^+)^+ = R^+}$

Once arbitrary repetition is already allowed, starring again adds nothing.

---

## 45. $\mu + RR^+$

---

A very useful identity:

$\boxed{R^+ = \epsilon + RR^+}$

Also:

$\boxed{R^+ = \epsilon + R^+R}$

Interpretation:

A starred sequence contains either:

- zero copies of  $\mu$
  - one R plus more R's
- 

## 46. $RR^+$

---

$\boxed{R^+R^+ = R^+}$

Concatenating two arbitrary sequences of R still produces an arbitrary sequence of R.

---

## 47. $\epsilon^+$

---

$\boxed{\emptyset^+ = \epsilon}$

Here this equality means the two REs denote the same language:

$\boxed{\{\epsilon\}}$

Strictly, it is useful to say:

$\boxed{L(\emptyset^+) = L(\epsilon)}$

---

## 48. $\epsilon^+$

---

$\boxed{\epsilon^+ = \epsilon}$

in language-equivalence sense.

---

## 49. IMPORTANT — EXPRESSION EQUALITY VS LANGUAGE EQUIVALENCE

---

Two different regular expressions may describe the same language.

Example:

---

$\emptyset$

and:

$R$

look different syntactically.

But:

$L(R \cup \emptyset) = L(R)$

So when we write:

$R \cup \emptyset = R$

we usually mean **equivalent regular expressions**, i.e. same language.

## 50. SOLVED EXAMPLE 1

What language does:

$0^*1$

describe?

$0^*$ :

TEXT

$\mu$   
 $\emptyset$   
 $\emptyset\emptyset$   
 $\emptyset\emptyset\emptyset$   
 $\dots$

Append mandatory  $1$ :

TEXT

$1$   
 $\emptyset 1$   
 $\emptyset\emptyset 1$   
 $\emptyset\emptyset\emptyset 1$   
 $\dots$

Therefore:

**zero or more 0s followed by exactly one 1.**

Formally:

$\boxed{0^n 1 \mid n \geq 0}$

## 51. SOLVED EXAMPLE 2

Interpret:

$01^*$

Since star applies only to  $1$ :



$\mathbb{V} [ 0(1^*) ]$

Strings:

TEXT

0  
01  
011  
0111  
...

NOT:

TEXT

μ  
01  
0101  
...

That second pattern would be:

$\mathbb{V} [ (01)^* ]$

---

## 52. SOLVED EXAMPLE 3

Interpret:

$\mathbb{V} [ (0+1)^*00 ]$

First part:

$\mathbb{V} [ (0+1)^* ]$

= any binary string.

Then mandatory 00.

Therefore:

$\mathbb{V} [ \boxed{\text{all binary strings ending in } 00} ]$

---

## 53. SOLVED EXAMPLE 4

Write RE for all strings over  $\{a, b\}$  starting with a.

Use:

$\mathbb{V} [ \Sigma^* = (a+b)^* ]$

Therefore:

$\mathbb{V} [ \boxed{a(a+b)^*} ]$

---

## 54. SOLVED EXAMPLE 5

Write RE for strings ending in ab.

$\mathbb{V} [ \boxed{(a+b)^*ab} ]$

## 55. SOLVED EXAMPLE 6

Write RE for strings containing `aba`.

$\boxed{(a+b)^*aba(a+b)^*}$

## 56. SOLVED EXAMPLE 7 — ONLY a's OR ONLY b's

Language:

strings consisting entirely of a's OR entirely of b's.

RE:

$\boxed{a^+b^+}$

Be careful.

This is not:

$\boxed{(a+b)^*}$

because  $(a+b)^*$  allows mixtures such as:

TEXT

ab  
ba  
aab  
baba

while:

$\boxed{a^+b^+}$

allows only:

TEXT

μ  
a  
aa  
...  
b  
bb  
...

## 57. SOLVED EXAMPLE 8 — a's FOLLOWED BY b's

Language:

$\boxed{a^mb^n \mid m,n \geq 0}$

RE:

$\boxed{a^+b^+}$

Examples:

TEXT

$\mu$   
a  
aaa  
b  
bbb  
ab  
aab  
abb  
aaabbb

But not:

TEXT

ba  
abab

## 58. GATE-STYLE QUESTION 1

Which language does:

$\Sigma^* (0+1)^* 1 \Sigma^*$

represent?

A. Strings starting with 1 B. Strings ending with 1 C. Strings containing exactly one 1 D. Strings containing no 1

Answer:

$\boxed{B}$

## 59. GATE-STYLE QUESTION 2

Which expression represents all binary strings containing substring 00?

A.

$\Sigma^* 00(0+1)^* \Sigma^*$

B.

$\Sigma^* (0+1)^* 00 \Sigma^*$

C.

$\Sigma^* (0+1)^* 00(0+1)^* \Sigma^*$

D.

$\Sigma^* 0^* 0^* \Sigma^*$

Answer:

$\boxed{C}$

## 60. GATE-STYLE QUESTION 3

Which of these includes  $\mu$ ?

A.

$\backslash [ab^* \backslash]$

B.

$\backslash [(ab)^* \backslash]$

C.

$\backslash [a+b \backslash]$

D.

$\backslash [ab \backslash]$

Answer:

$\backslash \boxed{B} \backslash$

Because star permits zero repetitions of the entire block  $ab$ .

## 61. GATE-STYLE QUESTION 4

What is:

$\backslash [L(\emptyset^*) \backslash]$

A. " B.  $\{ \mu C \} \{ " D \} \epsilon^*$

Answer:

$\backslash \boxed{B} \backslash$

## 62. GATE-STYLE QUESTION 5 — PRECEDENCE

Interpret:

$\backslash [a+bc^* \backslash]$

Using:

$\backslash [*> \text{concatenation} > + \backslash]$

we get:

$\backslash [a+(b(c^*)) \backslash]$

So strings are either:

TEXT

a

or:

TEXT

b

bc

bcc

bccc

...

---

## 63. TYPICAL GATE/PYQ PATTERNS

---

GATE commonly tests:

### Pattern 1 — Identify language of an RE

Translate symbolically into plain English.

---

### Pattern 2 — Choose correct RE for a language

Especially:

- begins with
  - ends with
  - contains substring
  - restrictions on symbols
  - count/parity patterns
- 

### Pattern 3 — Operator precedence

Especially differences like:

$\backslash [ ab^* ]$

versus:

$\backslash (ab)^* ]$

---

### Pattern 4 — $\mu$ and $\epsilon$

Examples:

$\backslash [ \emptyset^* ]$

$\backslash [ \epsilon^* ]$

$\backslash [ R \emptyset ]$

$\backslash [ R^+ \emptyset ]$

---

### Pattern 5 — Equivalent expressions

Determine whether two REs describe the same language.

---

---

## 64. GATE TRAPS

---

### Trap 1

$\backslash [ a+b=ab ]$

'LWrong.

$+$  means OR.

---

---

## Trap 2

---

$\nmid (ab)^n = a^n b^n$

'L Wrong.

---

## Trap 3

---

$\nmid ab^n = (ab)^n$

'L Wrong.

Star applies only to nearest expression unless parentheses expand its scope.

---

## Trap 4

---

$R^*$  excludes  $\mu$ .

'L Wrong.

Star always allows zero repetitions.

---

## Trap 5

---

$\nmid \emptyset^n = \emptyset$

'L Wrong.

It denotes:

$\nmid \{\varepsilon\}$

---

## Trap 6

---

$\nmid L(\varepsilon) = \varepsilon$

'L Type confusion.

Correct:

$\nmid L(\varepsilon) = \{\varepsilon\}$

---

## Trap 7

---

$\nmid (a+b)^n$

means either all a's or all b's.

'L Wrong.

It allows arbitrary mixing.

All a's OR all b's is:

$\nmid a^+b^+$

---

---

## Trap 8

Concatenation has lower precedence than union.

'L Wrong.

Correct:

$\text{[ } * \text{> \text{\texttt{concatenation}} \text{>+ } ]}$

---

## Trap 9

$\text{[ } \Sigma^* P \text{ ]}$

means strings starting with P.

'L Wrong.

It means ending with P.

Starting with P:

$\text{[ } P \Sigma^* \text{ ]}$

---

## Trap 10

Two syntactically different REs necessarily describe different languages.

'L Wrong.

Many different REs can be equivalent.

---

## 65. MASTER TEMPLATE TABLE

Assume:

$\Sigma = \{0, 1\}$

so:

$\Sigma^* = (0+1)^*$

| Requirement       | Regular Expression |
|-------------------|--------------------|
| Any binary string | $(0+1)^*$          |
| Starts with 1     | $1(0+1)^*$         |
| Ends with 1       | $(0+1)^*1$         |
| Contains 1        | $(0+1)^*1(0+1)^*$  |
| Starts with 01    | $01(0+1)^*$        |
| Ends with 01      | $(0+1)^*01$        |
| Contains 01       | $(0+1)^*01(0+1)^*$ |

| Requirement        | Regular Expression |
|--------------------|--------------------|
| Only 0s            | 0*                 |
| Only 1s            | 1*                 |
| All 0s then all 1s | 0*1*               |
| Only 0s OR only 1s | 0* + 1*            |

## 66. SHORTCUTS & MEMORY TRICKS

### Union

**+ = Pick one**

### Concatenation

**Touching = Followed by**

TEXT

ab

=

a followed by b

### Star

**Star Starts at Zero**

### Precedence

$\boxed{^* > \cdot > +}$

Memory:

**Star !' Stick !' Select**

### Standard templates

TEXT

Starts with P !' P £ \*  
Ends with P !' £ \* P



## 67. IMPORTANT RESULTS TO MEMORIZE

$$L(\emptyset) = \emptyset$$

$$L(\epsilon) = \{\epsilon\}$$

$$L(a) = \{a\}$$

$$L(R+S) = L(R) \cup L(S)$$

$$L(RS) = L(R)L(S)$$

$$L(R^*) = (L(R))^*$$

And:

$$L(\text{concatenation}) = L(R) \cup L(S)$$

## 68. RE ALGEBRA — MUST REMEMBER

$$R + \emptyset = R$$

$$R \epsilon = R$$

$$R \emptyset = \emptyset$$

$$R + R = R$$

$$(R^*)^* = R^*$$

$$R^* R^* = R^*$$

$$\emptyset^* \equiv \epsilon$$

$$\epsilon^* \equiv \epsilon$$

where  $\equiv$  means language-equivalent.

## 69. WRITE THIS IN YOUR NOTES

### Regular Expression

An RE is a symbolic representation of a regular language.

Basic REs:

$$\emptyset, \epsilon, a \in \Sigma$$

### Meaning

$$L(\emptyset) = \emptyset$$

$$L(\epsilon) = \{\epsilon\}$$

$$\mathbb{L}(a) = \{a\}$$

## Operators

### Union

$$\mathbb{L}(R+S)$$

means OR.

$$\mathbb{L}(R+S) = \mathbb{L}(R) \cup \mathbb{L}(S)$$

### Concatenation

$$\mathbb{L}(RS)$$

means R followed by S.

### Kleene Star

$$\mathbb{L}(R^*)$$

means zero or more repetitions.

## Precedence

$$\boxed{(\text{concatenation})^+}$$

Memory:

Star !' Stick !' Select

## Standard binary templates

$$\Sigma^{(0+1)^*}$$

Starts with P:

$$P\Sigma^*$$

Ends with P:

$$\Sigma^*P$$

Contains P:

$$\Sigma^*P\Sigma^*$$

## Important distinctions

$$(ab)^n \neq a^n b^n$$

$$a^n \neq (ab)^n$$

$$a^n + b^n \neq (a+b)^n$$

## 70. 60-SECOND REVISION

---

- RE describes a regular language.
  - Basic REs:  $\epsilon$ ,  $\mu$  alphabet symbols.
  - $L(\epsilon) = \{\epsilon\}$
  - $L(\mu) = \{\mu\}$
  - $+$  means union/OR.
  - Juxtaposition means concatenation.
  - $*$  means zero or more.
  - $R^*$  always contains  $\epsilon$ .
  - $R^+$  means one or more.
  - Precedence is  $*$  > concatenation >  $+$ .
  - $(ab)^*$  differs from  $a^*b^*$ .
  - $(a+b)^*$  means all strings over  $\{a, b\}$ .
  - $a^*+b^*$  means only a's OR only b's.
  - Starts with P !'  $P \epsilon^*$
  - Ends with P !'  $\epsilon^* P$
  - Contains P !'  $\epsilon^* P \epsilon^*$
  - $\epsilon$  denotes  $\{\epsilon\}$
  - Different REs may describe the same language.
- 

## 71. ACTIVE RECALL

---

Try before checking.

### Q1

What does  $+$  mean in a regular expression?

### Q2

What does concatenation mean?

### Q3

Does  $R^*$  always contain  $\mu$ ?

### Q4

Which has higher precedence:  $*$  or concatenation?

### Q5

Interpret:

$\epsilon[(0+1)^*01\epsilon]$

### Q6

Write an RE for binary strings starting with  $10$ .

### Q7

Write an RE for binary strings containing  $11$ .

### Q8

What language does:

$\bigcup \{0^*1^*\}$

represent?

### Q9

Is:

$\bigcup \{(ab)^n = a^n b^n \mid n \geq 0\}$

true?

### Q10

What does:

$\bigcup \{\emptyset^* \}$

denote?

---

## ANSWERS

1. Union / OR.
2. One generated string followed by another.
3. Yes.
4.  $*$ .
5. Binary strings ending in  $01$ .

6.

$\bigcup \{10(0+1)^*\}$

7.

$\bigcup \{(0+1)^*11(0+1)^*\}$

8. Any number of 0s followed by any number of 1s.

9. No.

10.

$\bigcup \{\epsilon\}$

---

## 72. MINI GATE TEST

### Q1 — MCQ

Which RE describes all strings over  $\{0, 1\}$  ending in  $10$ ?

A.

$\bigcup \{10(0+1)^*\}$

B.

$\bigcup \{(0+1)^*10\}$

C.

$$\{ (0+1)^{10}(0+1)^* \}$$

D.

$$\{ 10 \}$$

---

## Q2 — MCQ

Which expression includes  $\mu$  ?

- A.  $01^*$  B.  $(01)^*$  C.  $01$  D.  $0+1$
- 

## Q3 — MSQ

Which statements are true?

A.

$$\{ R + \emptyset \} \equiv R$$

B.

$$\{ R \emptyset \} \equiv R$$

C.

$$\{ R \vee \epsilon \} \equiv R$$

D.

$$\{ (R^*) \} \equiv R^*$$

---

## Q4 — MCQ

What does:

$$\{ (0+1)^{101}(0+1)^* \}$$

represent?

- A. Starts with 101 B. Ends with 101 C. Contains 101 D. Exactly equals 101
- 

## Q5 — MCQ

According to standard RE precedence, which is evaluated first?

- A. Union B. Concatenation C. Kleene star D. All equally
- 

## Q6 — TRICKY

Which pair is generally **not equivalent**?

A.

$$\{ R + \emptyset, R \}$$

---

B.

$\mathbb{R} \setminus \{\epsilon\}$

C.

$R^+ \cup R^*$

D.

$(RS)^+ \cup R^+ S^*$

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\boxed{B}$

### Q2

$\boxed{B}$

because  $(01)^*$  allows zero copies.

### Q3

Correct:

$\boxed{\{A, C, D\}}$

B is false because:

$\emptyset \in R$

### Q4

$\boxed{C}$

### Q5

$\boxed{C}$

### Q6

$\boxed{D}$

Generally:

$(RS)^+ \neq R^+ S^*$

## CHAPTER 4 PROGRESS

### TEXT

```
[ ' ] why regular expressions are needed
[ ' ] RE vs language
[ ' ] "
[ ' ] μ
[ ' ] Alphabet symbols
[ ' ] Union
[ ' ] Concatenation
[ ' ] Kleene star
```

```

[ ' ] P o s i t i v e   c l o s u r e
[ ' ] O p e r a t o r   p r e c e d e n c e
[ ' ] B a s i c   R E   i d e n t i t i e s
[ ' ] S t a r t s - w i t h   p a t t e r n
[ ' ] E n d s - w i t h   p a t t e r n
[ ' ] C o n t a i n s - s u b s t r i n g   p a t t e r n

[ ] A d v a n c e d   R E   c o n s t r u c t i o n
[ ] E x a c t l y / a t   l e a s t / a t   m o s t   o c c u r r e n c e s
[ ] E v e n / o d d   o c c u r r e n c e s
[ ] A v o i d i n g   s u b s t r i n g s
[ ] R e s t r i c t i o n s   o n   c o n s e c u t i v e   s y m b o l s
[ ] R E   e q u i v a l e n c e   t r i c k s
[ ] R e g u l a r   E x p r e s s i o n   ! '   μ - N F A
[ ] F i n i t e   A u t o m a t o n   ! '   R e g u l a r   E x p r e s s i o n

```

## NEXT TOPIC

### 0=10 ADVANCED REGULAR-EXPRESSION CONSTRUCTION

Next we will build GATE-level REs for conditions such as:

- exactly one 1
- exactly two 1s
- at least one / at least two occurrences
- even number of symbols
- odd number of symbols
- every 0 followed by 1
- no consecutive 1s
- strings not containing a pattern
- parity/count restrictions

This is where RE questions become tricky.

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing from your uploaded TOC master prompt and the Regular Expression fundamentals section.

## CHAPTER 4 — REGULAR EXPRESSIONS

### Topic 2: Advanced Regular Expression Construction for GATE

Priority: 0=10 VERY HIGH

This is where Regular Expressions become exam-oriented.

The main challenge is not remembering syntax.

The challenge is:

**Translate a language condition into a correct pattern.**

The safest method is:

TEXT

Understand condition

!"

Break into mandatory pieces

!"

Insert flexible parts

!"

Check edge cases

!"

Test accepted/rejected strings

## 1. GOLDEN METHOD FOR BUILDING AN RE

Whenever you see a language condition, ask:

### Step 1

What symbols are mandatory?

### Step 2

What can appear before them?

### Step 3

What can appear between them?

### Step 4

What can appear after them?

### Step 5

Can  $\mu$  occur?

### Step 6

Does the RE accidentally generate forbidden strings?

This last step is very important.

## PART A — EXACTLY ONE OCCURRENCE

## 2. EXACTLY ONE 1

Alphabet:

$\Sigma = \{0,1\}$

Language:



strings containing exactly one 1.

Before the one mandatory 1, only zeros may appear.

After it, again only zeros may appear.

Therefore:

$\boxed{0^*10^*}$

## Examples accepted

TEXT

1  
01  
10  
00100  
0001000

Rejected:

TEXT

μ  
0  
11  
101  
111

## MEMORY

Exactly one 1 = zeros + one 1 + zeros

$\boxed{0^*10^*}$

### 3. EXACTLY ONE 0

Symmetric:

$\boxed{1^*01^*}$

### 4. EXACTLY TWO 1s

We need two mandatory 1s.

Everything else must be 0.

Therefore:

$\boxed{0^*10^*10^*}$

## Accepted

TEXT

11  
011  
110  
0101  
00100100

Rejected:

TEXT

1  
111  
1011  
 $\mu$

## GENERAL PATTERN

Exactly  $k$  occurrences of **1**:

TEXT

$0^* 1 0^* 1 0^* \dots 1 0^*$

with exactly  $k$  mandatory **1**s.

## PART B — AT LEAST

### 5. AT LEAST ONE **1**

We need one mandatory **1**.

Before and after it, anything may appear.

So:

$\boxed{(0+1)^*1(0+1)^*}$

This is equivalent to:

strings containing **1**.

### 6. ANOTHER RE FOR AT LEAST ONE **1**

We can also write:

$0^*1(0+1)^*$

Why does this work?

Before the **first chosen 1**, only zeros need to occur.

After that, anything is allowed.

This is also correct.

## 7. AT LEAST TWO 1s

Need two mandatory ones, but arbitrary strings may appear around/between.

One natural RE:

$\boxed{(0+1)^1(0+1)^1(0+1)^*}$

This guarantees at least two 1s.

## But there is a more structured form

Choose first and second mandatory 1:

$\boxed{0^1(0+1)^1(0+1)^*}$

Both can work depending on interpretation of chosen occurrences.

The first version is easiest to reason about.

## 8. AT LEAST k OCCURRENCES

General idea:

place k mandatory symbols and allow arbitrary strings around them.

For at least three 1s:

$\boxed{(0+1)^1(0+1)^1(0+1)^1(0+1)^*}$

## PART C — AT MOST

### 9. AT MOST ONE 1

Means:

TEXT

zero 1s

OR

exactly one 1

Zero ones:

$\boxed{0^*}$

Exactly one:

$\boxed{0^10^*}$

Therefore:

$\boxed{0^+ + 0^+10^{+*}}$

Factorized:

$\boxed{0^{(\text{varepsilon}+10^+)}}$

Both are correct.

---

## 10. AT MOST TWO 1s

Could be:

- 0 ones
- 1 one
- 2 ones

Therefore:

$\boxed{0^+ + 0^+10^+ + 0^+10^+10^+}$

This is correct but somewhat long.

For GATE, correctness matters more than forcing a fancy form.

---

## MEMORY

At most  $k = \text{union of } 0,1,\dots,k \text{ occurrences}$

---

## PART D — EVEN NUMBER OF SYMBOLS

---

### 11. EVEN NUMBER OF 1s

This is a classic RE construction.

We need 1s to occur in pairs.

But zeros may occur anywhere.

One useful RE is:

$\boxed{0^+(10^+10^+)^*}$

Let's understand it.

Initial:

$0^{+*}$

allows zeros before any ones.

Inside:

$10^+10^+$

each block contributes exactly two 1s.

Repeating the block gives:

#### TEXT

0 ones  
2 ones  
4 ones  
6 ones  
...

Hence even number of 1s.

## Test

$\mu$

Choose zero repetitions !' accepted.

000

Initial 0\* !' accepted.

11

One pair !' accepted.

101

Two 1s !' accepted.

11001

Three 1s !' rejected.

## 12. ALTERNATIVE EVEN-1 RE

Another equivalent form:

$\boxed{0^{(10^{10})^*}}$

is usually the easiest exam form.

Remember the principle rather than memorizing only the formula:

**Pair the counted symbol. Let uncounted symbols float between pairs.**

## 13. ODD NUMBER OF 1s

Odd means:

**one mandatory 1 plus an even number of additional 1s.**

A clean form:

$\boxed{0^1 1 0^{(10^{10})^*}}$

Another logically equivalent construction may be produced by reorganizing where the mandatory 1 appears.

The key principle:

$$\text{Odd} = 1 + \text{Even}$$

## MEMORY

TEXT

Even !' pairs  
Odd !' one + pairs

## PART E — EVEN/ODD STRING LENGTH

### 14. EVEN LENGTH OVER $\{0,1\}$

Every two symbols form one block.

Any binary symbol is:

$$(0+1)$$

Two-symbol block:

$$(0+1)(0+1)$$

Repeat zero or more times:

$$\boxed{((0+1)(0+1))^*}$$

or compactly:

$$\boxed{((0+1)^2)^*}$$

## Examples

TEXT

|         |   |
|---------|---|
| $\mu$   | ' |
| 0 0     | ' |
| 0 1     | ' |
| 1 0     | ' |
| 1 1     | ' |
| 1 0 1 0 | ' |

Odd-length strings reject.

### 15. ODD LENGTH

One mandatory symbol plus any number of two-symbol blocks:

$$\boxed{(0+1)((0+1)(0+1))^*}$$

Examples:

TEXT

0  
1  
000  
101  
11111

## MEMORY

Length parity = group symbols in blocks of 2

## PART F — START/END COMBINATIONS

### 16. STARTS WITH 0 AND ENDS WITH 1

For strings of length at least 2:

$\boxed{0(0+1)^*1}$

Examples:

TEXT

01  
001  
011  
0101

### 17. STARTS AND ENDS WITH SAME SYMBOL

Two cases:

#### Start/end with 0

$\boxed{0(0+1)^*0}$

#### Start/end with 1

$\boxed{1(0+1)^*1}$

But what about strings of length 1?

TEXT

0  
1

They also start and end with the same symbol.

So full language:

$\boxed{0 + 1 + 0(0+1)^*0 + 1(0+1)^*1}$

If  $\mu$  is also considered to satisfy a particular convention, the question would need to specify it; normally "starts/ends"

presumes nonempty strings.

---

## & pGATE LESSON

---

Always check short strings separately.

RE constructions often accidentally miss:

TEXT

$\mu$   
length 1  
length 2

---

## 18. STARTS AND ENDS WITH DIFFERENT SYMBOLS

---

Cases:

TEXT

starts 0, ends 1  
OR  
starts 1, ends 0

RE:

$\boxed{0(0+1)^*1 + 1(0+1)^*0}$

---

## PART G — NO CONSECUTIVE 1s

---

## 19. LANGUAGE

---

Binary strings containing no substring:

$\boxed{11}$

Examples:

TEXT

$\mu$   
0  
1  
00  
01  
10  
0101  
1010

Rejected:

TEXT

11  
110  
011



1011

## BUILD THE RE

Every 1 except possibly the last must be followed by 0.

Think of safe blocks:

TEXT

0  
10

We may repeat them:

$\mathcal{V}[(0+10)^*]$

But this only generates strings ending in 0 or  $\mu$ .

We may optionally add one final 1.

So:

$\mathcal{V}[\boxed{(0+10)^*(\vepsilon+1)}]$

This is a very important standard RE.

## Test 101

Choose:

TEXT

10

from repeated block, then final optional 1.

Accepted.

## Test 1

Zero repeated blocks, then choose 1.

Accepted.

## Test 11

Impossible.

Good.

## MEMORY

No 11 !' every 1 is paired with following 0, except possibly the final 1

$\mathbb{N}[(0+10)^*(\text{varepsilon}+1)]$

## 20. NO CONSECUTIVE 0s

By symmetry:

$\mathbb{N}[\boxed{(1+01)^*(\text{varepsilon}+0)}]$

## PART H — EVERY 0 MUST BE FOLLOWED BY 1

### 21. CONDITION

Every occurrence of 0 must immediately have 1 after it.

Thus 0 cannot appear alone.

Treat:

TEXT

01

as one legal block.

A standalone 1 is also legal.

So legal blocks are:

TEXT

1

01

Any concatenation:

$\mathbb{N}[\boxed{(1+01)^*}]$

## Examples

TEXT

|         |   |
|---------|---|
| $\mu$   | ' |
| 1       | ' |
| 0 1     | ' |
| 1 0 1   | ' |
| 0 1 1   | ' |
| 0 1 0 1 | ' |

Rejected:

TEXT

0  
10  
010  
100

because some 0 is not immediately followed by 1.

## & WORDING TRAP

"Every 0 is followed by 1" may mean **immediately followed** in typical automata questions if phrased "every 0 is immediately followed by 1."

If it only says "followed somewhere later," that is a different language.

Read wording carefully.

## PART I — EVERY 1 IS FOLLOWED BY 0

By symmetry, legal blocks:

TEXT

0  
10

Therefore:

$\boxed{(0+10)^*}$

Notice this does **not** allow an unmatched final 1.

Compare with:

**no consecutive 1s**

which allows a final 1.

So:

**Every 1 immediately followed by 0**

$(0+10)^*$

**No consecutive 1s**

$(0+10)^*(\epsilon+1)$

Important distinction.

## PART J — STRINGS WITH NO 00

## 22. NO CONSECUTIVE ZEROS

We already saw symmetric formula:

$\boxed{(1+01)^*(\epsilon+0)}$

---

## PART K — CONTAINS AT LEAST ONE 0 AND AT LEAST ONE 1

---

### 23. CONDITION

---

We need both symbols somewhere.

There are two possible relative orders for their first useful pair:

TEXT

0 occurs before some 1

OR

1 occurs before some 0

So:

$\boxed{(0+1)^0(0+1)^1(0+1)^* + (0+1)^1(0+1)^0(0+1)^*}$

Correct but long.

A more structured form may be possible, but this is easy to verify.

---

### 24. ANOTHER WAY

---

All binary strings minus:

- all-zero strings
- all-one strings

Conceptually:

$\Sigma^*(0^+1^+)^*$

But **difference/complement is not a primitive regular-expression operator** in the standard formal definition.

So for a pure RE answer, use a union/concatenation/star form unless extended RE syntax is explicitly allowed.

---

## & pGATE TRAP

---

Do not casually use:

TEXT

NOT

intersection

complement

inside a formal basic regular expression unless the question permits extended regular expressions.

Primitive regular operations are:

- union
- concatenation
- star

---

## PART L — EXACTLY ONE OCCURRENCE OF SUBSTRING

---

This is trickier.

Be careful: saying "exactly one occurrence of substring 01" is not the same as exactly one 0 or 1.

Overlap and surrounding patterns matter.

For such questions, a direct RE can become complicated.

### GATE strategy

If construction becomes messy:

**Build a DFA/NFA first, then convert to RE if needed.**

Do not force an error-prone formula.

---

## PART M — STRINGS WHERE ALL 0s COME BEFORE ALL 1s

---

### 25. LANGUAGE

---

Examples:

```
TEXT
μ
0
00
1
11
01
001
011
000111
```

Not:

```
TEXT
10
101
010
```

RE:

$\boxed{0^*1^*}$

Very important.

---

### 26. STRINGS WHERE ALL 1s COME BEFORE ALL 0s

---

$\boxed{1^*0^*}$

---

## 27. EITHER ALL 0s THEN 1s OR ALL 1s THEN 0s

$\boxed{0^+1^+ + 1^+0^+}$

## PART N — ALTERNATING SYMBOLS

## 28. BINARY STRINGS WITH ALTERNATING 0 AND 1

No equal adjacent symbols.

Examples:

TEXT

μ  
0  
1  
01  
10  
010  
101  
0101  
1010

We need account for sequences starting with either symbol.

One correct RE:

$\boxed{(01)^{\varepsilon+0} + (10)^{\varepsilon+1}}$

Let's test.

First branch:

TEXT

μ  
0  
01  
010  
0101  
01010  
...

Second:

TEXT

μ  
1  
10  
101  
1010  
...

Together = alternating strings.

## 29. BE CAREFUL WITH DUPLICATE μ

Both branches include  $\mu$ .

That's fine.

Languages are sets, so duplicate generation does not matter.

---

## PART O — EVEN NUMBER OF 0s AND EVEN NUMBER OF 1s

---

This becomes much harder as a direct RE.

A product DFA is often much cleaner.

This is an important exam strategy:

**Not every regular language should be attacked directly with RE algebra.**

For combined parity conditions:

TEXT

DFA/product automaton

is usually easier.

Then if RE is required, convert the automaton later.

---

## 30. LANGUAGE IDENTIFICATION STRATEGY FOR RE QUESTIONS

---

When given an RE, read it in this order:

### 1. Parentheses

Identify groups.

### 2. Star

What can repeat?

### 3. Concatenation

What must occur in sequence?

### 4. Union

What are the alternative branches?

---

## Example

---

$\forall 0^*(10^*10^*)^*\forall$

Interpret:

**Initial**  $0^*$

any zeros.

**Inner block**

$\{ 10^*10^* \}$

contains exactly two mandatory 1s.

### Outer star

repeat that block any number of times.

Therefore:

**even number of 1s.**

## 31. RE CONSTRUCTION USING CASES

This is one of the strongest techniques.

Suppose condition can happen in mutually understandable cases.

Example:

**starts and ends with different symbols.**

Break:

TEXT

Case 1: starts 0 ends 1

Case 2: starts 1 ends 0

Construct each separately.

Then union.

$\{ 0\Sigma^*1+1\Sigma^*0 \}$

## MEMORY

**If language has cases !' construct each case + union them**

## 32. RE CONSTRUCTION USING BLOCKS

Another powerful method:

Identify a **legal repeating block**.

Examples:

### Even length

block = any two symbols.

$\{ (\Sigma\Sigma)^* \}$

### Even number of 1s



block contributes two 1s:

$\backslash [ 0^*(10^*10^*)^* \backslash ]$

### Every 0 followed by 1

legal blocks:

TEXT

1  
01

So:

$\backslash [ (1+01)^* \backslash ]$

## MEMORY

Find the smallest safe block, then star it.

## 33. RE CONSTRUCTION USING MANDATORY MARKERS

For "contains" or "at least":

Identify mandatory symbols/patterns.

Example:

at least two 1s

Use:

TEXT

anything  
1  
anything  
1  
anything

Thus:

$\backslash [ \backslash \text{Sigma}^1 \backslash \text{Sigma}^1 \backslash \text{Sigma}^* \backslash ]$

## 34. RE CONSTRUCTION USING OPTIONAL PARTS

For zero-or-one occurrence, use:

$\backslash [ \backslash \text{varepsilon} + R \backslash ]$

Example optional final 1:

$\backslash [ \backslash \text{varepsilon} + 1 \backslash ]$

This was useful for no-consecutive-1 language.

---

## PART P — SIMPLIFICATION IDENTITIES

---

### 35. DISTRIBUTIVE LAWS

---

Regular expression concatenation distributes over union.

$$\boxed{R(S+T)=RS+RT}$$

and:

$$\boxed{(R+S)T=RT+ST}$$

---

#### Example

---

$$a(b+c)$$

equivalent to:

$$ab+ac$$

---

### 36. FACTORING

---

Reverse direction:

$$ab+ac$$

can become:

$$a(b+c)$$

This often makes REs shorter.

---

### 37. UNION PROPERTIES

---

$$R+S=S+R$$

$$(R+S)+T=R+(S+T)$$

$$R+R=R$$

---

### 38. CONCATENATION PROPERTIES

---

Associative:

$$(RS)T=R(ST)$$

Generally not commutative:

$$RS \neq SR$$

---

### 39. STAR IDENTITIES

---

$$\nmid (R^+)^+ = R^+$$

$$\nmid R^+R^+ = R^+$$

$$\nmid \text{\textbackslash varepsilon} + R R^+ = R^+$$

$$\nmid \text{\textbackslash varepsilon} + R^+R = R^+$$

## 40. A VERY IMPORTANT NON-IDENTITY

In general:

$$\nmid \boxed{(R+S)^+ \neq R^+S^+}$$

Why?

Left side allows mixing.

Example:

$$\nmid (a+b)^+$$

contains:

TEXT

ab  
ba  
aba

But:

$$\nmid a^+b^+$$

does not.

## 41. ANOTHER NON-IDENTITY

$$\nmid \boxed{(RS)^+ \neq R^+S^+}$$

in general.

Example:

$$\nmid (ab)^+$$

versus:

$$\nmid a^+b^+$$

## 42. IMPORTANT EQUIVALENCE

$$\nmid (R+S)^+$$

is equivalent to:

$$\nmid (R^+S^+)^+$$

This works because the outer star permits arbitrary alternation among blocks from R and S.

This type of identity may appear in conceptual questions, but always reason using generated strings rather than

memorizing a huge algebra list.

---

## 43. HOW TO TEST WHETHER TWO RES ARE EQUIVALENT

---

Use three levels.

### Level 1 — Expand simple meanings

Example:

$\forall [a+a=a]$

obvious.

### Level 2 — Find a counterexample

If you suspect not equivalent, find one string generated by one but not the other.

Example:

$\forall (ab)^*$

vs:

$\forall a^*b^*$

String:

TEXT

a

belongs to  $a^*b^*$ , not  $(ab)^*$ .

Thus not equivalent.

### Level 3 — Convert to automata

For harder cases, automata equivalence may be safer.

---

## 44. COUNTEREXAMPLE SHORTCUT

---

To prove two REs are **not** equivalent:

**One distinguishing string is enough.**

This mirrors distinguishability in DFA.

---

## PART Q — SOLVED GATE-STYLE EXAMPLES

---

### 45. EXAMPLE 1

Write an RE for binary strings containing exactly one **1**.

$\forall \boxed{0^*10^*}$

## 46. EXAMPLE 2

Write an RE for binary strings containing exactly two 0s.

Between/before/after zeros, only 1s can occur.

$\boxed{1^*01^*01^*}$

## 47. EXAMPLE 3

Write an RE for strings over  $\{a, b\}$  containing at least one  $a$ .

$\boxed{(a+b)^*a(a+b)^*}$

## 48. EXAMPLE 4

Write an RE for strings where every  $a$  is immediately followed by  $b$ .

Legal blocks:

TEXT

$b$

$ab$

Therefore:

$\boxed{(b+ab)^*}$

## 49. EXAMPLE 5

Binary strings with no consecutive zeros.

$\boxed{(1+01)^*(\text{varepsilon}+0)}$

## 50. EXAMPLE 6

Binary strings of odd length.

$\boxed{(0+1)((0+1)(0+1))^*}$

## 51. EXAMPLE 7

Strings over  $\{0, 1\}$  starting and ending in the same symbol.

$\boxed{0+1+0(0+1)^*0+1(0+1)^*1}$

for nonempty strings.

## 52. EXAMPLE 8

Strings over  $\{0, 1\}$  with exactly one occurrence of 0.

$\boxed{1^*01^*}$

---

## 53. EXAMPLE 9

---

Strings containing at least two zeros.

$\boxed{(0+1)^*0(0+1)^*0(0+1)^*}$

---

## 54. EXAMPLE 10

---

Strings where all zeros precede all ones:

$\boxed{0^*1^*}$

---

## 55. TYPICAL GATE/PYQ PATTERNS

---

GATE often asks:

### Pattern 1

Choose RE satisfying a verbal condition.

### Pattern 2

Identify which strings belong to an RE.

### Pattern 3

Compare two REs for equivalence.

### Pattern 4

Find which RE excludes/permits  $\mu$ .

### Pattern 5

Restrictions like:

- no consecutive symbols
- exactly/at least occurrences
- start/end pattern
- parity
- alternating symbols

### Pattern 6

Convert between RE and finite automata.

That comes next.

---

## 56. & pGATE TRAPS

---

### Trap 1

---

Exactly two 1s:

$\text{\texttt{\\[ (0+1)^11(0+1)^1 \\}}$

'LWrong.

This guarantees substring **11** , not exactly two total 1s.

Correct:

$\text{\texttt{\\[ 0^10^10^10^* \\}}$

---

## Trap 2

At least two 1s:

$\text{\texttt{\\[ 0^10^10^10^* \\}}$

'LThis gives exactly two 1s.

At least two must allow additional 1s.

---

## Trap 3

No consecutive 1s:

$\text{\texttt{\\[ (0+10)^* \\}}$

'LThis misses strings ending in a lone 1, such as **1** and **101** .

Correct:

$\text{\texttt{\\[ (0+10)^*(\\text{\\texttt{\\textbackslash} varepsilon+1}) \\}}$

---

## Trap 4

Every 1 followed by 0 is same as no consecutive 1s.

'LFalse.

String:

TEXT

1

has no consecutive 1s but violates "every 1 immediately followed by 0."

---

## Trap 5

Even number of 1s means even string length.

'LFalse.

Example:

---

has zero 1s (even) but odd length.

## Trap 6

At most one 1 means exactly one 1.

'L False.

Zero 1s are also allowed.

## Trap 7

If two REs look different, they must define different languages.

'L False.

Always compare languages.

## Trap 8

Complement can always be written directly using  $\neg R$  in standard RE.

'L Not in the basic formal syntax unless extended notation is explicitly allowed.

## Trap 9

Every regular-language condition has an easy direct RE.

'L False.

Sometimes DFA/NFA construction is safer.

## 57. MASTER CONSTRUCTION TABLE

Assume:

$\Sigma = \{0, 1\}$

| Condition       | RE                |
|-----------------|-------------------|
| Exactly one 1   | $0^*10^*$         |
| Exactly two 1s  | $0^*10^*10^*$     |
| At least one 1  | $0^*10^*$         |
| At least two 1s | $0^*10^*10^*10^*$ |
| At most one 1   | $0^* + 0^*10^*$   |



| Condition                         | RE                               |
|-----------------------------------|----------------------------------|
| Even length                       | $(\epsilon \epsilon)^*$          |
| Odd length                        | $\epsilon (\epsilon \epsilon)^*$ |
| Starts with 0, ends 1             | $0 \epsilon^* 1$                 |
| No consecutive 1s                 | $(0 + 1 0)^* (\mu + 1)$          |
| No consecutive 0s                 | $(1 + 0 1)^* (\mu + 0)$          |
| Every 0 immediately followed by 1 | $(1+01)^*$                       |
| All 0s before all 1s              | $0^* 1^*$                        |
| Contains 01                       | $\epsilon^* 0 1 \epsilon^*$      |

where:

$\Sigma = \{0, 1\}$

in RE notation.

## 58. SHORTCUTS & MEMORY TRICKS

### Exactly k

Put k mandatory symbols; allow only the other symbol around them.

### At least k

***\*Put k mandatory symbols; allow  $\epsilon$  around them.\*\****

### At most k

Union cases 0 through k.

### Even count

Pair the counted symbol.

### Odd count

One counted symbol + pairs.

## No consecutive x

Pair x safely with the other symbol, allow optional final x.

## Every x followed by y

Treat xy as one legal block.

## 59. ' p WRITE THIS IN YOUR NOTES

### Advanced RE Construction

#### Exactly one 1

$\backslash [ 0^*10^* \backslash ]$

#### Exactly two 1s

$\backslash [ 0^*10^*10^* \backslash ]$

#### At least one 1

$\backslash [ \backslash \text{Sigma}^1 \backslash \text{Sigma}^* \backslash ]$

#### At least two 1s

$\backslash [ \backslash \text{Sigma}^1 \backslash \text{Sigma}^1 \backslash \text{Sigma}^* \backslash ]$

### Length parity

Even:

$\backslash [ (\backslash \text{Sigma} \backslash \text{Sigma})^* \backslash ]$

Odd:

$\backslash [ \backslash \text{Sigma} (\backslash \text{Sigma} \backslash \text{Sigma})^* \backslash ]$

### No consecutive 1s

$\backslash [ \boxed{ (0+10)^*(\backslash \text{varepsilon}+1) } \backslash ]$

### No consecutive 0s

$\backslash [ \boxed{ (1+01)^*(\backslash \text{varepsilon}+0) } \backslash ]$

## Every 0 immediately followed by 1

$\boxed{(1+01)^*}$

## All 0s before all 1s

$\boxed{0^*1^*}$

## Construction strategies

### TEXT

|              |   |                                |
|--------------|---|--------------------------------|
| Exactly      | ! | mandatory count                |
| At least     | ! | mandatory markers + $\epsilon$ |
| At most      | ! | union cases                    |
| Parity       | ! | pair blocks                    |
| Cases        | ! | build each + union             |
| Restrictions | ! | find safe repeating blocks     |

## 60. 60-SECOND REVISION

- Exactly one 1 !'  $0^*10^*$ .
- Exactly two 1s !'  $0^*10^*10^*$ .
- At least one 1 !'  $\epsilon^* 1 \epsilon^*$
- At least two 1s !'  $\epsilon^* 1 \epsilon^* 1 \epsilon^*$
- At most one = zero or exactly one.
- Even length =  $(\epsilon \epsilon)^*$
- Odd length =  $\epsilon (\epsilon \epsilon)^*$
- Even count !' pair counted symbols.
- Odd count !' one + pairs.
- No consecutive 1s =  $(0 + 10)^* (0 + 1)$
- Every 1 followed immediately by 0 =  $(0+10)^*$ .
- These two languages are different.
- All 0s before all 1s =  $0^*1^*$ .
- Build difficult languages using cases or safe repeating blocks.
- Test  $\mu$  and shortest strings before trusting an RE.

## 61. ACTIVE RECALL

### Q1

Write RE for exactly one  $0$ .

### Q2

Write RE for exactly three  $1$ s.

### Q3

Write RE for at least one 0.

### Q4

Write RE for even-length binary strings.

### Q5

Write RE for odd-length binary strings.

### Q6

Write RE for no consecutive 1s.

### Q7

What is the difference between:

$\mathcal{L}[(0+10)^*]$

and:

$\mathcal{L}[(0+10)^*(\epsilon+1)?]$

### Q8

Write RE for all zeros occurring before all ones.

### Q9

What general technique is useful when a condition has two separate cases?

### Q10

What must you always test after constructing an RE?

---

## ANSWERS

1.

$\mathcal{L}[1^*01^*]$

2.

$\mathcal{L}[0^*10^*10^*10^*]$

3.

$\mathcal{L}[(0+1)^*0(0+1)^*]$

4.

$\mathcal{L}[((0+1)(0+1))^*]$

5.

$\mathcal{L}[(0+1)((0+1)(0+1))^*]$

6.

$\mathcal{L}[(0+10)^*(\epsilon+1)]$

7. First requires every 1 to be followed by 0; second allows an unmatched final 1.

8.

$\{0^*1^*\}$

9. Construct each case and take union.

10.  $\mu$  shortest accepted strings, shortest rejected strings, and boundary cases.

---

## 62. MINI GATE TEST

---

### Q1 — MCQ

---

Which RE represents binary strings with exactly one 1?

A.

$\{(0+1)^*1(0+1)^*\}$

B.

$\{0^*10^*\}$

C.

$\{1^*01^*\}$

D.

$\{0^*1^*\}$

---

### Q2 — MCQ

---

Which represents binary strings with no consecutive 1s?

A.

$\{(0+10)^*\}$

B.

$\{(0+10)^*(\epsilon+1)\}$

C.

$\{(0+1)^*\}$

D.

$\{0^*1^*\}$

---

### Q3 — MSQ

---

Which strings belong to:

$\{0^*10^*\}$

A. 1 B. 00100 C. 101 D. 000

---

### Q4 — MCQ

---

Which RE represents all binary strings of even length?

A.

$\backslash [ (0+1)^* \backslash ]$

B.

$\backslash [ ((0+1)(0+1))^* \backslash ]$

C.

$\backslash [ (00+11)^* \backslash ]$

D.

$\backslash [ (01+10)^* \backslash ]$

---

## Q5 — TRICKY

Does:

$\backslash [ (00+11)^* \backslash ]$

represent every even-length binary string?

A. Yes B. No

---

## Q6 — MCQ

Which RE allows a string containing zero or one occurrence of **1** ?

A.

$\backslash [ 0^*10^* \backslash ]$

B.

$\backslash [ 0^+0^*10^* \backslash ]$

C.

$\backslash [ (0+1)^* \backslash ]$

D.

$\backslash [ 1^* \backslash ]$

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\backslash [ \boxed{B} \backslash ]$

---

### Q2

$\backslash [ \boxed{B} \backslash ]$

---

### Q3

1 '

00100 '

101 has two 1s '

000 has zero 1s '

Therefore:

$\boxed{A, B}$

### Q4

$\boxed{B}$

### Q5

$\boxed{B}$

For example:

TEXT

01

has even length but cannot be generated by  $(00+11)^*$ .

### Q6

$\boxed{B}$

## CHAPTER 4 PROGRESS

TEXT

```
[ ] RE fundamentals
[ ] Union / concatenation / star
[ ] Operator precedence
[ ] Basic identities
[ ] Starts / ends / contains
[ ] Exactly occurrences
[ ] At least / at most
[ ] Even / odd conditions
[ ] Length parity
[ ] No-consecutive-symbol patterns
[ ] Mandatory-follow patterns
[ ] Case-based construction
[ ] Block-based construction
[ ] RE equivalence basics

[ ] RE !'  $\mu$ -NFA
[ ] Thompson-style construction intuition
[ ] Union construction
[ ] Concatenation construction
[ ] Star construction
[ ] FA !' RE
[ ] State elimination
```

NEXT TOPIC

0=VERY HIGH PRIORITY REGULAR EXPRESSION !' μ - N F A

Next we will connect the two major representations:

$$\boxed{\text{Regular Expression}} \xrightarrow{\text{varepsilon}} \text{-NFA}$$

We will learn the reusable constructions for:

- symbol
- μ
- "
- union
- concatenation
- Kleene star

and then convert full expressions step-by-step.

Type NEXT.

USER  
NEXT

USER  
NEXT

ASSISTANT

Continuing exactly from your uploaded TOC master prompt and the completed advanced Regular Expression construction section.

CHAPTER 4 — REGULAR EXPRESSIONS

Topic 3: Regular Expression !' μ - N F A

Priority: 0=VERY HIGH

This topic proves one direction of the fundamental equivalence:

$$\boxed{\text{Regular Expression}} \rightarrow \text{-NFA}$$

Later we will prove the reverse direction:

$$\text{Finite Automaton} \rightarrow \text{Regular Expression}$$

Together:

$$\boxed{\text{RE} \iff \text{FA} \iff \text{Regular Language}}$$



## 1. WHY CONVERT RE TO AUTOMATON?

Suppose:

$[ R = (0+1)^*01 ]$

The RE clearly describes:

**binary strings ending in 01.**

But we may want an automaton that actually recognizes those strings.

So we systematically convert the RE into an  $\mu$ -NFA.

## 2. CORE IDEA

We do **not** invent the whole automaton at once.

Instead:

**Build tiny automata for tiny RE pieces, then combine them.**

The three main operations are:

1. Union
2. Concatenation
3. Kleene star

So if we know how to build automata for those operations, we can handle every regular expression.

## MEMORY

**RE !' break into pieces !' build small machines !' connect with  $\mu$**

## 3. THOMPSON-CONSTRUCTION INTUITION

A standard systematic method is commonly called **Thompson's construction**.

The key design idea:

**Each RE fragment has one entry and one exit.**

Then  $\mu$ -transitions connect fragments.

Conceptually:

TEXT

```
RE fragment
      !"
start ----> final
```

This makes union, concatenation, and star easy to combine.

For GATE, understanding the construction is more important than memorizing a particular state naming scheme.

---

## PART A — BASIC BUILDING BLOCKS

---

### 4. AUTOMATON FOR ONE SYMBOL **a**

---

For RE:

$\{a\}$

build:

TEXT

```
! ' q 0 - - a - - > q 1 *
```

That's all.

Language:

$\{a\}$

---

### 5. AUTOMATON FOR SYMBOL **0**

---

TEXT

```
! ' q 0 - - 0 - - > q 1 *
```

accepts exactly:

$\{0\}$

---

### 6. AUTOMATON FOR $\mu$

---

For RE:

$\{\epsilon\}$

we need to accept without consuming input.

So:

TEXT

```
! ' q 0 - - \mu - - > q 1 *
```

Language:

$\{\epsilon\}$

---

### 7. AUTOMATON FOR "

---

For RE:

$\emptyset$

we need an automaton accepting nothing.

One representation:

TEXT

! ' q 0 q 1 \*

with no path from  $q_0$  to  $q_1$ .

Therefore no input can reach the final state.

Language:

$\emptyset$

## & IMPORTANT

Do not confuse:

$\mu$

There is an  $\mu$  - path from start to final.

"

There is **no successful path at all**.

## PART B — UNION CONSTRUCTION

### 8. RE UNION

Suppose:

$R \cup S$

means:

**strings from R OR strings from S.**

We already have  $\mu$  - NFA for R and  $\mu$  - NFA for S.

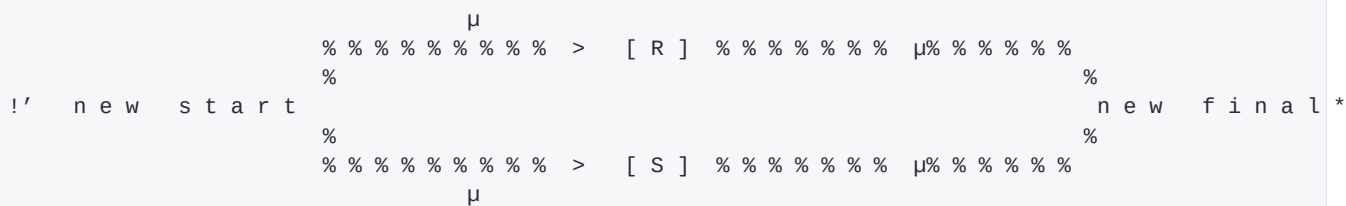
Create:

- new start state
- new final state

Then branch using  $\mu$ .

## GENERAL STRUCTURE

TEXT



## WHY?

At the new start, the machine can freely choose:

TEXT

R branch  
OR  
S branch

without consuming input.

Exactly what union requires.

## 9. EXAMPLE — **a+b**

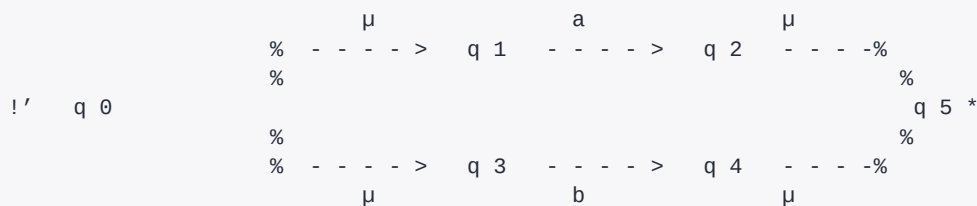
We want:

$\backslash [a+b]$

Create two symbol automata.

Conceptually:

TEXT



This accepts:

TEXT

a  
or  
b

## MEMORY TRICK

UNION = SPLIT and JOIN

TEXT

```
new start
    !"  μ  s p l i t
R      S
    !"  μ  j o i n
new final
```

## PART C — CONCATENATION CONSTRUCTION

### 10. RE CONCATENATION

Suppose:

$[RS]$

means:

**first match R, then match S.**

If we have automata:

TEXT

R-start ----> R-final

and:

TEXT

S-start ----> S-final

connect:

TEXT

R - f i n a l   - - μ - - >   S - s t a r t

### GENERAL STRUCTURE

TEXT

!' [ R ] - - μ - - > [ S ] !' f i n a l \*

No branching is needed.

### WHY $\mu$ ?

We need to move from the end of R to the beginning of S without consuming an extra symbol.

### 11. EXAMPLE — ab

Build:

TEXT

```
! ' q 0 - - a - - > q 1 - - μ - - > q 2 - - b - - > q 3 *
```

This accepts exactly:

$\backslash[ab]\backslash$

## 12. CAN WE SOMETIMES MERGE STATES INSTEAD?

Yes.

A simplified automaton for `ab` could be:

TEXT

```
! ' q 0 - - a - - > q 1 - - b - - > q 2 *
```

But Thompson-style construction often keeps fragments separate and connects with  $\mu$ .

Both may recognize the same language.

For systematic conversion, follow one method consistently.

## MEMORY

**CONCATENATION = chain machines**

## PART D — KLEENE STAR CONSTRUCTION

### 13. RE STAR

Suppose we have automaton for:

$\backslash[R]\backslash$

We want:

$\backslash[R^*]\backslash$

Meaning:

**zero or more repetitions of R.**

So we must allow:

1. zero repetitions
2. one repetition
3. two repetitions
4. infinitely many finite repetitions

## 14. GENERAL STAR STRUCTURE

Create a new start and new final.

Then add  $\mu$ transitions so that:

- we may skip R completely
- we may enter R
- after finishing R, we may repeat
- after finishing R, we may exit

Conceptually:

TEXT

```

                                % % % % % % % %      μ % % % % % % % %
                                %                               %
                                v                               %
!' newStart - - μ - - > [ R ] - - μ - - > newFinal *
                                ^
                                %
                                % % % % % % % μ % % % % % % % %
                                skip

```

More precisely, old final also has  $\mu$ back to old start for repetition.

## FULL IDEA

TEXT

```

                                % % % % % % % %      μ % % % % % % % %
                                %                               %
                                v                               %
!' newStart - - μ - - > oldStart . . . oldFinal - - μ - - > newFinal
                                %                               %
                                % % % % % % % % μ % % % % % % % % % % % % % % %
                                %                               %
                                % % % μ - - > oldStart

```

## 15. WHY DO WE NEED START !' FINAL $\mu$ ?

Because:

$\backslash R^* \backslash$

must accept zero repetitions.

Zero repetitions produce:

$\backslash \text{\textbackslash varepsilon} \backslash$

So  $\mu$  must be accepted.

## 16. WHY OLD FINAL !' OLD START $\mu$ ?

After completing one copy of R:

This gives:

 $\backslash RR, \backslash RRR, \backslash \text{dots} \backslash$ 

## MEMORY

- Skip !' zero repetitions
- Repeat !' many repetitions

### 17. EXAMPLE — $a^*$

Original **a**:

Star construction conceptually:

Main behavior:

- q0 !' q3 via  $\mu'$  accept  $\mu$
- q0 !' q1 !' read a !' q2 !' q3 !' accept a
- q2 !' q1 via  $\mu'$  repeat !' accept aa, aaa, etc.

Therefore:

$$\mathcal{L} = \{\varepsilon, a, aa, aaa, \dots\}$$

## PART E — BUILDING COMPLEX EXPRESSIONS



## 18. CONSTRUCTION ORDER FOLLOWS RE STRUCTURE

Suppose:

$\backslash (a+b)^* \backslash$

Do not start with star blindly.

Operator structure:

1. Build `a`
2. Build `b`
3. Union them !' `(a+b)`
4. Apply star

## SAME AS EXPRESSION TREE

TEXT



Bottom-up construction:

TEXT

a  
b  
!"  
a+b  
!"  
(a+b)\*

## 19. EXAMPLE — `ab+c`

Using precedence:

$\backslash ab+c \backslash$

means:

$\backslash (ab)+c \backslash$

So:

TEXT



Build:

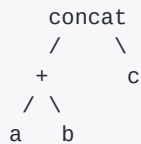
1. `a`

2. `b`
3. concatenate `!` `ab`
4. build `c`
5. union `!` `ab+c`

## 20. EXAMPLE — `(a+b)c`

Structure:

TEXT



Steps:

1. build `a`
2. build `b`
3. union `!` `a+b`
4. build `c`
5. concatenate

## PART F — COMPLETE EXAMPLE 1

### 21. Convert:

$\backslash [R=a+b \backslash]$

We already know union construction.

#### Fragment a

TEXT

`q1 --a--> q2`

#### Fragment b

TEXT

`q3 --b--> q4`

Add:

TEXT

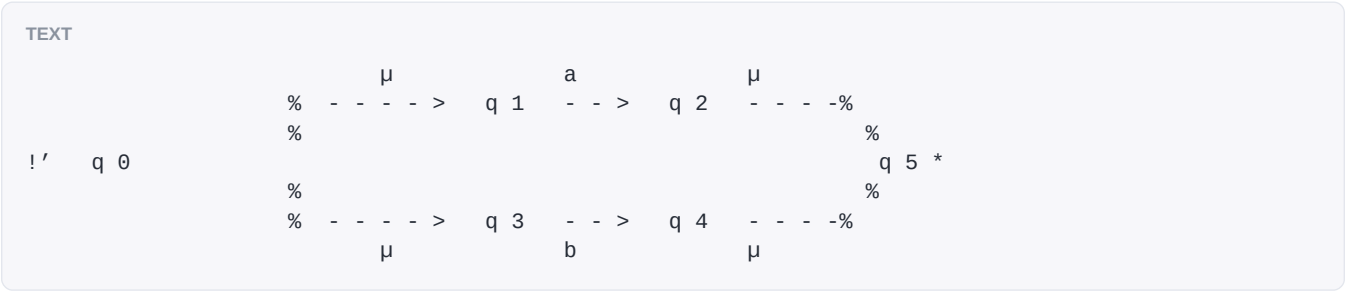
```

q 0 - - μ - - > q 1
q 0 - - μ - - > q 3

q 2 - - μ - - > q 5
q 4 - - μ - - > q 5
  
```

q0 start.  
q5 final.

Result



Accepted:

TEXT

a  
b

Rejected:

TEXT

$\mu$   
ab  
aa

PART G — COMPLETE EXAMPLE 2

22. Convert:

$\backslash [ab] \backslash$

Build:

TEXT

q0 --a--> q1

and:

TEXT

q2 --b--> q3

Connect:

TEXT

q 1 - -  $\mu$  - - > q 2

Result:

TEXT

! ' q 0 - - a - - > q 1 - - μ - - > q 2 - - b - - > q 3 \*

## PART H — COMPLETE EXAMPLE 3

### 23. Convert:

$\backslash [a^*]$

Need:

TEXT

zero or more a's

A very compact  $\mu$ -NFA could be:

TEXT

a  
% % % % %  
% % % % %  
! ' \* q 0 % % % % %

This DFA itself already recognizes  $a^*$ .

But Thompson construction may produce additional states and  $\mu$ -transitions.

Important lesson:

**Conversion is not unique.**

Many equivalent automata can represent the same RE.

## GATE WARNING

If GATE asks:

**Which automaton recognizes this RE?**

do not reject an option merely because it uses fewer/more states than your construction.

Check language equivalence.

## PART I — COMPLETE EXAMPLE 4

### 24. Convert:

$\backslash (a+b)^* \backslash$

Language:

$\Sigma^*$

A minimal DFA is simply:

TEXT

```

    a, b
    !
    * q 0
```

But Thompson's NFA is larger.

Both are correct automata for the language.

This demonstrates:

RE → NFA construction gives an automaton, not necessarily a minimal automaton.

## 25. WHY USE NFA RATHER THAN DIRECT DFA?

Because combining structures is easy.

For RE:

$\Sigma^+abb$

direct DFA construction may require careful suffix-memory reasoning.

But NFA construction is mechanical:

TEXT

```

a
b
!" union
(a+b)
!" star
(a+b)*
!" concatenate a
!" concatenate b
!" concatenate b
```

Then later convert NFA → DFA if needed.

## PART J — FULL GATE-STYLE EXAMPLE

### 26. Convert:

$R = (0+1)^*01$

We will think structurally rather than drawing dozens of states immediately.

### STEP 1 — Expression structure

$((0+1)^*), 0, 1$

Main operations:

```
TEXT
0 + 1
!"
star
!"
concatenate 0
!"
concatenate 1
```

## STEP 2 — Build 0+1

Use union:

```
TEXT
start      μ  !'  [ 0 ]  !'  μ
              final
              μ  !'  [ 1 ]  !'  μ
```

This generates:

```
TEXT
0 or 1
```

## STEP 3 — Apply star

Add  $\mu$ routes for:

- skip
- enter
- repeat
- exit

Now this fragment accepts:

$\mathbb{V}[(0+1)^*]$

i.e. any binary string.

## STEP 4 — Concatenate 0

Connect fragment final via  $\mu$ to:

```
TEXT
--0-->
```

## STEP 5 — Concatenate 1

Then via  $\mu$ to:

TEXT

--1-->

Result accepts:

$\Sigma^* (0+1)^* 01 \Sigma^*$

which is:

binary strings ending in 01.

## 27. DO WE HAVE TO DRAW EVERY STATE IN GATE?

Depends on the question.

If question explicitly asks for the constructed  $\mu - NFA$ , Yes.

But many GATE MCQs only ask:

- number of states
- accepted language
- correct structural construction
- equivalence

In those cases, understanding the templates can be enough.

## PART K — THREE UNIVERSAL CONSTRUCTION TEMPLATES

These are the heart of this topic.

## 28. UNION

$\Sigma^* R \cup S \Sigma^*$

TEXT

|           |       |   |   |   |           |   |   |   |       |
|-----------|-------|---|---|---|-----------|---|---|---|-------|
|           | $\mu$ | ! | ' | [ | R         | ] | ! | ' | $\mu$ |
| new start |       |   |   |   | new final |   |   |   |       |
|           | $\mu$ | ! | ' | [ | S         | ] | ! | ' | $\mu$ |

Memory:

split / join

## 29. CONCATENATION

$\Sigma^* R S \Sigma^*$

TEXT

[ R ] - - μ - - > [ S ]

Memory:

chain

## 30. STAR

$\backslash [ R^* ]$

Need  $\mu$ for:

- skipping R
- entering R
- repeating R
- exiting R

Memory:

skip / enter / repeat / exit

## PART L — POSITIVE CLOSURE

### 31. Construction for:

$\backslash [ R^+ ]$

Recall:

$\backslash [ R^+ = RR^* ]$

So unlike star, one occurrence of R is mandatory.

Conceptually:

TEXT

! ' [ R ] ! ' m a y r e p e a t R ! ' f i n a l

No direct  $\mu$ path from new start to final unless R itself can generate  $\mu$ .

## STAR VS PLUS CONSTRUCTION

### Star

Zero copies allowed.

So direct  $\mu$ bypass needed.

### Plus



At least one copy required.

No zero-copy bypass.

---

## PART M — OPERATOR PRECEDENCE DURING CONVERSION

---

### 32. Example

$\backslash [ ab^*+c \backslash ]$

Precedence:

$\backslash [$

- $> \text{\texttt{\textbackslash text{concatenation}}} > +$

$\backslash ]$

So:

$\backslash [ (ab^*)+c \backslash ]$

Construction order:

TEXT

```
b
!"  s t a r
b*
!"  c o n c a t e n a t e   w i t h   a
ab*
!"  u n i o n   w i t h   c
ab*+c
```

---

## & pGATE TRAP

Do not interpret:

$\backslash [ ab^* \backslash ]$

as:

$\backslash [ (ab)^* \backslash ]$

because the automata will be completely different.

---

## PART N — HOW MANY STATES?

---

### 33. THOMPSON-CONSTRUCTION STATECOUNTS

Depending on exact textbook convention, state counts can vary because implementations may:

- merge adjacent states
- use explicit new start/end states
- optimize  $\mu$ -transitions

Therefore:

**Do not memorize one universal state-count formula unless the construction convention is explicitly specified.**

For GATE:

- understand the language
- understand the structural construction
- follow the convention in the question

This avoids false memorization.

## 34. IMPORTANT THEOREM

Every regular expression can be converted into an  $\mu$  - N F A .

Why?

Because REs are recursively built only from:

- "
- $\mu$
- symbols
- union
- concatenation
- star

And we have an automaton construction for every one of them.

Therefore, by structural induction:

$\boxed{\text{Every RE has an equivalent finite automaton}}$

## 35. WHAT LANGUAGE CLASS DOES THIS PROVE?

If a language can be described by a regular expression, then it can be recognized by an  $\mu$  - N F A .

Since  $\mu$  - N F A can be converted to DFA:

$\text{RE} \rightarrow \epsilon\text{-NFA} \rightarrow \text{DFA}$

Therefore every regular-expression language is regular.

## CONNECTION

TEXT

Regular Expression  
! "  
 $\mu$  - N F A  
! "  
NFA  
! "  
DFA  
! "  
Regular Language

Later we show the reverse:

TEXT  
DFA  
! "  
Regular Expression

Then equivalence is complete.

## PART O — SOLVED GATE-STYLE QUESTIONS

### 36. Q1

Which construction correctly represents:

$\boxed{R+S}$

A. Run R then S B.  $\mu$  - b r a new start into either R or S C. Loop R forever D. Reverse R and S

**Answer**

$\boxed{\text{boxed B}}$

### 37. Q2

Which construction represents:

$\boxed{RS}$

A. Branch into R or S B. Connect R's end to S's start C. Apply star to R D. Run S before R

**Answer**

$\boxed{\text{boxed B}}$

### 38. Q3

Why does construction for:

$\boxed{R^*}$

need an  $\mu$ path allowing R to be skipped?

Because:

$\boxed{\forall \epsilon \in L(R^*)}$

always.

### 39. Q4

Which language is recognized by an automaton constructed for:

$\boxed{(0+1)^*01}$

Answer:

all binary strings ending in 01.

## 40. Q5

Suppose an  $\mu$ -NFA produced from an RE has many states.

Must it be minimal?

$\boxed{\text{No}}$

Construction guarantees equivalence, not minimality.

## 41. TYPICAL GATE/PYQ PATTERNS

Typical questions include:

### Pattern 1 — Choose correct construction

Given:

$R+S, RS, R^*$

identify correct  $\mu$ -NFA structure.

### Pattern 2 — Determine language of resulting automaton

Understand branches and  $\mu$ -loops.

### Pattern 3 — Identify whether $\mu$ is accepted

Particularly for star.

### Pattern 4 — Build conversion order

Use RE precedence/tree.

### Pattern 5 — Compare direct DFA vs Thompson $\mu$ -NFA

Remember both may recognize the same language with different state counts.

## 42. & pGATE TRAPS

### Trap 1

Union means run R and then S.

'LWrong.

That is concatenation.

Union means choose R or S.

---

## Trap 2

---

Concatenation uses a union branch.

'L Wrong.

It connects R to S sequentially.

---

## Trap 3

---

Star means one or more.

'L Wrong.

Star = zero or more.

---

## Trap 4

---

Star construction doesn't need to accept  $\mu$ .

'L Wrong.

It must.

---

## Trap 5

---

RE !'  $\mu$  - N F always produces minimal automaton.

'L Wrong.

---

## Trap 6

---

$\mu$  - t r a n s c o n s u m e s a character.

'L Wrong.

It consumes nothing.

---

## Trap 7

---

Automaton for **R+S** accepts only if both branches accept.

'L Wrong.

One branch is enough.

---

## Trap 8

---

Automaton for **(ab)\*** is same as **a\*b\***.

---

'LWrong.

Always respect parentheses and precedence before construction.

## Trap 9

There is only one valid automaton for an RE.

'LWrong.

Many equivalent automata can recognize the same language.

## 43. MASTER COMPARISON TABLE

| RE form | Automaton idea                        |
|---------|---------------------------------------|
| a       | one a edge                            |
| $\mu$   | $\mu$ edge to final                   |
| "       | no accepting path                     |
| R+S     | $\mu$ split into R/S, then $\mu$ join |
| RS      | connect R to S                        |
| R*      | skip + enter + repeat + exit          |
| R+      | one R mandatory, then repeat          |

## 44. MEMORY TRICKS

### Union

Split and Join

### Concatenation

Chain

### Star

Skip and Repeat

## Full construction

Break !' Build !' Connect

## 45. ' $\mu$ WRITE THIS IN YOUR NOTES

### RE !' $\mu$ - N F A

Every RE can be converted to an equivalent  $\mu$  - N F A .

#### Symbol a

TEXT

! ' q 0 - - a - - > q 1 \*

$\mu$

TEXT

! ' q 0 - -  $\mu$  - - > q 1 \*

"

No path from start to final.

### Union R+S

TEXT

new start  
% %  $\mu$  ! ' R %  $\mu$  % %  
% %  $\mu$  ! ' S %  $\mu$  % % 4 ! ' n e w f i n a l

Memory:

split + join

### Concatenation RS

TEXT

R - -  $\mu$  - - > S

Memory:

chain

## Star R\*

Allow:

TEXT

```
skip R
enter R
repeat R
exit R
```

Must accept:

$\epsilon$

## Conversion rule

Respect precedence:

$\text{concatenation} > +$

Build expression bottom-up.

## Equivalence chain

$\boxed{\text{RE}} \xrightarrow{\epsilon} \text{NFA} \xrightarrow{} \text{DFA}$

All describe regular languages.

## 46. 60-SECOND REVISION

- RE  $\rightarrow$   $\mu$  - NFA systematic.
- Build tiny automata and combine them.
- Symbol = one labeled transition.
- $\mu$  = transition without consuming input.
- " = no successful path.
- Union uses  $\mu$ split and  $\mu$ join.
- Concatenation chains automata.
- Star allows zero or more repetitions.
- Star must allow  $\mu$ .
- Star uses skip + repeat logic.
- Conversion follows RE operator structure.
- Respect  $\text{concatenation} > +$ .
- Resulting automaton need not be minimal.
- Different automata may recognize the same RE language.
- Every RE has an equivalent  $\mu$  - NFA.

## 47. ACTIVE RECALL



### Q1

What three operations must we know to convert arbitrary REs?

### Q2

How is symbol  $a$  represented?

### Q3

How is  $\mu$  represented?

### Q4

What does the automaton for  $^*$  need to guarantee?

### Q5

What is the memory phrase for union?

### Q6

What is the memory phrase for concatenation?

### Q7

What two main ideas make star work?

### Q8

Why must  $R^*$  construction accept  $\mu$ ?

### Q9

Is an RE-generated  $\mu$  - NFA necessarily minimal?

### Q10

Which should be built first in:

$\backslash [a+ b^*] \backslash$

---

## ANSWERS

1. Union, concatenation, Kleene star.
  2. One  $a$ -labelled transition from start to final.
  3.  $\mu$  - transition from start to final.
  4. No string reaches a final state.
  5. Split and join.
  6. Chain.
  7. Skip and repeat.
  8. Zero repetitions are allowed.
  9. No.
  10.  $b^*$ , because star has highest precedence.
- 

## 48. MINI GATE TEST

---

### Q1 — MCQ

For:

$\{ R+S \}$

the  $\mu$ -NFA should primarily:

- A. Run R then S B. Choose R or S C. Repeat R only D. Reject  $\mu$  always

### Q2 — MCQ

For:

$\{ RS \}$

the automata are combined using:

- A. Sequential connection B. Choice only C. Complement D. Reversal

### Q3 — MSQ

For a standard star construction  $R^*$ , which behaviors are required?

- A. Skip R completely B. Enter R C. Repeat R D. Require exactly one copy of R

### Q4 — MCQ

Which must belong to:

$\{ L(R^*) \}$

for every R?

- A.  $a$  B.  $\mu$  C.  $R$  as a literal symbol D.  $"$

### Q5 — TRUE/FALSE

Every  $\mu$ -NFA obtained from Thompson-style RE conversion is minimal.

### Q6 — TRICKY

The RE is:

$\{ ab^* \}$

Which structure should be constructed first?

- A.  $(ab)$  then star B.  $b^*$ , then concatenate with  $a$

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\boxed{B}$

## Q2

$\boxed{A}$

## Q3

$\boxed{A, B, C}$

## Q4

$\boxed{B}$

## Q5

$\boxed{\text{False}}$

## Q6

$\boxed{B}$

because:

$ab^* = a(b^*)$

## CHAPTER 4 PROGRESS

### TEXT

```
[ ] RE fundamentals
[ ] Union / concatenation / star
[ ] Operator precedence
[ ] Advanced RE construction
[ ] Exactly / at least / at most
[ ] Parity patterns
[ ] No-consecutive patterns
[ ] RE equivalence basics
[ ] RE !'  $\mu$ -NFA
[ ] Symbol construction
[ ]  $\mu$  / " construction
[ ] Union construction
[ ] Concatenation construction
[ ] Kleene-star construction
[ ] Thompson-construction intuition

[ ] Finite Automaton !' Regular Expression
[ ] State elimination method
[ ] Generalized transition labels
[ ] Arden's theorem intuition
[ ] Complete FA !' RE equivalence
```

## NEXT TOPIC

### ~~FINITE~~ FINITE AUTOMATON !' REGULAR EXPRESSION

Next we reverse the direction:

$\boxed{FA \rightarrow RE}$

We will learn:

- state elimination
- how transition labels become REs
- how loops are handled
- how to eliminate intermediate states safely
- the key formula

$$R_{ij} + R_{ik}(R_{kk})^*R_{kj}$$

with simple intuition rather than memorizing it blindly.

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing exactly from your uploaded TOC master prompt and the completed **RE !' μ - N F** Action.

## CHAPTER 4 — REGULAR EXPRESSIONS

### Topic 4: Finite Automaton !' Regular Expression

Priority: ~~0~~ = **VERY HIGH**

Now we prove the reverse direction:

$$\boxed{\text{Finite Automaton} \rightarrow \text{Regular Expression}}$$

Earlier we learned:

$$\text{RE} \rightarrow \epsilon\text{-NFA}$$

Now after this topic we get the full equivalence:

$$\boxed{\text{Regular Expression} \iff \text{Finite Automaton} \iff \text{Regular Language}}$$

This is one of the central results of Theory of Computation.

## 1. WHY CONVERT AN AUTOMATON TO AN RE?

Suppose a DFA recognizes:

**all strings ending in 01.**

We may want an algebraic description of exactly the same language.

That RE is:

$$(0+1)^*01$$

So both:

- automaton

- regular expression

describe the same set of strings.

---

## 2. MAIN METHODS

There are two common approaches:

### Method 1 — State Elimination

~~0=1~~ Most important for understanding conversion.

### Method 2 — Arden's Theorem

Useful when equations are formed from automaton states.

We will focus first on **state elimination**, then learn Arden's theorem intuition.

---

## PART A — STATE ELIMINATION

### 3. CORE IDEA

Suppose we have an automaton:

```
TEXT
start
  !"
states
  !"
final
```

We gradually remove intermediate states.

Whenever a state is removed, we replace all routes through that state with an equivalent **regular-expression-labelled transition**.

Eventually only:

```
TEXT
Start -----RE-----> Final
```

remains.

That RE is the answer.

---

## MEMORY

**Remove states, preserve paths.**

Every time a state disappears:

**all strings that previously passed through it must still be represented by the new transition labels.**

---

## 4. GENERALIZED TRANSITION GRAPH

---

Originally an automaton edge contains one symbol:

TEXT

```
q0 --a--> q1
```

During state elimination, an edge may contain an entire regular expression:

TEXT

```
q0 --(a+b*c)--> q1
```

This is often called a **Generalized NFA** or **Generalized Transition Graph** idea.

The transition label itself represents all strings capable of moving between those states.

---

## 5. PARALLEL EDGES

---

Suppose:

TEXT

```
q0 --a--> q1  
q0 --b--> q1
```

Instead of keeping two edges, combine them:

$\boxed{a+b}$

So:

TEXT

```
q0 --(a+b)--> q1
```

---

## MEMORY

---

Parallel paths != Union

---

## 6. SERIES EDGES

---

Suppose:

TEXT

```
q0 --a--> q1 --b--> q2
```

If q1 has no relevant complication and we remove it, the path means:

$\boxed{ab}$

---

So:

TEXT

```
q0 --ab--> q2
```

## MEMORY

Series paths !' Concatenation

## 7. SELF-LOOP

Suppose:

TEXT

```
      b
      !°
q1
```

If a path may go around this loop any number of times, it contributes:

$\bigvee b^{*}$

## MEMORY

Loop !' Star

These three rules are the heart of state elimination:

TEXT

```
P a r a l l e l    !'    +
S e r i e s        !'    c o n c a t e n a t i o n
L o o p            !'    *
```

## PART B — THE STATE-ELIMINATION FORMULA

## 8. GENERAL SITUATION

Suppose we want to eliminate state **k**.

There is:

TEXT

```
i --Rik--> k
k --Rkk--> k
k --Rkj--> j
```

and maybe an existing direct edge:

TEXT

$i \xrightarrow{R_{ij}} j$

A path from  $i$  to  $j$  can either:

1. go directly using  $R_{ij}$ , or
2. enter  $k$  using  $R_{ik}$ ,
3. loop at  $k$  zero or more times using  $R_{kk}^*$ ,
4. leave  $k$  using  $R_{kj}$

So the new transition becomes:

$$R_{ij} + R_{ik}(R_{kk})^*R_{kj}$$

## 9. DON'T MEMORIZE IT BLINDLY

Read it in English:

$R_{ij}$

means:

**old direct route.**

And:

$R_{ik}(R_{kk})^*R_{kj}$

means:

**go to  $k$  !' loop there any number of times !' leave  $k$ .**

Then union them.

## MEMORY

**Direct OR Via- $k$**

$$\boxed{\text{New path} = \text{Old direct} + \text{Enter} \cdot \text{Loop}^* \cdot \text{Exit}}$$

This is the easiest way to remember the formula.

## PART C — SIMPLE EXAMPLES

### 10. EXAMPLE 1 — ONE EDGE

Automaton:



TEXT

```
! ' q 0 - - a - - > q 1 *
```

It accepts exactly:

$\{a\}$

Therefore RE:

$\boxed{a}$

## 11. EXAMPLE 2 — TWO SYMBOLS IN SERIES

TEXT

```
! ' q 0 - - a - - > q 1 - - b - - > q 2 *
```

Only accepted string:

$\{ab\}$

Therefore:

$\boxed{ab}$

## 12. EXAMPLE 3 — PARALLEL CHOICE

TEXT

```
! ' q 0 - - a - - > q 1 *  
! ' q 0 - - b - - > q 1 *
```

Equivalent edge:

$\{a+b\}$

Hence RE:

$\boxed{a+b}$

## 13. EXAMPLE 4 — LOOP THEN EXIT

Automaton:

TEXT

```
      a  
      !  
! ' q 0 - - b - - > q 1 *
```

At  $q_0$  we can read any number of a's:

$\{a^*\}$

Then mandatory b:

$\{b\}$

Therefore:

$\boxed{a^*b}$

Accepted:

TEXT

b  
ab  
aab  
aaab  
...

## 14. EXAMPLE 5 — ENTRY + LOOP + EXIT

TEXT

```
! ' q 0 - - a - - > q 1
                                ! o b
q1 --c--> q2*
```

Path structure:

TEXT

a  
then  
zero or more b  
then  
c

Therefore:

$\boxed{ab^*c}$

## PART D — STATE ELIMINATION STEP-BY-STEP

### 15. EXAMPLE

Consider:

TEXT

```
! ' q 0 - - a - - > q 1
q1 --b--> q1
q1 --c--> q2*
```

We eliminate q1.

There is:

$R_{01}=a$

$R_{11}=b$

$R_{12}=c$

No direct  $q_0 \rightarrow q_2$ , so:

$R_2 = \emptyset$

Apply:

$R'_2 = R_2 + R_1(R_1)^*R_2$

$= \emptyset + ab^*c$

Thus:

$\boxed{ab^*c}$

## 16. ANOTHER EXAMPLE WITH DIRECT PATH

Suppose:

TEXT

```
! ' q 0 - - d - - > q 2 *
! ' q 0 - - a - - > q 1
q1 --b--> q1
q1 --c--> q2*
```

There are two ways to reach  $q_2$ :

**Direct**

$d$

**Via  $q_1$**

$ab^*c$

Therefore:

$\boxed{d+ab^*c}$

Exactly the formula:

$R_2 + R_1 R_1^* R_2$

## PART E — MULTIPLE INCOMING AND OUTGOING EDGES

### 17. IMPORTANT RULE

If eliminating state  $k$  has:

- several incoming edges
- several outgoing edges

then every incoming source must be connected to every outgoing destination.

Example:

TEXT

```
p --> k --> r
q --> k --> s
```

After eliminating  $k$ , potentially add:

#### TEXT

|   |   |   |   |
|---|---|---|---|
| p | ! | ' | r |
| p | ! | ' | s |
| q | ! | ' | r |
| q | ! | ' | s |

with appropriate RE labels.

## WHY?

Because every old path through  $k$  must still exist after  $k$  is removed.

## 18. ELIMINATION PROCEDURE

Whenever removing state  $k$ :

### Step 1

Identify all incoming states  $i$ .

### Step 2

Identify all outgoing states  $j$ .

### Step 3

For every  $(i, j)$  pair calculate:

$$\lceil R_{ij} + R_{ik}(R_{kk})^*R_{kj} \rceil$$

### Step 4

Replace/update  $i \rightarrow j$  transition.

### Step 5

Remove state  $k$  and its incident edges.

## PART F — SPECIAL NEW START / NEW FINAL

## 19. WHY ADD NEW START AND FINAL?

A clean state-elimination procedure often assumes:

- exactly one start state
- exactly one final state
- start has no incoming edges
- final has no outgoing edges

If original DFA does not satisfy this, we can introduce new states.

## NEW START

Create:

TEXT

```
newStart --μ--> oldStart
```

## NEW FINAL

For every old final state:

TEXT

```
oldFinal --μ--> newFinal
```

Then old final states can be treated as ordinary intermediate states if necessary.

## WHY THIS HELPS

At the end, we can eliminate all original states until only:

TEXT

```
newStart --R--> newFinal
```

remains.

Then:

$\boxed{R}$

is the resulting regular expression.

## 20. MULTIPLE FINAL STATES

Suppose original final states:

TEXT

```
q2*  
q4*
```

Create one new final:

TEXT

```
q2 --μ--> Fnew*  
q4 --μ--> Fnew*
```

Now state elimination becomes systematic.

## PART G — DOES ELIMINATION ORDER MATTER?

## 21. IMPORTANT RESULT

Yes and no.

### Does the final language change?

No.

Any valid elimination order gives an equivalent RE.

### Does the RE look the same?

No.

Different elimination orders may produce very different-looking expressions.

## EXAMPLE IDEA

One order may produce:

$\{ a^*b(a+b)^* \}$

Another may produce a longer algebraically equivalent expression.

So:

**Different REs can describe the same language.**

## GATE TRAP

Do not assume your answer is wrong just because it looks different from another equivalent RE.

Always compare languages.

## PART H — FULL EXAMPLE

## 22. AUTOMATON

Consider:

TEXT

0  
! ' q 0 - - 1 - - > q 1 \*

At  $q_0$ :

$\{ 0^* \}$

Then move on 1.

So:

$\{ 0^*1 \}$

No need for heavy elimination.

## 23. SLIGHTLY HARDER

TEXT

$$\begin{array}{ccccccc} & 0 & & & & & \\ & & ! & 0 & & & \\ ! & ' & q & 0 & - & - & 1 & - & - & > & q & 1 & * \\ & & & & & & & ! & 0 & & & 1 \end{array}$$

Interpret:

- zero or more 0s
- then at least one 1
- then possibly more 1s

Thus:

$\boxed{0^*11^+}$

or:

$\boxed{0^*1^+}$

Equivalent basic-RE form:

$\boxed{0^*11^+}$

## 24. ANOTHER EXAMPLE

TEXT

$$\begin{array}{ccccccc} & a & & & & & \\ & & ! & 0 & & & \\ ! & ' & q & 0 & - & - & b & - & - & > & q & 1 & * \\ & & & & & & & ! & 0 & & & c \end{array}$$

Language:

TEXT

any a's  
one b  
any c's

Therefore:

$\boxed{a^*bc^*}$

## PART I — ARDEN'S THEOREM

### 25. WHY ARDEN'S THEOREM?

Another way to derive an RE from an automaton is to write equations for languages associated with states.

Example:

Suppose:

TEXT

$q_0 \xrightarrow{a} q_0$   
 $q_0 \xrightarrow{b} q_1^*$

Let:

$\{X\}$

represent the language accepted starting from  $q_0$ .

From  $q_0$ :

- on  $a$ , return to  $q_0$
- on  $b$ , reach final

So:

$\{X = aX + b\}$

How do we solve this?

Using Arden's theorem.

## 26. ARDEN'S THEOREM — BASIC FORM

If:

$\{X = AX + B\}$

and:

$\forall \epsilon \notin L(A)$

then the unique solution is:

$\boxed{X = A^*B}$

## MEMORY

**$X = AX + B \Rightarrow AB^*$**

Think:

TEXT

Repeat A zero or more times  
then finish with B

## 27. WHY DOES THIS MAKE SENSE?

Take:

$\{X = aX + b\}$

Substitute X repeatedly:

$\{X = a(aX + b) + b\}$



$$\backslash [ =a^2X+ab+b \backslash ]$$

Again:

$$\backslash [ =a^3X+a^2b+ab+b \backslash ]$$

Continuing:

$$\backslash [ X=b+ab+a^2b+a^3b+\cdots \backslash ]$$

Factor:

$$\backslash [ X=a^*b \backslash ]$$

Exactly what Arden gives:

$$\backslash [ \boxed{X=a^*b} \backslash ]$$

## 28. EXAMPLE

Equation:

$$\backslash [ X=0X+1 \backslash ]$$

Using Arden:

$$\backslash [ \boxed{X=0^*1} \backslash ]$$

## 29. ANOTHER FORM

Sometimes equation is written:

$$\backslash [ X=XA+B \backslash ]$$

Then solution becomes:

$$\backslash [ \boxed{X=BA^*} \backslash ]$$

because A repeats on the right.

## MEMORY

### Left recurrence

$$\backslash [ X=AX+B \rightarrow A^*B \backslash ]$$

### Right recurrence

$$\backslash [ X=XA+B \rightarrow BA^* \backslash ]$$

## 30. ORDER MATTERS

Because RE concatenation is generally not commutative.

So:

$$\backslash [ A^*B \backslash ]$$

and:

$\{0^*1\}$

may be different.

Do not swap them.

---

## PART J — SOLVING AUTOMATON EQUATIONS

---

### 31. EXAMPLE

---

Automaton:

TEXT

```
! ' q 0 - - 0 - - > q 0
q0 --1--> q1*
```

Let:

$R_0$

be language from  $q_0$  to a final state.

Since  $q_0$ :

- can read 0 and return  $q_0$
- can read 1 and accept

$R_0 = 0R_0 + 1$

By Arden:

$R_0 = 0^*1$

So RE:

$\boxed{0^*1}$

---

### 32. FINAL STATE EQUATIONS AND $\mu$

---

If a state itself is final, then  $\mu$  should appear as one possibility.

Example:

TEXT

```
! ' q 0 *
q0 --a--> q0
```

Language:

$a^*$

Equation:

$R_0 = aR_0 + \epsilon$

Arden:

$R_0 = a^*\epsilon$

---

$$\Sigma^* = a^*$$

## VERY IMPORTANT

For a final state, equation often includes:

$$\Sigma^* \cup \{\epsilon\}$$

because from that state, we may accept immediately without consuming further input.

## PART K — MULTI-STATE ARDEN EXAMPLE

### 33. AUTOMATON

Suppose:

TEXT

```
! ' q 0 - - a - - > q 0
q0 --b--> q1
q1 --b--> q1
q1 is final
```

Let:

$$R_0$$

= strings from  $q_0$  to acceptance.

$$R_1$$

= strings from  $q_1$  to acceptance.

### Equation for $q_1$

$q_1$  is final and loops on b.

$$R_1 = bR_1 \cup \epsilon$$

By Arden:

$$R_1 = b^*$$

### Equation for $q_0$

$q_0$  loops on a and moves on b to  $q_1$ :

$$R_0 = aR_0 + bR_1$$

Substitute:

$$R_1 = b^*$$

$$R_0 = aR_0 + bb^*$$

By Arden:

$R_0 = a^+b^+$

So language:

$\boxed{a^+b^+}$

meaning:

zero or more a's followed by one or more b's.

## PART L — STATE ELIMINATION VS ARDEN

### 34. COMPARISON

| Feature         | State Elimination | Arden's Theorem            |
|-----------------|-------------------|----------------------------|
| Main idea       | Remove states     | Solve RE equations         |
| Visual          | Graph-based       | Algebraic                  |
| Loop handling   | $R^*$             | Arden recurrence           |
| GATE usefulness | High              | High conceptual usefulness |
| Best when       | Small graph       | Equations are easy         |
| Risk            | Messy RE          | Order/algebra errors       |

## RECOMMENDATION FOR EXAM

If automaton is small and diagram-friendly:

use state elimination/intuitive paths.

If equations appear naturally:

use Arden's theorem.

## PART M — COMPLETE FA !”RE EQUIVALENCE

### 35. WE NOW HAVE BOTH DIRECTIONS

Previously:

$\text{RE} \rightarrow \epsilon\text{-NFA}$

And:

$\epsilon\text{-NFA} \rightarrow \text{DFA}$

Now:

$\text{FA} \rightarrow \text{RE}$

Therefore:

$\boxed{\text{DFA} \equiv \text{NFA} \equiv \forall \epsilon \text{-NFA} \equiv \text{Regular Expression}}$

All describe exactly:

$\boxed{\text{Regular Languages}}$

## 36. THIS IS A MAJOR TOC RESULT

Remember this diagram:

TEXT



Different representation.

Same expressive power.

## PART N — TYPICAL GATE/PYQ PATTERNS

### 37. Pattern 1 — Direct path interpretation

Given a small automaton, identify the corresponding RE.

Examples:

TEXT

loop + exit

often directly gives:

$R^*S$

### 38. Pattern 2 — State elimination

Given generalized labels, eliminate one state and compute new transition.

Use:

$\boxed{R_{ij} + R_{ik}(R_{kk})^*R_{kj}}$

### 39. Pattern 3 — Arden equation

Given:

$$\{ X = aX + b \}$$

solve:

$$\{ \boxed{a^*b} \}$$

---

## 40. Pattern 4 — Equivalent representations

Recognize that a DFA, NFA,  $\mu$ -NFA and RE may all describe the same language.

---

## 41. Pattern 5 — Choose valid elimination result

Question may test:

- whether loop got starred
  - whether parallel path was unioned
  - whether entry/exit were concatenated
- 

## 42. & pGATE TRAPS

### Trap 1

Parallel edges are concatenated.

'L Wrong.

Parallel alternatives use union.

$$\{ a + b \}$$

---

### Trap 2

Series edges use union.

'L Wrong.

Series uses concatenation.

$$\{ ab \}$$

---

### Trap 3

Self-loop contributes  $R$ , not  $R^*$ .

'L Wrong when eliminating through that state.

Loop may be traversed zero or more times:

$$\{ R^* \}$$

---

## Trap 4

---

When eliminating  $k$ , only via- $k$  path matters.

'L Wrong.

Existing direct path must remain.

So:

$$\{ R_{ij} + R_{ik}R_{kk}^*R_{kj} \}$$

---

## Trap 5

---

Different elimination orders must produce identical-looking RE.

'L Wrong.

They only need to be equivalent.

---

## Trap 6

---

Arden's equation:

$$\{ X=AX+B \}$$

has solution:

$$\{ BA^* \}$$

'L Wrong orientation.

Correct:

$$\{ A^*B \}$$

---

## Trap 7

---

$$\{ X=XA+B \}$$

has solution:

$$\{ A^*B \}$$

'L Wrong.

Correct:

$$\{ BA^* \}$$

---

## Trap 8

---

Concatenation order can be rearranged.

'L Wrong.

Generally:

$$\{ AB \neq BA \}$$

---

---

## Trap 9

---

A final state equation does not need  $\mu$ .

'L Often wrong.

If state itself is accepting,  $\mu$  must represent accepting immediately.

---

## Trap 10

---

State elimination changes the language.

'L Wrong if performed correctly.

It preserves all possible paths.

---

## 43. SHORTCUTS & MEMORY TRICKS

---

### Parallel

---

Parallel = Plus

$\{ R+S \}$

---

### Series

---

Sequence = Stick

$\{ RS \}$

---

### Loop

---

Loop = Star

$\{ R^* \}$

---

### Eliminate state

---

Direct OR Enter–Loop–Exit

$\{ \boxed{R_{ij}} + R_{ik}R_{kk}^*R_{kj} \}$

---



## Arden

**\*AX before X !' A\ before B\*\***

$$\boxed{X = AX + B \Rightarrow A^*B}$$

## 44. IMPORTANT RESULTS TO MEMORIZE

### State elimination

$$\boxed{R'_{ij} = R_{ij} + R_{ik}(R_{kk})^*R_{kj}}$$

### Arden's theorem

$$\boxed{X = AX + B \Rightarrow X = A^*B}$$

if:

$$\forall \epsilon \notin L(A)$$

And:

$$\boxed{X = XA + B \Rightarrow X = BA^*}$$

## 45. 'p WRITE THIS IN YOUR NOTES

### FA !' RE

Two common methods:

1. State elimination
2. Arden's theorem

### State elimination rules

TEXT

|                 |   |                           |
|-----------------|---|---------------------------|
| P a r a l l e l | ! | U n i o n                 |
| S e r i e s     | ! | C o n c a t e n a t i o n |
| L o o p         | ! | S t a r                   |

When eliminating state k:

$$\boxed{R'_{ij} = R_{ij} + R_{ik}(R_{kk})^*R_{kj}}$$

Memory:

**\*Direct OR Enter !' Loop !' Exit\*\***

### Clean conversion setup

Add:

TEXT

n e w   s t a r t   - - μ - - >   o l d   s t a r t

and:

TEXT

o l d   f i n a l s   - - μ - - >   n e w   f i n a l

Then eliminate intermediate states.

Final start! final = RE.

---

## Arden's theorem

If:

$$X = AX + B$$

and:

$$\epsilon \notin L(A)$$

then:

$$X = A^*B$$

If:

$$X = XA + B$$

then:

$$X = BA^*$$

---

## Major equivalence

$$\{ DFA = NFA = \epsilon\text{-NFA} = RE \}$$

in expressive power.

All characterize:

$$\{ \text{Regular Languages} \}$$

---

## 46. 60-SECOND REVISION

- FA can be converted to RE.
- Main method = state elimination.
- Parallel paths become union.
- Series paths become concatenation.
- Loops become star.
- Elimination preserves all paths through removed state.
- Formula: direct + enter·loop\*·exit.
- New start/final states make elimination easier.

- Different elimination orders may give different equivalent REs.
- Arden solves equations like  $X=AX+B$ .
- $X = A X + B$  . ! '  $A * B$
- $X = X A + B$  . ! '  $B A *$
- Final-state equations often contain  $\mu$ .
- D F A / N F A /  $\mu$  - N A F A / R E - N A F A / R E

## 47. ACTIVE RECALL

### Q1

What operation combines parallel paths?

### Q2

What operation combines sequential paths?

### Q3

How is a self-loop represented when eliminating a state?

### Q4

State the elimination formula.

### Q5

What is the intuition behind the formula?

### Q6

Why might we add a new start state?

### Q7

Why add one new final state?

### Q8

Solve:

$$\backslash [ X=aX+b \backslash ]$$

### Q9

Solve:

$$\backslash [ X=Xa+b \backslash ]$$

### Q10

Do different elimination orders necessarily give the same syntactic RE?

## ANSWERS

1. Union.
2. Concatenation.
3. Kleene star.

4.

$$\mathbb{N}[R_{ij} + R_{ik}(R_{kk})^*R_{kj}]$$

5. Direct route OR go through the eliminated state.

6. To obtain a clean single-entry structure with no incoming edge to the start.

7. To combine multiple finals and obtain a clean single-exit structure.

8.

$$\mathbb{N}[a^*b]$$

9.

$$\mathbb{N}[ba^*]$$

10. No. They may be different but equivalent.

## 48. MINI GATE TEST

### Q1 — MCQ

Suppose:

TEXT

```
i --a--> k
k --b--> k
k --c--> j
```

and there is no direct  $i \rightarrow j$  edge.

After eliminating  $k$ , the  $i \rightarrow j$  label is:

A.  $a+b+c$  B.  $abc$  C.  $ab^*c$  D.  $a^*bc$

### Q2 — MCQ

Suppose there is already an edge:

$$\mathbb{N}[i \rightarrow j]$$

along with the previous path through  $k$ .

New label becomes:

A.

$$\mathbb{N}[dab^*c]$$

B.

$$\mathbb{N}[d+ab^*c]$$

C.

$$\mathbb{N}[(d+a)b^*c]$$

D.

$$\mathbb{N}[d^*ab^*c]$$

---

### Q3 — MCQ

Solve using Arden:

$$X = 0X + 1$$

A.  $01^*$  B.  $0^*1$  C.  $10^*$  D.  $(01)^*$

---

### Q4 — MCQ

Solve:

$$X = X0 + 1$$

A.  $0^*1$  B.  $10^*$  C.  $(10)^*$  D.  $1+0$

---

### Q5 — MSQ

Which statements are true?

A. Parallel transition labels are unioned. B. Loop labels are starred during elimination. C. Series transition labels are concatenated. D. Different state elimination orders can never produce different-looking REs.

---

### Q6 — TRICKY

A DFA and an RE look structurally very different. Can they still represent exactly the same language?

A. Yes B. No

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

$$\boxed{C}$$

$$ab^*c$$

---

### Q2

$$\boxed{B}$$

Direct route plus via-k route:

$$d + ab^*c$$

---

### Q3

$$\boxed{B}$$

---

### Q4

$$\boxed{B}$$

---

because:

$\boxed{X=XA+B \Rightarrow BA^*}$

## Q5

Correct:

$\boxed{\{A,B,C\}}$

D is false.

## Q6

$\boxed{A}$

Representation differs; language may be identical.

## CHAPTER 4 — COMPLETED '

### TEXT

```
[ ' ]  R E  f u n d a m e n t a l s
[ ' ]  "    /  μ    /  s y m b o l s
[ ' ]  U n i o n    /  c o n c a t e n a t i o n    /  s t a r
[ ' ]  O p e r a t o r  p r e c e d e n c e
[ ' ]  R E  c o n s t r u c t i o n  p a t t e r n s
[ ' ]  E x a c t    /  a t  l e a s t    /  a t  m o s t
[ ' ]  P a r i t y    a n d    r e s t r i c t i o n s
[ ' ]  R E  e q u i v a l e n c e
[ ' ]  R E  ! '  μ - N F A
[ ' ]  T h o m p s o n    c o n s t r u c t i o n    i n t u i t i o n
[ ' ]  F A  ! '  R E
[ ' ]  G e n e r a l i z e d    t r a n s i t i o n    l a b e l s
[ ' ]  S t a t e    e l i m i n a t i o n
[ ' ]  E l i m i n a t i o n    f o r m u l a
[ ' ]  A r d e n ' s    t h e o r e m
[ ' ]  C o m p l e t e    F A  ! "    R E    e q u i v a l e n c e
```

## MAJOR RESULT LOCKED Ø=Ý

You should now remember permanently:

$\boxed{\text{DFA} \iff \text{NFA} \iff \text{Regular Expression}}$

All recognize exactly:

$\boxed{\text{REGULAR LANGUAGES}}$

## NEXT CHAPTER

## CHAPTER 5 — REGULAR LANGUAGES & CLOSURE PROPERTIES

Next we will learn one of the most important conceptual GATE areas:

- closure under union
- intersection

- complement
- difference
- concatenation
- Kleene star
- reversal
- homomorphism
- quotient where relevant
- decision properties
- how closure properties help prove a language regular or non-regular

And we will build the **first master closure table**.

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing exactly from your uploaded TOC master prompt and the completed **Regular Expressions + Finite Automata equivalence** section.

## CHAPTER 5 — REGULAR LANGUAGES & CLOSURE PROPERTIES

### Topic 1: Closure Properties of Regular Languages

Priority: ~~0~~ = **VERY HIGH**

This chapter is extremely important because GATE often gives languages  $\setminus(L_1, L_2)$  and asks:

**If  $\setminus(L_1)$  and  $\setminus(L_2)$  are regular, which of the following languages are guaranteed to be regular?**

You should eventually be able to answer many such questions in **seconds**.

### 1. WHAT DOES "CLOSED UNDER" MEAN?

Suppose a class of languages  $\setminus(C)$  is **closed under union**.

That means:

If

$\setminus L_1 \in C \setminus$

and

$\setminus L_2 \in C \setminus$

then necessarily:

$\setminus L_1 \cup L_2 \in C \setminus$

## SIMPLE INTUITION

Think of a closed room.

If you take two objects from inside the room, perform the specified operation, and the result still stays inside the room:

**the class is closed under that operation.**

## Example

Regular languages are closed under union.

So:

TEXT

Regular + Regular  
! " u n i o n  
Regular

## 2. MASTER FACT

Regular languages have extremely strong closure properties.

They are closed under all of the following important GATE operations:

$$\left[ \begin{array}{l} \text{Union} \\ \text{Intersection} \\ \text{Complement} \\ \text{Difference} \\ \text{Symmetric difference} \\ \text{Concatenation} \\ \text{Kleene star} \\ \text{Positive closure} \\ \text{Reversal} \\ \text{Homomorphism} \\ \text{Inverse homomorphism} \end{array} \right]$$

For most basic GATE questions:

**Regular languages are the "closure-friendly" class.**

## 3. FIRST MASTER TABLE

| Operation            | Regular Languages Closed? |
|----------------------|---------------------------|
| Union                | ' Yes                     |
| Intersection         | ' Yes                     |
| Complement           | ' Yes                     |
| Difference           | ' Yes                     |
| Symmetric Difference | ' Yes                     |
| Concatenation        | ' Yes                     |
| Kleene Star          | ' Yes                     |



| Operation            | Regular Languages Closed? |
|----------------------|---------------------------|
| Positive Closure     | ' Y e s                   |
| Reversal             | ' Y e s                   |
| Homomorphism         | ' Y e s                   |
| Inverse Homomorphism | ' Y e s                   |

This table is worth memorizing.

But we will understand **why**.

## PART A — UNION

### 4. UNION

If  $L_1$  and  $L_2$  are regular:

$L_1 \cup L_2$  is regular

### WHY?

Suppose DFA  $M_1$  recognizes  $L_1$ .

Suppose DFA  $M_2$  recognizes  $L_2$ .

Use a product automaton.

A product state is:

$(p, q)$

where:

- $p$  tracks  $M_1$
- $q$  tracks  $M_2$

For union, accept when:

**first DFA accepts OR second DFA accepts.**

### Memory

$L_1 \cup L_2$

### Alternative RE proof

If:

$R_1$

describes  $L_1$ , and:

$R_2$

describes  $L_2$ ,

then:

$R_1 + R_2$

describes:

$L_1 \cup L_2$

So union is automatically regular.

---

## 5. EXAMPLE

---

Let:

$L_1 = \{w \mid \text{w\text{ends in } 0}\}$

and:

$L_2 = \{w \mid \text{w\text{ends in } 1}\}$

Both are regular.

Then:

$L_1 \cup L_2$

contains all non-empty binary strings.

Therefore it is regular.

---

## PART B — INTERSECTION

---

### 6. INTERSECTION

---

If  $L_1, L_2$  are regular:

$L_1 \cap L_2$  is regular

---

### WHY?

---

Again use product DFA.

For intersection:

**accept only when both component DFAs are accepting.**

So:

$L_1 \cap L_2 = \text{Intersection} = \text{AND}$

---

## Example

$L_1 = \{w \mid \text{is even}\}$

$L_2 = \{w \mid \text{is odd}\}$

Both regular.

Then:

$L_1 \cap L_2$

means:

**even number of 0s and odd number of 1s.**

We already know product DFA can recognize this.

Therefore regular.

## 7. PRODUCT-STATECOUNT

If:

$|Q_1| = m$

and:

$|Q_2| = n$

product construction has at most:

$\boxed{mn}$

state pairs.

Again:

**this does not mean the minimal DFA necessarily has  $(mn)$  states.**

## PART C — COMPLEMENT

### 8. COMPLEMENT

If:

$L$

is regular, then:

$\boxed{\overline{L} \text{ is regular}}$

where:

$\overline{L} = \Sigma^* - L$

## WHY?

Take a **complete DFA** for  $\neg(L)$ .

Swap:

TEXT

f i n a l !" n o n - f i n a l

The resulting DFA accepts exactly the strings rejected before.

Therefore complement is regular.

## FORMALLY

Original:

$M = (Q, \Sigma, \delta, q_0, F)$

Complement DFA:

$M' = (Q, \Sigma, \delta, q_0, Q - F)$

## MEMORY

**Complement = Flip Finals**

But:

**complete the DFA first.**

## 9. EXAMPLE

Language:

$L = \{w \mid w \text{ contains substring } 11\}$

is regular.

Therefore:

$\overline{L}$

= strings that do **not** contain 11

is also regular.

## PART D — DIFFERENCE

## 10. DIFFERENCE

If  $(L_1, L_2)$  are regular:

$\boxed{L_1 - L_2 \text{ is regular}}$

because:

$L_1 - L_2 = L_1 \cap \overline{L_2}$

We already know:

- complement preserves regularity
- intersection preserves regularity

Therefore difference does too.

---

## MEMORY

$\boxed{A - B = A \cap \overline{B}}$

---

## 11. EXAMPLE

Suppose:

$L_1 = \Sigma^*$

and:

$L_2 = \{w \mid w \text{ contains } 11\}$

Then:

$L_1 - L_2$

is exactly:

**strings not containing 11 .**

Regular.

---

## PART E — SYMMETRIC DIFFERENCE

---

### 12. SYMMETRIC DIFFERENCE

Strings appearing in exactly one of  $(L_1, L_2)$ .

Written:

$L_1 \triangle L_2$

It can be expressed as:

$(L_1 - L_2) \cup (L_2 - L_1)$

Since regular languages are closed under difference and union:

$\boxed{L_1 \triangle L_2 \text{ is regular}}$

## PART F — CONCATENATION

### 13. CONCATENATION

If:

$\{L_1, L_2\}$

are regular, then:

$\boxed{L_1 L_2}$  is regular

Recall:

$L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$

### WHY?

Suppose REs:

$\{R_1, R_2\}$

describe  $\{L_1, L_2\}$ .

Then:

$\{R_1 R_2\}$

is itself a regular expression.

Therefore:

$\{L_1 L_2\}$

is regular.

### AUTOMATON INTUITION

Take:

TEXT

Automaton  $L_1$

! "  $\mu$

Automaton  $L_2$

Connect finals of first to start of second using  $\mu$ -transitions.

Therefore an  $\mu$ -NFA recognizes the concatenation.

### 14. EXAMPLE

$\{L_1 = 0^*\}$

$\{L_2 = 1^*\}$

Then:

$\{L_1 L_2 = 0^* 1^*\}$

which is regular.

---

## PART G — KLEENE STAR

---

### 15. KLEENE STAR

---

If:

$\boxed{L}$

is regular, then:

$\boxed{L^*}$  is regular

---

### WHY?

---

If  $R$  is an RE for  $L$ , then:

$R^*$

is an RE.

Therefore  $L^*$  is regular.

---

### $\mu$ - NFA intuition

---

Allow:

- zero repetitions
- one repetition
- return to beginning for more repetitions

Exactly our Thompson-star construction.

---

### 16. POSITIVE CLOSURE

---

$L^+ = LL^*$

Since regular languages are closed under:

- concatenation
- star

we get:

$\boxed{L^+}$  is regular

---

## PART H — REVERSAL

---

### 17. REVERSAL

---

For language:

---

$\forall L$

define:

$L^R = \{w^R \mid w \in L\}$

If  $L$  is regular:

$\boxed{L^R \text{ is regular}}$

## SIMPLE RE INTUITION

If:

$R = ab^*$

describes:

TEXT

a  
ab  
abb  
abbb  
...

reversing the strings gives:

TEXT

a  
ba  
bba  
bbba  
...

which is described by:

$b^*a$

## AUTOMATON INTUITION

Given an automaton:

1. reverse every transition
2. swap start/final roles appropriately
3. if multiple new starting possibilities arise, use an  $\mu$ -NFA with a new start state

This gives an automaton for  $L^R$ .

## & IMPORTANT

You cannot always just:

**reverse the arrows and swap one start with one final**

without accounting for multiple final states.



A new  $\mu$ -st may be needed.

---

## PART I — HOMOMORPHISM

---

### 18. HOMOMORPHISM — INTUITION

---

A homomorphism replaces each alphabet symbol with a string.

Example:

$$\forall [h(a)=01]$$

$$\forall [h(b)=1]$$

Then:

$$\forall [h(ab)=h(a)h(b)]$$

$$\forall [=011]$$

---

#### General rule

---

$$\forall [h(xy)=h(x)h(y)]$$

So symbol substitution extends naturally to strings.

---

### 19. LANGUAGE HOMOMORPHISM

---

For language  $L$ :

$$\forall [h(L)=\{h(w) \mid w \in L\}]$$

If  $L$  is regular:

$$\forall [\boxed{h(L)} \text{ is regular}]$$

---

#### INTUITION

---

Take the regular expression for  $L$ .

Replace every symbol by the corresponding homomorphic string/RE.

The result remains a regular expression.

Therefore regularity is preserved.

---

### 20. EXAMPLE

---

Suppose:

$$\forall [L=(a+b)^*]$$

and:

$$\forall [h(a)=01]$$

---

$$\forall h(b)=1$$

Then strings in  $\mathcal{L}(h(L))$  are formed from repeated blocks:

TEXT

01  
or  
1

Thus:

$$\mathcal{L}(h(L)) = (01+1)^*$$

which is regular.

## PART J — INVERSE HOMOMORPHISM

### 21. INVERSE HOMOMORPHISM

For a language  $\mathcal{L}$ , inverse homomorphism contains strings whose images belong to  $\mathcal{L}$ :

$$h^{-1}(\mathcal{L}) = \{w \mid h(w) \in \mathcal{L}\}$$

Regular languages are also closed under inverse homomorphism:

$$\boxed{h^{-1}(\mathcal{L}) \text{ is regular}}$$

when  $\mathcal{L}$  is regular.

## GATE MEMORY

**Regular survives homomorphism in both directions.**

## PART K — QUOTIENT IDEA

This is less central than union/intersection/complement but useful where syllabus depth allows.

### 22. RIGHT QUOTIENT

For languages  $\mathcal{L}_1, \mathcal{L}_2$ :

$$\mathcal{L}_1 / \mathcal{L}_2 = \{x \mid \exists y \in \mathcal{L}_2, xy \in \mathcal{L}_1\}$$

Informally:

**strings  $x$  that can be completed by some string from  $\mathcal{L}_2$  to enter  $\mathcal{L}_1$ .**

Regular languages have useful closure properties under quotients; in particular quotient constructions involving regular languages preserve regularity in the standard settings relevant to automata theory.

For core GATE preparation, prioritize the earlier closure operations first.

## PART L — HOW CLOSURE HELPS IN PROOFS

---

Closure properties are not just facts to memorize.

They are powerful **proof tools**.

---

### 23. PROVING A NEW LANGUAGE IS REGULAR

---

Suppose:

$\{L_1, L_2\}$

are known regular.

Then:

$\{L_1 \cup L_2\}^*$

is automatically regular because:

1. union of regular languages is regular
2. star of regular language is regular

No DFA construction is necessary.

---

### 24. EXAMPLE

---

Suppose  $\{L_1, L_2, L_3\}$  are regular.

Is:

$\{(L_1 - L_2) \cap L_3^R\}$

regular?

Step by step:

$\{L_1 - L_2\}$

regular by difference closure.

$\{L_3^R\}$

regular by reversal closure.

**Intersection**

regular.

Therefore:

$\boxed{\text{Yes}}$

---

### 25. VERY IMPORTANT PROOF TECHNIQUE FOR NON-REGULARITY

---

Closure properties can also help prove a language is **not regular**.

Suppose you want to show language  $\{L\}$  is not regular.

Assume:

---

$\{L\}$

is regular.

Intersect it with some carefully chosen regular language  $\{R\}$ .

Because regular languages are closed under intersection:

$\{L \cap R\}$

would have to be regular.

But suppose:

$\{L \cap R\}$

becomes a known non-regular language.

Contradiction.

Therefore  $L$  was not regular.

---

## PROOF TEMPLATE

### TEXT

Assume  $L$  is regular

!"

Choose regular  $R$

!"

Then  $L \cap R$  must be regular

!"

But  $L \cap R = \text{known non-regular language}$

!"

Contradiction

!"

$L$  is not regular

This technique becomes very useful alongside pumping lemma.

---

## 26. EXAMPLE INTUITION

Suppose a complicated language includes many patterns.

We may intersect it with:

$\{a^n b^n\}$

which is regular.

This can filter away irrelevant strings and leave something like:

$\{a^n b^n \mid n \geq 0\}$

which is known non-regular.

Then contradiction proves the original language non-regular.

We will do formal examples later.

## PART M — IMPORTANT LOGICAL TRAP

### 27. CLOSURE ONLY GIVES GUARANTEES IN THE CORRECT DIRECTION

Suppose:

$$L_1 \cup L_2$$

is regular.

Can we conclude both  $L_1$  and  $L_2$  are regular?

$$\boxed{\text{NO}}$$

Closure says:

**if inputs are regular !' result is regular.**

It does **not** say:

**if result is regular !' inputs must be regular.**

### EXAMPLE

Take a non-regular language:

$$L$$

and its complement:

$$\overline{L}$$

Their union is:

$$L \cup \overline{L} = \Sigma^*$$

which is regular.

Yet  $L$  itself may be non-regular.

### MEMORY

**Closure works forward, not backward.**

This is a very common GATE trap.

## PART N — REGULAR LANGUAGE DECISION PROPERTIES

Regular languages also have strong **decidability properties**.

Given finite automata, many questions are algorithmically decidable.

## 28. MEMBERSHIP

Given DFA  $\langle M \rangle$  and string  $w$ :

Is  $w \in L(M)$ ?

Simply run the DFA.

Therefore decidable.

## 29. EMPTINESS

Given DFA  $\langle M \rangle$ :

Is:

$L(M) = \emptyset$ ?

Check whether any accepting state is reachable from the start state.

If no final state is reachable:

$L(M) = \emptyset$

So decidable.

## 30. FINITENESS

Given DFA:

Is  $L(M)$  finite?

Conceptually, language is infinite iff there is some reachable cycle that lies on a path from the start to an accepting state and can be traversed arbitrarily many times.

So finiteness is decidable.

## 31. EQUIVALENCE

Given DFAs  $\langle M_1, M_2 \rangle$ :

Do they accept the same language?

This is decidable.

One method:

Construct symmetric difference:

$L(M_1) \triangle L(M_2)$

Then test emptiness.

If symmetric difference is empty:

$$\setminus [L(M_1) \oplus L(M_2)]$$

## 32. INCLUSION

To check:

$$\setminus [L_1 \subseteq L_2]$$

observe:

$$\setminus [L_1 - L_2]$$

must be empty.

So:

$$\setminus [L_1 \subseteq L_2 \text{ iff } L_1 \cap \overline{L_2} = \emptyset]$$

Since all required operations and emptiness are decidable for regular languages, inclusion is decidable.

## MASTER DECISION TABLE

| Problem for Regular Languages | Decidable? |
|-------------------------------|------------|
| Membership                    | '          |
| Emptiness                     | '          |
| Finiteness                    | '          |
| Equivalence                   | '          |
| Inclusion                     | '          |

This becomes useful later when comparing regular languages with CFG/TM problems.

## PART O — GATE-STYLE QUESTIONS

### 33. QUESTION 1

Suppose  $\setminus (L_1, L_2)$  are regular.

Which is guaranteed regular?

A.

$$\setminus [L_1 \cup L_2]$$

B.

$$\setminus [L_1 \cap L_2]$$

C.

$$\setminus [L_1 - L_2]$$

D. All of these

### Answer

$\boxed{D}$

---

## 34. QUESTION 2

If  $L$  is regular, is:

$\Sigma^*L$

regular?

Yes.

It is the complement.

$\boxed{\text{Regular}}$

---

## 35. QUESTION 3

If  $L$  is regular, is:

$L^R$

regular?

$\boxed{\text{Yes}}$

---

## 36. QUESTION 4

Suppose  $(L_1, L_2)$  are regular.

Is:

$(L_1L_2)^*$

regular?

First concatenation:

$L_1L_2$

regular.

Then star:

$(L_1L_2)^*$

regular.

$\boxed{\text{Yes}}$

---

## 37. QUESTION 5 — TRICKY

Suppose:

$L_1 \cup L_2$

---



is regular.

Must  $(L_1)$  and  $(L_2)$  individually be regular?

$\boxed{\text{No}}$

Closure cannot be reversed.

---

## 38. QUESTION 6

---

If  $L$  is regular and  $R$  is regular, then:

$L \cap R$

is:

$\boxed{\text{Regular}}$

This simple fact becomes very powerful in contradiction proofs.

---

## 39. QUESTION 7

---

If  $L$  is non-regular and  $R$  is regular, is:

$L \cap R$

necessarily non-regular?

$\boxed{\text{No}}$

Example:

Take:

$R = \emptyset$

Then:

$L \cap R = \emptyset$

which is regular.

---

## HUGE GATE TRAP

---

**Non-regular combined with regular does not automatically produce non-regular.**

Closure rules guarantee behavior only when the required operands belong to the closed class.

---

## PART P — CLOSURE-PROOF SHORTCUTS

---

### 40. DIFFERENCE

---

Instead of memorizing separately:

$L_1 - L_2$

---

remember:

$$\overline{L_1 \cap L_2}$$

---

## 41. SYMMETRIC DIFFERENCE

---

Remember:

$$(L_1 - L_2) \cup (L_2 - L_1)$$

---

## 42. INCLUSION

---

$$L_1 \subseteq L_2$$

iff:

$$L_1 - L_2 = \emptyset$$

---

## 43. EQUIVALENCE

---

$$L_1 = L_2$$

iff symmetric difference:

$$L_1 \triangle L_2 = \emptyset$$

---

## PART Q — & pGATE TRAPS

---

### Trap 1

---

Regular languages are not closed under intersection.

'L Wrong.

They are.

---

### Trap 2

---

Regular languages are not closed under complement.

'L Wrong.

They are.

---

### Trap 3

---

If  $(L_1 \cup L_2)$  is regular, both operands must be regular.

'L Wrong.

Closure works forward only.

---

---

### Trap 4

If  $L$  is non-regular and  $R$  regular, then  $(L \cap R)$  must be non-regular.

'L Wrong.

It may even be empty.

---

### Trap 5

Difference requires a completely new proof.

'L Not necessary.

$[L_1 - L_2 = L_1 \cap \overline{L_2}]$

---

### Trap 6

Reversing a regular language can produce a non-regular language.

'L Wrong.

Regular languages are closed under reversal.

---

### Trap 7

Homomorphism can destroy regularity.

'L Not for regular languages.

Regular languages are closed under homomorphism.

---

### Trap 8

A DFA's complement can always be obtained by flipping final states without checking missing transitions.

'L Complete the DFA first.

---

### Trap 9

If two regular languages have product automata with  $(mn)$  state pairs, the minimal DFA must have  $(mn)$  states.

'L Wrong.

Some may be unreachable/equivalent.

---

### Trap 10

Regular-language equivalence is undecidable.

'L Wrong.

---

It is decidable.

## 44. MUST REMEMBER — MASTER CLOSURE TABLE

| Operation            | Regular |
|----------------------|---------|
| Union                | '       |
| Intersection         | '       |
| Complement           | '       |
| Difference           | '       |
| Symmetric Difference | '       |
| Concatenation        | '       |
| Kleene Star          | '       |
| Positive Closure     | '       |
| Reversal             | '       |
| Homomorphism         | '       |
| Inverse Homomorphism | '       |

### Memory

**Regular languages are closed under almost every standard operation you meet in GATE.**

Later, CFL closure will be much more selective.

That contrast is important.

## 45. COMPARISON PREVIEW — REGULAR VS CFL

Do not memorize the entire CFL table yet, but notice:

| Operation     | Regular | CFL          |
|---------------|---------|--------------|
| Union         | '       | '            |
| Concatenation | '       | '            |
| Star          | '       | '            |
| Intersection  | '       | 'L generally |
| Complement    | '       | 'L generally |

This difference is heavily tested.

We'll study CFL closure properly later.

## 46. SHORTCUTS & MEMORY TRICKS

### Boolean operations

Regular languages handle:

**OR, AND, NOT**

So:

TEXT

|                         |   |       |
|-------------------------|---|-------|
| U n i o n               | ! | O R   |
| I n t e r s e c t i o n | ! | A N D |
| C o m p l e m e n t     | ! | N O T |

And from these you get difference.

### Product DFA

**Pair memories**

Used for:

- union
- intersection
- difference
- symmetric difference

Only final-state selection changes.

### Complement

**Complete !' Flip**

### Closure proof

**Known regular pieces + closed operations = regular result**

### Non-regular contradiction

**Assume regular !' filter with regular language !' obtain known non-regular language**

## 47. 'p WRITE THIS IN YOUR NOTES

### Closure Property

A language class is closed under operation  $\setminus(O)$  if:

$$\forall L_1, L_2 \in C \Rightarrow O(L_1, L_2) \in C$$

### Regular languages are closed under

$\begin{array}{l} \text{Union} \\ \text{Intersection} \\ \text{Complement} \\ \text{Difference} \\ \text{Concatenation} \\ \text{Kleene Star} \\ \text{Positive Closure} \\ \text{Reversal} \\ \text{Homomorphism} \\ \text{Inverse Homomorphism} \end{array}$

### Key formulas

Difference:

$$L_1 - L_2 = L_1 \cap \overline{L_2}$$

Inclusion:

$$L_1 \subseteq L_2 \text{ iff } L_1 - L_2 = \emptyset$$

Equivalence:

$$L_1 = L_2 \text{ iff } L_1 \triangle L_2 = \emptyset$$

### Product DFA

TEXT

```
Union          !'  at least one component final
Intersection   !'  both components final
Difference      !'  first final, second non-final
```

### Closure contradiction template

TEXT

```
Assume L regular
! "
Choose regular R
! "
L " ) R must be regular
! "
But L " ) R = known non-regular language
! "
Contradiction
```

## 48. 60-SECOND REVISION

---

- Closed means operation keeps you inside the language class.
  - Regular languages are closed under union.
  - Closed under intersection.
  - Closed under complement.
  - Closed under difference.
  - Closed under concatenation.
  - Closed under star and plus.
  - Closed under reversal.
  - Closed under homomorphism and inverse homomorphism.
  - Union/intersection can use product DFA.
  - Complement = complete DFA, flip finals.
  - Difference = intersection with complement.
  - Closure works forward, not backward.
  - Regular intersection can be used to prove non-regularity by contradiction.
  - Membership, emptiness, finiteness, equivalence and inclusion are decidable for regular languages.
- 

## 49. ACTIVE RECALL

---

Try without looking above.

### Q1

What does "closed under union" mean?

### Q2

Are regular languages closed under intersection?

### Q3

Are they closed under complement?

### Q4

Express:

$\overline{L_1 \cap L_2}$

using intersection and complement.

### Q5

Are regular languages closed under reversal?

### Q6

If  $(L_1 \cup L_2)$  is regular, must both languages be regular?

### Q7

If  $L$  is non-regular and  $R$  regular, must  $(L \cap R)$  be non-regular?

### Q8

How can equivalence of two regular languages be tested using symmetric difference?

### Q9

Is emptiness decidable for DFA languages?

### Q10

What is the main closure-based contradiction technique for proving non-regularity?

---

## ANSWERS

---

1. Union of any two regular languages must also be regular.
  2. Yes.
  3. Yes.
  4.  
 $\overline{L_1 \cap L_2}$
  5. Yes.
  6. No.
  7. No.
  8. Check whether:  
 $L_1 \triangle L_2 = \emptyset$
  9. Yes.
  10. Assume regular, intersect with a carefully chosen regular language, and derive a known non-regular language.
- 

## 50. MINI GATE TEST

---

### Q1 — MSQ

If  $L_1, L_2$  are regular, which are guaranteed regular?

A.

$$L_1 \cup L_2$$

B.

$$L_1 \cap L_2$$

C.

$$L_1 - L_2$$

D.

$$L_1 L_2$$

---

### Q2 — MCQ

Regular languages are closed under:

A. Complement only B. Union only C. Intersection only D. All three

---



---

### Q3 — MCQ

---

If  $L$  is regular, which is guaranteed regular?

A.

$[L^R]$

B.

$[L^*]$

C.

$[\overline{L}]$

D. All of these

---

### Q4 — TRICKY

---

Suppose:

$[L_1 \cup L_2]$

is regular.

Which is guaranteed?

A.  $(L_1)$  regular B.  $(L_2)$  regular C. Both regular D. None of these conclusions necessarily follows

---

### Q5 — TRICKY

---

$L$  is non-regular and  $R$  is regular.

Which statement is always true?

A.  $(L \cap R)$  is non-regular B.  $(L \cup R)$  is non-regular C.  $(L - R)$  is non-regular D. None of the above is guaranteed

---

### Q6 — MCQ

---

To prove a language  $L$  is non-regular using closure, one useful strategy is:

A. Assume  $L$  regular and intersect with a suitable regular language B. Assume every subset of  $L$  is regular C. Convert  $L$  directly to a TM D. Reverse all strings only

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

All four:

$[\boxed{A,B,C,D}]$

---

### Q2

$\boxed{D}$

### Q3

$\boxed{D}$

### Q4

$\boxed{D}$

Closure cannot be reversed.

### Q5

$\boxed{D}$

Example:

$R = \emptyset$

makes:

$L \cap R = \emptyset$

regular.

### Q6

$\boxed{A}$

## CHAPTER 5 — COMPLETED '

### TEXT

```
[ ' ]   M e a n i n g   o f   c l o s u r e
[ ' ]   U n i o n
[ ' ]   I n t e r s e c t i o n
[ ' ]   C o m p l e m e n t
[ ' ]   D i f f e r e n c e
[ ' ]   S y m m e t r i c   d i f f e r e n c e
[ ' ]   C o n c a t e n a t i o n
[ ' ]   K l e e n e   s t a r
[ ' ]   P o s i t i v e   c l o s u r e
[ ' ]   R e v e r s a l
[ ' ]   H o m o m o r p h i s m
[ ' ]   I n v e r s e   h o m o m o r p h i s m
[ ' ]   P r o d u c t - a u t o m a t o n   r e a s o n i n g
[ ' ]   C l o s u r e - b a s e d   r e g u l a r i t y   p r o o f s
[ ' ]   C l o s u r e - b a s e d   c o n t r a d i c t i o n   i d e a
[ ' ]   M e m b e r s h i p   d e c i d a b i l i t y
[ ' ]   E m p t i n e s s   d e c i d a b i l i t y
[ ' ]   F i n i t e n e s s   d e c i d a b i l i t y
[ ' ]   E q u i v a l e n c e   d e c i d a b i l i t y
[ ' ]   I n c l u s i o n   d e c i d a b i l i t y
```

## NEXT CHAPTER — ~~0=VERY~~ VERY IMPORTANT

## CHAPTER 6 — PUMPING LEMMA FOR REGULAR LANGUAGES

This is where we will slow down considerably.

The most important thing will **not** be memorizing:

$\{ w=xyz \}$

It will be understanding the logic:

TEXT

Who chooses pumping length  $p$ ?  
Who chooses string  $w$ ?  
Who chooses  $x, y, z$ ?  
Who chooses pumping value  $i$ ?

We will use an **adversarial game method** so the quantifiers become crystal clear, and then prove languages such as:

$\{ \{ a^n b^n \mid n \geq 0 \} \}$

are not regular.

Type **NEXT**.

USER

NEXT

ASSISTANT

Continuing exactly from your TOC master prompt and the completed **Regular Languages & Closure Properties** section.

## CHAPTER 6 — PUMPING LEMMA FOR REGULAR LANGUAGES

### Topic 1: Pumping Lemma Intuition, Quantifiers, Game Method, and First Non-Regularity Proof

Priority: ~~0~~=**VERY HIGH**

This is one of the most confusing topics in TOC if learned only as:

$\{ w=xyz \}$

Do **not** memorize only the formula.

The real question is:

Who gets to choose  $\{p\}$ ? Who chooses  $\{w\}$ ? Who chooses  $\{x,y,z\}$ ? Who chooses the pumping value  $\{i\}$ ?

Once this is clear, pumping lemma becomes much easier.

### 1. WHY DO WE NEED PUMPING LEMMA?

Finite automata have only finitely many states.

So if a DFA reads a sufficiently long string, it must eventually repeat some state.

That repeated state creates a loop.

And a loop can be repeated:

```
TEXT
0 times
1 time
2 times
3 times
...
```

This gives the intuition behind **pumping**.

## 2. THE CORE DFA IDEA

Suppose a DFA has:

$\lfloor p \rfloor$

states.

Now it reads a string of length at least:

$\lfloor p \rfloor$

During the first  $\lfloor p \rfloor$  input symbols, the machine visits at least  $\lfloor p+1 \rfloor$  state occurrences including the starting state.

But there are only  $\lfloor p \rfloor$  distinct states.

So by the **Pigeonhole Principle**, some state must repeat.

Example:

```
TEXT
q 0 !'   q 1 !'   q 3 !'   q 2 !'   q 1 !'   . . .
                !'
          repeated state
```

The input consumed while going:

$\lfloor q_1 \rightarrow \dots \rightarrow q_1 \rfloor$

forms a loop.

That loop can be repeated.

## MEMORY

Finite states + long string = repeated state = loop = pump

## 3. FORMAL PUMPING LEMMA

If  $\Sigma^*(L)$  is a regular language, then there exists an integer:

$$\forall p \geq 1$$

called the **pumping length**, such that every string:

$$\forall w \in L$$

with:

$$\forall |w| \geq p$$

can be written as:

$$\boxed{w=xyz}$$

satisfying:

$$\boxed{|xy| \leq p}$$

$$\boxed{|y| > 0}$$

and:

$$\boxed{\forall x^i y^j z^k \in L \quad \text{for every } i \geq 0}$$

---

## 4. THE THREE CONDITIONS

---

You must know these perfectly.

### Condition 1

---

$$\forall |xy| \leq p$$

Meaning:

**x and y together lie within the first  $p$  symbols.**

---

### Condition 2

---

$$\forall |y| > 0$$

Meaning:

**y cannot be empty.**

Something must actually be pumped.

---

### Condition 3

---

$$\forall x^i y^j z^k \in L \quad \text{for all } i \geq 0$$

Meaning:

**y may be repeated any non-negative number of times and the resulting string must remain in L.**

---

## 5. WHAT DOES $\forall (y^i)$ MEAN?

If:

$$\forall [y=ab]$$

then:

$$\forall (i=0)$$

$$\forall [y^0=\varepsilon]$$

So:

$$\forall [xy^0z=xz]$$

We remove  $y$ .

---

$$\forall (i=1)$$

$$\forall [y^1=y]$$

Original string remains.

---

$$\forall (i=2)$$

$$\forall [y^2=yy]$$

Duplicate  $y$ .

---

$$\forall (i=3)$$

Triple  $y$ .

---

## MEMORY

TEXT

```
i = 0 !' delete y
i = 1 !' original string
i = 2 !' duplicate y
```

Most proofs use:

$$\forall [i=0]$$

or:

$$\forall [i=2]$$

because they are easiest.

---

## 6. THE MOST IMPORTANT PART — QUANTIFIERS

The lemma logically says:

---

$\forall \boxed{\forall \text{regular } L, \exists p, \forall w, \exists x,y,z, \forall i}$

More explicitly:

$\exists p$

such that:

$\forall w \in L, |w| \geq p$

there exists a split:

$w=xyz$

such that:

$|xy| \leq p, |y| > 0$

and for all:

$i \geq 0$

we have:

$xy^iz \in L$

---

## 7. WHO CHOOSES WHAT?

This is the heart of pumping lemma proofs.

Imagine a game.

---

## REGULARITY GAME

If someone claims:

**$L$  is regular**

then:

**Opponent chooses**

$p$

the pumping length.

You do **not** control  $p$ .

---

**YOU choose**

a carefully designed:

$w \in L$

such that:

$|w| \geq p$

## Opponent chooses

a decomposition:

$$\backslash[ w=xyz \backslash]$$

satisfying:

$$\backslash[ |xy|\leq p \backslash]$$

and:

$$\backslash[ |y|>0 \backslash]$$

You must handle **every valid decomposition**.

## YOU choose

a pumping value:

$$\backslash[ i \backslash]$$

usually  $\backslash(0\backslash)$  or  $\backslash(2\backslash)$ .

Your goal:

$$\backslash[ xy^iz\notin L \backslash]$$

for every possible valid decomposition.

## MASTER GAME TABLE

| Step                                     | Who chooses? |
|------------------------------------------|--------------|
| Pumping length $\backslash(p\backslash)$ | Opponent     |
| Long string $\backslash(w\backslash)$    | You          |
| Split $\backslash(x,y,z\backslash)$      | Opponent     |
| Pumping value $\backslash(i\backslash)$  | You          |

## MEMORY TRICK

**THEY choose p !' I choose w !' THEY split !' I pump**

Compact:

$$\backslash[ \boxed{\text{THEY} - \text{ME} - \text{THEY} - \text{ME}} \backslash]$$

This is the single most important pumping-lemma shortcut.

## 8. WHY CAN'T WE CHOOSE $y$ ?

This is a huge mistake.



To prove non-regularity, you cannot say:

Let  $(y=a)$ .

Why?

Because the lemma says that **some valid decomposition** must work for a regular language.

Your proof must show:

**no matter how the opponent chooses a valid  $(x,y,z)$ , pumping breaks the language.**

Therefore you must characterize all possible  $y$ 's imposed by:

$\forall |xy| \leq p$

and:

$\forall |y| > 0$

## 9. WHAT IS THE PURPOSE OF $(|xy| \leq p)$ ?

It restricts  $y$  to the beginning of the chosen string.

This is why we carefully choose  $(w)$ .

Suppose we choose:

$\forall w = a^p b^p$

The first  $(p)$  symbols are all:

$\forall a$

Since:

$\forall |xy| \leq p$

both  $x$  and  $y$  must lie entirely inside those first  $(p)$   $a$ 's.

Thus:

$\forall \boxed{y = a^k}$

for some:

$\forall 1 \leq k \leq p$

This is exactly what makes the proof work.

## 10. FIRST CLASSIC LANGUAGE

Consider:

$\forall \boxed{L = \{a^n b^n \mid n \geq 0\}}$

Examples:

TEXT

$\mu$   
ab  
aabb  
aaabbb  
aaaabbbb  
...

The number of a's must equal the number of b's.

We want to prove:

$\boxed{L \text{ is not regular}}$

## 11. PROOF — STEP 1: ASSUME REGULAR

Assume, for contradiction, that:

$L$

is regular.

Then pumping lemma applies.

Therefore there exists a pumping length:

$p \geq 1$

## 12. STEP 2: CHOOSE THE STRING

We choose:

$w = a^p b^p$

Why this string?

Because:

1.  $w \in L$
2.  $|w| = 2p \geq p$
3. the first  $p$  symbols are all a's

This third property traps  $y$  inside the a-region.

## 13. STEP 3: CONSIDER ANY VALID SPLIT

Let:

$w = xyz$

where:

$|xy| \leq p$

and:

$|y| > 0$

Since the first  $p$  characters of  $w$  are all a:

$\forall y$

must consist only of a's.

Therefore:

$\boxed{y = a^k}$

for some:

$k \geq 1$

---

## 14. STEP 4: PUMP

Choose:

$i=0$

Then:

$xy^0z = xz$

We remove  $k$  a's.

The new string contains:

$p-k$

a's but still:

$p$

b's.

Therefore:

$xy^0z = a^{p-k}b^p$

Since:

$p-k \neq p$

the number of a's and b's is unequal.

Thus:

$xy^0z \notin L$

Contradiction.

Therefore:

$\boxed{L = \{a^n b^n \mid n \geq 0\} \text{ is not regular} }$

---

## 15. THE ENTIRE PROOF IN ONE FLOW

TEXT

Assume  $L$  regular

!"

Pumping length  $p$  exists

!"

Choose  $w = a^p b^p$

!"

```

Any valid split has  $|xy| \leq p$ 
Therefore  $y = a^k$ ,  $k \leq 1$ 
Pump  $i = 0$ 
 $a^{(p-k)} b^p$ 
 $\# a^{\leq 1} \# b$ 
Not in  $L$ 
Contradiction
 $L$  non-regular

```

## 16. WHY $(w=a^p b^p)$ IS A SMART CHOICE

Suppose instead we chose some random string such as:

$\{ a^{p+10} b^{p+10} \}$

It may still work.

But  $(a^p b^p)$  is simplest because:

$\{ |xy| \leq p \}$

forces  $y$  entirely into the first block.

Good pumping proofs are built around:

**designing  $w$  so the decomposition restriction helps you.**

## 17. PUMPING WITH $(i=2)$

Same proof can also use:

$\{ i=2 \}$

Then:

$\{ xy^2 z \}$

adds another copy of  $y$ .

If:

$\{ y=a^k \}$

new string has:

$\{ p+k \}$

$a$ 's and:

$\{ p \}$

$b$ 's.

So again:

$\forall p+k \neq p$

and the string is outside L.

Therefore both  $\forall(i=0)$  and  $\forall(i=2)$  work.

## 18. WHICH $i$ SHOULD I CHOOSE?

There is no universal answer.

Use the value that breaks the property most easily.

Common choices:

| Pump value     | Effect                                   |
|----------------|------------------------------------------|
| $\forall(i=0)$ | Remove $y$                               |
| $\forall(i=2)$ | Duplicate $y$                            |
| Large $i$      | Increase imbalance                       |
| Special $i$    | Sometimes needed for modular constraints |

Exam shortcut:

Try  $\forall(i=0)$  first. If awkward, try  $\forall(i=2)$ .

## PART B — WHAT PUMPING LEMMA CAN AND CANNOT PROVE

### 19. VERY IMPORTANT LOGICAL FACT

The pumping lemma gives:

$\forall \boxed{\text{Regular}} \rightarrow \text{Pumping property}$

Therefore if a language **fails** the pumping property:

$\forall \boxed{\text{Not regular}}$

But the reverse implication is false.

A language satisfying some pumping-like behavior does not automatically become regular.

### CRITICAL RULE

Pumping lemma is primarily a tool to prove NON-REGULARITY.

It generally cannot be used to prove that a language is regular.

## 20. COMMON WRONG REASONING

---

Suppose you successfully find one decomposition that pumps.

Can you conclude  $L$  is regular?

$\boxed{\text{NO}}$

The lemma says if  $L$  is regular, **every sufficiently long string** must have **some** valid pumpable decomposition.

Showing one example is nowhere near enough.

---

## 21. CONTRAPOSITIVE LOGIC

---

We know:

$\text{Regular} \rightarrow \text{Pumping property}$

Contrapositive:

$\boxed{\text{No pumping property} \rightarrow \text{Not regular}}$

This is the direction we actually use.

---

## PART C — SECOND EXAMPLE

---

### 22. LANGUAGE

---

$\boxed{L = \{0^n 1^n \mid n \geq 0\}}$

This is the binary-symbol version of the same pattern.

Assume regular.

Let pumping length be  $p$ .

Choose:

$w = 0^p 1^p$

Since:

$|xy| \leq p$

$y$  contains only zeros:

$y = 0^k, k \geq 1$

Pump:

$i = 0$

Then:

$xy^0z = 0^{p-k}1^p$

Counts unequal.

Thus:

$xy^0z \notin L$

Therefore:

---

$\boxed{\text{L is non-regular}}$

## PART D — LANGUAGE $\{a^n \mid n \geq 0\}$

### 23. IS THIS REGULAR?

$L = \{a^n \mid n \geq 0\}$

This is simply:

$a^*$

So:

$\boxed{\text{Regular}}$

Don't confuse this with:

$a^n b^n$

The problem is not an unbounded count by itself.

A DFA can accept any number of a's.

The problem is:

**comparing two unbounded counts.**

## MEMORY

TEXT

|           |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $a^n$     | ! | e | a | s | f | o | r | D | F | A |   |   |   |   |   |   |   |   |   |   |   |   |   |
| $a^n b^n$ | ! | r | e | q | u | i | r | e | s | m | a | t | c | h | i | n | g | c | o | u | n | t | s |

## PART E — WHY FINITE AUTOMATA CANNOT HANDLE $\{a^n b^n\}$

A DFA would need to remember exactly how many a's appeared so it can check the same number of b's.

Possible number of a's:

$\{0, 1, 2, 3, \dots\}$

unbounded.

But DFA has finite states.

So it cannot store arbitrary exact count.

Pumping lemma formalizes this limitation.

## 24. LANGUAGE IDENTIFICATION HEURISTIC

These patterns often suggest non-regularity:

## Equal unbounded counts

$\{a^n b^n\}$

## Exact copying

$\{ww\}$

## Mirror comparison

$\{ww^R\}$

## Prime-length or certain arithmetic properties

sometimes non-regular depending on language

## Exact distance relationships

often require unbounded memory

But these are **heuristics**, not proofs.

You still need a valid proof.

---

## 25. BE CAREFUL

---

Not every language containing an  $\{a^n\}$  is non-regular.

Example:

$\{a^{2n} \mid n \geq 0\}$

means even number of a's.

This is regular:

$\{aa^*\}$

Likewise:

$\{a^{3n} \mid n \geq 0\}$

is regular:

$\{aaa^*\}$

Modulo counting requires only finite memory.

---

## KEY DISTINCTION

---

### Regular-friendly:

$\{n \bmod k\}$

Only finite remainder information needed.

### Often non-regular:

$\{\text{compare two arbitrary unbounded values}\}$

Requires unbounded memory.

---



## PART F — A MORE SUBTLE EXAMPLE

---

### 26. LANGUAGE

---

$$L = \{a^n b^m \mid n = m\}$$

This is just another way to write:

$$\{a^n b^n\}$$

Therefore non-regular.

---

### 27. WHAT ABOUT

---

$$L = \{a^n b^m \mid n \neq m\}$$

Interesting.

Over strings restricted to  $(a^*b^*)$ , this language is the complement of  $\{a^n b^n\}$  relative to  $(a^*b^*)$ .

Can this be regular?

Suppose it were regular.

We know:

$$a^*b^*$$

is regular.

Then:

$$a^*b^* - L$$

would be regular by difference closure.

But:

$$a^*b^* - L = \{a^n b^n\}$$

which is non-regular.

Contradiction.

Therefore:

$$\boxed{\{a^n b^m \mid n \neq m\} \text{ is non-regular}}$$

This is a nice example of combining closure with a known non-regular language.

---

## PART G — PUMPING LEMMA PROOF TEMPLATE

---

Use this exact structure in your notes/exam.

---

### TEMPLATE

---

To prove  $(L)$  is not regular:

#### 1. Assume

---

Assume  $\Sigma^*$  is regular.

## 2. Pumping length

Then by pumping lemma, there exists a pumping length:

$$p \geq 1$$

## 3. Choose string

Choose:

$$w = \dots$$

such that:

$$w \in L, \text{ and } |w| \geq p$$

## 4. Consider arbitrary split

For every decomposition:

$$w = xyz$$

satisfying:

$$|xy| \leq p, \text{ and } |y| > 0$$

derive the forced form of  $y$ .

## 5. Choose pump

Choose suitable:

$$i$$

usually 0 or 2.

## 6. Show failure

Prove:

$$xy^iz \notin L$$

## 7. Contradiction

This contradicts pumping lemma.

Therefore:

$$\boxed{L \text{ is not regular}}$$

# 28. EXAM MEMORY SKELETON

TEXT

ASSUME REGULAR

!"

GET  $p$

!"

CHOOSE  $w$

!"

FOR ANY VALID SPLIT

!"

LOCATE  $y$

```

! "
PUMP 0/2
! "
BREAK LANGUAGE PROPERTY
! "
CONTRADICTION

```

Mnemonic:

**A-P-W-Y-I-C**

You may remember:

**Assume – Pump length – Word – Y-location – i – Contradiction**

## PART H — BAD PROOF EXAMPLES

### 29. WRONG PROOF 1

**Assume  $L$  regular. Let  $p$  be pumping length. Choose  $\mid(w=a^p b^p)\mid$ . Let  $\mid(x=\mid w \mid \text{varepsilon}, y=a, z=a^{\mid p-1 \mid} b^p)\mid$ . Pump down. Not in  $L$ . Hence non-regular.**

Why is this incomplete?

Because **you chose the decomposition**.

The opponent may choose another valid  $y$ .

To prove non-regularity, you must show:

**every valid decomposition fails for some pumping value.**

Correct reasoning:

Since  $\mid(xy) \mid \leq p$ , **whatever the split**,  $y$  consists only of  $a$ 's.

That handles all decompositions at once.

### 30. WRONG PROOF 2

Choose:

$\mid w = ab \mid$

for language  $\mid(a^n b^n)\mid$ .

Why bad?

Pumping lemma only guarantees pumping for strings with:

$\mid w \mid \geq p$

We don't know  $p$ .

Maybe:

$\forall p=100 \forall$

Then  $ab$  is too short.

Your chosen string must depend on  $p$ .

### 31. WRONG PROOF 3

Pump with  $\forall(i=1)$ , resulting string is still in  $L$ , so pumping lemma holds.

$\forall(i=1)$  always gives:

$\forall xy^1z=xyz=w \forall$

So it tells you nothing.

Never use  $\forall(i=1)$  to try to break the language.

### 32. WRONG PROOF 4

We found one decomposition that cannot be pumped, therefore non-regular.

Not enough.

A regular language only requires **one good decomposition for each long string**.

To defeat the lemma, your selected  $w$  must fail for **every possible valid decomposition**.

### 33. WRONG PROOF 5

Pumping lemma works, therefore language regular.

False direction.

Pumping lemma gives a necessary condition, not a sufficient characterization of regularity.

## PART I — QUANTIFIER TABLE

### 34. THIS TABLE IS EXAM GOLD

For a regular language:

| Object             | Quantifier           |
|--------------------|----------------------|
| Pumping length $p$ | $\forall(\exists p)$ |
| Long strings $w$   | $\forall(\forall w)$ |

| Object                  | Quantifier         |
|-------------------------|--------------------|
| Valid decomposition xyz | $\exists (x,y,z)$  |
| Pump amount i           | $\forall i \geq 0$ |

So:

$\exists p \forall w \exists x,y,z \forall i$

## TO PROVE NON-REGULARITY

We effectively negate it:

$\forall p \exists w \forall x,y,z \exists i$

This corresponds exactly to the game:

| Non-regularity proof | Who controls? |
|----------------------|---------------|
| Any p                | Opponent      |
| Choose bad w         | You           |
| Every valid split    | Opponent      |
| Find breaking i      | You           |

## 35. NEGATED QUANTIFIER FORM

Regular pumping property:

$\exists p \forall w \exists (x,y,z) \forall i$

Negating:

$\forall p \exists w \forall (x,y,z) \exists i$

This is why the proof order matters so much.

## MEMORY

**To disprove: flip every quantifier.**

$\exists \rightarrow \forall$

## PART J — WHY $(|y| > 0)$ ?

## 36. SUPPOSE y COULD BE $\mu$

Then:

$\forall y = \text{varepsilon}$

and for every  $i$ :

$\forall y^i = \text{varepsilon}$

So pumping would never change the string.

Every language would trivially satisfy the condition.

Therefore:

$\forall \boxed{y > 0}$

is essential.

---

## PART K — WHY $\forall (|xy| \leq p)$ ?

---

### 37. DFA PROOF INTUITION

---

The repeated state is guaranteed within the first  $\text{varepsilon}$  steps.

Therefore the loop  $y$  can be selected entirely inside the first  $p$  input symbols.

That's where:

$\forall |xy| \leq p$

comes from.

---

### 38. IS $\forall (|y| \leq p)$ ?

---

It follows from:

$\forall |xy| \leq p$

because lengths are nonnegative.

Thus:

$\forall |y| \leq p$

But the standard lemma states:

$\forall |xy| \leq p$

because that gives stronger positional information.

---

## PART L — MINI PROOF: $\forall (a^n b \mid n \geq 0)$

---

### 39. IS THIS REGULAR?

---

Yes.

RE:

$\forall a^*b$

So do not try to prove it non-regular.

---

This illustrates an important habit:

**Before using pumping lemma, first check whether you can easily build a DFA/RE.**

## 40. PRE-PUMPING CHECKLIST

Before attempting a pumping proof, ask:

1. Can I write an obvious RE?
2. Can I build a finite-count/modulo DFA?
3. Is the language a standard regular pattern?
4. Is there actually an unbounded dependency?

Only then move to pumping lemma.

## PART M — GATE/PYQ PATTERNS

Typical questions include:

### Pattern 1 — Correct pumping lemma statement

Especially quantifiers and conditions.

### Pattern 2 — Identify valid decomposition constraints

Given  $w$  and  $p$ , determine where  $y$  can occur.

### Pattern 3 — Find flaw in a pumping proof

Very common conceptual style.

### Pattern 4 — Choose correct witness string $w$

Usually designed around the first  $p$  symbols.

### Pattern 5 — Choose useful pumping value

Often  $\lfloor i=0 \rfloor$  or 2.

### Pattern 6 — Distinguish necessary vs sufficient

Pumping property is necessary for regular languages, not generally sufficient.

## PART N — & pGATE TRAPS

### Trap 1

You choose pumping length  $p$ .

'L Wrong.

For contradiction proof, assume arbitrary  $p$  supplied by the lemma.

---

## Trap 2

---

You choose  $x, y, z$ .

'L Wrong.

You must handle every valid decomposition of your chosen  $w$ .

---

## Trap 3

---

Opponent chooses  $i$ .

'L Wrong in the non-regularity strategy.

After seeing the decomposition, you choose a breaking  $i$ .

---

## Trap 4

---

$\forall |y| \geq 0$

is enough.

'L Wrong.

Must have:

$\forall |y| > 0$

---

## Trap 5

---

$\forall |xyz| \leq p$

is required.

'L Wrong.

Correct:

$\forall |xy| \leq p$

while:

$\forall |w| = |xyz| \geq p$

---

## Trap 6

---

Pumping must work for some  $i$ .

'L Wrong.

For the good decomposition of a regular language, it must work for:

$\forall \boxed{\text{all } i \geq 0}$

---



---

## Trap 7

---

If pumping down fails for one decomposition, language is non-regular.

'L Wrong.

Must defeat all valid decompositions of your witness.

---

## Trap 8

---

A language that passes pumping lemma must be regular.

'L Wrong.

---

## Trap 9

---

Use  $\forall i \geq 1$  to obtain contradiction.

'L Impossible:  $\forall i \geq 1$  gives original string.

---

## Trap 10

---

Every language with counting is non-regular.

'L Wrong.

Modulo counts are regular.

---

## 41. MUST REMEMBER

---

### Pumping lemma

---

If  $L$  regular, then:

$\forall p \geq 1$

such that every:

$\forall w \in L, |w| \geq p$

can be written:

$w = xyz$

with:

$|xy| \leq p$

$|y| > 0$

and:

$xy^iz \in L \quad \forall i \geq 0$

---

## QUANTIFIER FORM

$\boxed{\exists p \forall w \exists xyz \forall i}$

For contradiction:

$\boxed{\forall p \exists w \forall xyz \exists i}$

## PLAYER ORDER

$\boxed{\text{Opponent} \rightarrow \text{You} \rightarrow \text{Opponent split} \rightarrow \text{You}}$

## 42. 'p WRITE THIS IN YOUR NOTES

### Pumping Lemma for Regular Languages

If  $L$  is regular, then:

$\exists p \geq 1$

such that every string:

$w \in L, |w| \geq p$

can be split as:

$w = xyz$

where:

$|xy| \leq p$

$|y| > 0$

and:

$xy^iz \in L \forall i \geq 0$

### Game method

TEXT

Opponent chooses  $p$   
I choose  $w$   
Opponent chooses valid  $x, y, z$   
I choose  $i$

Memory:

THEY – ME – THEY – ME

### Proof template

#### TEXT

1. Assume  $L$  regular
2. Let  $p$  be pumping length
3. Choose smart  $w \in L$ ,  $|w| \geq p$
4. Consider ANY valid split  $xyz$
5. Use  $|xy| \leq p$  to determine form of  $y$
6. Choose  $i = 0$  or  $2$
7. Show  $xy^iz \notin L$
8. Contradiction
9. Therefore  $L$  non-regular

## Classic result

$\boxed{\{a^n b^n \mid n \geq 0\}}$  is non-regular

Witness:

$w = a^p b^p$

Since:

$|xy| \leq p$

$y = a^k, k > 0$

Pump  $(i=0)$ :

$a^{p-k} b^p$

not in  $L$ .

## 43. 60-SECOND REVISION

- Pumping lemma is mainly for proving non-regularity.
- Regular DFA has finite states.
- Long strings force repeated states.
- Repeated states create loops.
- Pumping length is  $p$ .
- Choose  $w$  with length at least  $p$ .
- Split  $w = xyz$ .
- Must have  $|xy| \leq p$ .
- Must have  $|y| > 0$ .
- Pumped string  $xy^iz$  must remain in  $L$  for every  $i \geq 0$
- In contradiction proof: opponent chooses  $p$ .
- You choose  $w$ .
- Opponent chooses split.
- You choose  $i$ .
- Never choose  $y$  yourself.
- Usually try  $i=0$  or  $2$ .
- One failing decomposition is not enough.
- Must defeat every valid decomposition.

- Passing pumping lemma does not prove regularity.

## 44. ACTIVE RECALL

### Q1

Why must a long string in a DFA contain a pumpable loop?

### Q2

State the three pumping conditions.

### Q3

Who chooses  $p$  in a non-regularity proof?

### Q4

Who chooses  $w$ ?

### Q5

Who chooses  $x, y, z$ ?

### Q6

Who chooses  $i$ ?

### Q7

Why choose  $(w = a^p b^p)$  for  $(a^n b^n)$ ?

### Q8

What does  $(i=0)$  do?

### Q9

Can pumping lemma normally prove a language regular?

### Q10

What is the regular pumping quantifier sequence?

## ANSWERS

1. Finite states plus sufficiently many transitions force a repeated state by the pigeonhole principle.

2.

$\forall [ |xy| \leq p, \quad |y| > 0, \quad xy^iz \in L \quad \forall i \geq 0 ]$

3. Opponent / pumping lemma.

4. You.

5. Opponent.

6. You.

7. It forces  $y$  into the initial block of  $a$ 's because  $(|xy| \leq p)$ .

8. Deletes  $y$ .

9. No.

10.

$\forall [ \exists x \exists y \forall z (x \neq y \wedge x \neq z \wedge y \neq z) \rightarrow \exists w (w \neq x \wedge w \neq y \wedge w \neq z) ]$

---

## 45. MINI GATE TEST

---

### Q1 — MCQ

For regular language pumping lemma, which condition is correct?

A.

$\forall [ |xyz| \leq p ]$

B.

$\forall [ |xy| \leq p ]$

C.

$\forall [ |xz| \leq p ]$

D.

$\forall [ |y| = p ]$

---

### Q2 — MCQ

Which is mandatory?

A.

$\forall [ |y| = 0 ]$

B.

$\forall [ |y| > 0 ]$

C.

$\forall [ |x| > 0 ]$

D.

$\forall [ |z| > 0 ]$

---

### Q3 — MCQ

In proving  $(a^n b^n)$  non-regular, choose:

$\forall [ w = a^p b^p ]$

Why must y contain only a's?

A. Because z must contain b's B. Because  $\forall [ |xy| \leq p ]$  C. Because  $\forall [ |y| = p ]$  D. Because p is even

---

### Q4 — MSQ

---

Which are commonly useful pumping values for contradiction?

A.  $\forall(i=0)$  B.  $\forall(i=1)$  C.  $\forall(i=2)$  D. Sometimes larger specially chosen  $i$

---

## Q5 — TRUE/FALSE

If one valid decomposition of  $w$  can be pumped successfully, then  $L$  must be regular.

---

## Q6 — GATE TRAP

To prove  $L$  non-regular using pumping lemma, which order is correct?

A. Choose  $p$  !' opponent chooses  $w$  !' choose split !' opponent chooses  $i$  B. Opponent  $p$  !' choose  $w$  !' opponent split !' choose  $i$  C. Choose  $p$  !' choose  $w$  !' choose split !' choose  $i$  D. Opponent chooses everything

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

### Q1

$\boxed{B}$

### Q2

$\boxed{B}$

### Q3

$\boxed{B}$

### Q4

Correct:

$\boxed{A,C,D}$

$\forall(i=1)$  leaves the string unchanged, so it is normally useless for contradiction.

### Q5

$\boxed{\text{False}}$

### Q6

$\boxed{B}$

Memory:

THEY – ME – THEY – ME

## CHAPTER 6 PROGRESS

### TEXT

```
[' ] Why pumping occurs
[' ] DFA - loop intuition
[' ] Formal pumping lemma
[' ] Three pumping conditions
[' ] Quantifier order
[' ] Adversarial game method
[' ] Correct choice of witness w
[' ] Meaning of  $|xy| \leq d \cdot p$ 
[' ] Pumping values  $i=0$  and  $i=2$ 
[' ] Classic proof for  $a^n b^n$ 
[' ] Common invalid proofs
[' ] Necessary vs sufficient condition
```

```
[ ] More difficult pumping-lemma proofs
[ ] Unary non-regular languages
[ ] Pattern-based witness selection
[ ] Cases where y may cross regions
[ ] Closure + pumping lemma combined
[ ] Pumping lemma limitations
```

## NEXT TOPIC

### 0=VERY MORE GATE-LEVEL PUMPING LEMMA PROOFS

Next we will solve different patterns, including languages where the witness choice is less obvious and learn **how to design  $w$**  instead of memorizing only  $(a^p b^p)$ .

Type NEXT.

### USER

NEXT

### ASSISTANT

Next we'll focus on **how to invent the pumping witness  $w$**  for unfamiliar GATE questions, not just repeat the  $(a^p b^p)$  proof.

### ASSISTANT

Continuing from your TOC master structure and the pumping-lemma fundamentals.

## CHAPTER 6 — PUMPING LEMMA FOR REGULAR LANGUAGES

### Topic 2: GATE-Level Pumping Proofs & How to Design the Witness String

Priority: 0=VERY HIGH

You already know the game:

$\boxed{\text{THEY choose } p \rightarrow \text{YOU choose } w \rightarrow \text{THEY split} \rightarrow \text{YOU pump}}}$

$\backslash$

Now the real skill is:

How do I choose the correct  $\backslash(w)$  for a new language?

## 1. THE GOLDEN WITNESS RULE

A good witness  $\backslash(w)$  should satisfy three things:

1.  $\backslash(w \text{ in } L)$
2.  $\backslash(|w| \geq p)$
3. The first  $\backslash(p)$  symbols should make  $\backslash(y)$  predictable.

The third point is the most important.

Because:

$\backslash[xy] \leq p \backslash$

means  $\backslash(y)$  lies entirely inside the first  $\backslash(p)$  symbols.

So whenever possible, make the first  $\backslash(p)$  symbols identical.

## MEMORY

Trap  $\backslash(y)$  inside a simple region.

Example:

$\backslash[ w = a^k p b^k p ]$

First  $\backslash(p)$  symbols:

TEXT  
aaaaaaaa...a

Therefore automatically:

$\backslash[ y = a^k, \text{ followed by } k \text{ } \backslash \leq p \backslash$

No need to know exactly what  $\backslash(k)$  is.

## 2. MASTER WITNESS-DESIGN PATTERNS

| Language dependency          | Smart witness idea           |
|------------------------------|------------------------------|
| $\backslash(a^n b^n)$        | $\backslash(a^p b^p)$        |
| $\backslash(a^n b^{\{2n\}})$ | $\backslash(a^p b^{\{2p\}})$ |
| $\backslash(a^n b^n c^n)$    | $\backslash(a^p b^p c^p)$    |



| Language dependency             | Smart witness idea                     |
|---------------------------------|----------------------------------------|
| $\setminus(n > m)$              | choose counts differing by only 1      |
| Square lengths $\setminus(n^2)$ | choose length $\setminus(p^2)$         |
| Prime lengths                   | choose prime $\setminus(q > p)$        |
| Multiplicative relation         | choose values involving $\setminus(p)$ |
| Complicated language            | use closure to filter it first         |

## EXAMPLE 1 — $\setminus(a^n b^{2n})$

Consider:

$$\setminus[L = \setminus\{a^n b^{2n} \mid n \geq 0\} \setminus]$$

Examples:

TEXT

$\mu$   
 abb  
 aabbbb  
 aaabbbbb  
 ...

The number of b's must be exactly twice the number of a's.

We claim:

$$\setminus[\boxed{L \text{ is not regular}} \setminus]$$

## Step 1 — Assume regular

Assume  $\setminus(L)$  is regular.

Let pumping length be:

$$\setminus[p \setminus]$$

## Step 2 — Choose witness

Choose:

$$\setminus[\boxed{w = a^p b^{2p}} \setminus]$$

Clearly:

$$\setminus[w \in L \setminus]$$

and:

$$\setminus[|w| = 3p \geq p \setminus]$$

### Step 3 — Analyze every valid decomposition

---

Let:

$$\forall w=xyz$$

such that:

$$\forall |xy| \leq p$$

and:

$$\forall |y| > 0$$

The first  $(p)$  symbols are all  $(a)$ 's.

Therefore:

$$\boxed{y = a^k}$$

where:

$$1 \leq k \leq p$$

---

### Step 4 — Pump down

---

Choose:

$$\forall i=0$$

Then:

$$\forall xy^0z = a^{p-k}b^{2p}$$

For this new string to belong to  $L$ , we would require:

$$\forall \#b = 2\#a$$

But:

$$\forall 2\#a = 2(p-k) = 2p - 2k$$

while:

$$\forall \#b = 2p$$

Since  $(k > 0)$ :

$$\forall 2p - 2k \neq 2p$$

Thus:

$$\forall xy^0z \notin L$$

Contradiction.

Therefore:

$$\boxed{\{a^n b^{2n} \mid n \geq 0\} \text{ is non-regular}}$$

---

### WHAT DID WE LEARN?

---

The dependency does not need to be:

$$\forall n = n \quad \forall$$

Even relationships like:

$$\forall m = 2n \quad \forall$$

still require comparing unbounded counts.

---

## EXAMPLE 2 — STRICT INEQUALITY

Consider:

$$\forall L = \{0^n 1^m \mid n > m\} \quad \forall$$

Examples:

TEXT

0  
00  
001  
00011  
0001  
...

We claim it is non-regular.

This requires a smarter witness.

---

## 3. DON'T CHOOSE COUNTS FAR APART

Suppose we chose:

$$\forall 0^{\{10p\}} 1^p \quad \forall$$

and pumped away a few zeros.

The inequality:

$$\forall n > m \quad \forall$$

might still remain true.

That would not help.

Instead:

**Choose the counts as close to the boundary as possible.**

Since condition is:

$$\forall n > m \quad \forall$$

make:

$$\forall n = m + 1 \quad \forall$$

---

## Smart witness

Choose:

$$\boxed{w = 0^{p+1}1^p}$$

Then:

$$p+1 > p$$

so:

$$w \in L$$

---

## Valid decomposition

Because:

$$|xy| \leq p$$

$y$  lies inside the first  $p$  zeros.

Hence:

$$y = 0^k$$

for:

$$k \geq 1$$

---

## Pump down

Choose:

$$i = 0$$

New string:

$$0^{p+1-k}1^p$$

Since  $k \geq 1$ :

$$p+1-k \leq p$$

Therefore:

$$|w| \leq 1$$

which violates:

$$|w| > 1$$

Thus:

$$xy^0z \notin L$$

Hence:

$$\boxed{\{0^n1^m \mid n > m\} \text{ is non-regular}}$$

---

## HUGE WITNESS SHORTCUT

For inequalities:

Choose a string just one unit away from the boundary.

Examples:

$\forall (n > m)$

choose:

$\forall [n = m + 1]$

$\forall (n < m)$

choose:

$\forall [m = n + 1]$

This makes a small pumping change enough to cross the boundary.

## & pGATE TRAP

For:

$\forall [n > m]$

do not automatically choose:

$\forall [0^{2p}1^p]$

Pumping out a short block of zeros may still leave:

$\forall [n > m]$

and your proof may fail.

Witness choice matters.

## EXAMPLE 3 — THREE EQUAL COUNTS

Consider:

$\forall [L = \{a^n b^n c^n \mid n \geq 0\}]$

This language is certainly not regular.

Later we will learn it is not even context-free.

For now, prove non-regularity.

## Choose

$\forall [w = a^p b^p c^p]$

Since:

$\forall [xy \leq p]$

$\forall (y)$  is entirely inside the  $\forall (a)$ -block.

Therefore:

$$y = a^k, \text{quad } k > 0$$

Pump down:

$$i = 0$$

gives:

$$a^{p-k} b^p c^p$$

Now:

$$a \neq b = c$$

So the new string is not in  $L$ .

Hence:

$$\boxed{L \text{ is non-regular}}$$

---

## IMPORTANT

Pumping lemma only needs us to break **one required property**.

We don't have to analyze every condition.

Once:

$$a \neq b$$

membership has already failed.

---

## EXAMPLE 4 — UNARY LANGUAGE WITH SQUARE LENGTH

Now let's study a very different type.

$$\boxed{L = \{a^{n^2} \mid n \geq 0\}}$$

Lengths are:

$$0, 1, 4, 9, 16, 25, \dots$$

Question:

**Is this regular?**

It is not.

---

## 4. WHY THIS IS INTERESTING

There is only **one alphabet symbol**.

So there are no separate  $a/b$  regions.

Instead, the language depends on an arithmetic property of the length.

For unary regular languages, eventually the accepted lengths must behave periodically.

Perfect squares do not have that kind of fixed periodic behavior.

Pumping lemma can show this.

---

## Step 1 — Assume regular

---

Let pumping length be:

$$\forall p \in \mathbb{N}$$

---

## Step 2 — Choose a square length

---

Choose:

$$\boxed{w = a^{p^2}}$$

Then:

$$\forall w \in L$$

for  $(n=p)$ .

Also:

$$\forall p^2 \geq p$$

for  $(p \geq 1)$ .

---

## Step 3 — Valid split

---

Since the whole string is  $a^s$ :

$$\forall y = a^k$$

where:

$$\forall 1 \leq k \leq p$$

Why  $(k \leq p)$ ?

Because:

$$\forall |xy| \leq p$$

---

## Step 4 — Pump up

---

Choose:

$$\forall i \geq 2$$

Length becomes:

$$\forall p^2 + k$$

Now:

$$\forall 1 \leq k \leq p$$

so:

$$\forall p^2 < p^2 + k \leq p^2 + p$$

---

But the next perfect square after  $(p^2)$  is:

$$(p+1)^2$$

which equals:

$$p^2 + 2p + 1$$

And:

$$p^2 + p < p^2 + 2p + 1$$

Therefore:

$$p^2 < p^2 + k < (p+1)^2$$

So  $(p^2 + k)$  lies strictly between two consecutive squares.

Thus it cannot itself be a perfect square.

Therefore:

$$xy^2z \notin L$$

Contradiction.

Hence:

$$\boxed{\{a^{n^2} \mid n \geq 0\} \text{ is non-regular} }$$

---

## MEMORY TRICK FOR SQUARES

Choose:

$$p^2$$

Then pumping adds at most:

$$p$$

But the gap to the next square is:

$$2p + 1$$

So you stay between squares.

---

## EXAMPLE 5 — PRIME LENGTH

Consider:

$$\boxed{L = \{a^n \mid n \text{ is prime}\}}$$

Examples:

TEXT

|         |          |
|---------|----------|
| aa      | length 2 |
| aaa     | length 3 |
| aaaaa   | length 5 |
| aaaaaaa | length 7 |
| ...     |          |

This language is non-regular.



This proof uses a clever pumping value rather than simply 0 or 2.

---

## Step 1 — Assume regular

---

Let pumping length be:

$$\forall p \in \mathbb{N}$$

---

## Step 2 — Choose a prime larger than p

---

There are infinitely many prime numbers.

So choose a prime:

$$\forall q > p$$

and let:

$$\boxed{w = a^q}$$

Then:

$$\forall w \in L$$

and:

$$\forall |w| = q \geq p$$

---

## Step 3 — Valid split

---

Since w contains only a's:

$$\forall y = a^k$$

for some:

$$\forall 1 \leq k \leq p$$

---

## Step 4 — Choose a special pumping value

---

This time choose:

$$\boxed{i = q + 1}$$

The pumped length is:

$$\forall$$

$$xy^{q+1}z$$

$$\forall$$

Original length was q.

We replace one copy of y by  $(q+1)$  copies, adding q extra copies of y.

Each y has length k.

So new length:

$$|q + qk|$$

Factor:

$$|q(1+k)|$$

Since:

$$|q| > 1$$

and:

$$|1+k| > 1$$

this number is composite.

Thus the pumped string has composite length.

Therefore:

$$xy^{q+1}z \notin L$$

Contradiction.

So:

$$\boxed{\{a^n \mid n \in \mathbb{P}\} \text{ is non-regular} }$$

## IMPORTANT LESSON

You are **not limited** to  $|i|=0$  or  $|i|=2$ .

The lemma lets you choose:

$$\boxed{\text{any } i \geq 0}$$

Use whatever value exploits the language property.

## WITNESS/PUMPING STRATEGY TABLE

| Property         | Useful strategy                          |
|------------------|------------------------------------------|
| Equality         | pump only one side                       |
| Inequality       | choose near boundary, pump across        |
| Fixed ratio      | alter only one component                 |
| Perfect squares  | pump into gap between squares            |
| Prime length     | pump to force composite length           |
| Modulo condition | be suspicious—it may actually be regular |

## PART B — UNARY LANGUAGES: IMPORTANT GATE INTUITION

A **unary language** has alphabet:

$\Sigma = \{a\}$

So strings are determined only by their lengths.

Examples:

### Regular

$\{a^{2n} \mid n \geq 0\}$

even lengths.

RE:

$(aa)^*$

---

### Regular

$\{a^{3n+1} \mid n \geq 0\}$

length:

$1 \bmod 3$

A DFA only needs three remainder states.

---

### Non-regular

$\{a^{n^2}\}$

---

### Non-regular

$\{a^n \mid n \text{ is prime}\}$

---

## KEY HEURISTIC

---

For unary languages:

### Eventually periodic/modulo-type length behavior

! often regular.

### Irregularly increasing gaps

such as squares/primes

! often non-regular.

This is intuition, not a substitute for proof.

---

## PART C — BAD WITNESS VS GOOD WITNESS

---

### 6. LANGUAGE

---

Again:

$L = \{0^n 1^m \mid n > m\}$

---

---

## Poor witness

---

$\forall w = 0^{100p} 1^p$

If:

$\forall y = 0$

and pump down:

$\forall 0^{100p-1} 1^p$

still:

$\forall 100p-1 > p$

So the string stays in L.

Your proof fails.

This does **not** mean the language is regular.

It means:

**your witness was weak.**

---

## Good witness

---

$\forall w = 0^{p+1} 1^p$

Now removing any non-empty y from first zeros gives:

$\forall \#0^k \#1$

immediately.

---

## MUST REMEMBER

---

**A failed pumping proof does not prove regularity.**

It may only prove that you selected a poor witness or poor pumping value.

---

## PART D — CHOOSING THE FIRST p SYMBOLS

---

### 7. WHY HOMOGENEOUS PREFIXES ARE SO POWERFUL

---

Suppose:

$\forall w = a^p b^p c^p$

Because:

$\forall |xy| \leq p$

we immediately know:

$y = a^k$

One line handles every valid split.

## Compare with a bad witness

Suppose the first  $\epsilon$  symbols alternate:

TEXT

ababababab...

Then  $y$  could be:

TEXT

a  
b  
ab  
ba  
aba  
bab  
...

Now you may need many cases.

So whenever the language permits it:

**design  $\epsilon$  with an easy first- $\epsilon$  region.**

## MEMORY

**Control the prefix to control  $y$ .**

## PART E — WHAT IF $y$ CAN CROSS A REGION?

Sometimes your witness or the lemma variant may force you to consider several possible forms for  $y$ .

Then use **case analysis**.

Example reasoning structure:

TEXT

Case 1:  $y$  contains only  $a$ 's  
!' p u m p a n d b r e a k c o u n t .

Case 2:  $y$  contains only  $b$ 's  
!' p u m p a n d b r e a k a n o t h e r c o u n t .

Case 3:  $y$  contains both  $a$  and  $b$   
!' p u m p i n g m a y d e s t r o y r e q u i r e d o r d e r i n g / p a t t e r n .

The key requirement remains:

**Every valid y-case must fail for some suitable i.**

## 8. DO NOT SKIP A CASE

Suppose y might be:

- only a's
- only b's
- mixture of a's and b's

and you only prove the first case fails.

That is not a complete pumping proof.

The opponent could choose another decomposition.

## GATE PROOF CHECK

Before saying "contradiction," ask:

**Have I covered all possible valid positions of y?**

If yes !' proof may be complete.

If no !' proof is incomplete.

## PART F — COMBINING CLOSURE WITH PUMPING

Sometimes the original language is complicated.

Instead of pumping it directly:

**Filter it using a regular language.**

## 9. EXAMPLE

Suppose:

$$L = \{0^n 1^n \mid n \geq 0\} \cup 0^*$$

Direct pumping may become annoying because some strings belong through the  $\{0^*\}$  part.

We can use closure.

Assume L were regular.

Choose regular language:

$$R = 0^* 1^+ \cup \{0^*\}$$

Then:

$$L \cap R = \{0^n 1^n \mid n \geq 1\}$$

Why does the  $\{0^*\}$  branch disappear?

Because strings in  $\{0^*\}$  contain no 1, whereas  $R$  requires at least one 1.

## Now pump the filtered language

$$\{0^n 1^n \mid n \geq 1\}$$

is non-regular by the standard proof.

But if  $L$  and  $R$  were both regular:

$$L \cap R$$

would have to be regular.

Contradiction.

Therefore:

$$\boxed{L \text{ is non-regular}}$$

## POWERFUL GATE TECHNIQUE

TEXT

```
Complicated L
    !"
Intersect with simple regular filter R
    !"
Known hard core
    !"
Use pumping lemma / known result
```

## MEMORY

**FILTER first, PUMP second.**

## PART G — WHAT CAN WE USE AS KNOWN NON-REGULAR LANGUAGES?

Once established, these become useful contradiction targets:

$$\{a^n b^n \mid n \geq 0\}$$

$$\{0^n 1^n \mid n \geq 0\}$$

and later other languages we prove non-regular.

For GATE,  $\{a^n b^n\}$  is the most useful "known hard core."

## PART H — PUMPING-LEMMA LIMITATIONS

---

### 10. PUMPING LEMMA IS NOT A MAGIC TEST

---

The pumping lemma says:

$\forall \text{Regular} \rightarrow \text{Pumping property}$

It is a **necessary property** of regular languages.

It is not generally used as a complete iff test.

---

#### If your pumping attempt fails

---

Possible reasons:

1. Language is regular.
2. Your chosen witness is poor.
3. Your pumping value is poor.
4. You chose the decomposition yourself incorrectly.
5. You failed to cover every decomposition.

So:

**Never conclude regularity simply because your contradiction attempt did not work.**

---

## PART I — SOLVED IDENTIFICATION QUESTIONS

---

### 11. LANGUAGE

---

$L_1 = \{a^{5n+2} \mid n \geq 0\}$

Regular or non-regular?

Lengths satisfy:

$|w| \equiv 2 \pmod{5}$

A DFA needs only five remainder states.

Therefore:

$\boxed{\text{Regular}}$

---

### 12. LANGUAGE

---

$L_2 = \{a^{n^2} \mid n \geq 0\}$

Square lengths.

$\boxed{\text{Non-regular}}$

---



## 13. LANGUAGE

---

$$L_3 = \{a^n b^m \mid n, m \geq 0\}$$

This is simply:

$$a^* b^*$$

Therefore:

$$\boxed{\text{Regular}}$$

No relationship between  $n$  and  $m$ .

---

## 14. LANGUAGE

---

$$L_4 = \{a^n b^m \mid n = m\}$$

Same as:

$$a^n b^n$$

Therefore:

$$\boxed{\text{Non-regular}}$$

---

## 15. LANGUAGE

---

$$L_5 = \{a^n b^m \mid n \equiv m \pmod{3}\}$$

This may look like comparison, but only the difference modulo 3 matters.

A DFA needs finite remainder memory.

So:

$$\boxed{\text{Regular}}$$

---

## IMPORTANT HEURISTIC

---

Compare:

### Exact equality

$$n = m$$

requires arbitrary exact matching !' non-regular.

### Equality modulo fixed $k$

$$n \equiv m \pmod{k}$$

requires only finitely many remainders !' regular.

---

## 16. LANGUAGE

---

$$L_6 = \{a^n b^m \mid n = 2m\}$$

---

Exact unbounded ratio.

This is non-regular.

A pumping witness can be:

$a^{2p}b^p$

But be careful with the first  $(p)$  symbols.

They're all a's, so:

$y = a^k$

Pump down and the exact relation:

$\#a = 2\#b$

breaks.

Thus:

$\boxed{\text{Non-regular}}$

## PART J — GATE WITNESS-SELECTION FLOWCHART

When you suspect a language is non-regular:

### TEXT

Does it compare unbounded counts?

YES

!"

Can I choose first  $p$  symbols all same?

YES

!"

Choose witness using  $p$

!"

Force  $y$  into first block

!"

Try  $i = 0$

Works?

/ YES

\ NO

!"

!"

finish Try  $i=2$  or special  $i$

For arithmetic unary languages:

### TEXT

Look at allowed lengths

!"

Choose a strategically special allowed length

!"

Use  $|y| \neq p$

!"

Pump into a forbidden length

## PART K — COMMON WITNESS CHOICES

---

### Equal blocks

---

$$\forall [a^n b^n]$$

choose:

$$\forall [a^p b^p]$$

---

### Ratio (2:1)

---

$$\forall [a^{2n} b^n]$$

choose:

$$\forall [a^{2p} b^p]$$

---

### Three equal blocks

---

$$\forall [a^n b^n c^n]$$

choose:

$$\forall [a^p b^p c^p]$$

---

### Greater than

---

$$\forall [n > m]$$

choose:

$$\forall [n = m + 1]$$

such as:

$$\forall [0^{p+1} 1^p]$$

---

### Perfect square length

---

choose:

$$\forall [a^{p^2}]$$

---

### Prime length

---

choose:

$$\forall [a^q]$$

where  $q$  is a prime greater than  $p$ .

---

## PART L — & pGATE TRAPS

---

### Trap 1

---

Always choose:

$$\forall w=a^p b^p$$

for every problem.

'LWrong.

Witness must reflect the language.

---

### Trap 2

---

For an inequality, choose counts very far apart.

'LOften a poor strategy.

Choose close to the boundary.

---

### Trap 3

---

Only  $\forall(i=0)$  and  $\forall(i=2)$  are allowed.

'LWrong.

Any:

$$\forall i \geq 0$$

is allowed.

---

### Trap 4

---

Unary languages are automatically regular.

'LWrong.

Examples:

$$\{a^{n^2}\}$$

and prime-length language are non-regular.

---

### Trap 5

---

Any arithmetic property of length is non-regular.

'LWrong.

Modulo-fixed-k properties are regular.

---

### Trap 6

---

If the first pumping attempt fails, language is regular.

'L Wrong.

Your witness may simply be weak.

---

## Trap 7

Once one possible form of  $y$  fails, proof is complete.

'L Wrong.

Every valid decomposition must be handled.

---

## Trap 8

Closure and pumping lemma cannot be combined.

'L Wrong.

They are very powerful together.

---

## Trap 9

For  $\forall (n \equiv m \pmod{3})$ , DFA needs exact  $n$  and  $m$ .

'L Wrong.

Only remainders matter.

---

## Trap 10

To prove non-regularity you need to pump both sections.

'L No.

Changing one section can be enough to violate the required relation.

---

## PART M — MUST REMEMBER

### Smart witness principles

---

#### Equality

$\forall [n = n]$

Choose equal  $p$ -sized blocks.

---

#### Fixed ratio

Choose counts exactly satisfying the ratio using  $p$ .

---

## Inequality

Choose values just inside the allowed region.

---

## Unary arithmetic

Choose a special accepted length and pump into a forbidden gap.

---

## Complicated language

Use:

$\mathbb{L} \cap \mathbb{R}$

with a carefully chosen regular filter.

---

## 17. 'p WRITE THIS IN YOUR NOTES

---

### How to choose $\mathbb{L}(w)$

---

A good witness:

$\mathbb{L}(w) = \{w \in L, |w| \geq p\}$

and should make:

$|xy| \leq p$

force  $y$  into a predictable region.

### Main rule

Control the first  $p$  symbols to control  $y$ .

---

## Equality example

---

$L = \{a^n b^{2n}\}$

Choose:

$w = a^p b^{2p}$

Then:

$y = a^k, k > 0$

Pump  $(i=0)$ :

$a^{p-k} b^{2p}$

ratio broken.

---

## Inequality trick

---

For:

---

$$\forall n > m$$

choose:

$$\forall n = m + 1$$

Example:

$$\forall w = 0^{p+1} 1^p$$

Then pumping down crosses the boundary.

---

## Square-length trick

$$\forall w = a^{p^2}$$

Pump to length:

$$\forall p^2 + k$$

where:

$$\forall 1 \leq k \leq p$$

and:

$$\forall p^2 < p^2 + k < (p+1)^2$$

so not square.

---

## Prime-length trick

Choose prime:

$$\forall q > p$$

$$\forall w = a^q$$

If:

$$\forall y = a^k$$

choose:

$$\forall i = q + 1$$

New length:

$$\forall q(1+k)$$

composite.

---

## Closure + pumping

TEXT

Assume  $L$  regular

!"

Choose regular filter  $R$

!"

$L \cap R$  must be regular

!"

Show L ") R is known non-regular  
!"  
Contradiction

Memory:

## FILTER !' PUMP

## 18. 60-SECOND REVISION

- Good pumping proofs depend heavily on witness choice.
- Control the first  $p$  symbols.
- Equality !' use equal  $p$ -sized blocks.
- Fixed ratio !' choose counts satisfying ratio using  $p$ .
- Inequality !' choose just one away from boundary.
- $\{a^n b^{2n}\}$  is non-regular.
- $\{0^n 1^m, n > m\}$  is non-regular.
- $\{a^n b^n c^n\}$  is non-regular.
- Unary does not imply regular.
- Square-length unary language is non-regular.
- Prime-length unary language is non-regular.
- Modulo length conditions are usually regular.
- Pumping value may be any  $\geq 0$ .
- If  $y$  can have multiple forms, handle every case.
- A failed proof attempt does not prove regularity.
- Closure can simplify a language before pumping.

## 19. ACTIVE RECALL

### Q1

What is the most useful property of the first  $p$  symbols in a witness?

### Q2

For:

$\{a^n b^{3n}\}$

what natural witness would you choose?

### Q3

For:

$\{0^n 1^m, \text{quad } n > m\}$

why is  $\{0^{p+1} 1^p\}$  better than  $\{0^{100p} 1^p\}$ ?



#### Q4

For square-length unary language, what witness do we use?

#### Q5

Why can't  $(p^2+k)$ , where  $(1 \leq k \leq p)$ , be a square?

#### Q6

For prime-length language, why choose  $(i=q+1)$ ?

#### Q7

Are all unary languages regular?

#### Q8

Is:

$$\{ \{a^{5n} \mid n \geq 0\} \}$$

regular?

#### Q9

What should you do if a direct pumping proof becomes messy?

#### Q10

If  $y$  might occur in three different kinds of regions, how many cases must your proof handle?

---

## ANSWERS

1. They should preferably force  $y$  into a predictable simple region.

2.

$$\{ a^p b^{3p} \}$$

3. Pumping down any nonempty  $y$  crosses the inequality boundary.

4.

$$\{ a^{p^2} \}$$

5. Because:

$$\{ p^2 < p^2+k < (p+1)^2 \}$$

6. It makes the pumped length:

$$\{ q+qk=q(1+k) \}$$

composite.

7. No.

8. Yes:

$$\{ (a^5)^* \}$$

9. Try a closure-based regular filter or redesign the witness.

10. All three.

## 20. MINI GATE TEST

---

### Q1 — MCQ

---

For:

$$\forall L = \{a^n b^{3n} \mid n \geq 0\}$$

which is the best standard witness?

A.  $\{a^p b^p\}$  B.  $\{a^p b^{3p}\}$  C.  $\{a^{3p} b^{3p}\}$  D.  $\{ab\}$

---

### Q2 — MCQ

---

For:

$$\forall L = \{0^n 1^m \mid n > m\}$$

which witness is particularly useful?

A.

$$\{0^{p+1} 1^p\}$$

B.

$$\{0^p 1^{p+1}\}$$

C.

$$\{0^p 1^p\}$$

D.

$$\{01\}$$

---

### Q3 — MSQ

---

Which languages are regular?

A.

$$\{a^{2n} \mid n \geq 0\}$$

B.

$$\{a^{n^2} \mid n \geq 0\}$$

C.

$$\{a^{3n+1} \mid n \geq 0\}$$

D.

$$\{a^n \mid n \text{ is prime}\}$$

---

### Q4 — MCQ

---

For:

$$w = a^{p^2}$$

---

and:

$$\forall 1 \leq |y| \leq k \leq p \quad \forall$$

pumping with  $\forall (i=2)$  produces length:

A.  $\forall (2p^2)$  B.  $\forall (p^2+k)$  C.  $\forall (p^2-k)$  D.  $\forall ((p+k)^2)$

---

## Q5 — TRUE/FALSE

---

In pumping-lemma proofs,  $\forall (i)$  must always be either 0 or 2.

---

## Q6 — GATE TRAP

---

You attempt a pumping proof and your chosen string can be pumped without leaving L.

What can you conclude?

A. L is definitely regular. B. L is definitely finite. C. Your particular proof attempt may be inadequate. D. Pumping lemma is false.

---

## SOLUTIONS — CHECK ONLY AFTER ATTEMPTING

---

### Q1

$$\forall \boxed{B} \quad \forall$$

---

### Q2

$$\forall \boxed{A} \quad \forall$$

---

### Q3

A:

$$\forall a^{\{2n\}} \quad \forall$$

is regular.

C:

$$\forall a^{\{3n+1\}} \quad \forall$$

is regular.

Squares and primes are non-regular.

Therefore:

$$\forall \boxed{\{A,C\}} \quad \forall$$

---

### Q4

One original y was already included.

Pumping  $\forall (i=2)$  adds one extra copy of y.

---

So:

$\boxed{p^2+k}$

Answer:

$\boxed{B}$

### Q5

$\boxed{\text{False}}$

Any:

$\geq 0$

may be chosen.

### Q6

$\boxed{C}$

## CHAPTER 6 — COMPLETED '

### TEXT

```
[ ' ] Pumping intuition
[ ' ] Formal pumping lemma
[ ' ] Three conditions
[ ' ] Quantifiers
[ ' ] THEY-ME-THEY-ME game
[ ' ] Classic  $a^n b^n$  proof
[ ' ] Witness design
[ ' ] Equality patterns
[ ' ] Fixed-ratio patterns
[ ' ] Inequality/boundary trick
[ ' ] Three-block dependencies
[ ' ] Unary square-length proof
[ ' ] Unary prime-length proof
[ ' ] Special pumping values
[ ' ] Bad-witness diagnosis
[ ' ] Multiple-y case awareness
[ ' ] Closure + pumping combination
[ ' ] Pumping lemma limitations
```

## MAJOR REGULAR-LANGUAGE SECTION COMPLETE Ø=Ý

We have now covered:

$\boxed{\text{DFA} \rightarrow \text{NFA} \rightarrow \text{NFA} \rightarrow \text{Regular Expressions}}$

plus:

- minimization
- closure properties
- proving non-regularity

## NEXT MAJOR CHAPTER

---

### CHAPTER 7 — CONTEXT-FREE GRAMMARS (CFG)

---

Next we move from **regular languages** to **context-free languages**.

We will start completely from zero:

- why finite automata become insufficient
- grammar intuition
- terminals
- non-terminals / variables
- productions
- start symbol
- formal 4-tuple
- how a grammar generates a string
- language generated by a grammar
- first CFGs such as

$\boxed{\{a^n b^n \mid n \geq 0\}}$

which DFA cannot recognize but CFG can generate.

Type **NEXT**.